.scratch compiled

1. talent-scout-hld
2. kernel-lld
3. kernel-build
4. gateway-lld
5. gateway-build
6. runtime-lld
7. runtime-build
8. request-plan-lld
9. request-plan-build
10. collection-lld
11. collection-build
12. analysis-lld
13. analysis-build
14. brief-lld
15. brief-build
16. web-ui
17. web-ui-build

talent-scout-hld

talent-scout-hld/map.md

Talent Market Scout: HLD

Label: wayfinder:map

Destination

`docs/HLD.md` for the Talent Market Scout Agent (Joveo assessment, due Mon 2 Nov 2026, 9:00 AM) plus an ordered component breakdown, where each component becomes its own later LLD/build effort. No LLD or code decisions on this map.

Notes

- Domain: LangGraph/LangChain agent (Python) that turns a Hiring Request into an evidence-backed Market Brief. Spec: `Joveo_Talent_Market_Agent- Assessment.pdf`. Vocabulary: `GLOSSARY.md`.
- Stack fixed by the user: Python, LangChain + LangGraph, OpenRouter for LLMs (models picked in LLD), LangSmith tracing, Adzuna (25 req/day) + USAJOBS keys in root `.env`, Jev (on OpenRouter) for probabilistic routing.
- Skills: `langchain-router` first for any framework question (per `CLAUDE.md`), then the skills it names; `langgraph-decision-models` for Jev; `grilling` + `domain-modeling` for grilling tickets.
- Standing preferences: the reference flow in the PDF is a suggestion, not a contract. Infra (API, queues, workers, threading, concurrency, MiniStack) is a future phase after core requirements. Durable resume at any Pause is non-negotiable.

Decisions so far

- What does the HLD cover, and what is deferred to LLD?: HLD = components, graph shape, state ownership, infra topology, build order; field types, prompts, limits, models deferred to per-component LLD.
- What is the build priority order?: must → recommended → streaming → infra shell (future phase) → critic → long-term memory.
- Which jobs sources are used, and how are tools executed?: Adzuna primary, USAJOBS for US; `@tool`s run by a custom executor node instead of ToolNode.
- How does the Jobs Data Gateway cache, budget and expire data?: L1 exact-query cache + L2 region × Domain Posting store, hard/soft expiry with filter-on-read and per-Run sweep, budget

guard, replay mode.

- What does the LLM do versus deterministic code?: LLM extracts per-Posting facts; a separate `stats` node computes every number; Sufficiency is a rule.
- What is the node-level graph shape?: full mermaid graph with three Pauses, fetch/broaden loop, Send fan-out, critic loop, zero-Postings edge.
- Where does Jev make routing decisions?: clarity check, Posting relevance, critic.
- How are LLMs tiered, and how are failures handled?: OpenRouter, tier concept + fallback chains, validation-error retry; models deferred.
- Which clients exist, and how do they talk to the graph?: CLI core, optional Streamlit, all via a Run Controller seam.
- How is state persisted so a Run survives the user leaving?: SQLite checkpointer + Store; pure Pause nodes, run registry, re-shown Pause payloads.
- How are Runs traced and explained?: LangSmith plus a run log in state feeding Run Notes and samples.
- Research: Jev on OpenRouter: `typesafe/jev-1.14` on OpenRouter returns full Noul/Choice/Score probabilities; `TypeSafeClassifier` works via `base_url`, same OpenRouter key; keep requests under ~32 KB.
- Research: Adzuna API facts: no expiry field (soft expiry + `max_days_old`), per-country categories endpoint, local-currency salaries with unstated period, descriptions truncated to 500 chars, quota unconfirmed (check dashboard), commercial-use licence caveat.
- Research: LangGraph persistence and resume semantics: SqliteStore exists; durable resume needs `durability="sync"`; one `interrupt()` per Pause node; registry in a Store namespace; Run Controller must guard concurrent resumes; ToolNode churn supports the custom executor.
- Research: USAJOBS API facts: 3-header auth, 500/page, salary with rate-interval codes, close date present (hard expiry works); federal-only, and its pay-table salaries must not mix unlabelled into market stats.
- Component breakdown and build order: 10 concern-cut components (Kernel, Gateway, Request & Plan, Collection, Analysis, Brief, Runtime, Clients, Long-term memory, Infra shell); slices declared in Kernel, owned per stage; built in tier passes, must-have walking skeleton first.
- Assemble docs/HLD.md: written at docs/HLD.md; map destination reached.

Not yet specified

Empty: the component breakdown settled state ownership and handed the remaining fog to component LLDs (see Out of scope).

Out of scope

- Supervisor multi-agent setup (assessment stretch goal): not on the user's requirement list; may be revisited as a separate effort.
- LLD-level questions handed to their component's own LLD effort by Component breakdown and build order, since the destination stops at the HLD:

- **Analysis**: currency/period normalisation (Adzuna local currency with no stated period, USAJOBS `RateIntervalCode`, `salary_is_predicted`), multi-source stats (USAJOBS pay tables vs Adzuna), batching for Samples over 500 (snippet-based skills evidence is a known write-up limitation).
- **Brief**: Confidence Tag and Hiring Outlook rules.
- **Jobs Data Gateway**: whether Adzuna histogram / top_companies / history feed the brief (no Posting IDs, so not strict Evidence).
- **Request & Plan + Clients**: which nodes or prompts each `--focus` flag changes.
- **Runtime**: streaming event model.
- **Long-term memory**: which preferences are remembered, learned vs asked.
- **Infra shell**: API surface, queue on MiniStack, worker model, threading vs async, shared daily Adzuna budget across concurrent Runs, persistence migration off SQLite, web UI.
- **Delivery task**: sample-run strategy for the three `samples/` requests within the Adzuna budget.

What does the HLD cover, and what is deferred to LLD?

Type: grilling Status: resolved

Question

What is the end state of this map, and where is the line between HLD (decided here) and LLD (decided later, per component)?

Answer

Resolved during charting.

- Destination: `docs/HLD.md` plus an ordered component breakdown.
- HLD covers: system context, components with responsibilities and boundaries, node-level graph shape (nodes, edges, Pauses, loops), concept-level state ownership, infra topology, where Jev sits, build order.
- Deferred to one LLD/build effort per component: exact state fields and types, prompt text, retry counts and limits, thresholds, library and model picks.
- Supervisor multi-agent stretch goal is out of scope for now (may be revisited later).

What is the build priority order when time runs short?

Type: grilling Status: resolved

Question

In what order are requirement tiers built, given a deadline of Mon 2 Nov 2026, 9:00 AM, and where does the infra layer sit?

Answer

Resolved during charting.

1. Must-haves (assessment minimum bar)
2. Recommended (Send map-reduce, plan-approval and low-data Pauses, retries/fallbacks, malformed-output recovery, CLI/Streamlit, `--focus` flags)
3. Streaming progress
4. Infra shell (API, queues, workers, concurrency, threading, MiniStack): explicitly a **future phase** after core requirements are done
5. Critic pass
6. Long-term memory

The core graph must never depend on the infra shell; the shell wraps it later through the Run Controller seam (see Client surfaces and the Run Controller seam).

Which jobs sources are used, and how are tools executed?

Type: grilling Status: resolved

Question

Which public jobs APIs feed the agent, how are they exposed to the model, and what runs the tool calls?

Answer

Resolved during charting.

- **Adzuna** is the primary source (multi-country, salary fields; 25 requests/day budget).
- **USAJOBS** is a secondary tool, used by the model only for US locations (federal roles only).
- Both are standard LangChain `@tool`s bound to the fetch model, so the model decides which to call.
- Tool calls run through a **custom tool executor node** (reads `tool_calls`, dispatches by name, returns `ToolMessage`s) instead of the prebuilt `ToolNode`, to insulate from LangChain internals churn. This satisfies "ToolNode or equivalent" and is fully explainable live.
- Tools never call the APIs directly; they go through the Jobs Data Gateway (see Jobs Data Gateway caching and expiry).

Update from Gateway query interface (2026-10-30): the two source tools become one `search_jobs` tool. The Gateway picks sources by rule (Adzuna always, USAJOBS when the country is `us`), fans out, merges and collapses Duplicates, so the model no longer chooses the source.

Update (2026-10-31), reversing the update above: back to two tools, `search_adzuna` and `search_usajobs`, because the spec evaluates tool calling and a router inside one tool hides the model's choice. See the Collection LLD §5.

How does the Jobs Data Gateway cache, budget and expire data?

Type: grilling Status: resolved

Question

With 25 Adzuna requests/day covering dev, samples and the live walkthrough, how does the data layer avoid wasting calls, and how does stale data leave the cache?

Answer

Resolved during charting. A **Jobs Data Gateway** sits outside the graph, between the tools and the APIs.

- **L1, exact-query cache**: normalised request params → raw response. Makes repeat runs, tests and replay free. Short TTL of its own, since live results drift. Must-have.
- **L2, region × Domain Posting store**: every fetched Posting is stored, deduped by ID, tagged with region (from Adzuna's own location hierarchy + lat/long, no custom geohash) and Domain (from Adzuna's category tag; USAJOBS job series mapped onto the same list). Before calling the API, the Gateway checks whether L2 already holds enough fresh Postings for that region + Domain + role; if so it skips the call, otherwise it merges new results in. Built later than L1.
  - Rejected: a pure region + Domain cache as a *substitute* for role queries, because Adzuna returns at most ~50 results per page, so a broad Domain pull rarely contains enough of a specific role.
- **Expiry**: hard expiry when the source gives a close date that has passed (USAJOBS `ApplicationCloseDate`); soft expiry when the Posting is older than a freshness window (window length is LLD). Expired rows are filtered on read (never served) and purged by a sweep at the start of each Run. No background process in core; in the infra phase the sweep becomes a scheduled job.
- **Budget guard**: tracks daily Adzuna calls and refuses before the limit.
- **Replay mode**: the whole graph runs from cached responses; doubles as the test harness.
- **Update from Research: Adzuna API facts**: the ~50 per page limit is unconfirmed (no documented max). The L2 rationale still holds because each extra page of a broad Domain pull costs quota. Adzuna has no expiry field, so Adzuna Postings only ever soft-expire.
- **Update from L1 cache, budget guard and replay**: L1 stores mapped Postings, not raw responses. A 24-hour TTL triggers a refetch, and a row is deleted only once every Posting in it has closed. Adzuna's limit is confirmed at 25 calls a day.

- **Update from L2 store, expiry and sweep**: L2 holds Adzuna Postings only, skips a call only when it can fill the requested page, and also backs failed live calls. Soft expiry is 7 days after Adzuna last returned a Posting. The sweep runs after each live write, not at Run start.
- **Prompt caching** is a prompt-construction principle, not a component: stable instructions and schema first so batch calls share a cached prefix.

What does the LLM do versus deterministic code?

Type: grilling Status: resolved

Question

Where is the line between LLM work and deterministic code, given that every number in the Market Brief must trace back to real Postings? Should analysis be a tool the LLM calls?

Answer

Resolved during charting.

- **LLM extracts per-Posting facts**: normalised skills, seniority, must-have vs nice-to-have, years of experience.
- **Python computes every number**: counts, salary median and middle 50%, pay-disclosure share, top employers, posting-date trend.
- The brief writer receives precomputed stats plus Posting IDs; it phrases and selects Insights, never invents figures.
- Analysis runs as a **separate deterministic `stats` node**, not as a tool: there is no decision for the model to make (stats always run), and a tool would add nondeterminism, tokens and a failure mode while hiding a fixed step inside the model loop.
- **Sufficiency** is a code/rule decision over API-returned data (Relevant Posting count vs the Search Plan target, capped attempts), not an LLM opinion. Exact numbers are LLD.

What is the node-level graph shape?

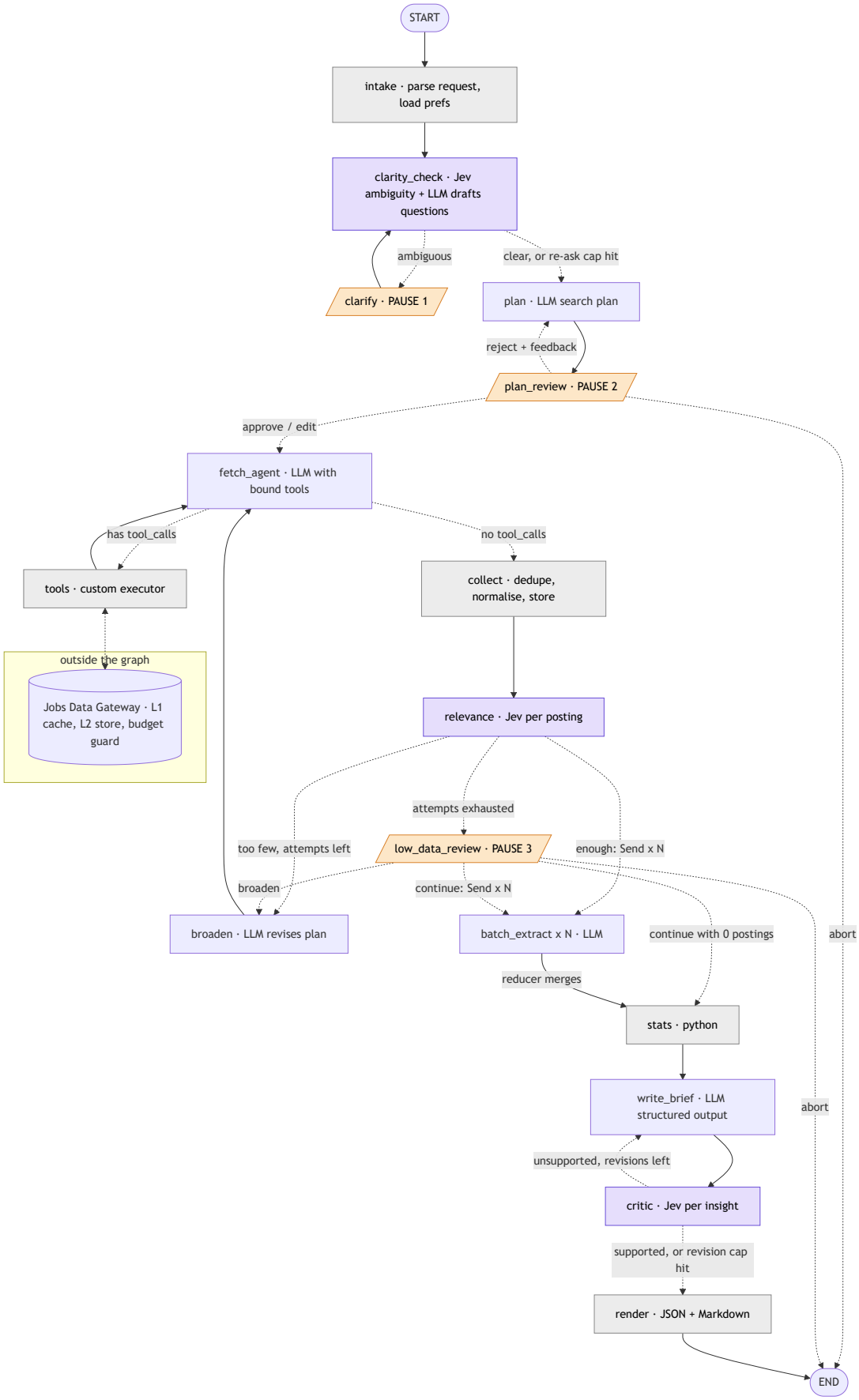
Type: grilling Status: resolved

Question

What nodes, fixed edges, conditional edges and Pauses make up the StateGraph?

Answer

Resolved during charting. Solid = fixed edge, dotted = conditional edge (label is the router outcome). Exact caps (re-asks, attempts, revisions) are LLD.


START
intake · parse request, load prefs
clarity_check · Jev ambiguity + LLM drafts questions
ambiguous
clear, or re-ask cap hit
clarify · PAUSE 1
plan · LLM search plan
reject + feedback
plan_review · PAUSE 2
approve / edit
fetch_agent · LLM with bound tools
has tool_calls
no tool_calls
tools · custom executor
collect · dedupe, normalise, store
outside the graph
Jobs Data Gateway · L1 cache, L2 store, budget guard
relevance · Jev per posting
attempts exhausted
too few, attempts left
low_data_review · PAUSE 3
enough: Send x N
broaden
continue: Send x N
broaden · LLM revises plan
batch_extract x N · LLM
continue with 0 postings
abort
reducer merges
stats · python
write_brief · LLM structured output
abort
unsupported, revisions left
critic · Jev per insight
supported, or revision cap hit
render · JSON + Markdown
END

Notes:

- The `continue with 0 postings → stats` edge exists because a zero-length `Send` fan-out never reaches `stats`; without it the Run would stall silently.
- A vague answer to a Clarifying Question is re-asked at most once, then the Run proceeds with a recorded Assumption.
- Plan rejection returns to `plan` with the user's feedback; abort ends the Run cleanly.

Where does Jev make routing decisions?

Type: grilling Status: resolved

Question

Which decisions use Jev's probabilistic outputs instead of an LLM, limited to semantic judgements that cannot be computed?

Answer

Resolved during charting. Three placements:

1. **Clarity check**: is the role, location or seniority too ambiguous to search on? Drives clarify vs skip. An LLM still drafts the actual Clarifying Questions.
2. **Posting relevance**: is this Posting actually the researched role (vs keyword collisions)? Only Relevant Postings count toward Sufficiency.
3. **Critic** (stretch): is each Insight supported by the Postings it cites? Drives the revise loop.

Optional later: staffing agency vs direct employer.

Everything else is computed in code or produced by the LLM. Thresholds live in one place and are tuned per model (Jev calibration does not transfer between models). How Jev is reached on OpenRouter is open: see Research: Jev on OpenRouter.

How are LLMs tiered, and how are failures handled?

Type: grilling Status: resolved

Question

What LLM structure does the HLD commit to, and what is deferred?

Answer

Resolved during charting.

- All LLM calls go through **OpenRouter**.
- HLD keeps only the *concept* of tiers (a cheaper tier for batch extraction and question drafting, a stronger tier for planning, brief writing and critique) and a `.with_fallbacks()` chain per tier.
- Malformed structured output: retry with the validation error fed back, then fall back.
- Specific models are decided later, in LLD.

Which clients exist, and how do they talk to the graph?

Type: grilling Status: resolved

Question

Which user-facing clients does the core ship, and what seam keeps the later infra phase from being a rewrite?

Answer

Resolved during charting.

- **CLI is the core client**: handles all three Pauses, `--focus` flags, `--resume`, streamed progress.
- **Streamlit** is an optional recommended-tier client; its rerun-per-interaction model fits checkpointing (each rerun resumes the thread).
- A **Run Controller** (start / resume / stream events / get state) is the only thing clients call; none touch the graph directly. The CLI, Streamlit and the future API worker all sit on it, which makes the infra shell an addition rather than a rewrite.

How is state persisted so a Run survives the user leaving?

Type: grilling Status: resolved

Question

Which persistence backs the core, and how does the system guarantee (non-negotiable) that a user can close the CLI or browser at any Pause and resume exactly where they left off?

Answer

Resolved during charting.

Backend: SQLite checkpointer for threads, SQLite-backed Store for long-term preferences (Store availability to be verified, see Research: LangGraph persistence and resume semantics).

- MemorySaver rejected: dies with the process, fails resume-later.
- SQLite: first-party, zero infra, a file that survives restarts; evaluators can clone and run.
- Postgres: right for concurrent writers but needs a server; candidate for the infra phase.
- Redis / DynamoDB-on-MiniStack: need a server, less mature savers.
- Known ceiling: single writer. Fine for CLI; replaced in the infra phase.

Durable resume rules (HLD-level, non-negotiable):

1. Checkpoints are on disk after every node, including the pending Pause payload (the exact Clarifying Questions). Closing or crashing loses at most the in-flight node.
2. **Pause nodes are pure**: `clarify`, `plan_review`, `low_data_review` only call `interrupt()` and write the answer. Anything generative (e.g. drafting questions) happens in the node before, because the paused node re-runs from its top on resume.
3. Returning re-shows the same Pause: the Run Controller loads the thread, finds the pending Pause, re-displays its stored payload, resumes with `Command(resume=answer)`.
4. A **run registry** (thread, request summary, status, updated time) means users never need to remember a thread ID; CLI `--resume` lists resumable Runs, Streamlit keeps the thread ID in the URL.
5. A kill mid-node re-runs that node from the last checkpoint; repeated fetches are served from the L1 cache, costing no Adzuna quota.

Amendments from Research: LangGraph persistence and resume semantics:

- Rules 1 and 5 hold only with `durability="sync"` on every graph call (default `async` can roll a just-answered Pause back). The Run Controller always passes `sync`.
- Checkpoints are per super-step, not strictly per node; parallel `batch_extract` branches share one.
- Rule 2 tightened: exactly one `interrupt()` per Pause node, never inside `try/except`.
- Rule 4: the run registry lives in a `("runs",)` namespace of the same SQLite Store (the checkpointer cannot list threads cheaply).
- New: the Run Controller guards against two processes resuming the same thread at once, since LangGraph does not.

How are Runs traced and explained?

Type: grilling Status: resolved

Question

How are routing decisions made visible to evaluators, who may not have access to the LangSmith project?

Answer

Resolved during charting.

- **LangSmith** traces everything.
- The graph also keeps its own **run log in state**: routing decisions with reasons, Clarifying Questions and answers, the final Search Plan. It feeds the Market Brief's Run Notes and the `samples/` transcripts, so samples stand on their own without LangSmith access.

Research: Jev on OpenRouter

Type: research Status: resolved Findings: branch `research/jev-on-openrouter`, file `research/jev-on-openrouter.md`

Question

How is Jev (TypeSafe's decision model) exposed on OpenRouter, and can it serve the three placements in Where does Jev make routing decisions??

Specifically:

- Model id(s) on OpenRouter, pricing, rate limits.
- API shape: OpenAI-style chat completions only, or the TypeSafe `/v1/systemone` classifier contract (Noul / Choice / Score with probabilities)?
- Does it return probabilities / confidence through OpenRouter, or only text? If only text, how are probabilities obtained (logprobs?), and what is lost?
- Can `langchain-typesafe`'s `TypeSafeClassifier` be pointed at OpenRouter via `base_url`, or is a thin adapter needed?
- Fallback: if Jev on OpenRouter cannot return probabilities, what are the realistic options (direct TypeSafe API, LangSmith Gateway BYOK, LLM structured-output classifier)?

Answer

Full findings on branch `research/jev-on-openrouter` (commit `d596f2a`), file `research/jev-on-openrouter.md`. Verified against source and a mocked request; no live call yet (no key in env), so the first real call is the confirmation step.

- **Available, with full probabilities.** `typesafe/jev-1.14` (alias `~typesafe/jev-latest`; `jev-1.13` still listed). $0.042/M input, output free, cost in `usage.cost`. No published rate limit.
- **Not a chat model**: no `/chat/completions`, no logprobs involved. Hidden from the default model list (`?output_modalities=decisions` shows it).
- **Two endpoints, one OpenRouter key**: `POST /api/v1/systemone` (TypeSafe-native contract) and `POST /api/alpha/decisions` (OpenRouter's cross-vendor decisions format). Both return Noul probability, Choice per-label distribution + confidence, Score expected value + per-level distribution + confidence.
- **`TypeSafeClassifier` works unchanged**: `base_url="https://openrouter.ai/api"`, OpenRouter key, pin `model="typesafe/jev-1.14"`.
- **Watch-outs for LLD**: the classifier drops `usage.cost` and request id; set both or neither Noul criteria sides; keep each request under ~32 KB of text (above that Jev switches to a slower, less

accurate long-context mode), which affects long Posting descriptions and the critic.
- **Fallback order**: OpenRouter decisions endpoint via thin wrapper → direct TypeSafe → LangSmith Gateway BYOK (skill-sourced, unconfirmed in docs) → LLM structured output (loses distributions).

Consequence for the HLD: the Decision Provider component reaches Jev through OpenRouter with the same key as the LLMs; no extra credential.

Research: Adzuna API facts

Type: research Status: resolved Findings: branch `research/adzuna-api-facts`, file `research/adzuna-api-facts.md`

Question

What does the Adzuna jobs API actually provide, as needed by Which jobs sources are used? and Jobs Data Gateway caching and expiry?

Specifically:

- Search endpoint params: `what`, `what_or`, `where`, `distance`, `max_days_old`, `category`, `salary_*`, sort options; supported countries.
- Max `results_per_page` and practical pagination depth; does the response report a total `count`?
- Rate limits for the free tier (confirm 25/day and any per-minute limit), and how limit errors surface.
- Posting fields: id, title, company, location (`area` hierarchy, lat/long), category tag, `created` date, salary min/max, `salary_is_predicted`, contract type/time, description length (truncated?).
- Does a posting carry any expiry / closing signal?
- Category taxonomy endpoint and the list of category tags (for Domain mapping).
- Other endpoints useful for market stats (histogram, top companies, history) and whether they cost the same quota.
- Currency and salary period per country.

Answer

Full findings on branch `research/adzuna-api-facts` (commit `39cd737`), file `research/adzuna-api-facts.md`. Sourced from Adzuna's OpenAPI spec, developer guides, legacy field docs and API Terms; no live call.

- **Paging**: no documented max for `results_per_page` or page depth; every response has `count` (total matches) and `mean` salary. The "~50 per page" in Jobs Data Gateway caching and expiry is unconfirmed; cap at 50 until a real call settles it.
- **Rate limits**: 25/day not confirmed by any primary source. Limits are per account, shown in the Adzuna dashboard; a third-party SDK cites "25/minute, 250/day". No 429 documented (only 400/404/410/500 with `{exception, display, doc}`). Assume stats/categories calls share the quota.
- **Expiry: no signal.** Only `created` (source post date, or first-seen). Soft expiry only; pass `max_days_old` = freshness window on every search.

- **Categories**: `GET /jobs/{country}/categories` → `{tag, label}`, per country, full list not published. One call per country needed before Domain mapping (incl. USAJOBS family mapping) is final.
- **Salary**: local currency, no currency field (derive from country); period not stated; `salary_is_predicted` marks Adzuna-estimated salaries (string "0"/"1" in spec, int in examples, so accept both).
- **Location**: `location.area` hierarchy round-trips as `location0..7` params, so it works directly as the region key. Default radius 5 km, so always pass `distance`.
- **Descriptions are truncated to 500 characters.** Skills/requirements extraction works on snippets, not full ads.
- **19 countries.** Stats endpoints: histogram (semantics ambiguous), top_companies (top 5), geodata, history (1–12 months).
- **Licensing**: commercial organisations get a 14-day evaluation; aggregated use (vacancy counts, average salaries) beyond that needs written consent. Fine for the assessment demo; production use would need a licence. Terms require attribution "The Adzuna API", deletion of Adzuna data on termination, one account per entity.

Research: USAJOBS API facts

Type: research Status: resolved Findings: branch `research/usajobs-api-facts`, file `research/usajobs-api-facts.md`

Question

What does the USAJOBS Search API provide as a secondary US source, as needed by Which jobs sources are used? and Jobs Data Gateway caching and expiry?

Specifically:

- Auth headers (`Host`, `User-Agent` email, `Authorization-Key`) and rate limits.
- Search params: keyword, position title, location name, radius, date posted, job category / occupational series, pay grade, remote.
- Page size limits and total-count reporting.
- Posting fields: id, title, organisation, locations, `PositionRemuneration` (min/max, rate interval), `PublicationStartDate`, `ApplicationCloseDate`, qualifications / description text available.
- Job series / category codes and how they could map onto Adzuna-style Domains.
- Coverage caveat: federal-only roles; what kinds of private-sector queries return nothing.

Answer

Full findings on branch `research/usajobs-api-facts` (commit `b387f01`), file `research/usajobs-api-facts.md`. Response shape is from docs, not a live call; confirm on first real call.

- **Auth**: `Host: data.usajobs.gov`, `User-Agent: <registered email>`, `Authorization-Key: <key>` on `GET https://data.usajobs.gov/api/Search`. Code-list endpoints need no auth.
- **Limits**: no published per-second/per-day quota; 10,000 rows per query, 500 per page. OPM may add quotas without notice, so the Gateway budget guard should still count calls.
- **Paging**: `ResultsPerPage` up to 500; totals in `SearchResultCountAll` and `UserArea.NumberOfPages`. One call usually covers a role + location.
- **Salary**: `PositionRemuneration[]` with `MinimumRange`/`MaximumRange` (strings) and `RateIntervalCode` (PA, PH, PD, BW, PM, ...).
- **Expiry**: `ApplicationCloseDate` and `PublicationStartDate` (ISO 8601 UTC) present, so the hard-expiry rule holds. Dedupe key: `MatchedObjectId`.
- **Remote**: query filter only (`RemoteIndicator`), no per-posting flag.
- **Domains**: 822 occupational series, each with a `JobFamily` parent (60 groups); draft family→Adzuna category mapping in the findings, Adzuna tags still to be cross-checked against Research: Adzuna API facts.

- **Coverage**: federal-only (some overseas, filter by `CountryCode`); private-sector titles likely return little. Federal pay comes from fixed pay tables, so it is not a market signal and must not be mixed unlabelled into Adzuna salary stats.
- **Unconfirmed**: `Radius` units; multi-value separator (semicolon vs colon in docs).

Research: LangGraph persistence and resume semantics

Type: research Status: resolved Findings: branch `research/langgraph-persistence-and-resume`, file `research/langgraph-persistence-and-resume.md`

Question

Do current LangGraph releases support the guarantees in How is state persisted so a Run survives the user leaving??

Specifically:

- SQLite checkpointer package, sync vs async variants, and its concurrency limits.
- Is there a SQLite-backed `Store` (for long-term memory), or only InMemory / Postgres? If not, what is the closest option?
- When checkpoints are written (per super-step?), durability modes, and what survives a process kill mid-node.
- Resume semantics: does the interrupted node re-run from its top? How are multiple `interrupt()` calls in one node matched on resume?
- How to discover pending interrupts and their payloads for a thread after restart (`get_state`, `tasks`, `interrupts`).
- Interaction of `interrupt()` with `Send` fan-out and with parallel branches.
- Listing threads from a checkpointer (to back a run registry), or whether a separate registry is needed.
- Current status of prebuilt `ToolNode` (location, recent breaking changes) to confirm the custom executor decision.

Answer

Full findings on branch `research/langgraph-persistence-and-resume` (commit `7eabba5`), file `research/langgraph-persistence-and-resume.md`. Verified from package source and a cross-process experiment on langgraph 1.2.18 + langgraph-checkpoint-sqlite 3.1.2 (fresh process per step, hard kill mid-node).

- **SQLite Store exists**: `langgraph.store.sqlite.SqliteStore` (+ async) in `langgraph-checkpoint-sqlite` 3.1.2, maintained though absent from the docs index. Survived across processes in the test.

- **Checkpoint timing**: one checkpoint per super-step (parallel nodes share one). Durability modes: `async` (default; a kill right after answering a Pause rolled back to that Pause), `sync` (persisted before the next step; only the killed node re-ran), `exit` (crash lost the answer). In every mode the Pause payload is on disk before the CLI shows it.
- **Resume re-runs the paused node from its top**: confirmed. Multiple `interrupt()` calls in one node are matched by call order.
- **Finding a pending Pause after restart**: `graph.get_state(cfg)` → `.interrupts[0].value` (payload), `.tasks[0].name` (node), empty `.next` when finished. Resume with `Command(resume=answer)`; parallel interrupts take a `{interrupt_id: answer}` map.
- **Run registry still needed**: the checkpointer cannot list threads cheaply, and metadata sticks only to checkpoints of the call that set it. Cheapest: a `("runs",)` namespace in the same Store. LangGraph does not prevent two processes resuming one thread at once.
- **ToolNode**: still `langgraph.prebuilt.ToolNode` (`langgraph-prebuilt` 1.1.1), not deprecated, but churn around it (`create_react_agent` deprecated, `langchain` 1.4.7 no longer re-exports `ToolNode`). Supports the custom executor decision.
- **Untested**: two processes writing one SQLite file at once (CLI + Streamlit); skipping completed parallel siblings when one pauses (docs say yes).

Amendments to How is state persisted so a Run survives the user leaving? recorded there.

Component breakdown and build order

Type: grilling Status: resolved Assignee: smayankulkarni05 Blocked by: 12, 13, 14, 15

Question

Into which components does the system split (each to become its own LLD/build effort), what does each own (responsibility, boundary, the slice of state it reads and writes, its seams to neighbours), and in what order are they built under What is the build priority order??

Inputs: the graph in What is the node-level graph shape?, the decisions in tickets 01–11, and the research findings from tickets 12–15. The breakdown should also place the infra shell (API, queues, workers, concurrency, MiniStack) as a future-phase component with its seam to the Run Controller.

Answer

Resolved by grilling. Components are cut by **concern** (one per graph stage plus supporting parts), not per node or per tier. Each becomes its own LLD/build effort.

#	Component	Owns	State slice it writes
A	**Kernel**	state schema + reducers, domain types, LLM tier factory (OpenRouter, fallbacks, validation retry), Jev classifier wrapper + thresholds	`run_log` reducer (append-only, every node writes)
B	**Jobs Data Gateway**	Adzuna + USAJOBS clients, L1 cache, budget guard, replay mode, expiry + sweep; L2 store as a later pass	none (outside the graph)
C	**Request & Plan**	`intake`, `clarity_check`, `clarify` (Pause), `plan`, `plan_review` (Pause)	Hiring Request, Clarifying Questions + answers, Assumptions, Search Plan
D	**Collection**	`@tool`s, `fetch_agent`, `tools` executor, `collect`, `relevance`, Sufficiency rule, `broaden`, `low_data_review` (Pause)	tool messages, Postings, Sample, attempt count
E	**Analysis**	`batch_extract` + Send fan-out + reducer, `stats`	per-Posting extractions, stats
F	**Brief**	`write_brief`, `critic`, `render`, Run Notes assembly from `run_log`	brief draft, critic verdicts, final JSON + Markdown

#	Component	Owns	State slice it writes
G	**Runtime**	StateGraph wiring and compile, SQLite checkpointer + Store, run registry, Run Controller, event stream, Pause protocol, concurrent-resume guard	nothing in graph state; owns threads + registry
H	**Clients**	CLI (Pause UX, `--resume`, `--focus`, progress); Streamlit later	none (Run Controller only)
I	**Long-term memory**	preferences namespace in the Store, read by `intake`	preferences
J	**Infra shell** (future phase)	API, queue, workers on MiniStack; wraps the Run Controller, never the graph	none

Boundary rules

- Each stage owns its slice of state; slices are *declared* in the Kernel, and no component writes another's slice. This is the state ownership map.
- Pause nodes live with the stage whose decision they gate; Runtime owns the Pause *protocol* (payload shape, re-show on return, `Command(resume=…)`, `durability="sync"`).
- Graph wiring lives in Runtime; stage components export only nodes and router functions.
- Critic, streaming and Run Notes are passes over existing components (Brief, Runtime + Clients, Brief), not new boundaries.
- `samples/`, README and write-up are a final delivery task, not a component.

Build order: tier passes across components, under [What is the build priority order?]. Each component's LLD covers all its tiers; building happens in passes.

1. **Must-have walking skeleton**, in order: Kernel → Gateway (L1, budget, replay, Adzuna) → Runtime + CLI (SQLite and Pause protocol from day one, since durable resume is non-negotiable) → Request & Plan (clarify only) → Collection → Analysis (sequential extract) → Brief (no critic).
2. **Recommended**: `plan_review`, `low_data_review`, Send fan-out, fallbacks, `--focus`, USAJOBS, Streamlit.
3. Streaming → 4. Infra shell (J) → 5. Critic (F) → 6. Long-term memory (I). Gateway L2 lands in pass 2 or later.

Fog reassigned to component LLDs (LLD-level, so beyond this map): currency/period normalisation, multi-source stats, batching → Analysis; Confidence Tag + Hiring Outlook rules → Brief; Adzuna aggregate endpoints → Gateway; `--focus` effects → Request & Plan + Clients; streaming event model → Runtime; memory contents → Long-term memory; infra internals and shared daily budget → Infra shell; sample-run strategy → delivery task.

Assemble docs/HLD.md

Type: task Status: resolved Assignee: smayankulkarni05 Blocked by: 16

Question

Write `docs/HLD.md` from the resolved tickets: system context, component list and boundaries, graph diagram, concept-level state ownership, Pauses and durable resume, Jev placements, data gateway, infra topology (future phase), build order. Gist and link each decision to its ticket; do not introduce new decisions.

Answer

Done: docs/HLD.md. Twelve sections, each gisting and linking its source tickets; no new decisions. The infra topology diagram only shows what tickets 02, 04, 09 and 10 already decided (shell wraps the Run Controller; sweep becomes a scheduled job; SQLite replaced). Queue technology and the other infra internals are explicitly left to the infra shell's LLD.

kernel-lld

kernel-lld/map.md

Kernel LLD

Label: wayfinder:map Parent: Talent Market Scout: HLD (component A in docs/HLD.md §3)

Destination

`docs/lld/kernel.md`: an LLD of the Kernel precise enough to build straight from (shared domain types, state schema + reducers, `run_log`, LLM tier factory, Jev wrapper + thresholds), with nothing left to decide. Building the code is a separate effort after this map.

Notes

- Domain: the Kernel is the first thing built in the must-have walking skeleton; every other component imports it. Vocabulary: `GLOSSARY.md`. Spec: `Joveo_Talent_Market_Agent-Assessment.pdf`. Deadline Mon 2 Nov 2026, 9:00 AM.
- Skills: `langchain-router` first for any framework question (per `CLAUDE.md`), then what it names (likely `langgraph-fundamentals`, `langgraph-persistence`, `langchain-middleware`); `langgraph-decision-models` for Jev; `grilling` + `domain-modeling` for grilling tickets.
- Scope rules (decided at charting):
  - **Shared contract only.** The Kernel fixes cross-component domain types, every state slice's key + owner + reducer, and `run_log`. Stage-private fields are added to the Kernel's schema file by that stage's own LLD.
  - **Jev split.** Kernel owns the classifier wrapper, threshold config (per model, per placement), first-cut values + tuning procedure, fallback chain. Stages own their criteria/labels.
  - **LLM tiers.** Model picks, fallback chains and validation-retry policy are decided here.
  - **LangSmith.** Kernel owns tracing settings, run names/tags on LLM tier calls, and making Jev calls traceable. Per-Run metadata (thread, request) belongs to Runtime.
- Standing preferences (decided at charting):
  - **Pydantic v2 everywhere**: domain types *and* the graph state schema (`BaseModel` with `Annotated` reducers), not `TypedDict`.
  - **Layout/tooling**: `uv`, Python 3.12, `src/talent_scout/` package with `kernel/` as first subpackage, `pytest`, settings loaded from root `.env` (OpenRouter, Adzuna, USAJOBS, LangSmith keys).
  - Live calls through the `.env` OpenRouter key are fine for smoke tests (cents, not dollars).

Decisions so far

<!-- one line per closed ticket -->

- Research: OpenRouter models for the LLM tiers: cheap gpt-6-luna → haiku-5.5, strong sonnet-5.5 → gpt-6.1-sol; `ChatOpenRouter` with `json_schema` strict + `require_parameters`; binding limit is in-flight spend (402), so cap concurrency and retry.
- Research: LangGraph/LangChain contract for Pydantic state: Pydantic state works on langgraph 1.2.x if nodes return partial dicts, `Send` payloads are models, writes are validated by us, and `LANGGRAPH_STRICT_MSGPACK` is on; structured-output failures surface differently per mode, so retry is ours; fallbacks compose.
- Research: langchain-typesafe API surface: alpha 0.0.1a3 Runnable, one request per Posting, `abatch` needs `max_concurrency`; already traced in LangSmith (cost/id via an HTTP hook); explicit fallback exception list; set the timeout and size-check inputs yourself.
- Live smoke test: OpenRouter LLM, Jev and LangSmith: live calls work as the research predicted; `json_schema` beats `function_calling` (cheaper, steadier); Jev's id and cost come back only through an HTTP hook, but LangSmith traces and prices Jev on its own; `ChatOpenRouter` runs trace without cost, so the Kernel supplies it.
- Jev wrapper and thresholds: Noul-only `judge(placement, state, questions)` returning `Judgements`; one cut per (model, placement), all 0.5 until a JSONL sweep tunes them, no uncertain band; jev-1.14 with retry → cheap LLM tier (bool + reason); truncate longest field to 30 KB; global semaphore of 8; no HTTP hook; injected classifier + `fake_classifier` for tests.
- Shared domain types: Kernel holds only cross-boundary types (Posting, Evidence, HiringRequest, Clarifying Question, Assumption, Search Plan, Market stats as a flat list of Stats with ids); the Gateway maps raw JSON to Postings keyed `source:source_id`; a type doubles as an LLM schema only when the model fills every field.
- State schema, reducers and run_log: one flat frozen Pydantic `State` in `kernel/state.py`; every slice `LastValue` except `extractions` (merge, for the `Send` fan-out); `broaden` appends to Collection's `broadened_plans` (attempts = its length, `current_plan` derived) and resets `messages`; `run_log` is narrative only, no timestamps, appended via `log()`; an `owned(node, owner)` guard validates every write, owner and `Send` payload.
- LLM tier factory: async `Tiers.structured` (system fixed at build, so the prefix rule is the signature) + `chat` for `fetch_agent`; both tiers DeepSeek v4.1 Flash → MiniMax M3.1 Flash while testing, reasoning `low`/`medium`, `max_tokens` 2,048/8,192 (floor 1,000); we parse, 1 feedback retry per model, `LLMExhausted` propagates; new **Caveat** `run_log` kind that Brief must render, backend errors never shown; semaphore 20, `check_balance`, cost copied into LangSmith; injected chains + `GenericFakeChatModel`; SQLite replay cache for LLM and Jev behind `REPLAY_CACHE` (pass 2), with a replay banner.
- Write docs/lld/kernel.md: written at docs/lld/kernel.md, replacing the working notes; adds settings, replay, test seams and the pass-1 build checklist.

Not yet specified

Empty: the LLM tier factory ticket settled replay for LLM and Jev calls (a SQLite replay cache behind `REPLAY_CACHE`, shared with the Gateway). Map destination reached at docs/lld/kernel.md.

Out of scope

- Final calibrated Jev threshold values: need live labelled runs; this map stops at first-cut values + the tuning procedure.
- Stage-private state fields and each stage's Jev criteria text: owned by that stage's LLD.
- Per-Run LangSmith metadata and `config` at invoke time: Runtime LLD.
- `SqliteStore` needing its own autocommit connection and JSON-dumped values (found by Research: LangGraph/LangChain contract for Pydantic state): Runtime LLD.
- Per-Posting extraction type and the Stat catalogue (Analysis LLD); Insight, Confidence Tag, Client Recommendation, Hiring Outlook and Market Brief types (Brief LLD): each is read only inside its own component, so Shared domain types leaves them to those LLDs.
- Writing the Kernel code: separate build effort after this map.
- `preferences` state slice: Long-term memory LLD adds it (found by State schema, reducers and run_log).
- Deliberate deferrals (Jev Choice/Score, `jev-1.13` hop, per-call Jev ids, `run_log` timestamps, calibrated cuts) are tracked with their triggers in docs/lld/future-scope.md.

Research: OpenRouter models for the LLM tiers

Type: research Status: resolved Assignee: smayankulkarni05 Blocked by: Findings: branch `research/openrouter-llm-tiers` (commit `d9681f5`), file `research/openrouter-llm-tiers.md`

Question

Which OpenRouter models are realistic candidates for the Kernel's two LLM tiers, and how does LangChain reach them?

- **Cheap tier** (per-Posting batch extraction, Clarifying Question drafting; high volume) and **strong tier** (Search Plan, brief writing, critique): 2–3 candidates each with current input/output price, context length, and a sensible fallback partner from a different provider.
- Structured output support per candidate on OpenRouter: native `json_schema` response format vs tool calling only; how reliable each is; OpenRouter's `require_parameters` / provider routing to avoid providers that drop the schema.
- Prompt caching on OpenRouter per candidate provider (automatic vs `cache_control`), minimum prefix size, and the discount.
- LangChain connection: `ChatOpenAI(base_url="https://openrouter.ai/api/v1")` vs the `langchain-openrouter` package (does it exist, is it maintained, what does it add). Which one keeps `with_structured_output` and `.with_fallbacks()` working.
- Rate limits / credits behaviour that would affect a batch fan-out of ~50–200 parallel calls.

Answer

Full findings on branch `research/openrouter-llm-tiers` (commit `d9681f5`), file `research/openrouter-llm-tiers.md`. Sourced from OpenRouter's live models/endpoints APIs, docs and package source; no live call (the worktree had no `.env`), so the smoke test confirms.

- **Cheap tier candidates**: `openai/gpt-6-luna` → fallback `anthropic/claude-haiku-5.5` (both $0.10/$0.50 per M, ~1M ctx); reserve `google/gemini-3.1-flash-lite` ($0.25/$1.50).
- **Strong tier candidates**: `anthropic/claude-sonnet-5.5` → fallback `openai/gpt-6.1-sol` (both $2/$10, ~1M ctx); reserve `google/gemini-4-argon`. Most are under six weeks old: pin exact ids.
- **Structured output**: all support native `json_schema` on their first-party endpoint; not on Bedrock/AWS for Claude or Bedrock for gpt-6.1-sol, so set `require_parameters: true`. Claude 5.5 endpoints report forced tool choice unsupported, so use `method="json_schema", strict=True` explicitly (metadata-based, unverified live).
- **Prompt caching**: OpenAI/Gemini automatic (OpenAI min 1,024 tokens); Claude needs top-level `cache_control`, min 512 tokens on 5.x. Use endpoint-API prices, not the stale docs. Cheap tier input is ~$0.02/run, so caching only pays off in the strong tier's critique loop.

- **LangChain**: `langchain-openrouter` (`ChatOpenRouter` 0.2.9), the official package; keeps per-call cost and provider name, which `ChatOpenAI` loses. `with_structured_output` and `.with_fallbacks()` work either way.
- **Fan-out limits**: no request-rate cap on paid models; the binding limit is the in-flight spend budget (402). The SDK doesn't retry 402/429, so the LLD needs `max_tokens` on every call, LangGraph `max_concurrency` (~20), `.with_retry()` for 429/in-flight 402, and a balance check via `GET /api/v1/key` before a Run.

Research: LangGraph/LangChain contract for Pydantic state and structured output

Type: research Status: resolved Assignee: smayankulkarni05 Blocked by: Findings: branch `research/langgraph-pydantic-state-contract` (commit `4172fde`), file `research/langgraph-pydantic-state-contract.md`

Question

What do current LangGraph and LangChain versions guarantee for a Pydantic-everywhere Kernel?

- Current stable versions of `langgraph`, `langgraph-checkpoint-sqlite`, `langchain-core`, `langchain-openai`, and Python support.
- **Pydantic `BaseModel` as the graph state schema**: how `Annotated` reducers work on model fields; what nodes receive (model instance) vs return (partial dict or model); when validation runs and its cost; known caveats (defaults, `Send` payloads with a Pydantic state, subgraphs, `add_messages` on a Pydantic field).
- **Checkpoint serialization**: do Pydantic v2 models (nested, with enums, datetimes, `Decimal`) round-trip through `SqliteSaver` / `JsonPlusSerializer` and `SqliteStore`? Any msgpack allowlist / "unregistered type" deserialization warnings or settings in recent versions, and what must be configured.
- **Structured output failures**: behaviour of `with_structured_output(schema, include_raw=True)` on a malformed response (what `parsing_error` holds), and the idiomatic way to retry with the validation error fed back.
- **Fallbacks**: does `.with_fallbacks()` compose with `with_structured_output` (fallback per tier model, each structured), and which exceptions trigger it.
- **Tracing**: confirm LangSmith picks up chat-model calls from env vars alone (`LANGSMITH_TRACING`, `LANGSMITH_API_KEY`, `LANGSMITH_PROJECT`) and how to set `run_name`/tags on a bound model.

Answer

Full findings on branch `research/langgraph-pydantic-state-contract` (commit `4172fde`), file `research/langgraph-pydantic-state-contract.md` (ends with an 8-point LLD recommendation). Sourced from package source and run in a scratch venv against a mock OpenAI endpoint. Not tested: real OpenRouter models and async variants.

- **Versions**: langgraph 1.2.18, langgraph-checkpoint 4.3.1, checkpoint-sqlite 3.1.2, langchain-core 1.7.2, langchain-openai 1.7.3, langsmith 0.14.16, pydantic 2.14.2; Python 3.10+ (target 3.12). Pydantic everywhere works, with rules.

- **State validation**: `Annotated` reducers on model fields work, and each node receives a freshly validated instance (on every node's input, contrary to the docs). Writes are never validated, so a bad write fails at the next node's read (wrong blame) or leaks silently from the last node.
- **Node return rule**: return partial dicts. Returning the whole model double-appends reducer lists. `Send` payloads aren't validated, so build them as model instances. `invoke()` returns a raw dict missing never-written fields.
- **Checkpoints**: nested models, enums, datetimes, `Decimal`, UUID and Pydantic interrupt payloads round-trip across processes. By default each type logs an "unregistered type … will be blocked" warning. `LANGGRAPH_STRICT_MSGPACK=true` makes `compile()` allowlist every type reachable from the schema; `Any` fields come back as plain dicts.
- **Store**: `SqliteStore` can't take models or `Decimal`s (`model_dump(mode="json")` / `model_validate`), and needs its own autocommit connection (sharing with `SqliteSaver` raises a nested-transaction error). Mainly a Runtime LLD concern.
- **Structured output**: in `ChatOpenAI`'s json_schema mode a malformed response raises `ValidationError` inside the call, so `include_raw` can't catch it. In function_calling mode failures give `parsed=None, parsing_error=None`, so check `parsed is not None`. No built-in retry; feeding the error back as a `ToolMessage` worked on the second try. Behaviour under `ChatOpenRouter` is not verified.
- **Fallbacks**: `.with_fallbacks()` composes with `with_structured_output` in either order and every fallback gets the schema. Any exception triggers it by default, but `include_raw`-caught parse failures don't.
- **Tracing**: env vars alone trace a model call, but they're read once and cached, so set them before the first call. `.with_config(run_name, tags)` on a structured runnable names only the outer chain, so set it on the model before `with_structured_output`.

Research: langchain-typesafe API surface

Type: research Status: resolved Assignee: smayankulkarni05 Blocked by: Findings: branch `research/langchain-typesafe-api` (commit `0d5608a`), file `research/langchain-typesafe-api.md`

Question

What does `langchain-typesafe`'s `TypeSafeClassifier` actually expose, so the Kernel's Jev wrapper can be designed against it? Builds on the HLD's Research: Jev on OpenRouter (OpenRouter access already confirmed).

- Current version; methods for Noul, Choice and Score; their argument and return types (where the probabilities, per-label distribution and confidence live).
- Async and batch support: can relevance check N Postings concurrently or in one request? Concurrency limits.
- Error types and which should trigger the fallback chain (OpenRouter decisions endpoint → direct TypeSafe → LLM structured output).
- Is it a LangChain `Runnable`, and does a call show up in LangSmith on its own, or does the wrapper need `@traceable`? Can `usage.cost` and the request id be recovered (the HLD research says the classifier drops them)?
- How input over ~32 KB is signalled or handled, and how to measure request size before sending.

Answer

Full findings on branch `research/langchain-typesafe-api` (commit `0d5608a`), file `research/langchain-typesafe-api.md`. Sourced from the 0.0.1a3 package code, TypeSafe docs/OpenAPI and OpenRouter's reference; behaviour checked offline against a mocked HTTP layer, no live call. Unsettled points are marked unconfirmed in the file.

- **Surface**: `langchain-typesafe` 0.0.1a3 (alpha); `TypeSafeClassifier` (beta) is a Runnable with `invoke`/`ainvoke`, one `/v1/systemone` request each. Answers: `nouls[id].noul` (P(yes)), `choices[id]` (`choice`, `probabilities`, `confidence`), `scores[id]` (`score`, `probabilities`, `legend`, `confidence`). Error classes live in `langchain_typesafe.client` (the README's top-level import fails).
- **Concurrency**: one state per request, so N Postings means N requests. `abatch` is LangChain's generic version, unbounded unless `max_concurrency` is set. No built-in retries.
- **Errors → fallback**: retry then fall back on 429, 5xx (incl. 524/529), connection errors and timeouts; fall back on 402, 404 and malformed 200s. Never fall back on 400/422 (it fails everywhere). A 400 `max_tokens_exceeded` or a 413 means trim, not switch. 402/413/415 surface as plain `TypeSafeAPIError`, so `with_fallbacks` needs an explicit `exceptions_to_handle`.

- **Tracing**: already traced as an `llm` run with tokens, nested under the calling node, so no `@traceable` needed. Cost and request id are dropped but recoverable via a response hook on an injected HTTP client writing to run metadata; add a LangSmith model price for `typesafe/jev-1.14`.
- **Timeout trap**: an injected HTTP client ignores the classifier's 30 s timeout and falls back to 5 s, so set it explicitly.
- **Large input**: OpenRouter silently switches to long-context mode above ~32 KB. The classifier never checks size, so the wrapper measures `json.dumps(state, ensure_ascii=False, separators=(",", ":"))` and truncates before sending.
- **Research recommendation** (input to the Jev wrapper ticket, not yet a decision): one long-lived classifier with injected clients (timeout + cost/id hook), `abatch` with `max_concurrency` ~8, size check, retries and fallbacks from `with_retry`/`with_fallbacks`; skip the `/api/alpha/decisions` hop since it fails the same ways.

Live smoke test: OpenRouter LLM, Jev and LangSmith

Type: task Status: resolved Assignee: smayankulkarni05 Blocked by:

Question

Make one real call of each kind with the keys in root `.env`, so the tier factory and Jev wrapper are designed against observed behaviour rather than docs (the HLD research made no live call). AFK; spending a few cents is approved.

- One chat call through OpenRouter with structured output (a tiny Pydantic schema).
- One Jev Noul, one Choice and one Score call to `typesafe/jev-1.14` via `TypeSafeClassifier` with `base_url="https://openrouter.ai/api"`.
- Confirm both appear in the LangSmith project, and note whether the Jev call is traced at all.

Record: exact response shapes (redact nothing structural, no secrets), latency, `usage.cost`, any surprises. Throwaway script lives in the scratchpad, not the repo.

Answer (2026-10-30)

Run with `langchain-openrouter` 0.2.9, `langchain-typesafe` 0.0.1a3, `langsmith` 0.14.16. The chat calls used `minimax/minimax-m3.1-flash`, the user's choice for tests, not the tier models. Total spend was about $0.0002.

OpenRouter chat, structured output (`require_parameters: true`, `strict=True`, `include_raw=True`):

- `json_schema`: 2.05 s, parsed cleanly, `finish_reason: stop`. 221 input / 27 output tokens, `usage_metadata.input_token_details.cache_read` = 128, cost $0.000032.
- `function_calling`: 1.63 s, parsed from `tool_calls`, `content` is empty. It used 114 reasoning tokens, so it cost 4.5x as much ($0.000146). Field values also drifted: "Data Engineer"/"senior" against "Senior Data Engineer"/"Senior" from `json_schema`. That supports making `json_schema` the default.
- `response_metadata` carries `id` (`gen-...`), `model_name`, `model_provider: "openrouter"`, `finish_reason`, `cost` and `cost_details`. It does **not** name the upstream provider. `usage_metadata` carries the cache-read and reasoning token counts.

Jev (`TypeSafeClassifier`, `base_url="https://openrouter.ai/api"`, one question per call):

- Noul 1.15 s, Choice 0.74 s, Score 1.10 s. Each costs about $0.000014 (roughly 330-360 input tokens).
- The resolved model is `typesafe/jev-1.14-20261014`. The shapes match the research: `noul: 0.98`; Choice has `choice`, `probabilities` and `confidence`; Score has `score: 2.0`, `legend` and `probabilities` keyed by int.
- The raw body has `id` (`gen-dec-...`), `provider: "TypeSafe"` and `usage.cost`. The classifier drops all three, and `request_id` is `null` because OpenRouter doesn't send `x-typesafe-request-id`. Headers include `x-generation-id`, which equals the body `id`, and `x-provider-name`. **Confirmed: a response hook on an injected `httpx2.Client` gets both `id` and cost.**
- The probabilities on this easy input were saturated (1.0/0.0, confidence 1.0). Tuning thresholds needs harder examples.

LangSmith: traced once the key was fixed. The first run's 403 came from a stray extra character at the end of `LANGSMITH_API_KEY` in `.env`; the region and workspace were not the problem.

- Every call appears under the `@traceable` parent with no extra wiring. Jev shows as `TypeSafeClassifier`, `run_type: llm`, model `typesafe/jev-1.14`, with tokens **and cost**. LangSmith's cost exactly matches OpenRouter's `usage.cost`: it prices input tokens at $0.042/M and output at zero. **No `@traceable` and no custom model price are needed for Jev.**
- `ChatOpenRouter` shows as `run_type: llm` with tokens and model name, but **cost is empty**, even though `response_metadata.cost` has it. The Kernel has to supply chat cost itself (copy `response_metadata.cost` into run metadata or `usage_metadata`, or add LangSmith model prices for each tier model). The production tier models weren't tested, so it's unknown whether LangSmith already prices them.
- `with_structured_output(include_raw=True)` makes about 10 nested chain runs per call (`RunnableSequence`, `RunnableParallel`, `RunnableAssign`, parser), so `run_name` and tags belong on the outer runnable.

Surprises: the deprecation warning on `list_runs()` (use `client.runs.query()`), and the `TypeSafeClassifier` beta warning on every construction.

Shared domain types

Type: grilling Status: resolved Assignee: smayankulkarni05 Blocked by: 02

Question

What are the fields, types and invariants of the Pydantic domain types that cross component boundaries?

Hiring Request, Clarifying Question (+ answer), Assumption, Search Plan, Posting, per-Posting extraction, Insight with Evidence and Confidence Tag, Hiring Outlook.

Particular pressure points:

- **Posting identity**: `source` + `source_id` as the key; what the dedupe key looks like.
- **Posting salary**: carried raw (amount range, currency, Adzuna's unstated period, USAJOBS `RateIntervalCode`, `salary_is_predicted`, federal pay-table flag) so the Analysis LLD can normalise later without a schema change.
- **Domain and region**: Adzuna category / `location.area`, USAJOBS job family mapping hook (needed later by Gateway L2).
- **Expiry fields**: close date (hard) vs created date (soft).
- **Description**: Adzuna's 500-char snippet vs USAJOBS full text; does the type say which?
- Which types are also LLM structured-output schemas (Search Plan, extraction, brief), and what that implies for field descriptions and optionality.

Inputs: Adzuna and USAJOBS research, `GLOSSARY.md`.

Answer (2026-10-30)

Grilled in three rounds. Everything below goes in `talent_scout/kernel/types.py`.

What the Kernel holds. A type lives here only if one component writes it and another reads it (the Gateway counts as a component):

- **Kernel**: Posting (with Salary, Source and PayPeriod), Evidence, HiringRequest, Clarifying Question, Assumption, Search Plan, and Market stats (Stat, Dimension).
- **Analysis LLD**: the per-Posting extraction and the Stat catalogue.
- **Brief LLD**: Insight, Confidence Tag, Client Recommendation, Hiring Outlook and Market Brief.

Conventions:

- Every model uses `ConfigDict(frozen=True, extra="forbid")`.

- Enums are `StrEnum`, money is `float`, and dates are `date`.

LLM-schema policy:

- A type is used directly as a `with_structured_output` schema only when the model writes every field. When code adds fields, the stage defines a narrower draft schema.
- Every field of an LLM-facing type has a `Field(description=...)` giving its meaning and units, never instructions.
- Optional fields are `T | None` with no default, because strict `json_schema` makes every property required anyway.
- Validator errors are what the LLM tier factory feeds back on retry.

Raw JSON to Posting. The Gateway's mapper converts raw responses into Postings. L1 caches the raw response, so replay stays faithful. Tools and Collection see only `list[Posting]`. The mapper handles every source quirk:

- derives the currency from the Adzuna country
- maps `RateIntervalCode` to a period
- maps `location.area` to a region
- maps the USAJOBS family to a Domain
- accepts `salary_is_predicted` as either "0"/"1" or an int
- caps descriptions at `MAX_DESCRIPTION_CHARS = 4000`
- swaps a reversed salary range and drops a salary that has neither bound
- skips any Posting that still fails validation, and counts the skips

```
class Source(StrEnum): adzuna, usajobs
class PayPeriod(StrEnum): annual, monthly, biweekly, weekly, daily, hourly, unknown

class Salary(BaseModel):
    min: float | None          # at least one of min/max; min <= max
    max: float | None
    currency: str              # ISO 4217, ^[A-Z]{3}$
    period: PayPeriod          # Adzuna always `unknown`
    predicted: bool            # Adzuna salary_is_predicted; False for USAJOBS

class Posting(BaseModel):
    source: Source
    source_id: str
    title: str
    company: str | None
    location: str              # display string
    region: tuple[str, ...]    # Adzuna location.area; USAJOBS (country, state, city) of
first location
    domain: str | None         # Adzuna category tag; USAJOBS family mapped onto it, None
until mapped
    posted_at: date            # soft expiry, trend
    closes_at: date | None     # USAJOBS only, hard expiry
```

```
    description: str           # Adzuna <=500-char snippet; USAJOBS capped full text
    url: str
    salary: Salary | None      # None = pay not disclosed
    # computed: key = f"{source}:{source_id}"
```

Notes on Posting:

- **Identity and dedupe.** `key` is a plain `str`. Postings with the same key merge as a no-op. Collapsing the same job across locations or sources by fingerprint is a Collection rule, not a field.
- **USAJOBS salary.** If a posting lists several `PositionRemuneration` entries, the mapper takes the first.
- **No federal-pay flag.** It is equivalent to `source == usajobs`.
- **Fields left out.** There is no snippet flag, no contract type or time, and no remote flag, because no stage needs them.

```
class WorkMode(StrEnum): onsite, hybrid, remote
class RequestField(StrEnum): role, location, seniority, employment_type, work_mode, currency,
purpose

class HiringRequest(BaseModel):          # intake's LLM schema; role/location non-blank
    role: str
    location: str
    seniority: str | None                # free text as given
    employment_type: str | None
    work_mode: WorkMode | None
    currency: str | None                 # ISO 4217 for the brief
    purpose: str | None

class ClarifyingQuestion(BaseModel):     # drafted via a narrower stage schema (answer is
code-filled)
    about: RequestField
    text: str
    options: list[str]
    answer: str | None

class Assumption(BaseModel):
    about: RequestField
    value: str
    reason: str

class SearchPlan(BaseModel):             # plan's (and broaden's) LLM schema; replaced whole,
never patched
    titles: list[str]                    # non-empty, primary first
    country: str                         # ^[a-z]{2}$
    location: str
    radius_km: int                       # >= 0
    max_days_old: int                    # >= 1; also the soft-expiry window
```

```
    target_postings: int                # >= 1; Sufficiency target
    category: str | None
```

Notes on these types:

- **HiringRequest** has no raw text field (the original wording goes in `run_log`) and no `--focus` (that's a Run option, decided in ticket 06).
- **Re-asks** are counted from earlier questions with the same `about`; there is no attempt counter.
- **SearchPlan** has no sources field (the fetch_agent model picks the tools) and no rationale field (the reasoning goes in `run_log`).

```
class Dimension(StrEnum): demand, compensation, skills, competition, titles_seniority

class Evidence(BaseModel):
    posting_keys: list[str]             # before-validator sorts + dedupes
    # computed: count

class Stat(BaseModel):
    id: str                             # unique within MarketStats, e.g. "comp.median"
    dimension: Dimension
    label: str
    value: float | str
    unit: str | None
    evidence: Evidence

class MarketStats(BaseModel):
    sample_size: int
    stats: list[Stat]
```

Market stats are a flat list, so the Kernel fixes only the shape of a Stat:

- **Analysis owns the Stat catalogue.** Its LLD decides which ids exist, so adding a figure never changes the Kernel.
- **Brief cites Stat ids.** Insights point to Stat `id`s, and code expands those ids into Evidence. The LLM never types Posting keys, so Evidence comes from the Sample by construction.

Validators:

- **Reject**: Salary (the rules noted on its fields), duplicate Stat ids in MarketStats, and the SearchPlan and HiringRequest rules above. Rejections on LLM-filled types feed the retry.
- **Normalise**: Evidence keys are sorted and deduplicated.
- **Left to the stages**: cross-object checks, such as Evidence keys being in the Sample or 1–3 questions per Pause.

Glossary: added **Region** and **Stat**.

Handed to [State schema, reducers and run_log]:

- `broaden` (Collection) revises the Search Plan, but Request & Plan owns that slice, which breaks the single-writer rule.
- Where the `--focus` Run option lives in state.

State schema, reducers and run_log

Type: grilling Status: resolved Assignee: smayankulkarni05 Blocked by: 05

Question

What is the Pydantic graph state: each slice's key, owning component, type and reducer, and the shape of the `run_log` the Kernel owns?

- Slices per docs/HLD.md §3: Request & Plan, Collection (messages, Postings, Sample, attempt count), Analysis (extractions keyed by Posting, stats), Brief (draft, critic verdicts, rendered output), preferences, `run_log`.
- **Reducers**: Postings across broaden attempts (merge by key vs replace); extraction merge from the `Send` fan-out; messages (`add_messages`), and whether they reset between broaden attempts; `run_log` append-only.
- **Reset semantics**: what clears when `broaden` or a `plan_review` reject loops back.
- **`run_log` entry**: fields (node, kind, message, data, timestamp?), the kinds Run Notes need (routing decision with reason, Clarifying Question + answer, final Search Plan, Assumption), and whether timestamps break deterministic replay.
- How the "stage-private fields are added by the stage's LLD" rule is expressed in code (one schema file each stage extends, or composed sub-models).

Input from Shared domain types:

- **Search Plan ownership.** `broaden` (Collection) produces a new Search Plan, but Request & Plan owns the plan slice. Decide who writes it: a Collection-owned "current plan" slice, moving `broaden` into Request & Plan, or something else.
- **`--focus`.** It is a Run option, not part of the Hiring Request, so it needs its own slice.
- **Type facts.** Posting identity is `key` (`source:source_id`), so a Postings reducer can merge by key. All domain types are frozen.

Answer (2026-10-30)

Grilled in three rounds. Everything below goes in `talent_scout/kernel/state.py`.

Layout. One flat `State(BaseModel)`, `ConfigDict(frozen=True, extra="forbid")`, fields grouped in comment blocks by owner. Stage-owned types (Extraction, MarketBrief, ...) live in that stage's `types.py`, which may import only `kernel.types`; `kernel/state.py` imports them, so there is no cycle. Nested slice sub-models were rejected: LangGraph puts reducers only on top-level fields, so a nested slice is

`LastValue` and the parallel `batch_extract` writes would raise `InvalidUpdateError`. Frozen because mutating `state` in place is a silent no-op in LangGraph; a node must return an update.

Reducers. Every slice is `LastValue`: a node returns the whole new value (`{"postings": {**state.postings, **new}}`). The only reducer is `extractions` (merge by key), because the `Send` fan-out has N branches writing it in one super-step and a branch sees only its payload. `Send` stays (dropping it loses per-branch retry on resume and changes the HLD graph).

Slices.

Key	Owner	Type (default)	Written by	Reset
`request_text`	input	`str` (required)	Run Controller	never
`focus`	input	`list[Dimension]` (`[]` = all)	Run Controller	never
`hiring_request`	Request & Plan	`HiringRequest | None`	`intake` (refined after answers per that LLD)	
`questions`	Request & Plan	`list[ClarifyingQuestion]` (`[]`)	`clarity_check` adds, `clarify` fills answers	never; re-asks counted from it
`assumptions`	Request & Plan	`list[Assumption]` (`[]`)	Request & Plan nodes	
`search_plan`	Request & Plan	`SearchPlan | None`	`plan`	rewritten on `plan_review` reject
`messages`	Collection	`list[AnyMessage]` (`[]`)	`fetch_agent`, `tools`	`[]` on each `broaden`
`postings`	Collection	`dict[str, Posting]` (`{}`)	`collect`; first copy seen wins	kept across attempts
`sample`	Collection	`list[str]` (`[]`), Posting keys in order	`relevance`	recomputed each attempt
`broadened_plans`	Collection	`list[SearchPlan]` (`[]`)	`broaden` appends	
`extractions`	Analysis	`Annotated[dict[str, Extraction], merge]` (`{}`)	`batch_extract` x N	
`stats`	Analysis	`MarketStats | None`	`stats`	
`brief`	Brief	`MarketBrief | None`	`render`	
`brief_markdown`	Brief	`str | None`	`render`	
`run_log`	Kernel (all nodes)	`list[RunLogEntry]` (`[]`)	everyone via `log()`	never; prefix-checked

Read-only `@property` views: `current_plan` (last of `broadened_plans`, else `search_plan`) and `sample_postings`. Attempt count is `len(broadened_plans)`; there is no counter slice. Stage-private fields (Brief draft, critic verdicts, `plan_review` feedback, each stage's decision fields) are added here by their stage LLDs. `preferences` is left to the Long-term memory LLD.

Search Plan and `broaden`. Request & Plan owns the approved `search_plan`; Collection owns `broadened_plans`. One writer per slice, and every revision reaches Run Notes without extra logging.

Messages reset. `broaden` returns `{"messages": []}`, so the next `fetch_agent` round starts from the new `current_plan`; fetched Postings stay in `postings`.

Routing. Routers are pure functions of state. The node before a router stores its outcome in a slice it owns (e.g. a clarity `Judgement`, a Sufficiency verdict) and writes a `route` entry to `run_log`; the router only reads that slice. No extra LLM or Jev call.

`run_log`. Narrative only: routing decisions and reasons, Jev outcomes, fallbacks used, skip counts, the original request wording. Domain objects are not copied in; Brief builds Run Notes from `run_log` plus the slices (questions, assumptions, `current_plan`, `sample`). This narrows HLD §9's wording; Run Notes content is unchanged.

```
class LogKind(StrEnum): route, pause, warning, note

class RunLogEntry(BaseModel):   # frozen, extra="forbid"
    node: str
    kind: LogKind
    message: str
    data: dict[str, JsonValue]  # plain JSON: strict msgpack blocks models in untyped fields
```

No timestamp: list order is event order, LangSmith has the times, and replay transcripts stay byte-identical. `batch_extract` branches never log; `stats` writes one summary entry. Helper: `log(state, node, kind, message, **data) -> {"run_log": [*state.run_log, entry]}`.

Write guard. Each field carries `Field(json_schema_extra={"owner": ...})` (`"input"`, `"kernel"`, or a component). The Kernel exports `owned(node, owner)`; Runtime applies it to every node at wiring. It raises `StateWriteError(node, key, reason)` when:

- an update key's owner is not this node's owner (`run_log` is writable by all; `"input"` by none),
- a value fails its field's type, checked with a cached `TypeAdapter` (the `ValidationError` is chained),
- a `run_log` update does not start with the current log.

Routers are wrapped the same way and raise `StateWriteError` for a `Send` whose `arg` is not a model instance, since LangGraph validates neither node outputs nor `Send` payloads.

Left out: recorded in docs/lld/future-scope.md.

LLM tier factory

Type: grilling Status: resolved Assignee: smayankulkarni05 Blocked by: 01, 02, 04

Question

What is the Kernel's LLM tier factory: its interface, models, fallbacks and failure policy?

- **Interface** stages call (e.g. a tier name → chat model; a tier + schema → structured runnable), and what stays out of it (prompts belong to stages).
- **Model picks**: primary + fallback chain for the cheap and strong tiers, from the OpenRouter research.
- **Validation retry**: how many retries with the validation error fed back before falling to the next model; what's raised when the chain is exhausted and how the node should treat it.
- **Prompt-prefix rule** (HLD §5): how the factory or a helper enforces stable instructions + schema first so batch calls share a cached prefix.
- **Tracing**: `run_name`, tags (tier, model) and metadata on every tier call.
- **Test seam**: how tests and replay substitute a fake model without touching stage code.
- Cost/token accounting, if any, beyond LangSmith.

Input from Live smoke test: OpenRouter LLM, Jev and LangSmith: `ChatOpenRouter` runs trace in LangSmith **without cost**, so the cost/token accounting point is live. Also, structured-output calls nest about 10 chain runs, so set `run_name` and tags on the outer runnable.

Input from Jev wrapper and thresholds: Jev's last fallback hop is an LLM adapter on the **cheap** tier. It needs a structured-output call returning `{question_id: {answer: bool, reason: str}}`, so the factory must give it a tier + schema → structured runnable.

Answer (2026-10-30)

Grilled with the user, with a live check of the new models (about $0.0002 spent). The factory lives in `kernel/llm.py`, with `kernel/settings.py` and `kernel/replay.py` beside it.

- **Interface**: a `Tiers` object with two async entry points.
  - `tiers.structured(tier, Schema, *, system, name, max_tokens=None) -> Runnable[str, Schema]` serves intake, plan, extraction, brief writing, the critic and Jev's LLM fallback. `system` is fixed at build time and the input is only the variable user content, so the HLD's prompt-prefix rule (stable instructions and schema first) is enforced by the signature, with no helper.

    - `tiers.chat(tier, *, tools, name, max_tokens=None) -> Runnable[list[AnyMessage], AIMessage]` exists only for `fetch_agent`. Tools are bound to each model *before* `with_fallbacks`, because `RunnableWithFallbacks` has no `bind_tools`.
    - Async-only, to match `jev.judge`. Claude `cache_control` is left out, since caching only pays off in the critic loop (future-scope).
- **Models (testing phase)**: both `cheap` and `strong` run `deepseek/deepseek-v4.1-flash` ($0.045/$0.50 per M) → `minimax/minimax-m3.1-flash` ($0.15/$0.60), two providers, 1M context, both with strict `json_schema` under `require_parameters: true`. The tier names stay in code, so one `.env` change (`LLM_CHEAP=a,b`, `LLM_STRONG=a,b`) restores the research picks, which are parked in future-scope as production defaults.
- **Reasoning and output room**: every call sends `reasoning: {effort}`, `low` for cheap and `medium` for strong, both overridable in settings. Live, DeepSeek at default effort spent 178 of 200 output tokens reasoning on a 3-field extraction. MiniMax refuses `reasoning.enabled: false` (400, "Reasoning is mandatory"), but `effort: "low"` gave clean JSON on both with 0 reasoning tokens. `max_tokens` defaults to 2,048 (cheap) and 8,192 (strong), overridable per call, and settings refuse anything below 1,000. It is a ceiling, not a charge. Worst case, with every call using ~1,000 output tokens, is about $0.12 per heavy Run against about $0.02 at low effort.
- **Transport failures**: each model is wrapped in `with_retry` (3 attempts, jittered backoff, on 429, 5xx, timeouts, connection errors and 402, since a 402 is usually in-flight spend that clears), then `with_fallbacks` to the next model with an explicit exception list. It never falls back on 400, 401 or 422 (our bug) or 402 (same account). 60 s timeout per call.
- **Validation retry**: the factory parses the response itself (`Schema.model_validate_json` on the raw content) instead of trusting `with_structured_output`'s parser, which raises inside the call and loses the raw output. On a `ValidationError` it resends the same prompt plus the bad output as an `AIMessage` and a `HumanMessage` carrying the Pydantic error text and "return corrected JSON". **One feedback retry per model**, so at most 4 attempts per call over the 2-model chain.
- **Chain exhausted**: raises `LLMExhausted(tier, name, attempts)`, chained to the last error. Nodes don't catch it by default: the Run stops at that node, the checkpoint stays at the last committed step, and `--resume` reruns only that node. A stage may catch it where losing one item is acceptable (e.g. Analysis skipping one Posting), and must then log a Caveat.
- **Transparency (Caveat)**: a new glossary term and a new `LogKind.caveat` in `run_log`, written with `log()`, with optional `dimension` and `posting_keys` in `data`. Brief must render every Caveat in Run Notes; `warning` and `note` stay internal. Stages decide when to raise one, and each stage LLD lists its Caveats. Backend errors (a Run killed by `LLMExhausted`, a 401, a bug) are never Caveats: the CLI shows "Run stopped at <node>, resume with `--resume`", and the details go to logs and LangSmith. A successful LLM fallback isn't shown either, because the fallback's output passes the same schema validation and can't change a job-related conclusion. LLM failures reach the user only through their effect: a skipped Posting, or Jev falling back to the LLM (judged without probabilities). Both are Caveats.
- **Concurrency and balance**: one process-wide `asyncio.Semaphore(20)` inside the factory (size from settings), so Runtime doesn't need LangGraph's `max_concurrency`. The Kernel exports `check_balance(settings)` (`GET /api/v1/key`). The Run Controller calls it before starting or resuming a Run and refuses below a floor (default $0.25).

- **Cost**: the factory copies OpenRouter's `response_metadata["cost"]` into the LLM run's `usage_metadata` cost fields so LangSmith shows the exact cost. Which field LangSmith reads gets confirmed at build; the fallback is LangSmith model prices for the pinned ids. No cost in `run_log`.
- **Tracing**: the outer runnable gets `run_name=f"llm:{name}"`; each model is tagged `tier:<tier>` and `model:<id>` before wrapping; metadata carries `tier`, `name` and the copied cost. The structured runnable returns only the `Schema` instance. Which model answered and how many retries ran is visible only in LangSmith.
- **Test seam**: constructor injection, `Tiers(chains: dict[Tier, list[BaseChatModel]], settings)`. `make_tiers(settings)` builds the real `ChatOpenRouter` chains. Tests pass langchain-core's `GenericFakeChatModel` with canned JSON strings, which runs the real parse, feedback-retry and fallback code. A few-line fake subclass whose `bind_tools` returns itself covers `chat()`. No custom fake framework.
- **Replay recording** (resolves the map's open item): `kernel/replay.py`, off by default, switched on by `REPLAY_CACHE=<path>` and shared with the Gateway's replay switch.
    - LLM: a ~25-line `BaseCache` on stdlib `sqlite3`, passed to `set_llm_cache()`. It is keyed on messages + model id + params, so prompt edits, model changes and feedback retries all miss correctly. Writing it avoids the `langchain-community` dependency.
    - Jev: ~20 lines in `jev.judge`, keyed on (model chain, hash of the request JSON), in the same SQLite file.
    - A replayed Run is made obvious. `render` puts a replay banner at the top of the Market Brief, read from settings. Collection logs a Caveat with the age of the oldest cached jobs response used, because replay ignores L1's TTL.
    - Built in pass 2, not the walking skeleton. Since LLM and Jev now replay too, HLD §7's "the whole graph runs from cached responses" holds as written.
- **Settings**: `kernel/settings.py`, one `Settings(BaseSettings)` (adds `pydantic-settings`) read from the root `.env`. It holds keys, tier chains, reasoning efforts, `max_tokens` defaults, semaphore sizes, the balance floor, the LangSmith project and `REPLAY_CACHE`. `get_settings()` is cached, and on first load it exports the LangSmith env vars to `os.environ` before any LangChain call reads them. `make_jev(settings)` and `make_tiers(settings)` take the same object.

Jev wrapper and thresholds

Type: grilling Status: resolved Assignee: smayankulkarni05 Blocked by: 03, 04

Question

What is the Kernel's Jev wrapper: interface, threshold config, fallbacks, tracing and tuning?

- **Interface** for the three placements (clarity check, Posting relevance, critic): what a stage passes (criteria/labels, text) and what it gets back (outcome, probability/distribution, threshold applied, reason for `run_log`).
- **Threshold config**: shape keyed by model and placement; where it lives; first-cut values; the "uncertain" band and what it routes to.
- **Tuning procedure**: how thresholds get calibrated per model (labelled examples, optionally a LangSmith dataset), given final values are out of scope for this map.
- **Fallback chain**: OpenRouter `systemone` → decisions endpoint → direct TypeSafe → LLM structured output (loses distributions; how thresholds apply then).
- **32 KB guard**: truncate, split, or refuse oversized input.
- **Batching/concurrency** for N relevance calls per Run.
- **Tracing**: `@traceable` (if needed) with criteria, probabilities, threshold, outcome, cost and request id.
- **Test seam**: a fake classifier for tests and replay.

Input from Live smoke test: OpenRouter LLM, Jev and LangSmith: LangSmith already traces and prices Jev (`llm` run, cost matches OpenRouter exactly), so no `@traceable` and no custom price. The OpenRouter id and cost still need the HTTP hook if the `run_log` wants them. Easy inputs return saturated probabilities, so tuning needs hard examples.

Answer (2026-10-30)

Grilled with the user. The Kernel's Jev wrapper is a thin class over one long-lived `TypeSafeClassifier` (`langchain-typesafe==0.0.1a3`, `model="typesafe/jev-1.14"`, OpenRouter `base_url` and key).

- **Interface**: Noul only, since all three placements are yes/no questions. Choice and Score get added when a stage needs one. `Placement = Literal["clarity", "relevance", "critic"]`. One method, `await jev.judge(placement, state: dict, questions: dict[str, Noul]) -> dict[str, Judgement]`. Several questions share one request (e.g. clarity asks about role, location and seniority together). `Judgement` holds `outcome: bool`, `p_yes`, `cut`, `model`, `source` (`"jev"` or `"llm_fallback"`), `truncated` and a one-line `reason` for `run_log`. Stages own the question text and phrase it so that "yes" is the side that has to clear the bar.

- **Thresholds**: a single cut per `(model, placement)`, `outcome = p_yes >= cut`, with **no uncertain band**. No Pause exists to route "uncertain" to, and collapsing it to one side is the same as a single cut. `p_yes` and `cut` are logged so tuning can see close calls. Cuts live in a dict literal in `kernel/thresholds.py`: no loader, no env overrides. **Every first-cut value is 0.5**, marked untuned until the sweep runs, and that includes the `llm:cheap` key, where the cut doesn't matter because p is always 0 or 1.
- **Picking the cut**: the cut comes from `response.model` by the longest config key that is a prefix of it, since Jev returns a dated snapshot like `typesafe/jev-1.14-20261014`. `source` comes from the same match. If no key matches, it fails loudly, so a new Jev version never runs on stale cuts. At construction, the wrapper checks that every placement has a cut for each model in its chain.
- **Fallback chain**: `jev-1.14` with `with_retry` (3 attempts, backoff, on 429, 5xx, connection errors and timeouts), then `with_fallbacks` to an LLM adapter on the tier factory's **cheap** tier, triggered by exhausted retries, a malformed 200 or a 404, with an explicit `exceptions_to_handle`. The adapter sends the same state and Noul text and gets back `{id: {answer: bool, reason: str}}` via structured output. It maps that to `p_yes` 0 or 1 and builds a `ClassifierResponse` with `model="llm:cheap"`, so the routers don't change. It asks for no verbalised probability, because that would be uncalibrated noise. It **never** falls back on 400 or 422 (our bug) or on 402 (the LLM is on the same OpenRouter account). Those exceptions, and any raised after the chain runs out, propagate unchanged. Dropped: the `/api/alpha/decisions` endpoint (same failure modes as `/v1/systemone`) and direct TypeSafe (no key). Adding `jev-1.13` as a middle hop waits until 1.14 proves flaky, since it would need its own cuts.
- **32 KB guard**: measure `len(json.dumps(state, ensure_ascii=False, separators=(",",":")).encode())` against a 30,000-byte budget. If it's over, cut the longest string value, ending it with `…[truncated]`, until it fits, and set `truncated=True`. Never raise. Stages may pre-trim more cleverly.
- **Concurrency**: one `asyncio.Semaphore` in the wrapper, size 8 from settings. It bounds every Jev call in the process, across placements and concurrent Runs. Relevance just `asyncio.gather`s `judge` calls.
- **Tracing**: no `@traceable` and no injected `httpx2` client or response hook. LangSmith already traces and prices Jev, and keeping the classifier's own client keeps its 30 s timeout, which sidesteps the 5 s injected-client trap. Each call passes `config={"run_name": f"jev:{placement}", "tags": ["jev", placement], "metadata": {"cut": ..., "model": ...}}`. The OpenRouter generation id and cost stay out of `run_log`; add the hook if per-call ids are ever needed.
- **Test seam**: constructor injection, `Jev(classifier: Runnable[ClassifierRequest, ClassifierResponse], cuts)`. `make_jev(settings)` builds the production chain. A `fake_classifier({"q_id": 0.9, ...})` helper in the Kernel's test utilities returns canned `ClassifierResponse`s.
- **Tuning procedure**: per placement, 20–40 *hard* labelled examples in `evals/jev/<placement>.jsonl` (easy inputs saturate at 1.0/0.0). A script runs them through the real `Jev`, sweeps the cut from 0.05 to 0.95 and prints precision, recall and F1 per cut. Relevance optimises for precision and clarity for recall of "ambiguous". A human then edits `thresholds.py`. Re-run whenever the model id changes. A LangSmith dataset is optional.

Write docs/lld/kernel.md

Type: task Status: resolved Assignee: smayankulkarni05 Blocked by: 05, 06, 07, 08

Question

Write `docs/lld/kernel.md` from the resolved tickets and the map's Notes (layout, Pydantic-everywhere, scope rules): module layout, shared domain types, state schema + reducers, `run_log`, LLM tier factory, Jev wrapper + thresholds, settings, tracing, test seams, and the build checklist for the walking skeleton. Gist and link each decision to its ticket; introduce no new decisions.

Input from State schema, reducers and run_log: the doc should note that Run Notes are built from `run_log` plus state slices, which narrows HLD §9's wording, and link docs/lld/future-scope.md.

Working notes so far: `docs/lld/kernel-notes.md`. Build the final doc from them once the LLM tier factory ticket closes.

Answer (2026-10-30)

Written at docs/lld/kernel.md. The working notes were turned into the final doc and removed, so there's one source. §12 (LLM tier factory), §13 (settings), §14 (replay cache), §15 (test seams) and §16 (pass-1 build checklist) are new. §9 gained the Caveat kind and its render rule. Items marked ⏳ belong to a named stage or Runtime LLD, not the Kernel. Future-scope gained the production tier models and Claude `cache_control`.

kernel-build

kernel-build/issues/01-project-skeleton-and-settings.md

01: Project skeleton and settings

What to build: a working Python project that every later ticket builds in. A developer runs `uv run pytest` from a clean checkout and it passes, and the Kernel's settings load from the real root `.env`. Spec: Kernel LLD §2, §13, §16 steps 1-2.

Pin the versions from the research: `langgraph` 1.2.18, `langgraph-checkpoint-sqlite` 3.1.2, `langchain-core` 1.7.2, `langchain-openrouter` 0.2.9, `langchain-typesafe` 0.0.1a3, `langsmith` 0.14.16, `pydantic` 2.14.2, plus `pydantic-settings` and `pytest` at their current versions.

Decisions made when these tickets were cut, not in the LLD:

- **Adzuna key name.** `.env` has `ADZUNA_API_KEY`, while the LLD's field is `adzuna_app_key`. Keep the LLD's field name and read the existing variable through an alias. Don't change `.env`.
- **`LANGSMITH_TRACING`.** It is in `.env` but missing from the LLD's settings table. Add it and export it with the other LangSmith vars, because tracing depends on it. Update §13 to match.
- **uv on PATH.** `uv` was installed with `pip --user` into Python's user Scripts folder, which may not be on PATH. Make sure `uv` resolves from a normal shell.

Blocked by: None (can start immediately).

Status: done

- [x] `uv run pytest` passes from a clean checkout on Python 3.12, with the dependencies above pinned
- [x] The `talent_scout` package imports from the `src` layout
- [x] `get_settings()` loads the real `.env`, is cached, and exports the LangSmith vars to the environment on first load, before any LangChain call
- [x] Every setting and default in LLD §13 exists
- [x] Test: a `max_tokens` value of 999 is rejected
- [x] Test: a comma list in `LLM_CHEAP` parses into a model chain
- [x] Test: `ADZUNA_API_KEY` populates `adzuna_app_key`
- [x] `.gitignore` covers `.venv`, `.cache` and `graphify-out/`
- [x] A `.graphifyignore` excludes `.claude/`, so skills and agent worktrees stay out of the code graph
- [x] `graphify update .` builds a graph containing the new package and nothing from `.claude/`

02: Shared domain types

What to build: the cross-component vocabulary every stage and the Gateway import: Posting (with Salary, Source, PayPeriod), Hiring Request, Clarifying Question, Assumption, Search Plan, Evidence, Stat and Market stats. Invalid values are refused at construction, and those validator errors are what later feed the LLM's retry. Spec: Kernel LLD §2, §3, §16 step 3.

Types read inside a single component (Extraction, Insight, Market Brief and so on) stay out of this ticket.

Blocked by: 01: Project skeleton and settings.

Status: done

- [x] All types and fields in LLD §3 exist, each model frozen with `extra="forbid"`
- [x] Enums are `StrEnum`, money is `float`, dates are `date`
- [x] LLM-filled types carry a `Field(description=...)` on every field, and optional fields are `T | None` with no default
- [x] Test: Posting `key` is `source:source_id`
- [x] Test: Salary rejects no bounds, `min > max` and a bad currency code
- [x] Test: Evidence sorts and dedupes its keys, and `count` matches
- [x] Test: Market stats reject duplicate Stat ids
- [x] Test: Search Plan rejects empty titles, a bad country code and out-of-range numbers; Hiring Request rejects a blank role or location

03: State, log() and write guard

What to build: the graph state contract, proven on a real graph. A small test graph runs a `Send` fan-out through the guard, merges `extractions`, checkpoints to SQLite under strict msgpack, and rehydrates. A bad write from any node fails at that node with a clear `StateWriteError`, instead of blowing up later and blaming the wrong one. Spec: Kernel LLD §4, §5, §8, §9, §10, §16 step 4.

The graph itself is test-only: graph wiring, the real checkpointer setup and the Store belong to Runtime.

Decision made when these tickets were cut, not in the LLD:

- **Stage type stubs.** `State` imports `Extraction` and `MarketBrief`, which their stage LLDs haven't defined yet. Add minimal frozen stubs at their final module paths in the Analysis and Brief packages, because checkpoints store class paths and moving a model later breaks rehydration. Those LLDs fill in the fields.

Blocked by: 02: Shared domain types.

Status: done

- [x] `State` has every field in LLD §4 with its owner, plus the `current_plan` and `sample_postings` properties
- [x] `extractions` is the only field with a reducer
- [x] `log()` appends an entry, and `LogKind` includes `caveat`
- [x] `RunLogEntry.data` accepts plain JSON only and has no timestamp
- [x] Test: each of the four guard rules raises `StateWriteError` (wrong owner, bad value type with the `ValidationError` chained, wiped `run_log`, `Send` with a non-model `arg`)
- [x] Test: a node writing `run_log` is allowed whatever its owner, and no node can write an `"input"` field
- [x] Test: mutating state in place raises, because it is frozen
- [x] Test: N parallel `Send` branches through the guard merge into `extractions`
- [x] Test: with `LANGGRAPH_STRICT_MSGPACK=true`, state round-trips through a SQLite checkpoint with no unregistered-type warnings

04: Tiers.structured, end to end

What to build: a stage can ask a tier for a validated Pydantic object and get one, from a real model through OpenRouter, with bad output corrected by feedback, transport failures retried and fallen back, and cost visible in LangSmith. If the chain runs out, the caller gets `LLMExhausted`. Spec: Kernel LLD §12, §15, §16 steps 6-7.

Start of the live smoke script (not in CI), which ticket 06 also adds to. Live calls use `minimax/minimax-m3.1-flash` and spend cents.

Blocked by: 01: Project skeleton and settings.

Status: done

- [x] `Tiers(chains, settings)` takes injected chains, and `make_tiers(settings)` builds real `ChatOpenRouter` chains with strict `json_schema` and `require_parameters: true`
- [x] `structured` takes its `system` text at build time and only the variable user content as input
- [x] The factory parses raw content itself, and every call sends the tier's reasoning effort and `max_tokens`
- [x] Test: a bad-then-good fake response passes on the feedback retry, and the retry carries the bad output and the Pydantic error text
- [x] Test: two bad responses on one model reach the next model
- [x] Test: the chain running out raises `LLMExhausted` with `tier`, `name` and `attempts`, chained to the last error
- [x] Test: 429, 5xx, timeouts and connection errors are retried and then fall back; 400, 401, 422 and 402 never fall back
- [x] Test: concurrent calls never exceed the process-wide semaphore
- [x] Runs carry `run_name=llm:<name>`, each model is tagged `tier:<tier>` and `model:<id>`, and metadata carries `tier` and `name`
- [x] Live smoke: one `structured` call per tier returns a valid object and shows in the LangSmith project
- [x] Cost shows in LangSmith. Record which `usage_metadata` field it reads in LLD §12, or fall back to LangSmith model prices for the pinned ids and record that

05: Tiers.chat and check_balance

What to build: `fetch_agent`'s entry point: a tier call that takes a message history and tools and returns an `AIMessage`, with tools still bound after a fallback. Also the pre-Run balance check, which refuses to spend when the OpenRouter balance is below the floor. Spec: Kernel LLD §12, §15.

Calling `check_balance` before a Run starts or resumes is the Run Controller's job, not this ticket's.

Blocked by: 04: Tiers.structured, end to end.

Status: done

- [x] `chat` binds tools to each model before `with_fallbacks`, and shares the transport retry, fallback, semaphore and tracing from ticket 04
- [x] `FakeToolChatModel` is a few-line `GenericFakeChatModel` subclass whose `bind_tools` returns itself
- [x] Test: `chat` returns an `AIMessage` with tool calls from the fake
- [x] Test: when the first model fails, the fallback model answers with its tools still bound
- [x] `check_balance(settings)` reads `GET /api/v1/key` and returns the balance
- [x] Test: with a mocked response, a balance below `balance_floor_usd` raises and one above it returns
- [x] Live smoke: `check_balance` returns a real number, and one `chat` call with a trivial tool on `minimax/minimax-m3.1-flash` returns a tool call

06: Jev wrapper and thresholds

What to build: a stage asks Jev yes/no questions for a placement and gets back `Judgement`s with an outcome, probability and cut, from the real `typesafe/jev-1.14` through OpenRouter. Oversized input is truncated rather than rejected, and a Jev model with no configured cut fails loudly. Spec: Kernel LLD §11, §15, §16 steps 5 and 7.

The LLM fallback hop is ticket 07. Here, exhausted retries propagate, so this ticket doesn't depend on the LLM tier factory. Add the Jev call to the live smoke script, creating the script if ticket 04 hasn't landed.

The tuning sweep (`evals/jev`) is not built here: it is in neither build pass, and all cuts ship at 0.5 marked untuned (future scope).

Blocked by: 01: Project skeleton and settings.

Status: done

- [x] `Jev(classifier, cuts)` takes an injected classifier, and `make_jev(settings)` builds the real one with OpenRouter's `base_url` and key
- [x] `judge(placement, state, questions)` returns a `Judgement` per question, with `outcome = p_yes >= cut`
- [x] `fake_classifier({...})` returns canned answers
- [x] Test: the cut comes from the longest config key that prefixes the returned model id, so a dated snapshot resolves
- [x] Test: an unknown model fails loudly
- [x] Test: construction fails when a placement has no cut for a model in the chain
- [x] Test: state over 30,000 bytes has its longest string truncated with the marker until it fits, `truncated=True` is set, and nothing raises
- [x] Test: concurrent calls never exceed the process-wide semaphore
- [x] Test: 429, 5xx, connection errors and timeouts are retried; 400 and 422 are not
- [x] Each call passes `run_name=jev:<placement>`, tags, and the cut and model as metadata
- [x] Live smoke: one Noul call returns a `Judgement` with a dated model id and shows in LangSmith

07: Jev LLM fallback hop (pass 2)

What to build: when Jev is down, a Run keeps going on the cheap LLM tier instead of stopping. Routers don't change, because the fallback still yields a `Judgement`, marked as made without Jev's probabilities so the calling stage can log a Caveat. Spec: Kernel LLD §9, §11.

Pass 2: not needed for the walking skeleton. Raising the Caveat is each stage's job, not this ticket's.

Blocked by: 04: Tiers.structured, end to end; 06: Jev wrapper and thresholds.

Status: done

- [x] `make_jev` chains `jev-1.14` with retry, then falls back to an adapter on `tiers.structured("cheap", ...)` with an explicit exception list
- [x] The adapter sends the same state and question text and gets `{id: {answer, reason}}` back
- [x] Answers map to `p_yes` of 0 or 1 with model `llm:cheap`, and `source="llm_fallback"` on the `Judgement`
- [x] Test: exhausted retries, a malformed 200 and a 404 reach the adapter
- [x] Test: 400, 422 and 402 propagate without touching the LLM
- [x] Test: a failure after the adapter also runs out propagates unchanged
- [x] Test: the existing cut lookup resolves `llm:cheap`

08: Replay cache (pass 2)

What to build: with `REPLAY_CACHE=<path>` set, LLM and Jev calls replay from a SQLite file, so running the same calls twice makes no live calls the second time. Changing a prompt or model misses the cache and calls live. Unset, nothing changes. Spec: Kernel LLD §11, §14.

Pass 2: not needed for the walking skeleton. Out of scope: Gateway replay of jobs data (Gateway reads the same setting), the replay banner (Brief) and the cached-data age Caveat (Collection).

Blocked by: 04: Tiers.structured, end to end; 06: Jev wrapper and thresholds.

Status: done

- [x] LLM replay is a small `BaseCache` on stdlib `sqlite3` installed with `set_llm_cache()`, with no `langchain-community` dependency
- [x] Jev replay is a second table in the same file, keyed on the model chain and the sha256 of the request JSON, looked up before the call and stored after
- [x] Test: with a counting fake model, a repeated `structured` call hits the model once
- [x] Test: a changed prompt or model id misses the cache
- [x] Test: a feedback retry misses the cache and reaches the model, so a cached bad response can't loop
- [x] Test: with a counting `fake_classifier`, a repeated `judge` hits it once, and a changed question misses
- [x] Test: with `REPLAY_CACHE` unset, no cache is installed and no file is created

gateway-lld

gateway-lld/map.md

Jobs Data Gateway LLD

Label: wayfinder:map Parent: Talent Market Scout: HLD (component B in docs/HLD.md §3, §7)

Destination

`docs/lld/gateway.md`: an LLD of the Jobs Data Gateway precise enough to build straight from, covering every tier (pass 1: Adzuna client, L1 cache, budget guard, replay; pass 2: USAJOBS, L2 region × Domain store, expiry and sweep), with nothing left to decide. Building the code is a separate effort after this map.

Notes

- Domain: the Gateway sits outside the graph, between Collection's `@tool`s and the jobs APIs, and protects a small daily Adzuna quota. Second in the must-have walking skeleton, after the Kernel. Vocabulary: `GLOSSARY.md`. Spec: `Joveo_Talent_Market_Agent- Assessment.pdf`. Deadline Mon 2 Nov 2026, 9:00 AM.
- Fixed upstream: docs/lld/kernel.md §3 (Posting, Salary, Source, mapper duty, `source:source_id` key), §9 (Caveats the Gateway feeds), §13 (one `Settings` object), §14 (`REPLAY_CACHE` switches jobs-data replay too), §15 (Jobs APIs test seam left to this LLD). HLD §7 and Gateway caching and expiry. API facts: Research: Adzuna API facts, Research: USAJOBS API facts.
- Skills: `langchain-router` first for any framework question (per `CLAUDE.md`); `grilling` + `domain-modeling` for grilling tickets.
- Scope rules (decided at charting):
  - **Boundary with Collection.** The Gateway owns a typed Python interface (`Fetch` in, `FetchResult` out) and everything behind it: source choice and fan-out, HTTP clients, mappers, Duplicate coalescing, L1, L2, budget guard, expiry, sweep, replay, test fake. Collection owns the one LLM-facing `search_jobs` tool schema and the tool-args → `Fetch` translation. Both types live in the `gateway/` package, not the Kernel.
  - The Gateway supplies the data a Caveat needs; Collection decides when to raise one and its wording.
- Standing preferences: inherited from the Kernel map (Pydantic v2 everywhere, `uv`, Python 3.12, `src/talent_scout/`, `pytest`, root `.env`). Live Adzuna calls are budgeted: about 5 approved for the smoke test; anything more needs the user's OK.

Decisions so far

<!-- one line per closed ticket -->

- Live smoke test: Adzuna and USAJOBS: Adzuna silently caps pages at 50, sends no rate-limit headers, leaks our app id in `redirect_url`, and returned only predicted point salaries; USAJOBS items are ~85 KB with up to 1,176 locations, `;` separates values, Radius acts like miles, and a per-Posting remote flag exists; fixtures in `tests/fixtures/jobs/`; quota still unreported.
- Gateway query interface: plain async Python `JobsGateway.fetch(Fetch) -> FetchResult`. It fans out to Adzuna (always) and USAJOBS (`country == "us"`) and merges them behind one `search_jobs` tool, one page per source per call, one title per Fetch. It coalesces Duplicates with a pure title + employer + description-prefix rule. Per-source failures are a result field; auth errors stop the Run.
- Adzuna client and mapper: `title_only` + `where`/`distance` (blank location = country-wide), `sort_by=date`, 50 per page; a 19-country → currency table that `Fetch` validates against; four non-occupational category tags → `None` Domain; `utm_*` stripped from the URL; drops are per-result mapping errors; 10 s timeout, 2 retries, 429 → `BudgetExhausted`, 401/403 propagate.
- L1 cache, budget guard and replay: own SQLite file; L1 stores mapped Postings, refetches after 24 h and deletes a row only when all its Postings have closed; Adzuna limit is 25/day, spent per HTTP attempt and locked for the UTC day on a 429; replay is a read-through file in front of L1; tests use `httpx.MockTransport` and `FakeGateway`.
- Adzuna aggregate endpoints: `histogram`, `top_companies`, `history` and the search `mean` are excluded from every pass, since they carry no Posting IDs and add little; `SourceOutcome.total` may appear only as labelled sampling disclosure, never as a Stat, Evidence or demand figure.
- USAJOBS client and mapper (pass 2): `PositionTitle` (matches any word) in relevance order, 50 per page, `DatePosted` capped at 60, category ignored; description starts with `JobSummary`, cut at 4,000; `source` alone labels federal pay; multi-location Region is the common prefix; a static family → Adzuna tag table with `None` for no match; a 429 is `SourceUnavailable`, no lockout.
- L2 store, expiry and sweep (pass 2): an Adzuna-only table that serves page N only if it can fill it, reuses a Posting only for a `Fetch` no broader than the one that found it, and also backs failed live calls with `cache=store`; expires 7 days after Adzuna last returned a Posting; purges after each live write, not at Run start; `Fetch` needs no target.
- Write docs/lld/gateway.md: docs/lld/gateway.md written, with the pass 1 and pass 2 build checklists; Kernel §15's Jobs APIs row filled; errors must never use `raise_for_status()`, which would leak the Adzuna keys.

Not yet specified

Out of scope

- Shared daily Adzuna budget across concurrent Runs, persistence migration off SQLite, moving L1 and the budget guard to MiniStack if required: Infra shell.

- Whether the replay Caveat also reports live calls made during replay (`SourceOutcome.cache == miss`): Collection LLD.
- Sample-run strategy for the three `samples/` requests within the Adzuna budget: delivery task.
- Caveat wording and when Collection raises one: Collection LLD.
- Rendering the "The Adzuna API" attribution in Run Notes (the Gateway only exports the string, per Adzuna client and mapper): Brief LLD.
- LLM-facing tool schemas: Collection LLD.
- Re-applying `coalesce` across Fetches in `collect`, and the `run_log` note for merged Duplicates: Collection LLD.
- Keeping federal pay (`source == usajobs`) apart from Adzuna salary Stats: Analysis LLD, per USAJOBS client and mapper (pass 2).
- Telling the Relevance stage about FAA "Public Notice" talent-pool announcements and `PositionTitle`'s any-word matches: Collection LLD.
- Wording and placement of the `SourceOutcome.total` sampling disclosure, which can't use `total` when `cache == store`: Collection and Brief LLDs.
- Whether an L2 answer's age (`cache == store`, `fetched_at`) raises a Caveat: Collection LLD.
- Listing the aggregate exclusion in the README's known limitations: delivery task.
- Writing the Gateway code: separate build effort after this map.

Live smoke test: Adzuna and USAJOBS

Type: task Status: resolved Assignee: smayankulkarni05 Blocked by:

Question

Make real calls with the keys in root `.env` so the Gateway is designed against observed responses, not docs (the HLD research made no live call). AFK except one HITL step. About 5 Adzuna calls are approved; USAJOBS is free.

- Adzuna (country `gb` or `us`, a common role like "data engineer"):
    1. One search with `what`, `where`, `distance`, `max_days_old`. Record the full response shape, `count`, `mean`, the type of `salary_is_predicted`, `location.area`, `category.tag`, `created` format, description length.
    2. One search at a large `results_per_page` (e.g. 100) to find the real per-page cap.
    3. `GET /jobs/{country}/categories`: save the full tag list (input to Domain mapping).
    4. One `histogram` or `top_companies` call (input to Adzuna aggregate endpoints).
    5. Spare, e.g. to see what an error body looks like.
- USAJOBS: one search with keyword + location + `Radius`, one with a multi-value param to settle the separator; record `Radius` units, `PositionRemuneration`, `ApplicationCloseDate`, `JobCategory`/family fields, `SearchResultCountAll`.
- **HITL**: the user reads the Adzuna dashboard and reports the daily and per-minute limits.

Save raw responses (no secrets) as test fixtures under `tests/fixtures/jobs/`. Throwaway script lives in the scratchpad, not the repo. Record exact shapes, latency, response headers that matter (rate-limit hints), and surprises against the research.

Answer (2026-10-30)

Run with stdlib `urllib` from a scratchpad script, Adzuna country `us`, "data engineer" around New York / Washington DC. 5 Adzuna calls used (the cap), 6 free USAJOBS calls, all HTTP 200. Fixtures: tests/fixtures/jobs/. Each file holds `{status, latency_ms, headers, body}` with the app id, keys and email replaced by `REDACTED`. **The dashboard quota was not reported**; it moves to L1 cache, budget guard and replay.

Adzuna

- **Page cap is 50, enforced silently.** `results_per_page=500` returned the same 50 ids as `=50`, with no error. Body: `results`, `count` (3,200), `mean`, `__CLASS__`.

- **No rate-limit headers** of any kind, so the guard can't learn the remaining quota from responses. Latency 0.8–2.5 s.
- **`redirect_url` carries our app id** as `utm_source=<app_id>`. Mapping it straight into `Posting.url` would leak the app id into state, LangSmith and the Market Brief.
- **Salary:** `salary_is_predicted` is the **string** `"1"`. In this sample, **all 50 salaries were predicted, with `salary_min == salary_max`**: Adzuna point estimates, not advertised ranges.
- `created` is ISO with `Z`. `location.area` is 4–5 levels deep, `["US", "New York", "New York City", "Manhattan", "Grand Central"]`, plus `latitude`/`longitude`. `company.display_name` was always present. No `contract_time` or `contract_type` in any of the 50 results. Descriptions run 200–500 chars. `adref` is an opaque JWT.
- **Categories (US): 30 tags.** They include non-Domain buckets: `graduate-jobs`, `part-time-jobs`, `other-general-jobs`, `unknown`. The search came back 47 `it-jobs`, 2 `scientific-qa-jobs`, 1 `engineering-jobs`.
- **`histogram`:** `{lower_bound_str: count}` in 20k bands up to an open `140000` bucket. It sums to 1,397 against `count` 3,200, so it covers only part of the matches (which part is undocumented). `where` resolved to the NYC level. **`top_companies`:** the top 5 by `canonical_name` with counts. No Posting ids in either.

USAJOBS

- **Responses are huge.** 25 results came to 2.0 MB, about 85 KB per item, mostly `UserArea.Details` (`MajorDuties`, `Education`, `QualificationSummary` up to 7.9 KB). At 500 per page that's about 40 MB per call.
- **Multi-location announcements.** 7 of 25 Postings list **1,176 locations**, and others list 2–44. `PositionLocationDisplay` summarises them.
- **Multi-value separator is `;`.** `2210;1550` returned 424, `2210:1550` returned 0, `2210` alone returned 385.
- **`Radius` behaves like miles:** 25 → 65 results, 100 → 74. Units aren't stated anywhere in the response. `UserArea` gives `NumberOfPages`, `IsRadialSearch` and `ResolvedLocation`.
- **There is a per-Posting remote flag**: `UserArea.Details.RemoteIndicator`, plus `TeleworkEligible`. This corrects the research, which said query-only.
- Salary: `PositionRemuneration` had one `PA` entry in all 25, with the ranges as strings. `JobGrade` (`GS`) plus `LowGrade`/`HighGrade` mark pay-table jobs. `JobCategory` usually has 1 entry, up to 11.
- Dates have no timezone and 4-digit fractions (`2027-01-31T23:59:59.9970`). Python 3.12's `fromisoformat` parses them. Id: `MatchedObjectId` (e.g. `759326100`). URL: `PositionURI`.
- Code list `occupationalseries`: 822 series, each with a `JobFamily` and an `IsDisabled` flag. No auth needed.

Gateway query interface

Type: grilling Status: resolved Assignee: smayankulkarni05 Blocked by: 01

Question

What is the Gateway's public interface that Collection's tools call?

- **Query type**: fields (title variants, location, distance, freshness / `max_days_old`, source(s), how many Postings or pages), validation, and how it normalises for cache keys.
- **Result**: Postings, mapper-dropped count, cache provenance (hit or miss, fetched-at age, replayed), per-source failure (feeds Collection's "one source failed" Caveat).
- **Shape of the call**: one `search(query)` across sources vs one per source; async (`httpx.AsyncClient`, already a LangChain dependency) to match the Kernel; who pages (Gateway to a target vs one page per tool call).
- **Errors**: budget refused, source down, bad credentials; which propagate and which become a result field.
- **Tracing**: whether Gateway calls are LangSmith runs, and what metadata (source, cache hit, calls left).
- **API style**: a plain typed Python function/class vs a GraphQL layer federating the sources. Recommendation at charting was plain Python: the Gateway is an in-process library with one caller and no network boundary, every caller wants the same flat `list[Posting]`, and source unification happens in the mapper either way, so GraphQL would add a schema server and resolvers without removing any work. GraphQL could be reconsidered for the Infra shell's external API (out of scope here).

Input from Live smoke test: Adzuna and USAJOBS: Adzuna pages cap at 50 silently (`count` 3,200 for a common role), so "who pages" decides how many quota calls one tool call can burn. Adzuna takes 0.8–2.5 s per call; USAJOBS returns ~85 KB per item.

Answer (2026-10-30)

Grilled with the user. One Adzuna call spent (call 6) on "data engineer", Washington DC, to see real cross-source overlap: adzuna_search_dc.json.

Shape

- **Plain typed Python, not GraphQL.** One class, `JobsGateway`, built from the Kernel's `Settings`, with its HTTP clients as constructor arguments (the exact test seam is decided in L1 cache,

budget guard and replay). GraphQL would add a schema server and resolvers with one in-process caller and no work removed.

- **Async.** `async def fetch(f: Fetch) -> FetchResult`, one `httpx.AsyncClient` per source client, opened and closed with `async with JobsGateway(...)`. Sources run concurrently.
- **The Gateway fans out and merges.** Collection exposes **one** `search_jobs` tool, not one per source. The Gateway picks sources by rule: Adzuna always, USAJOBS when `country == "us"`. This reverses HLD §2's "the model decides which to call" (HLD and its ticket updated). **Reversed again (2026-10-31):** Collection exposes one tool per source and the Gateway call is `fetch(f, source)`, so the model's tool choice shows in the trace. The Gateway keeps one `Fetch` per call and no fan-out.
- **One page per call.** A `Fetch` asks for page N of each source, so one call costs at most one API call per source. Page size is a client constant (Adzuna 50; USAJOBS set in its mapper ticket). Collection decides whether another page is worth it, using each source's `total` and `calls_left`.
- **Tracing.** Each call is a LangSmith child run `gateway:<source>`, with metadata `source`, `page`, `cache`, `calls_left` and a summary output (count, dropped, total), not the Postings. The exact `traceable` API gets checked at build time.

Types (`gateway/types.py`, frozen, `extra="forbid"`). Named `Fetch`/`FetchResult` because the glossary avoids "query" and "search" for the Hiring Request.

```
class Fetch(BaseModel):
    title: str                   # one Search Plan title; non-blank
    country: str                 # ^[a-z]{2}$; "us" adds USAJOBS
    location: str
    radius_km: int               # >= 0
    max_days_old: int            # >= 1
    category: str | None         # Adzuna tag
    page: int = 1                # >= 1, same page of each source

class CacheStatus(StrEnum): hit, miss, replay

class SourceOutcome(BaseModel):
    total: int                   # source's match count
    dropped: int                 # mapper drops (Caveat input)
    cache: CacheStatus
    fetched_at: datetime         # replay Caveat age (Kernel §14)
    calls_left: int | None       # budget guard; None = unmetered

class FetchResult(BaseModel):
    postings: list[Posting]      # coalesced survivors
    merged: dict[str, list[str]] # kept key -> Duplicate keys
    sources: dict[Source, SourceOutcome]  # sources that answered
    failures: dict[Source, str]  # sources that failed
```

- **One title per Fetch.** Adzuna's `what_or` ORs single words, so title variants can't be combined; each variant is its own Fetch and its own L1 key.

- A validator lowercases, strips and collapses whitespace in `title`, `location` and `category`, so equal Fetches are equal objects. The L1 key is derived in the L1 ticket.
- No `sources` field (the rule above picks them), no work mode (`Posting` can't carry it), no target count (the L2 fog stays open).

Errors

- `BudgetExhausted` and `SourceUnavailable` (timeout or 5xx after the client's retries) subclass `GatewayError`. A per-source one is recorded in `failures` and the other source's Postings still return, which is the input for Collection's "one source failed" Caveat.
- `GatewayError` is raised only when every source failed, and Collection's tool turns it into a `ToolMessage`.
- Auth failures (401/403) and bugs propagate and stop the Run (Kernel §9).

Duplicates (new glossary term). `coalesce(postings) -> (survivors, merged)`, a pure function in `gateway/`. The Gateway applies it to each merged result, and Collection's `collect` re-applies it over everything gathered across Fetches. One rule, owned by the Gateway; Kernel §3 updated.

- **Rule:** same normalised title, same normalised employer, and descriptions agree: after normalising (lowercase, non-word characters to spaces, trailing partial word dropped), the shorter is a prefix of the longer. They must also have the same Salary, unless either salary is `predicted`. Location and date are ignored. No LLM or Jev.
- **Survivor:** USAJOBS over Adzuna (close date, full text, real pay range), then the most recent `posted_at`, then the lowest key. A multi-location ad counts as one job; the dropped locations are recoverable from `merged`.
- Collapsing is a `run_log` `note`, not a Caveat.
- **Evidence from the fixtures.**
  - The rule collapses Deloitte "Data Engineer III" (3 locations, same snippet), Capital One "Data Engineer 4" (3 reposts weeks apart, all predicted salaries differ) and NAVSEA "Supervisory General Engineer" (3 hiring paths, same pay).
  - It keeps apart Booz Allen's 15 DC "Data Engineer" requisitions (every description differs) and the FAA FV/FG pairs (different pay plans).
  - Title + employer alone would have wrongly collapsed Booz Allen.
- **Cross-source overlap was zero.** All 50 DC Adzuna results are contractors with no federal agency, so Adzuna doesn't appear to carry USAJOBS ads. The prefix form still covers cross-source at no extra cost, *if* the USAJOBS mapper's description starts with the announcement's opening text (handed to USAJOBS client and mapper (pass 2)).

Adzuna client and mapper

Type: grilling Status: resolved Assignee: smayankulkarni05 Blocked by: 01, 02

Question

How does the Gateway call Adzuna and turn its JSON into Kernel Postings?

- **Params**: query fields → `what` / `what_or` / `title_only`, `where` vs `location0..n`, `distance` (always passed, default is 5 km), `max_days_old`, `results_per_page`, `sort_by`, `category`; country selection from location.
- **Mapper** (Kernel §3 duty): currency from country, `period = unknown`, `salary_is_predicted` (string or int), reversed ranges, Region from `location.area`, Domain from `category.tag`, `posted_at` from `created`, description (500-char snippet), `company` absence. What counts as a drop, and how drops are counted.
- **Transport**: timeout, retries on which statuses, how quota exhaustion surfaces (no 429 documented), attribution requirement ("The Adzuna API").

Input from Live smoke test: Adzuna and USAJOBS: `redirect_url` contains `utm_source=<app_id>`, so the mapper must strip it (or build the URL another way) before it reaches `Posting.url`. `salary_is_predicted` is the string `"1"`; every salary in the sample was predicted with min `==` max. No `contract_time`/`contract_type` in the US sample. US categories include non-Domain buckets (`graduate-jobs`, `part-time-jobs`, `other-general-jobs`, `unknown`): decide what Domain they map to. Page cap is 50.

Input from Gateway query interface: map the `Fetch` fields (one `title`, `country`, `location`, `radius_km`, `max_days_old`, `category`, `page`) onto params; page size is fixed at the cap. Failures surface as `SourceUnavailable` / `BudgetExhausted`, and auth errors propagate. Fill `SourceOutcome.total` from `count`. `company` and `description` feed the Duplicate rule, so the mapper's normalisation of them matters.

Answer (2026-10-30)

Grilled with the user; all recommendations accepted. No live call spent; checked against tests/fixtures/jobs/.

Params (`GET /jobs/{country}/search/{page}`)

- `title_only=<Fetch.title>`, never `what` or `what_or`: matching the title only cuts keyword collisions per 50-result page; synonyms come from the Search Plan's title variants.
- `where=location`, `distance=radius_km` (Adzuna's unit is km). A blank `location` means country-wide: both are omitted. `location0..n` is never used.

- `sort_by=date`, so page N walks back in time inside the window; repeats from shifting pages are caught by `key`.
- Always `results_per_page=50`, `max_days_old`; `category` when set. No salary or `contract_*` filters.
- `SourceOutcome.total = count`.

Countries: one table of Adzuna's 19 countries → ISO currency in the Adzuna client. The `Fetch` validator rejects any other country (a planning bug, not a source failure); the table is exported so the Planning LLD can constrain the Search Plan to it.

Mapper (one raw result → `Posting`)

- `source_id = id`; `title`, `company` (`company.display_name`, missing → `None`) and `description` are trimmed and whitespace-collapsed, nothing more. The trailing … stays (`coalesce` normalises it away). No HTML stripping: none seen in 100 results; add it if it appears.
- **Salary:** both bounds missing or ≤ 0 → `None`; one bound → just that bound; reversed → swapped. `currency` from the country table, `period = unknown`, `predicted = str(v) == "1"` (accepts `"1"` and `1`). Predicted salaries are kept; Analysis decides how to treat them.
- `region = tuple(location.area)` as given (country first, e.g. `"US"`); missing → `(country.upper(),)`. `location = location.display_name`. `posted_at` = UTC date of `created`. `closes_at = None`.
- `domain = category.tag`; `graduate-jobs`, `part-time-jobs`, `other-general-jobs`, `unknown` or a missing category → `None`.
- `url`: `redirect_url` with `utm_source` and `utm_medium` removed, every other param kept (`se`/`v` may be needed by the landing page).
- **Drop** = a result whose mapping raises `KeyError`, `TypeError`, `ValueError` or `ValidationError`; counted in `SourceOutcome.dropped`, logged once with its id. No salary, a non-Domain category or no company is not a drop. A body without a `results` list → `SourceUnavailable`.

Transport

- `httpx`, 10 s timeout; up to 2 retries (0.5 s, 1 s) on timeout, connect error and 5xx, then `SourceUnavailable`. Every attempt sent counts as one budget call (handed to L1 cache, budget guard and replay).
- `429` → `BudgetExhausted`, no retry. `401`/`403` → auth error, propagates and stops the Run (if over-quota turns out to be a 403, it fails loudly with the body logged and the guard's limit gets lowered). Any other 4xx → propagates as a bug (our params are wrong).
- `app_id`/`app_key` are query params and `httpx` logs URLs at INFO, so the `httpx` logger is set to WARNING and trace metadata never carries the URL.

Attribution: the Gateway exports `ATTRIBUTION[Source.adzuna]` ("The Adzuna API"); rendering it is the Brief LLD's job.

L1 cache, budget guard and replay

Type: grilling Status: resolved Assignee: smayankulkarni05 Blocked by: 02

Question

How do L1, the budget guard and replay work, and how do tests fake the Gateway?

- **L1**: key (normalised query params per source), value (raw response vs mapped Postings), TTL, storage. Same SQLite file as the Kernel's replay cache (§14), the checkpointer's, or its own; sync `sqlite3` vs `aiosqlite`.
- **Hard-expiry filter on read**: L1 hits drop Postings whose close date has passed.
- **Budget guard**: per-source counters, persisted across process restarts, day boundary (UTC vs Adzuna's), limits from settings, refuse-before-limit behaviour, whether cache hits and categories calls count, what a Run sees when refused.
- **Replay** (`REPLAY_CACHE`): L1 read with TTL ignored; what a miss does (fail vs live call); the fetched-at age Collection needs for its replay Caveat.
- **Test seam** (Kernel §15's open row): fake Gateway vs fake HTTP transport fed from Live smoke test: Adzuna and USAJOBS's fixtures.
- New settings rows for Kernel §13.

Input from Live smoke test: Adzuna and USAJOBS: the user never reported the Adzuna dashboard limits, and responses carry no rate-limit headers, so the limits must come from settings. Ask the user for the dashboard numbers at the start of this ticket. USAJOBS raw responses are ~85 KB per item (2 MB for 25), which bears on storing raw vs mapped in L1.

Input from Gateway query interface: L1 is keyed per source from a normalised `Fetch` (it's already normalised by its validator). Fill `SourceOutcome.cache` (`hit` / `miss` / `replay`), `fetched_at` and `calls_left` (`None` = unmetered); decide what a replay miss raises. The test seam must fit `JobsGateway`'s constructor-injected clients.

Input from Adzuna client and mapper: every HTTP attempt sent, retries included (up to 3 per call), counts as one budget call. A `429` raises `BudgetExhausted`, so the guard should also stop spending for the rest of the day when it sees one. Credentials are query params, so the L1 key must be built from the `Fetch`, never from the request URL.

Answer (2026-10-30)

Grilled with the user. No live call spent. The user delegated all replay decisions to the agent. The Adzuna dashboard limit is **25 calls a day**; no per-minute limit was reported.

Storage

- The Gateway has its own SQLite file, `gateway_db`, holding the L1 and budget tables. It uses stdlib `sqlite3` in autocommit mode, called directly from async code, like the Kernel's replay cache. A row is committed before `fetch` returns, so a kill mid-node re-serves it at no quota cost.
- MiniStack was considered at the user's request and deferred. It needs Docker, which this machine doesn't have on its PATH and evaluators would have to run, and it conflicts with Persistence and durable resume's clone-and-run rule. Revisit in the Infra shell if required.

L1

- **Key:** primary key `(source, fetch.model_dump_json())`. The `Fetch` validator has already normalised it, and it contains no credentials.
- **Value:** one source's mapped Postings plus `total`, `dropped` and `fetched_at`, stored before `coalesce` runs. This reverses the HLD's raw response: the cache stays lean and nothing in it can leak the Adzuna app id.
  - A mapper fix doesn't reach rows already cached. They stay as mapped until refetched; delete `gateway_db` if the fix can't wait.
  - A row that no longer validates against `Posting` counts as a miss.
- **What's stored:** valid 2xx bodies, including empty result lists. Failures and bodies without a `results` list are not stored.
- **Refetch vs delete** are separate rules:
  - `l1_ttl_hours` (24, one setting for both sources) decides when a row stops being served and the next identical Fetch calls the API and overwrites it.
  - A row is deleted only when every Posting in it has a `closes_at` and all have passed: store `expires_at = max(closes_at)`, `NULL` if any Posting has none or the row is empty, and run `DELETE ... WHERE expires_at < today` when the Gateway opens. Adzuna rows and empty rows are never purged, only overwritten.
- **Close-date filter:** on normal hits and fresh responses, drop each Posting past its `closes_at`. These don't count in `dropped`, which is the mapper-drop Caveat input.
- A row past its TTL is not served when the budget is exhausted: that source fails with `BudgetExhausted` as usual. Serving stale data would need its own status and Caveat, and L2 is the better place for that reuse.

Budget guard

- Table `(source, utc_day, used)`. Both sources are counted; only Adzuna has a limit (`adzuna_daily_limit`, default 25), so USAJOBS reports `calls_left=None`. To hold calls back for a walkthrough, lower the setting in `.env`; there is no reserve setting.
- Day boundary is UTC, since Adzuna's reset time is undocumented. If it differs, a `429` catches it.
- One atomic statement runs before every HTTP attempt, retries included: `UPDATE ... SET used = used + 1 WHERE used < limit RETURNING used`. No returned row raises `BudgetExhausted`.
- Cache hits don't count. L1 is checked first, so a fresh row is still served once the budget runs out.

- **On a `429`:** set today's `used` to the limit, so the refusal survives restarts and no more calls go out until the next UTC day. Deleting the row clears it by hand.
- **What a Run sees:** the path in Gateway query interface. `failures[adzuna]` is filled and USAJOBS still returns for `us`; outside the US every source has failed, so `GatewayError` becomes a `ToolMessage`.

Replay (`REPLAY_CACHE` set)

- The replay file gets its own L1 table, in front of the normal one and next to the LLM and Jev tables, so one file is a self-contained recorded Run. Budget counters always stay in `gateway_db`, otherwise a new replay path would reset the count.
- **Read-through, like Jev (Kernel §11):** replay file, then fresh normal L1, then the guarded live call. The result is written to the replay file. A failing miss was rejected because a recording Run could then never fill the file without a second setting.
- A replay hit reports `cache=replay`, with `fetched_at` from the row. A live call during replay reports `miss`.
- The replay table stores mapped Postings too, is never purged, ignores the TTL and skips the close-date filter. Filtering would make Postings vanish as dates pass, change the prompts and miss every LLM cache entry. The age Caveat discloses the staleness.

Test seam (Kernel §15's open row)

- **Gateway tests:** `make_gateway(settings, transport=None)` takes an `httpx.MockTransport` serving `tests/fixtures/jobs/`, with `gateway_db` under `tmp_path`. This runs the real clients, mappers, L1 and guard.
- **Stage tests:** `FakeGateway(results)` in `gateway/testing.py` returns canned `FetchResults` from `fetch`, matched by duck typing. No `Protocol`.

New settings (added to Kernel §13): `gateway_db` (`.cache/gateway.sqlite`), `adzuna_daily_limit` (25), `l1_ttl_hours` (24).

Update from L2 store, expiry and sweep (pass 2): the L1 delete no longer runs when the Gateway opens. It runs after each live write, in the same `_purge()` as L2's sweep. The expired-row rule itself is unchanged.

USAJOBS client and mapper (pass 2)

Type: grilling Status: resolved Assignee: smayankulkarni05 Blocked by: 01, 03

Question

How does the Gateway call USAJOBS and map its JSON onto Kernel Postings, following the Adzuna mapper's conventions?

- **Client**: three auth headers, params (`Keyword` / `PositionTitle`, `LocationName`, `Radius` units, `DatePosted`, `ResultsPerPage` up to 500), when to call it at all (settled: when `Fetch.country == "us"`).
- **Mapper**: `MatchedObjectId` as `source_id`, `PositionRemuneration` + `RateIntervalCode` → Salary period, multiple remuneration entries or locations per Posting, `ApplicationCloseDate` for hard expiry, description cap (4000), Region from the location fields.
- **Domain mapping**: USAJOBS family → Adzuna category tag, from the research draft and the categories list fetched in Live smoke test: Adzuna and USAJOBS.
- **Federal pay labelling**: how a Posting marks federal pay-table salaries so Analysis never mixes them unlabelled into Adzuna stats (Kernel `Posting` is fixed, so via `source` alone or something else).

Input from Live smoke test: Adzuna and USAJOBS: multi-value separator is `;`; `Radius` behaves like miles; announcements can list up to 1,176 locations (Region and `Posting.location` need a rule); per-Posting `RemoteIndicator` and `TeleworkEligible` exist; `JobGrade`/`LowGrade`/`HighGrade` mark pay-table jobs; `JobCategory` can hold up to 11 series; description candidates (`JobSummary`, `MajorDuties`, `QualificationSummary`) run to 7.9 KB against the 4,000 cap; dates have no timezone. Page size 500 would mean ~40 MB per response. Code list fixture has all 822 series with `JobFamily`.

Input from Gateway query interface: for the cross-source Duplicate rule to work, `Posting.description` must **start with** the announcement's opening text (likely `JobSummary`), so an Adzuna snippet of the same ad is a prefix of it. `company` should be the employer name Adzuna would show (`OrganizationName` likely). NAVSEA's 3 same-job announcements in the fixture are a test case.

Input from Adzuna client and mapper: conventions to mirror: text fields only trimmed and whitespace-collapsed; a drop is a per-result mapping error (`KeyError`/`TypeError`/`ValueError`/`ValidationError`), and missing optional data never is; non-occupational buckets map to `None` Domain; 10 s timeout, 2 retries on timeout/connect/5xx, 401/403 propagate, other 4xx propagate as bugs; credentials never in logs or trace metadata.

Input from L1 cache, budget guard and replay: L1 stores mapped Postings, so raw response size only affects the transfer, not the cache. `closes_at` drives both the close-date filter and when an L1 row is

deleted, so it must be set whenever `ApplicationCloseDate` parses. USAJOBS calls are counted but unlimited (`calls_left=None`).

Answer (2026-10-30)

Grilled with the user; all recommendations accepted. 15 free USAJOBS calls made (no Adzuna quota) from a scratchpad script, not saved as fixtures.

New live facts

- `PositionTitle` matches **any** word in the title: "data engineer" returned 99 results around DC, including Data Analyst and Civil Engineer. A quoted phrase returned 0. `Keyword` needs all words but searches the whole announcement: 33 results, including `GENERAL ENGINEER`.
- `DatePosted` above 60 is capped silently: "engineer" returned 426 at both 60 and 90.
- Country-wide "data engineer" (`DatePosted=60`) timed out on both tries, at 25 s and 20 s. "engineer" and "nurse" returned in about a second with no location.
- An unresolvable `LocationName` returns 200 with 0 results.
- `ResultsPerPage=50` is about 1 MB. In 125 sampled Postings, every one had a single remuneration entry; one used `SY`.

Params (`GET https://data.usajobs.gov/api/Search`; headers `Host`, `User-Agent`, `Authorization-Key`)

- `PositionTitle=<Fetch.title>`, in the API's default relevance order. `sort=opendate` (Adzuna's rule) was rejected because it puts unrelated engineers ahead of real matches. The Relevance stage removes the extra matches.
- `ResultsPerPage=50`, `Page=Fetch.page`, `DatePosted=min(max_days_old, 60)`. `Fields` isn't sent, because the default response already carries `Details`.
- `LocationName=location`, `Radius=ceil(radius_km / 1.609)`. A blank location omits both.
- `Fetch.category` is ignored. Translating a tag into series codes needs long `;` lists, and filtering after mapping would shorten pages, falsify `total` and lose real matches where the Domain table is coarse. A USAJOBS Posting can therefore return with a different Domain from the one asked for.
- `SourceOutcome.total = SearchResultCountAll`.

Mapper (one `SearchResultItems` entry → `Posting`)

- `source_id = MatchedObjectId`; `title = PositionTitle`; `company = OrganizationName` (the agency, not the department); `url = PositionURI` with `:443` removed. Text fields are trimmed and whitespace-collapsed, nothing more.
- `posted_at` = date of `PublicationStartDate`; missing or unparseable is a drop. `closes_at` = date of `ApplicationCloseDate`; unparseable is `None`. The timestamps carry no timezone, so the date is taken as written.
- **Description:** `JobSummary`, `MajorDuties` (list joined), `QualificationSummary`, in that order, collapsed and cut at 4,000 characters. All three blank is a **drop**: an empty description is a prefix of every

other, so `coalesce` would merge it with any Posting of the same title and employer.

- **Salary:** the first `PositionRemuneration` entry only. `PA`/`PH`/`PD`/`BW`/`PM` → annual, hourly, daily, biweekly, monthly. `SY`/`PW`/`FB`/`ST`/`WC` → `salary=None`, because those figures can't be compared. Bounds ≤ 0 are ignored, reversed ones swapped, as for Adzuna. `currency="USD"`, `predicted=False`.
- **Federal pay label:** `source == usajobs`. Every USAJOBS salary is federal pay, so a new field would only repeat it. Keeping it apart from Adzuna salaries is the Analysis LLD's rule.
- **Location:** `location = PositionLocationDisplay`. For `region`, each `PositionLocation` becomes `(country, CountrySubDivisionCode, city)`, with `United States` written `US` and the `, State` suffix removed from `CityName`. One location uses its hierarchy; several use the common prefix (two Virginia sites → `("US", "Virginia")`, FAA's 1,176 → `("US",)`). Mixed countries or no locations → `("US",)`. Overseas Postings are kept, country first.
- `RemoteIndicator` and `TeleworkEligible` are ignored: `Posting` has no field for them, and the display string already reads "Anywhere in the U.S. (remote job…".
- FAA "Public Notice" talent-pool announcements stay normal Postings, since the mapper doesn't judge relevance.
- A body without `SearchResult` → `SourceUnavailable`. Drops follow the Adzuna rule otherwise.

Domain: a static dict in the USAJOBS client, using the first `JobCategory` entry. Series overrides win over the family. An unknown series → `None`, logged once. All tags checked against the 30 live Adzuna tags.

Tag	Families	Series overrides into this tag
`social-work-jobs`	0100	
`hr-jobs`	0200	
`admin-jobs`	0300	
`scientific-qa-jobs`	0400, 1300, 1500, 1900	0028
`accounting-finance-jobs`	0500	1510
`healthcare-nursing-jobs`	0600, 0700	0180
`engineering-jobs`	0800	
`legal-jobs`	0900, 1200	
`creative-design-jobs`	1000, 3900, 9000	
`pr-advertising-marketing-jobs`		1035
`consultancy-jobs`	1100	0110, 0343
`property-jobs`		1170
`maintenance-jobs`	1600, 4800, 5300, 5800, 8200, 8600, 8800	
`teaching-jobs`	1700	

Tag	Families	Series overrides into this tag
`logistics-warehouse-jobs`	2000, 2100, 5700, 6900, 7000	
`it-jobs`	2200	0391, 0854, 1550, 1560
`trade-construction-jobs`	2500, 2600, 2800, 3600, 3800, 4100, 4200, 4600, 4700	
`manufacturing-jobs`	3100, 3300, 3400, 3700, 4000, 4300, 4400, 5200, 5400, 6500, 6600	
`domestic-help-cleaning-jobs`	7300	
`hospitality-catering-jobs`	7400	
`None`	0000, 1400, 1800, 2300, 3500, 5000, 7600, 9900	1102, 1105

The draft's local `law-enforcement` Domain is rejected: Domain stays an Adzuna tag. No USAJOBS Posting gets `sales-jobs`, `retail-jobs`, `customer-services-jobs`, `energy-oil-gas-jobs` or `charity-voluntary-jobs`.

Transport: as Adzuna (`httpx`, 10 s timeout, 2 retries at 0.5 s and 1 s on timeout, connect error and 5xx, then `SourceUnavailable`; 401/403 propagate; other 4xx propagate as bugs). The differences:

- A `429` → `SourceUnavailable`, with no day-long lockout, because USAJOBS has no limit to protect.
- The timeout stays at 10 s. A multi-word country-wide search that times out ends in `failures[usajobs]`, and Adzuna's results still return. Raising it wouldn't help, since those calls also failed at 20 s and 25 s.
- The key travels in a header, not the URL, so it can't reach `httpx`'s URL logs. It still never goes into logs or trace metadata.

Adzuna aggregate endpoints

Type: grilling Status: resolved Assignee: smayankulkarni05 Blocked by: 01

Question

Handed down by HLD §10: do Adzuna's `histogram`, `top_companies` and `history` endpoints feed the Market Brief, and if so how?

- They carry no Posting IDs, so they aren't strict Evidence. Allowed as labelled context (e.g. "Adzuna-wide salary distribution"), or excluded?
- Quota cost per call against the daily budget, and which build pass (if any).
- If in: what the Gateway returns for them, and which component turns them into Stats or Insights.

Input from Live smoke test: Adzuna and USAJOBS: `histogram` is `{lower_bound: count}` in 20k bands up to an open 140k bucket, and sums to 1,397 against 3,200 matches, so it covers an undocumented subset. `top_companies` is the top 5 with counts. Both resolved `where` to the city level. Each costs one quota call.

Input from L1 cache, budget guard and replay: the daily Adzuna limit is 25, and any aggregate call would go through the same budget guard. L1 rows hold Postings, so aggregate responses would need their own cache shape.

Answer (2026-10-30)

Grilled with the user; both recommendations accepted. No live call spent; `history` was never called, so its real shape is unobserved.

Excluded from every build pass: `histogram`, `top_companies`, `history` and the search response's `mean`. The Gateway exposes only Postings and `SourceOutcome`, so no aggregate client, cache shape, `Fetch`/`FetchResult` type or budget spend is needed, and no component turns them into Stats or Insights.

- **Traceability:** none of them has Posting IDs, and the spec expects every number to trace back to real Postings.
- **`histogram`:** the New York fixture covers 1,397 of 3,200 matches (44%), an undocumented subset, and 1,059 of those (76%) sit in the open 140k+ band. It can't produce a median or middle 50%.
- **`top_companies`:** the Sample already gives top employers with Evidence, and the direct vs agency split needs per-Posting data.
- **`history`:** the spec asks for a trend in posting dates, which the Sample provides.
- **`mean`:** every salary in the smoke-test sample was predicted, so it may be an average of estimates.

- **Cost avoided:** one call per endpoint per Run would be up to 3 of the 25 daily calls, or 150 fewer Postings. Both calls made also resolved `where` to the city level, so their scope may not match the Sample's.

`SourceOutcome.total` stays, as disclosure only. Collection uses it for paging. It may appear in Run Notes or a Caveat, labelled as the source's own match count (for example "Analysed 50 of about 3,200 Adzuna matches"). It is never a Stat, an Insight's Evidence or a demand figure: it is an unverified keyword count that hasn't passed the relevance check. Wording and placement belong to the Collection and Brief LLDs.

Follow-ups: the histogram and top-companies fixtures stay as evidence for Live smoke test: Adzuna and USAJOBS, and the exclusion goes in the README's known limitations (delivery task).

L2 store, expiry and sweep (pass 2)

Type: grilling Status: resolved Assignee: smayankulkarni05 Blocked by: 04, 05

Question

How does the L2 region × Domain Posting store work?

- **Schema**: Postings keyed `source:source_id`, Region and Domain columns, first-seen / last-seen, raw vs mapped.
- **Read-before-call rule**: what "enough fresh Postings for region + Domain + role" means (role matching without an LLM, count threshold, freshness), and whether it needs a target from the query (see the map's Not yet specified). `Fetch` carries no target today (Gateway query interface).
- **Merge**: dedupe on key, updating changed Postings.
- **Expiry**: soft-expiry freshness window length, hard expiry from close dates, filter on read, sweep at Run start (who calls it: Runtime or the Gateway lazily).

Input from L1 cache, budget guard and replay: L2 can live in `gateway_db`. L1 rows now hold mapped Postings and are purged only when every Posting has closed, with a 24-hour TTL deciding refetch, and the purge runs when the Gateway opens. The user's intent of keeping Postings until they end and reusing them across queries belongs here. Decide what L2 still adds over L1, and whether the replay path (never purged, no close-date filter) must bypass it so a replayed Run stays reproducible.

Input from USAJOBS client and mapper (pass 2): a multi-location USAJOBS Posting's Region is the common prefix of its locations, so a nationwide notice sits at `("US",)`, not in any city. USAJOBS Postings can carry a different Domain from the Fetch's `category`, or `None`, because the category isn't sent to that source. `PositionTitle` matches any word in the title, so stored Postings aren't all real matches for the title that fetched them.

Answer (2026-10-30)

Grilled with the user; all recommendations accepted, except that the user rejected sweeping when the Gateway opens, which led to the sweep-on-write rule below. No API calls spent.

What L2 is for

- L2 skips Adzuna calls on an L1 miss, and serves as a fallback when a live call fails. It adds two savings over L1: the same search after 24 hours, and overlapping searches.
- **Adzuna only.** USAJOBS calls are free, and its Postings match poorly (nationwide notices at `("US",)`, Domains that differ from the one asked for, any-word titles). USAJOBS stays on L1 and

live calls. Adzuna's `closes_at` is always `None`, so L2 needs soft expiry only, and hard expiry stays in L1. This reverses the HLD's "every Posting is stored".

Read order (Adzuna, per `Fetch`)

1. Replay file, when `REPLAY_CACHE` is set.
2. Fresh L1 row.
3. L2, if it can fill the requested page.
4. Guarded live call. The result is written to L1 and L2.
5. On a failed live call, L2's partial page, if it has any matches.

A replay hit returns before L2, so a replayed Run stays reproducible. During a recording Run, an L2 answer is written to the replay file like any other. L2 answers are never written to normal L1, which would restart the 24-hour TTL on old data.

Reuse rule (guards against a skewed Sample) A stored Posting remembers the `Fetch` that last returned it (`via`), and is reused only if `via` was at least as broad as the new `Fetch`. Otherwise "senior data engineer" Postings would fill a later "data engineer" request and push its salary Stats up.

- `via` **must have:** the same country; every word of `via.title` in the new title; the same location, or a blank `via.location`; `via.radius_km` at least the new radius; `via.category` of `None` or the same category.
- **The Posting must have:** every word of the new title in `Posting.title` (what `title_only` would require); `posted_at` within `max_days_old`; `domain` equal to `category`, if one was given; and, if `via` was country-wide and the new location isn't blank, the new location equal to one Region segment (case-insensitive).
- **Known limits:** a wider stored radius can return a few Postings outside the new radius. A later, narrower `Fetch` overwrites `via`, which can cost an extra call but never skews the Sample.

Stored `via`	New `Fetch`	Result
"data engineer", London	"senior data engineer", London	Reused, only the senior jobs
"senior data engineer", London	"data engineer", London	Live call
"data engineer", Camden	"data engineer", London	Live call
"data engineer", UK-wide	"data engineer", Camden	Reused, Camden jobs only

"Enough" means a full page Matches sort newest first, then by key, like `sort_by=date`. L2 serves page N only if slice `(N-1)*50 .. N*50` is full, so it never returns less than a live call would. Otherwise the call goes live, and any overlap is removed by key in `collect`. The 50 is the client's page-size constant, not a setting. `Fetch` needs no target count, which closes the map's open question without changing the interface. Only the failure fallback serves a partial page.

Freshness

- `l2_fresh_days` (default 7) measures time since Adzuna last returned the Posting (`last_seen_at`). Only a live call moves it, so serving from L2 doesn't extend it.
- `Fetch.max_days_old` separately limits how old the ad is, per request.
- A search can therefore go about a week without spending a call.

Schema and merge (in `gateway_db`)

```
l2_postings(key TEXT PRIMARY KEY, country TEXT, via TEXT, posting TEXT, last_seen_at TEXT)
-- plus one index on last_seen_at
```

- **Write:** every live Adzuna response, from any page, before `coalesce`, with `INSERT ... ON CONFLICT(key) DO UPDATE`. The newest version wins.
- **Read:** `WHERE country = ? AND last_seen_at >= ?`, then the reuse rule in Python, checking `via` before validating the Posting. A row that no longer validates is skipped.
- **Dropped:** Region and Domain columns, `first_seen` (nothing reads it) and raw responses (same reasoning as L1).
- **Ceiling:** about 8,750 rows (25 calls × 50 Postings × 7 days), so a scan is fine. The read gets a `ponytail:` comment naming this ceiling; the upgrade is Region and Domain columns.

Outcome fields `CacheStatus` gains `store`. An L2 answer reports `cache=store`, `total` = L2's match count, `dropped=0`, `fetched_at` = the oldest `last_seen_at` served, and the current `calls_left`. A read-before-call hit spends no budget. On the failure fallback, `sources[adzuna]` is set as above and `failures[adzuna]` is also filled, so Collection's "one source failed" Caveat still fires. `GatewayError` is raised only when no source returned anything. With `store`, `total` is not Adzuna's match count, so it can't feed the sampling disclosure.

Sweep

- A single `_purge()` runs straight after each live write: `DELETE ... WHERE last_seen_at < cutoff` for L2, plus L1's existing delete. The tables only grow on a live write, so nothing else needs to trigger it, and cache hits and opening the Gateway do no sweep work. This reverses HLD §7's "sweep at the start of each Run".
- The read filter hides expired rows whether or not they've been deleted yet.
- The Infra shell's scheduled job can call the same method later.
- Measured on a worst-case table (8,750 rows, default SQLite durability, D: drive): sweep with nothing to delete 0.01 ms, deleting 350 rows 4 ms, read scan over 7,000 rows 23 ms checking `via` only and 39 ms validating every Posting, against 800–2,500 ms for an Adzuna call.

New setting (added to Kernel §13): `l2_fresh_days` (7).

Write docs/lld/gateway.md

Type: task Status: resolved Assignee: smayankulkarni05 Blocked by: 01, 02, 03, 04, 05, 06, 07

Question

Write `docs/lld/gateway.md` from the resolved tickets and the map's Notes: module layout, query interface, Adzuna and USAJOBS clients and mappers, L1, budget guard, replay, L2, expiry and sweep, settings rows, test seams, and the build checklist per pass. Gist and link each decision to its ticket; introduce no new decisions. Fill Kernel §15's Jobs APIs row by linking here. Include `coalesce` and the source-choice rule.

Answer (2026-10-30)

Written: docs/lld/gateway.md. Kernel §15's Jobs APIs row now links to it, and future-scope.md has a Gateway section. No API calls spent.

Details the tickets left open, settled while writing, none changing a ticket's decision:

- **`SourceRejected`, and no `raise_for_status()`.** Confirmed on `httpx` 0.28.1: its message includes the full URL, so a 401 would put `app_id` and `app_key` in logs and traces. The credential rule in Adzuna client and mapper needs a plain exception that carries only status and body.
- **`ADZUNA_CURRENCY` lives in `gateway/types.py`,** not the Adzuna client as Adzuna client and mapper says, because the `Fetch` validator needs it and the client imports `Fetch`.
- **Series `8999` gets an override into `maintenance-jobs`.** It's the one entry of 822 in the code list whose `JobFamily` (`8800`) isn't its first two digits plus `00`.
- **Replay rows are written as served,** after the close-date filter, so a replay returns what the recording Run returned. They go in a `jobs` table on the Kernel's `Replay`.

Noted, not changed:

- L2 store, expiry and sweep (pass 2) writes every L2 answer to the replay file, including a failure fallback's partial page. A replay of that Fetch then reports `replay` with no `failures[adzuna]`, so the "one source failed" Caveat doesn't repeat.
- The L2 radius check also applies to a country-wide `via`, so a small `radius_km` on one costs an extra call. Listed as a known limit.
- A blank Adzuna description isn't a drop, unlike USAJOBS, so it could over-merge in `coalesce`. Not seen in the 100 results sampled.

gateway-build

gateway-build/issues/01-adzuna-through-fetch.md

01: Adzuna through fetch

What to build: a stage can call `make_gateway(settings)`, open it with `async with`, and `fetch` one page for a `Fetch`, getting coalesced Postings back. A source failure comes back as a result field or a clear error, and no failure path can leak the Adzuna keys. Spec: Gateway LLD §2, §3, §5, §6, §7.1, §7.2, §16 pass 1 steps 1-3.

Out of scope: the budget guard and L1 (ticket 02), so nothing here is cached or counted yet. Tests use `httpx.MockTransport` and spend no live quota.

Blocked by: None (can start immediately).

Status: done

- [x] `httpx` is pinned directly at the installed version
- [x] Test: the `Fetch` validator normalises text, equal Fetches dump equal JSON, and a country outside `ADZUNA_CURRENCY` is rejected
- [x] Test: the Adzuna search fixture gives 50 Postings through `fetch`, with no `utm_` params in `url`, `predicted` true, and non-Domain tags mapped to `None`
- [x] Test: reversed and missing salaries map as §7.2 says, and a blank `location` omits `where` and `distance`
- [x] Test: a bad result is counted in `dropped` and the rest still return, while no salary, no company or a non-Domain category isn't a drop
- [x] Test: Deloitte and Capital One collapse into one survivor each with `merged` filled, Booz Allen stays apart, and the survivor order holds
- [x] Test: a timeout, connect error or 5xx retries twice then ends in `failures[adzuna]`, and `fetch` raises `GatewayError` when no source returned anything
- [x] Test: a body without `results` is `SourceUnavailable`
- [x] Test: a `401` raises `SourceRejected` carrying source, status and body, with no key or URL in its message
- [x] `raise_for_status()` is never called, `SourceUnavailable` is raised `from None` with fixed text, and `make_gateway` sets the `httpx` logger to WARNING
- [x] `FakeGateway(results)` returns canned `FetchResults` from `fetch`, so stage tests can start against it
- [x] `ATTRIBUTION[Source.adzuna]` is exported

02: Budget guard and L1

What to build: the Gateway protects the daily Adzuna quota. A repeat `Fetch` is served from `gateway_db` at no spend, every HTTP attempt is counted first, and once the limit is reached or Adzuna answers `429`, no more calls go out until the next UTC day. A killed process re-serves what it already fetched. Spec: Gateway LLD §4 steps 2, 4 and 6, §8, §9, §16 pass 1 steps 4-5.

Out of scope: the close-date filter and deleting L1 rows (ticket 04), though `expires_at` is written from the start so the schema doesn't change then. L2 (ticket 05).

Blocked by: 01: Adzuna through fetch.

Status: done

- [x] Test: a repeat `Fetch` is a `hit` with no HTTP call and no spend, and a first call is a `miss`
- [x] Test: a fresh row is still served after the budget has run out
- [x] Test: a row past `l1_ttl_hours` misses and is overwritten, and isn't served when the budget has run out
- [x] Test: a row that no longer validates against `Posting` is a miss
- [x] Test: an empty result list is cached, and failures and bodies without `results` aren't
- [x] Test: the guard refuses at the limit, and the count survives reopening `gateway_db`
- [x] Test: retries spend too, one budget call per attempt
- [x] Test: an Adzuna `429` raises `BudgetExhausted` with no retry and locks the UTC day across a reopen
- [x] Test: a refusal ends in `failures[adzuna]` and, with no other source, `GatewayError`
- [x] `calls_left` is reported on every outcome, and the spend is one atomic statement
- [x] The L1 key holds no credentials, and rows are committed before `fetch` returns
- [x] TTL tests backdate rows directly, with no injected clock

03: Replay, tracing and live smoke (pass 1 exit)

What to build: with `REPLAY_CACHE` set, jobs data replays from the same file as the LLM and Jev responses, so a recorded Run is one self-contained file. Each source call shows in LangSmith without exposing a URL or key. One live `fetch` proves the real stack works, which finishes the walking skeleton. Spec: Gateway LLD §4 step 1, §10, §13, §16 pass 1 steps 6 and 8.

Check the `traceable` API through the `langchain-router` skill before writing the tracing. The smoke script is not in CI and spends one Adzuna call, so **get the user's OK before running it**.

Blocked by: 02: Budget guard and L1.

Status: done

- [x] The Kernel's `Replay` gains a `jobs` table with `get_jobs` and `put_jobs`, and the Gateway gets it from `open_replay`
- [x] Test: a recording Run's answer replays with `cache=replay` and the row's `fetched_at`, past its TTL and with no HTTP call
- [x] Test: a replay miss goes live, reports `miss`, and is written to the replay file as served
- [x] Test: budget counters stay in `gateway_db`, whatever the replay path
- [x] Test: with `REPLAY_CACHE` unset, no replay file is created
- [x] Each source call is a LangSmith child run `gateway:<source>`, with metadata `source`, `page`, `cache` and `calls_left`, and a summary output of count, dropped and total
- [x] Trace metadata and outputs never carry the Postings, a URL or a key
- [x] Live smoke: with the user's OK, one `fetch` returns Postings through the real stack and shows in the LangSmith project

04: USAJOBS, fan-out and close dates (pass 2)

What to build: a `us` Fetch returns Adzuna and USAJOBS Postings merged and coalesced, while other countries call Adzuna only. One source failing still returns the other. Postings past their close date never reach a stage, and L1 rows whose Postings have all closed get deleted. Spec: Gateway LLD §4, §5, §7.3, §7.4, §8, §12, §16 pass 2 steps 1-3.

Tests use the fixtures and spend no live quota. Out of scope: L2 (ticket 05), and keeping federal pay apart from Adzuna salaries (Analysis).

Blocked by: 03: Replay, tracing and live smoke, because replay rows must skip the close-date filter and the purge.

Status: done

- [x] Test: on the USAJOBS search fixture, the description starts with `JobSummary`, `:443` is gone from `url`, and `SY` gives `salary=None`
- [x] Test: FAA's 1,176 locations give region `("US",)`, and two Virginia sites give `("US", "Virginia")`
- [x] Test: a blank description and an unparseable `posted_at` are drops, and an unparseable `closes_at` is `None`
- [x] Test: the Domain table applies series overrides over families, including `8999` to `maintenance-jobs`, and an unknown series gives `None`
- [x] Test: `DatePosted` is capped at 60, `Radius` is `radius_km` in miles rounded up, and `Fetch.category` isn't sent
- [x] Test: a USAJOBS `429` is `SourceUnavailable` with no lockout, and USAJOBS reports `calls_left=None`
- [x] Test: `country == "us"` calls both sources concurrently, and other countries call Adzuna only
- [x] Test: one source failing fills `failures` and still returns the other
- [x] Test: a `SourceRejected` from one source wins over a result from the other
- [x] Test: a re-announced ad collapses, while NAVSEA's two offices (different duties) and FAA FV/FG stay apart, through `coalesce`
- [x] Test: an expired Posting is filtered on hits and fresh responses, and isn't counted in `dropped`
- [x] Test: `_purge()` runs after a live write and deletes an L1 row only when all its Postings have closed
- [x] Test: a replay row skips both the filter and the purge

05: L2 store (pass 2)

What to build: on an L1 miss, an Adzuna Fetch is answered from stored Postings when they can fill the page without skewing the Sample, spending no budget. A failed live call falls back to whatever partial page L2 has, and Collection still learns the source failed. Stored Postings expire 7 days after Adzuna last returned them. Spec: Gateway LLD §4 steps 3 and 5, §11, §12, §16 pass 2 steps 4-5.

Adzuna only. Whether an L2 answer's age raises a Caveat is Collection's.

Blocked by: 04: USAJOBS, fan-out and close dates, for the source loop and `_purge()`.

Status: done

- [x] `CacheStatus` gains `store`, and an L2 answer reports `total` as L2's match count, `dropped=0`, `fetched_at` as the oldest `last_seen_at` served, and the current `calls_left`
- [x] Test: each live Adzuna response, from any page, is written before `coalesce` with the newest version winning, and `via` and `last_seen_at` are updated
- [x] Test: the four rows of the reuse table in §11 give the stated result
- [x] Test: the Posting checks hold for title words, `posted_at` within `max_days_old`, `domain` against `category`, and a country-wide `via` against a `region` segment
- [x] Test: page N is served only when its slice is full, otherwise the call goes live
- [x] Test: a read-before-call hit spends no budget
- [x] Test: on a failed live call, a partial page fills both `sources[adzuna]` and `failures[adzuna]`, and with no matches only `failures[adzuna]`
- [x] Test: an L2 answer isn't written to L1, and during recording is written to the replay file
- [x] Test: a replay hit returns before L2
- [x] Test: USAJOBS Postings are never stored in L2
- [x] Test: a row that no longer validates is skipped
- [x] Test: rows past `l2_fresh_days` are hidden on read and deleted after the next live write, and serving from L2 doesn't move `last_seen_at`
- [x] The read carries the `ponytail:` comment naming the 8,750-row ceiling and the upgrade path

runtime-lld

runtime-lld/map.md

Runtime and CLI LLD

Label: wayfinder:map Parent: Talent Market Scout: HLD (components G and H in docs/HLD.md §3, §8)

Destination

`docs/lld/runtime.md`: an LLD of Runtime and Clients precise enough to build straight from, covering pass 1 (wiring, SQLite, registry, Run Controller, Pause protocol, resume guard, CLI, stub-node skeleton), pass 2 (`--focus` plumbing, Streamlit's mapping onto the Run Controller) pass 3 (event model) and how pass 6 opens the Store, with nothing left to decide. Building the code is a separate effort after this map.

Notes

- Domain: Runtime wires and runs the graph and owns threads and the registry; Clients call only the Run Controller. Vocabulary: `GLOSSARY.md`. Spec: `Joveo_Talent_Market_Agent- Assessment.pdf`. Deadline Mon 2 Nov 2026, 9:00 AM, so this map stays lean.
- Fixed upstream: docs/lld/kernel.md §7 (routing), §8 (`owned()`), §10 (persistence rules), §12 (`check_balance`), docs/lld/gateway.md, HLD §8, Research: LangGraph persistence and resume.
- Skills: `langchain-router` first for any framework question (per `CLAUDE.md`); `grilling` + `domain-modeling` for grilling tickets.
- Standing preferences: inherited from the Kernel map (Pydantic v2 everywhere, `uv`, Python 3.12, `src/talent_scout/`, `pytest`, root `.env`).
- Scope rules (decided at charting):
  - **Boundary with stages.** Runtime owns the export contract, dependency handover, the Pause envelope, the wiring table from HLD §4 and a stub-node skeleton. Stage LLDs own node bodies and each Pause's payload and answer fields.
  - **Dependencies.** Each stage exports `make_nodes(deps: Deps) -> dict[str, Node]` plus plain router functions, closing over a frozen `Deps(tiers, jev, gateway, settings)`. No `context_schema`.
  - **Async throughout.** Async Run Controller on `AsyncSqliteSaver`, `AsyncSqliteStore` and `astream`; the CLI calls `asyncio.run`; Streamlit bridges in its own layer.
  - **Storage.** Checkpointer and registry share `.cache/runs.sqlite` (setting `runs_db`), separate from `gateway.sqlite`. The registry is a plain `runs` table, not a Store namespace, and the Store is opened only when preferences land (pass 6), per Run Controller API and wiring.

    - **Run status.** The registry stores `running | paused | done | aborted | failed`, thread ID, request summary and updated time. It is an index: `get_state` is the truth and disagreements reconcile on read, so no heartbeat. A killed Run stays resumable.
    - **Resume guard.** A per-thread OS-level lock: a stdlib SQLite connection holding `BEGIN IMMEDIATE` on `.cache/locks/<thread>.lock`. A second process gets an immediate "this Run is open elsewhere" error. Confirmed by the research ticket.
    - **Pause envelope.** Every `interrupt()` carries `Pause(kind, payload)`, `kind` being `clarify | plan_review | low_data_review`. The answer is a per-kind model the Controller validates before `Command(resume=...)`, so a bad answer re-prompts. Clients render by `kind`.
    - **CLI.** Stdlib `argparse`: `scout run "<request>" [--focus dim ...]`, `scout resume [thread_id]` (lists resumable Runs with no ID), `scout show <thread_id>`. The Market Brief is printed and written as JSON and Markdown to `runs/<thread_id>/`.
    - **Events (pass 3).** Clients get a chat-assistant-style live view: LLM tokens as they arrive, tool and API calls as they happen, and a reasoning display. The event stream yields a `RunEvent` union and clients never see LangGraph chunks. The exact events and their sources are decided in Clients flow, after Research: live streaming sources.
    - **Streamlit is disposable.** A simple web UI is future scope, so Streamlit stays a thin client that shapes nothing in Runtime.
- On 2026-10-31 the user told the agent to make every remaining decision itself and finish the map. Tickets 06 and 07 were decided by the agent, and each says so.

Decisions so far

<!-- one line per closed ticket -->

- Research: async persistence and the resume guard: the lock-file guard holds, including after a hard kill; saver and Store share one file on separate connections; `Pause.payload` loses its class across processes unless concretely typed; `setup()` needs retry on a fresh file, and registry read-then-write needs `BEGIN IMMEDIATE`.
- Research: live streaming sources: the chat-style view is practical; `messages` mode needs two fixes in `kernel/llm.py` first; reasoning is optional per turn; structured output streams JSON fragments and stays provisional until the node finishes; closing a stream cancels the Run.
- Run Controller API and wiring: the Controller owns the graph task and hands out a `RunHandle` with an in-memory event buffer; the registry is a `runs` table written after the checkpoint and reconciled on read; `Pause` is a discriminated union by kind; the full graph compiles from day one with stubs for missing stages; the skeleton proves resume with a kill-and-resume test across processes.
- Clients flow: `rich` CLI with `$EDITOR` plan edits and `Ctrl-C` cancelling to a resumable Run; `RunEvent` union with tokens only from `fetch_agent` and a new `tasks` stream mode to verify; tokens stay out of the replay buffer; Streamlit is a thin client with no Controller changes.
- Write docs/lld/runtime.md: the LLD is written and the Kernel, Gateway, HLD and future-scope docs agree with it; the `tasks` mode passed an offline check, which also removed the need for a custom extraction signal.

- Opening the Store: from pass 6 the Controller opens it on `runs_db` on its own autocommit connection, and nodes reach it through `runtime.store`, not `Deps`.
- Bring the LLD in line with the code: three stale details fixed in the doc, the code was right each time, and nothing else in HLD §3's Runtime row is undecided.

Not yet specified

Nothing: the destination is reached.

Out of scope

- Infra shell internals: only the seam at the Run Controller is stated here.
- Node bodies, Pause payload and answer fields, what each `--focus` value changes: stage LLDs.
- What the Store holds and which nodes read it: Long-term memory LLD.
- Writing the Runtime and CLI code: separate build effort after this map.

Research: async persistence and the resume guard

Type: research Status: resolved Findings: local branch `research/runtime-async-persistence` (commit `4c64c6f`, not pushed: the repo has no remote), file `research/runtime-async-persistence.md` Assignee: smayankulkarni05 Blocked by: none

Question

Do the charting decisions in Runtime and CLI LLD hold on the pinned LangGraph versions, on Windows?

- `AsyncSqliteSaver` and `AsyncSqliteStore`: construction, lifecycle, and whether both can share one file.
- `LANGGRAPH_STRICT_MSGPACK=true` with `Pause(kind, payload)` as an interrupt value: does the payload rehydrate, and what goes in `allowed_msgpack_modules`?
- How a Pause surfaces through `astream(stream_mode="updates")`, and reading it back with `aget_state` in a fresh process.
- The guard: does a stdlib SQLite connection holding `BEGIN IMMEDIATE` on a lock file refuse a second process at once, and release on a hard kill?
- Two processes writing the same `runs.sqlite` file at once (untested in the HLD research): what fails, and is WAL or a busy timeout needed?

Answer

Verified by cross-process experiments on Windows.

- **Construction.** The saver and Store share one file on separate connections. An explicit allowlist needs raw `AsyncSqliteSaver(conn, serde=JsonPlusSerializer(allowed_msgpack_modules=[...]))`, because `from_conn_string` takes no `serde`. Construct inside the running loop. Set `LANGGRAPH_STRICT_MSGPACK` before importing langgraph.
- **Pause payload.** The serializer keeps only the outermost model's class, so `payload: Any` or `dict` returns a dict in a fresh process, and `payload: BaseModel` silently loses all fields. Same-process tests hide this. A concrete model or discriminated union rebuilds. Allowlist only the outermost classes: `Pause` and any Pydantic resume value.
- **Surfacing.** `astream` updates end with `{"__interrupt__": (Interrupt(value=...),)}`, with no exception. In a fresh process, `aget_state` gives `next` and `.interrupts`; both are empty after a resume.

- **Guard holds.** A second process is refused in about 0.5 ms at `timeout=0`, and the lock is free about 1 ms after `taskkill /F`. Keep a dedicated connection referenced for the whole Run and never commit on it: GC or `commit()` releases the lock. Probe only with `BEGIN IMMEDIATE`, and check `sqlite_errorname`. Keep it out of `runs.sqlite`.
- **Two writers.** WAL (set by the saver's `setup()`) and a busy timeout are needed. At the 5 s default: no failures at 2×100 or 4×150, 1 of 1,200 at 4×300, no corruption. Racing `setup()` on a fresh file fails, and retry with backoff fixed it (18 of 18). Registry read-then-write must use `BEGIN IMMEDIATE`, or it fails at once with `SQLITE_BUSY_SNAPSHOT`. Stale `running` rows mean the lock file is the liveness test.
- **Untested:** an `Enum` or custom class inside a payload; one connection shared by saver and Store.

Run Controller API and wiring

Type: grilling Status: resolved Assignee: smayankulkarni05 Blocked by: none

Question

What is the Run Controller's exact API, and how is the graph wired?

- Signatures of start, resume, stream events and get state, and what each returns.
- Status transitions, and when the registry is written relative to the checkpoint.
- Failure handling: `LLMExhausted`, auth errors and bugs stopping a Run; where `check_balance` refuses.
- Wiring table from HLD §4, `owned()` wrapping, and the stub-node skeleton that proves durable resume before any stage exists.
- `Pause.payload` typing, given the research: `payload: dict` plus `kind` with the stage calling `model_validate` (keeps ownership in the stage), or a discriminated union in Runtime (typed, but Runtime imports every stage's payload).
- Running the graph in its own task, since closing a stream cancels the Run, and whether the Controller stores events so a reloaded client has history.
- Lifecycle of `Deps` and the Gateway's `async with`.

Answer

Decided by grilling.

- **Identity and storage.**
  - Thread ID is `uuid4().hex[:8]` (typable, filename-safe, URL-safe).
  - The registry is a plain `runs` table in `runs.sqlite` on its own connection, not a `("runs",)` Store namespace (this changes HLD §8): `thread_id` PK, `summary` (first 80 chars of the request), `status`, `error`, `created_at`, `updated_at`. A transition is one atomic `UPDATE`, so no read-then-write.
  - `AsyncSqliteStore` is not opened until the preferences slice (pass 6); the Kernel §10 Store items move there.
  - On `done`, the Controller writes `runs/<thread_id>/brief.json` and `brief.md` (idempotent) and puts the paths on the `Done` event.
- **API.** `async with RunController.open(settings) as rc`:
  - `start(request_text, focus=[]) -> RunHandle`
  - `resume(thread_id, answer=None) -> RunHandle`
  - `get_pause(thread_id) -> Pause | None`
  - `get_run(thread_id) -> RunView` (record plus `State.model_validate(values)`)

    - `list_runs(resumable_only=False) -> list[RunRecord]`, reconciled on read
    - Errors, all raised before any state change: `RunLocked`, `NotResumable(status)`, `InvalidAnswer(errors)`, `BalanceTooLow`.
- **Task and events.**
    - The Controller owns the graph task, so closing a subscription never cancels the Run.
    - `RunHandle`: `thread_id`, `events()` (replays the buffer, then live), `wait()` (returns the terminal event), `cancel()`.
    - History is an in-memory buffer per active Run in this process, at most 5,000 events, oldest dropped, terminal events kept. Nothing is persisted: a reloaded client rebuilds from `get_run` (`run_log`, pending Pause, status).
    - Pass 1 events: `NodeDone(node)`, `Paused(pause)`, `Done(status, paths)` with status `done | aborted`, `Stopped(node, reason)`. Tokens, tool calls and reasoning are decided in Clients flow.
    - `cancel()` cancels the task, writes `failed` with error `cancelled`, and releases the lock.
- **Status and registry timing.**
    - `running` is written after the lock and `check_balance`, just before the task starts. The final status is written after the task ends, from `aget_state`: pending interrupt → `paused`; `next` empty with a `brief` → `done`; `next` empty without → `aborted`; exception → `failed`. The registry lags the checkpoint and never leads it.
    - Reconcile on read: a `running` row with a free lock is stale, so the real status is derived from `aget_state` and written back.
    - Resumable: `paused` and `failed`. A `failed` Run resumes with `astream(None, ...)`, rerunning only the node that stopped.
- **Resume order.** Take the lock, read the pending Pause, validate the answer against that kind's answer model, `check_balance`, write `running`, spawn the task. The lock is held until the task ends. The `get_*` and `list_runs` calls are read-only and never lock.
- **Failures.** `check_balance` runs at the top of `start` and `resume` and raises `BalanceTooLow` before anything changes. Inside the task, any `Exception` (`LLMExhausted`, 401, `StateWriteError`, bugs) writes `failed` with a short `error` (class and message, about 200 chars), logs the traceback, and emits `Stopped(node, reason)` with `node` from `aget_state().next`. Nothing goes in `run_log`. There is no balance check between nodes.
- **Pause typing.** `Pause` is a discriminated union by `kind` (`ClarifyPause | PlanReviewPause | LowDataPause`), each with a concrete payload model; the Controller validates answers per kind and resumes with the Pydantic answer model. Payload and answer models live in the owning stage's `types.py`, and the skeleton uses stubs in Runtime's `testing.py`. `allowed_msgpack_modules` lists `Pause` and the answer classes.
- **Wiring.**
    - Each stage exports `make_nodes(deps) -> dict[str, Node]` and `ROUTERS: dict[str, Router]` keyed by the node they follow.
    - The whole HLD §4 graph is compiled from one edge table in `runtime/graph.py` from day one, with `nodes = {**stub_nodes, **real_nodes}`, so a missing stage runs as a stub and landing a stage needs no wiring change.

    - One owner table wraps every node and router in `owned(name, owner)`. Owners: `request_plan` for `intake`, `clarity_check`, `clarify`, `plan`, `plan_review`; `collection` for `fetch_agent`, `tools`, `collect`, `relevance`, `broaden`, `low_data_review`; `analysis` for `batch_extract`, `stats`; `brief` for `write_brief`, `critic`, `render`.
- **Lifecycle.** `RunController.open` builds Deps (`make_tiers`, `make_jev`, `async with make_gateway`), then the saver (raw `AsyncSqliteSaver` with an explicit serde, `setup()` retried with backoff) and the registry connection, and compiles the graph once. It closes everything in reverse order. The CLI opens one Controller per `asyncio.run`; Streamlit holds one on a background loop for the process lifetime. Fixed per-call config: `thread_id`, `run_name="scout"`, tags, `thread_id` in LangSmith metadata, `durability="sync"`, `stream_mode=["updates","messages","custom"]`. `recursion_limit` stays at the default (10007), so there is no cap to set.
- **Skeleton proof.** Stub nodes each write one `log` note and a minimal valid slice, and the three Pause stubs interrupt once each. A `pytest` test with no LLM or network (fake Deps, stubbed `check_balance`) starts a Run in a subprocess, kills it with `taskkill /F` at each Pause, then fetches the typed Pause with `get_pause` and resumes in a fresh process through all three Pauses to `done`. It also checks that a second process resuming a running thread gets `RunLocked`. A separate live smoke script runs the same flow with the real `check_balance`.

Clients flow

Type: grilling Status: resolved Assignee: smayankulkarni05 Blocked by: none

Question

How do the CLI and Streamlit behave on top of the Run Controller?

- How each Pause kind renders and takes an answer, including re-prompting on a bad answer.
- What `scout resume` lists, what `scout show` prints, and what a stopped Run tells the user.
- Progress display before and after the pass 3 event stream.
- The live view: the exact `RunEvent` union, how tokens, tool and API calls and reasoning display, which nodes show tokens versus a status line (structured JSON and parallel extraction may not read well), and whether the terminal UI needs a library such as `rich`.
- Streamlit's mapping: thread ID in the URL, rerun per interaction, bridging to the async Controller.

Answer

Decided by grilling.

- **Layout.** `clients/cli.py`, `clients/streamlit_app.py`, `clients/streamlit_bridge.py`, and `scout` as a `[project.scripts]` entry point. The two clients share only `clients/pauses.py`, the plain text formatting of each Pause payload.
- **Terminal.** `rich` for output from pass 1 (spinner, dim reasoning, tables, Markdown), plain `input()` for prompts.
- **Pauses in the CLI.** Rendered by `kind`.
  - `clarify`: print the questions, take a typed answer to each in turn.
  - `plan_review`: print the Search Plan, take a one-letter choice: approve / edit / reject (then ask for feedback) / abort.
  - `low_data_review`: print the counts, take a choice: broaden / continue / abort.
  - `InvalidAnswer(errors)` prints the errors and re-asks the same Pause, with no attempt cap.
  - `Ctrl-C` at a prompt leaves the Run `paused` and exits with "resume with `scout resume <id>`".
- **`edit` at `plan_review`.** The CLI writes the Search Plan as JSON to a temp file and opens `$EDITOR` (`notepad` as the Windows fallback). It validates on save against the plan model, and on an error re-opens the editor with the error at the top.
- **`Ctrl-C` during a live Run.** Calls `handle.cancel()` (`failed` / `cancelled`, lock released), prints "Stopped at <node>. Resume with `scout resume <id>`" and exits 130. There is no detach, because the Controller dies with the CLI process.
- **`scout resume`.**

    - No ID: a table of `paused` and `failed` Runs, newest first, with `#`, thread ID, status, summary, relative updated time, and a hint (pending Pause kind or stopped node). Prompt for a number or ID, first row on Enter. "No resumable Runs" if empty.
    - With an ID: `paused` re-shows its Pause; `failed` prints "Stopped at <node>: <error>. Retrying." and streams; `done` or `aborted` prints "Run already finished, see `scout show <id>`"; a locked Run prints "This Run is open elsewhere".
- **`scout show <thread_id>`.** Read-only, never locks. A header (ID, status, summary, updated time), then by status: `done` prints the Market Brief Markdown and the `brief.md` path; `paused` prints the Pause read-only plus the resume hint; `failed` prints the stopped node, error and resume hint; `aborted` prints the abort reason from `run_log`. `--log` also prints `run_log` as a table.
- **Progress before pass 3.** One line per `NodeDone`, which also carries `notes: list[str]`: the messages of the `run_log` entries that node appended, found by diffing the log length in the `updates` chunk. Raw node names, no friendly labels. Example: `✓ relevance — 38 of 142 Postings relevant (route: enough)`.
- **Live view (pass 3).**
    - `fetch_agent` streams prose tokens, plus dim reasoning when it exists.
    - Structured nodes (`clarity_check`, `plan`, `broaden`, `write_brief`) show a spinner "drafting <node>…" plus dim reasoning. JSON fragments are never shown, and there is no partial-field parsing.
    - `batch_extract` shows only a count, "extracted 37/142".
    - Each tool and API call gets one line when called and one when done.
    - Jev and plain-code nodes get nothing beyond the `NodeDone` line.
- **`RunEvent`.** A discriminated union of Pydantic models in `runtime/events.py`:
    - `NodeStarted(node)` from `tasks` mode, and only once for `batch_extract`.
    - `NodeDone(node, notes)` from `updates`.
    - `Token(node, text)` from `messages`, `fetch_agent` only.
    - `Reasoning(node, text)` from `messages` via `content_blocks`, dropped for parallel branches.
    - `ToolCalled(name, summary)`, `ToolReturned(name, summary)`, `ApiCalled(source, cached, count)`, `ExtractProgress(done, total)` from custom events.
    - `Paused`, `Done`, `Stopped` from the Controller, as decided in Run Controller API and wiring.
    - No event carries a branch ID.
- **`stream_mode` gains `"tasks"`.** This adds to the modes fixed in Run Controller API and wiring. `tasks` was not part of the streaming research, so the pass 3 build must check that interrupts still behave with it.
- **Custom event emitters.** The guarded `emit()` lives in `kernel/`. The `tools` executor emits `ToolCalled` and `ToolReturned` with its own summaries, the Gateway emits `ApiCalled` per source once the source call finishes, and `batch_extract` emits nothing: the Controller counts its task starts and `updates` chunks, as settled in Write docs/lld/runtime.md, which replaced a "one done" signal. Stages emit plain dicts with a `type` key, and the Controller maps them to `RunEvent` and drops unknown types with a log line, so Kernel and stages never import Runtime.
- **Replay buffer.** `Token` and `Reasoning` go only to live subscribers, never into the 5,000-event buffer. A client attaching mid-Run gets every node, tool and API line, and tokens from then on.

- **Streamlit stays thin and disposable.** A simple web UI is future scope and will replace it, so Streamlit gets no Controller changes and no event types of its own. It renders a subset of what the CLI does.
    - **Bridge.** `st.cache_resource` holds a background thread running an event loop, with one `RunController` open on it for the process lifetime. A sync wrapper uses `asyncio.run_coroutine_threadsafe`. A subscriber coroutine feeds a `queue.Queue` that the script reads.
    - **Pages.** The thread ID lives in `st.query_params["thread"]`. With no ID: a request box, a focus multiselect, and a table of Runs linking to `?thread=<id>`. With an ID: render from `get_run`, so `paused` shows its form, `failed` shows the error and a Resume button, `done` shows the Brief with JSON and Markdown downloads.
    - **Live view.** Node, tool and API lines in `st.status`, and `st.progress` for extraction. Tokens and reasoning are ignored.
    - **Pauses.** One `st.form` per kind, with a JSON `text_area` for `edit`. `InvalidAnswer` shows `st.error` above the same form.
    - **Cut on purpose.** No `attach()`, no Cancel button, no token placeholder. Reloading a running Run shows "Run in progress" with a Refresh button until it reaches a Pause or finishes. Closing the tab doesn't stop the Run, since the Controller owns the task.

Write docs/lld/runtime.md

Type: task Status: resolved Assignee: smayankulkarni05 Blocked by: 01, 02, 03, 05

Question

Is `docs/lld/runtime.md` written, with per-pass build checklists, future-scope entries, and the Kernel's ⏳ Runtime items closed? The two `kernel/llm.py` fixes from Research: live streaming sources are already patched, and Kernel LLD §12 is updated. The guarded `emit()` and a check that interrupts still behave with the `tasks` stream mode go on the pass 3 checklist. The simple web UI that replaces Streamlit gets a future-scope entry.

Answer

Yes. docs/lld/runtime.md is written, with checklists for passes 1, 2, 3 and 6, and a Runtime and Clients section in `future-scope.md`. The `kernel/llm.py` fixes and Kernel §12 were already in (commit `f600628`), and `tests/test_llm.py` passes. The Kernel's ⏳ Runtime items are closed, and HLD §3, §8 and §12 are updated.

Offline run on the pinned versions: `tasks` mode with a pause, a 3-branch `Send` with one failure, and a rerun, on `AsyncSqliteSaver` with `durability="sync"`. Interrupts and resume behave. All task starts arrive before the first `updates` chunk. On a rerun, finished branches arrive as `updates` marked `cached` with no task finish event.

Decisions made while writing, beyond the tickets:

- **Extraction progress needs no custom signal.** `total` counts task starts and `done` counts `updates` chunks, per that run. This replaces `batch_extract`'s "one done" signal from Clients flow.
- **Stub payload and answer models sit in the stage `types.py`,** not `runtime/testing.py`, since checkpoints store class paths and `runtime/pause.py` imports them. Stub nodes stay in `testing.py`.
- **CLI prompts run outside `asyncio.run`,** one Controller per call, because `asyncio.run`'s SIGINT handler won't end a blocked `input()`.
- **`check_balance` is skipped when `replay_cache` is set,** so replay works without OpenRouter.
- **Small additions:** `UnknownRun`, `RunView.stopped_at`, `cancel()` returning `Stopped`, `ApiCalled.failed`, `aborted` for a Run killed before its first checkpoint, and reconcile also writing the Brief files.
- **Enum inside a Pause payload** was untested in research. The skeleton proof settles it through `ClarifyingQuestion`.

Research: live streaming sources

Type: research Status: resolved Findings: local branch `research/live-streaming-sources` (commit `9d2b7fd`), file `research/live-streaming-sources.md` Assignee: smayankulkarni05 Blocked by: none

Question

What can actually be streamed to a client for a chat-assistant-style view of a Run (tokens, tool and API calls, reasoning), given the Kernel as built?

- Does `astream(stream_mode=["updates", "messages", "custom"])` deliver tokens from `Tiers.structured` and `Tiers.chat`, which call models inside their own retry and fallback loop? Does it need code changes in `kernel/llm.py`?
- Does `ChatOpenRouter` expose reasoning deltas, for which models, and in what shape?
- What do strict `json_schema` calls stream: raw JSON fragments only? What happens to already-streamed tokens on a validation retry or a fallback?
- How are tokens from parallel `Send` branches told apart?
- How can a node or the Gateway emit custom events (`get_stream_writer`), and does that work from an async node on Python 3.12 and through `owned()`?
- Do interrupts and `durability="sync"` still behave with multiple stream modes?

Answer

The chat-style view is practical. Basis: package source, offline runs of the real `Tiers`, `owned()` and `AsyncSqliteSaver`, and about 8 live minimax calls. Provider failures were only simulated, and `deepseek/deepseek-v4.1-flash`, first in both chains, was not tested.

- **Tokens.** They arrive through `Tiers` with no wiring changes, but `messages` mode moves every LLM call onto `_astream`, so two fixes in `_CostedOpenRouter` must land first (code in the findings):
  - An `_astream` override raising `_ProviderError` on `finish_reason == "error"` (today `Tiers.chat` returns truncated text as success), on `ValueError` from the stream, and on an empty stream (both crash today with no retry or fallback).
  - `total_cost` set after the merge in an `_agenerate_with_cache` override, a private method to re-check on upgrades. Streaming otherwise loses the cost.
  - Provider-failure tests must also run under `astream`. The 60 s timeout becomes per-read (from source, untested).
- **Reasoning.** Available on minimax as `additional_kwargs.reasoning_content`, but not on every turn: a tool turn at low effort had none. Treat it as optional.
- **Structured calls.** They stream raw JSON fragments, and `parse_partial_json` handled every sampled step. Streamed tokens are never retracted. Each attempt gets a new message id, and a

validation retry looks like a finished message, so structured output stays provisional until the node's `updates` chunk.

- **Parallel branches.** Told apart by `metadata["langgraph_checkpoint_ns"]`, then message id. `custom` and `updates` chunks carry no branch identity, so the writer must stamp it.
- **Custom events.** `get_stream_writer()` works from async nodes on 3.12, through `owned()`, and in `Send` branches. It raises outside a graph, so the Gateway and `Tiers` need a small guarded `emit()`.
- **Interrupts.** They work with all three modes under `durability="sync"`, arriving as an `updates` chunk. Resume re-runs the whole node, which repeats tokens and cost for any LLM call before `interrupt()`, so the pure Pause node rule matters more.

Limits to design around:

- Replay-cache hits give one whole message, not a typewriter effect, and identical hits may dedupe (tested with `InMemoryCache` only).
- Most calls are structured, so they show partial fields or a drafting state, not prose.
- Tokens aren't persisted, so a reloaded client has no history unless the Run Controller stores events.
- Closing the stream cancels the Run, so the graph needs its own task, apart from any client connection.
- Extraction fan-out of 50 to 200 streams should show as a progress count.

Opening the Store

Type: grilling Status: resolved Assignee: smayankulkarni05 Blocked by: 02

Question

HLD §3 gives Runtime the SQLite Store, but Run Controller API and wiring only says it isn't opened until pass 6. How does `RunController.open` open it, and how does a node reach it, given that nodes close over `Deps` and go through `owned()`? What a preference contains stays with the Long-term memory LLD.

Answer

Decided by the agent, on the user's instruction, after an offline probe on the pinned versions.

- **Opening.** From pass 6, `RunController.open` opens a raw `AsyncSqliteStore` on `runs_db`, on its own `aiosqlite` connection, between the registry and `build_graph`, and closes it in reverse order. Its `setup()` shares the saver's retry.
- **The connection must be autocommit** (`isolation_level=None`), the way `from_conn_string` opens it, with a 5 s busy timeout. On a default connection `setup()` leaves a transaction open, so in the probe the saver's next checkpoint write failed with "database is locked", and the Store's own next write fails with "cannot start a transaction within a transaction".
- **Reaching a node.** `build_graph(deps, saver, store=None)` passes it to `builder.compile(checkpointer=saver, store=store)`. A node that needs it declares `runtime: Runtime` and uses `runtime.store`. It does not go in `Deps`: this is LangGraph's own route, and nodes that don't use it don't change.
- **`owned()` needs no change.** Its `functools.wraps` keeps the signature LangGraph reads, so `runtime` is still injected.
- **Values** stay as already decided: written with `model_dump(mode="json")`, read with `model_validate`. They are plain JSON, so nothing joins `ALLOWED_MSGPACK`.
- **Nothing is opened before pass 6.** No stage reads it yet, so `store` stays `None` until then.

Probe results:

Check	Result
A node behind `owned()` reads and writes through `runtime.store`	Works
A preference written on one thread is read by another, after both connections reopen	Works

Check	Result
A Pause and resume with the saver and Store on one file, on separate connections	Works
Default connection instead of autocommit	Fails, as above
4 processes racing `setup()` on a fresh file, then 100 writes each	No failures in the one run. The retry stays, since the saver's race did fail

Bring the LLD in line with the code

Type: task Status: resolved Assignee: smayankulkarni05 Blocked by: 06

Question

The Runtime build and the later stages have landed since `docs/lld/runtime.md` was written. Where does the doc no longer describe the code, and is anything else in HLD §3's Runtime row undecided?

Answer

Compared the LLD with `runtime/graph.py`, `deps.py`, `guard.py`, `registry.py`, `pause.py`, `events.py` and `controller.py`. The code was right each time, so only the doc changed:

Section	The doc said	The code does
§3	`Deps` has four fields	A fifth, `stub_stages`, lets tests run a landed stage as its stub
§4	`relevance` has three targets	A fourth, `stats`, because `fan_out` sends an empty Sample there
§6	`cancel()` always writes `failed`	A Run with no checkpoint yet has nothing to rerun, so it writes `aborted`

Also added from Opening the Store: §1, §4, §11, §16 and §17.

Nothing else in the row is undecided: wiring, checkpointer, registry, Run Controller, event stream, Pause protocol and guard are all specified and built.

The Runtime tests gave 66 passed on three runs in a row. An earlier run had 2 failures in `test_skeleton_proof.py`, while another session was part-way through its Brief edits. The 2 passed on an immediate rerun, so they weren't investigated further. No code changed here.

runtime-build

runtime-build/issues/01-pause-protocol-guard-registry-stub-graph.md

01: Pause protocol, guard, registry and the stub graph

What to build: the foundations of Runtime. The full HLD §4 graph compiles from one wiring table with stubs for every missing stage, and a stub Run can be driven in-process to `render` through all three Pauses on an `AsyncSqliteSaver`. Pauses are typed and survive a checkpoint round trip, a per-thread lock refuses a second process, and the registry derives a Run's status from a checkpoint snapshot. Spec: Runtime LLD §2-5, §7, §8, §15 (stubs), §16 pass 1 steps 1-5.

Blocked by: None (can start immediately).

Status: done

- [x] `LANGGRAPH_STRICT_MSGPACK` is set in the package `__init__.py`, `rich` is added, `aiosqlite` is pinned, `runs_db` and `runs_dir` settings exist with the LLD defaults, and `runs/` is in `.gitignore`
- [x] Stage `types.py` stubs for the three payload and answer pairs sit at their final paths, and `Pause` is a discriminated union by `kind` with the `ANSWERS` table
- [x] Test: each Pause and answer survives a dump and load through the allowlisted serializer, and `model_validate` rejects a bad answer
- [x] Per-thread lock: a stdlib SQLite connection holding `BEGIN IMMEDIATE`, busy mapped to `RunLocked` by `sqlite_errorname`, held for the whole Run
- [x] Test: a second subprocess is refused, and the lock frees after a kill
- [x] `runs` table with atomic single-`UPDATE` transitions, and `derive_status` as a pure function
- [x] Test: `derive_status` for all five outcomes on a fake snapshot, and a transition updates `updated_at`
- [x] `Deps`, `stub_nodes`, `STUB_ROUTERS` and `fake_deps` exist, and `build_graph(deps, saver)` wraps every node and router in `owned()` and compiles once from the edge, conditional and owner tables
- [x] Test: the full graph compiles, and a stub Run reaches `render` with the expected three Pauses

Notes for ticket 02: `RunLocked` lives in `guard.py`, and `RunStatus` and `RunRecord` in `registry.py`, because the Controller imports both modules. `Registry.get` exists; `list_runs` and the conditional reconcile `UPDATE` are still to add. The `PlanReviewAnswer` cross-field validator is left to Request & Plan step 2.

02: Run Controller and the skeleton proof

What to build: a caller can `start` a Run, get its typed Pause, `resume` it with a validated answer and reach `done`, with the Run surviving a hard kill and resuming from a fresh process. A failing node leaves a resumable `failed` Run. Spec: Runtime LLD §6, §7, §9-11, §15, §16 pass 1 steps 6-7.

Pass 1 events only: `NodeDone`, `Paused`, `Done`, `Stopped`. Runtime is not done until the skeleton proof passes.

Blocked by: 01: Pause protocol, guard, registry and the stub graph.

Status: done

- [x] `RunController.open` builds the Deps, the saver (raw `AsyncSqliteSaver` with the allowlisted serde, `setup()` retried with backoff), the registry connection and the compiled graph
- [x] `start`, `resume`, `get_pause`, `get_run` and `list_runs` match the §6 API, with `check_balance` skipped when `replay_cache` is set
- [x] `RunHandle` replays the buffer then goes live, `wait()` returns the terminal event, and `cancel()` returns `Stopped`
- [x] Test: the happy path through the stub Pauses, and each refusal (`UnknownRun`, `RunLocked`, `NotResumable`, `InvalidAnswer`, `BalanceTooLow`) changes nothing
- [x] Test: a raising node gives `failed` and `Stopped`, and resume reruns only that node
- [x] Test: `cancel()` gives a resumable `failed` Run
- [x] Test: a stale `running` row reconciles on read, and a live task's row stays `running`
- [x] Test: closing a subscriber doesn't stop the Run
- [x] Test: `done` writes `brief.json` and `brief.md` with the paths on `Done`, and reconcile writes them too
- [x] Skeleton proof: a subprocess Run killed with `taskkill /F` at each Pause is resumed in a fresh process through all three Pauses to `done`, and a second process resuming a running thread gets `RunLocked`
- [x] The proof settles the `StrEnum`-in-payload question: it passed, an enum in a payload comes back typed in a fresh process, so the field stays an enum

03: CLI and `--focus`

What to build: a developer drives a whole Run from the terminal: `scout run "<request>" [--focus dim ...]`, answering each Pause, then `scout resume` and `scout show` to pick it up or read it later. `Ctrl-C` leaves a resumable Run. `--focus` values land in the checkpoint. Spec: Runtime LLD §12, §16 pass 1 steps 8-9 and pass 2 step 1.

Blocked by: 02: Run Controller and the skeleton proof.

Status: in-progress, waiting on the hand Ctrl-C check

- [x] `scout` entry point with `run`, `resume [thread_id]` and `show <thread_id> [--log]`, prompts run outside the event loop, and exit codes as in §12
- [x] A shared plain-text renderer for each Pause payload, used by the CLI and later Streamlit
- [x] `edit` opens `$EDITOR` (`notepad` fallback) on the Search Plan as JSON, and re-opens with the error on top if validation fails
- [x] `resume` with no ID lists resumable Runs and prompts, and with an ID handles each status as in §12
- [x] `show` is read-only and prints per status, and `--log` adds the `run_log` table
- [x] Test: scripted `input` drives `scout run` through the stub Pauses, a bad answer re-asks the same Pause, and `resume` and `show` output matches each status
- [x] Test: `--focus` is rejected for unknown values, and the chosen values are in `State.focus` in the checkpoint
- [ ] `Ctrl-C` checked by hand on Windows, at a prompt and during a live Run
- [x] One live smoke script (not in CI) runs the same flow with the real `check_balance`

Notes: menu letters are `a`/`e`/`r`/`x` for plan review and `b`/`c`/`x` for low data review. `main` sets stdout to replace unencodable characters, because the live smoke crashed on ✓ with piped output on Windows. `Ctrl-C` is the one open item: it needs a person at a Windows terminal, at a prompt and during a live Run (`python scripts/smoke_cli.py` gives a Run to interrupt).

04: Streamlit client

What to build: a disposable web page over the Run Controller: start a Run with a focus selection, answer each Pause in a form, reload at a Pause and carry on, see failed Runs and resume them, and download the Brief. No Controller changes. Spec: Runtime LLD §13, §16 pass 2 step 2.

Blocked by: 02: Run Controller and the skeleton proof, 03: CLI and `--focus` (for the shared Pause renderer and `--focus`).

Status: done

Run it with `uv run --group streamlit streamlit run src/talent_scout/clients/streamlit_app.py`. Not exercised by hand: the failed-Run view and "Run in progress" on reload, since a stub Run is too fast to catch either.

- [x] A `streamlit` dependency group, a bridge holding one Controller on a background loop for the process lifetime, and the thread ID in `st.query_params`
- [x] No ID shows the request box, the focus multiselect and a table of Runs linking to each; an ID renders from `get_run` by status
- [x] One form per Pause kind, a JSON `text_area` for `edit`, and `InvalidAnswer` shown above the same form
- [x] Node lines in `st.status` while a Run is live, and "Run in progress" with a Refresh button on reload
- [x] `done` shows the Brief with JSON and Markdown downloads
- [x] Checked by hand through a stub Run: start, each Pause form, a bad answer, reload at a Pause, and download. No automated tests.

05: Event stream

What to build: the Controller turns LangGraph's stream into the full `RunEvent` set, so a client can show LLM tokens, reasoning, tool and API calls and extraction progress as they happen, and never sees a LangGraph chunk. Spec: Runtime LLD §10, §16 pass 3 steps 1-3.

The `tools` executor's own `tool_called` and `tool_returned` emits are Collection's, and the Controller drops unknown types with a log line until they land.

Blocked by: 02: Run Controller and the skeleton proof.

Status: done

- [x] `emit()` is a no-op outside a graph and writes to the stream inside one
- [x] Test: `emit()` is delivered inside a graph and through `owned()`
- [x] The Gateway's `fetch` emits one `api_called` per source after the `gather`, with `cached` meaning a cache status other than `miss`
- [x] Test: the real Gateway on a mock transport, inside a one-node graph, emits one `api_called` per source, including a failed one
- [x] The full chunk mapping in `runtime/events.py`: `NodeStarted`, `Token`, `Reasoning`, `ToolCalled`, `ToolReturned`, `ApiCalled` and `ExtractProgress` per the §10 table
- [x] Test on a scripted graph: tokens only from `fetch_agent`, reasoning optional, and `Token` and `Reasoning` never in the replay buffer
- [x] Test: a `Send` run with one failing branch counts `ExtractProgress` correctly across the rerun, and `NodeDone` for `batch_extract` fires once

Notes: The Gateway's `fetch` is per source now, so it emits one `api_called` per call rather than per source after a `gather`; a raised error emits `failed=True` with `count=0`, then re-raises. `Token` and `Reasoning` are in `LIVE_ONLY` and skip the buffer. `ExtractProgress` is emitted on each `batch_extract` update, and the mapper is per stream, so the rerun counts 1 to 3 again. Not done: the `tools` executor's own emits, which are Collection's.

06: Live view in the clients

What to build: the CLI and Streamlit show the chat-assistant-style live view: prose tokens and dim reasoning from `fetch_agent`, a spinner for structured nodes, an extraction count, and a line per tool and API call. Spec: Runtime LLD §12 (live view), §13, §16 pass 3 step 4.

Blocked by: 03: CLI and `--focus`, 04: Streamlit client, 05: Event stream.

Status: done

- [x] The CLI follows the §12 live-view table, and the spinner stops on the first `Token` or `Reasoning` and restarts on the next `NodeStarted`
- [x] Streamlit adds tool and API lines to `st.status` and `st.progress` for extraction, and ignores tokens and reasoning
- [x] Test: the CLI's rendering of a scripted event sequence, including a turn with no reasoning
- [x] One live run (not in CI) to check how the view reads
- [x] Check whether `deepseek/deepseek-v4.1-flash` sends readable reasoning, and record the answer in LLD §10

07: Open the Store and reach it from a node

What to build: the Run Controller opens the Store on the runs database, and a node behind `owned()` can read and write it through `runtime.store`, so a value written during one Run is there for the next. The core Run path doesn't change. What the Store holds and which nodes read it belong to the Long-term memory LLD. See Opening the Store and `docs/lld/runtime.md` §4, §11 and §16.

Blocked by: None (can start immediately). It changes the Run Controller and the graph wiring, so run it after any session editing those.

Status: done

- [x] A value written through `runtime.store` on one thread is read on another after the Controller reopens
- [x] A Pause still resumes correctly with the Store open
- [x] The Store's connection is autocommit, and a test fails if it isn't
- [x] The Store's `setup()` uses the saver's retry
- [x] Values are stored as plain JSON, and nothing joins the msgpack allowlist

request-plan-lld

request-plan-lld/map.md

Request & Plan LLD

Label: wayfinder:map Parent: Talent Market Scout: HLD (component C in docs/HLD.md §3, §4)

Destination

`docs/lld/request-plan.md`: an LLD of Request & Plan precise enough to build straight from, covering pass 1 (`intake`, `clarity_check`, `clarify`, `plan`, a pass-through `plan_review`) and pass 2 (the real `plan_review`, `focus`), with a stated seam for pass 6 preferences and nothing left to decide. Building the code is a separate effort after this map.

Notes

- Domain: Request & Plan turns `request_text` into a final Hiring Request and an approved Search Plan, pausing for the user only when it has to. Vocabulary: `GLOSSARY.md`. Spec: `Joveo_Talent_Market_Agent- Assessment.pdf`. Deadline Mon 2 Nov 2026, 9:00 AM, so this map stays lean.
- Fixed upstream: docs/lld/kernel.md §3 (LLM-schema policy), §7 (routing), §11 (Jev), §12 (Tiers), docs/lld/runtime.md §3 (stage contract), §5 (Pause protocol), docs/lld/gateway.md §3 (`Fetch`, `ADZUNA_CURRENCY`), HLD §4, §6.
- Skills: `langchain-router` first for any framework question (per `CLAUDE.md`); `grilling` + `domain-modeling` for grilling tickets.
- Standing preferences: inherited from the Kernel map (Pydantic v2 everywhere, `uv`, Python 3.12, `src/talent_scout/`, `pytest`, root `.env`).
- Tiers are fixed by the HLD: cheap for parsing and question drafting, strong for planning.
- All tickets were resolved in one grilling session, in two rounds, on 2026-10-31.

Decisions so far

<!-- one line per closed ticket -->

- Request parsing and the clarify loop: `intake` parses into a nullable `RequestDraft` and `clarify` returns to it, `clarity_check` alone writes the final `hiring_request`, routers need no decision field, and `HiringRequest` gains an inferred `country`, so unsupported countries are caught up front.

- Clarity check and the re-ask cap: code flags missing fields, Jev judges role, location and any given seniority with "yes" meaning ambiguous, only flagged fields are asked, and at the cap a field settles with an Assumption or the Run aborts.
- Search Plan and its limits: the strong tier fills `SearchPlan` directly, with limits on the Kernel type so the LLM, an edit and `broaden` share one set of checks.
- Pause payloads and answers: `clarify` answers match by position with blanks counting as vague; `plan_review` shows the plan and Assumptions, and validators require `plan` on edit and `feedback` on reject.
- What focus changes: nothing in the search; Brief uses it for emphasis, and the CLI accepts `salary` as an alias.
- Prompts: the LLD fixes each prompt's inputs and rules, and the exact wording lives in `request_plan/prompts.py`, tuned during the build.
- Write docs/lld/request-plan.md: the LLD is written and the HLD, Kernel, Runtime, Gateway and future-scope docs agree with it.

Not yet specified

Out of scope

- `broaden`, `low_data_review` and their fields: Collection LLD.
- Insight emphasis from `focus`, and rendering Assumptions in the Run Notes: Brief LLD.
- What preferences contain and how they fill draft fields: Long-term memory LLD.
- Writing the Request & Plan code: separate build effort after this map.

Request parsing and the clarify loop

Type: grilling Status: resolved Assignee: smayankulkarni05 Blocked by: none

Question

How does free text become a Hiring Request when the text may be incomplete, and where do Clarifying Question answers get applied?

- What `intake` parses into, given `HiringRequest` requires `role` and `location`.
- Which node applies answers. Kernel §4 says `intake` refines `hiring_request` after answers, while HLD §4 and Runtime §4 wire `clarify → clarity_check`.
- Which node writes `hiring_request`, and how the router after `clarity_check` decides.
- Where an unsupported country is caught.

Answer

Decided by grilling.

- **`RequestDraft`.** `intake` parses into a private model in `request_plan/types.py` where every field can be null. Parsing "Data analyst" straight into `HiringRequest` would fail validation on every retry until `LLMExhausted` stopped the Run, so an incomplete request would look like a backend failure.
- **`clarify` returns to `intake`.** `intake` re-parses `request_text` plus every answered question and writes a new draft. `clarity_check` then only judges and drafts questions, and doesn't also have to parse. This changes one edge in Runtime §4 and the HLD §4 diagram, and resolves their disagreement with Kernel §4.
- **`hiring_request` means final.** Only `clarity_check` writes it, once nothing is left to ask. Everything before that reads `request_draft`. This changes the owner table in Kernel §4.
- **No decision field.** The router reads state directly: any unanswered question → `clarify`, `hiring_request` set → `plan`, neither → END. Runtime §4 gains the END target for `clarity_check`.
- **`country`.** `RequestDraft` and `HiringRequest` gain a `country` the LLM infers ("Texas" → `us`, "Bangalore" → `in`). A null or unsupported country counts as an ambiguous location. Without this, a bad country fails only when `Fetch` validates it, after the user has approved a plan.
- **`ADZUNA_CURRENCY` moves to `kernel/types.py`,** with `gateway/types.py` importing it, since stage `types.py` modules may import only `kernel.types`.
- A location of just "Remote" gets asked which country. "India" is a valid location with `country = in`.

Clarity check and the re-ask cap

Type: grilling Status: resolved Assignee: smayankulkarni05 Blocked by: 01

Question

What does `clarity_check` judge, how are questions chosen, and what happens when a field is still unclear after the one re-ask the HLD allows?

- Which Jev questions it sends, and which side is "yes".
- How the spec's 1–3 question limit is met, and how a clear request skips the Pause.
- What happens at the cap for a vague value, a missing location, an unsupported country and an unreadable role.

Answer

Decided by grilling.

- **Code flags what is plainly missing,** with no Jev call: `role` null, `location` null, `country` null, or `country` outside `ADZUNA_CURRENCY`.
- **Jev judges the rest** in one `jev.judge("clarity", ...)` request, one Noul per field that exists and isn't already flagged: `role`, `location`, and `seniority` only if the user gave one. "Yes" means too ambiguous to search on. The state sent is the raw request, the draft and the Q&A so far.
- **Only flagged fields are asked,** so a round has at most 3 questions and a clear request gets none. A missing `seniority` isn't ambiguous, because a search can cover all levels. That lets "Data Analyst, Bangalore" go straight through, as the spec's first sample run requires.
- **Drafting.** The cheap tier fills a private `QuestionDraft(text, options)`, and code sets `about` and `answer=None`. A question about an unsupported country lists the supported ones, from code.
- **Counting.** Earlier questions with the same `about` are counted: fewer than 2 means ask, 2 means settle.
- **Settling at the cap:**

Field state after one re-ask	Result
Readable but still vague ("Engineer" twice)	Keep the draft's value, and code writes an Assumption. No LLM call.
`location` or `country` null	Country-wide in a new `default_country` setting (`us`), with an Assumption
`country` named but unsupported ("Tokyo")	Abort: no source has data there

Field state after one re-ask	Result
`role` null ("asdf")	Abort: nothing to search for

- **An abort** writes no `hiring_request`, logs a `route` entry with the reason, and goes to END. Runtime's `derive_status` reports `aborted`.

Decided while writing the LLD:

- **Assumptions are written only when `clarity_check` finishes,** so a field settled in one pass and cleared in a later one leaves no stale Assumption.
- **`location` falls back to the country code** when defaulted, since `HiringRequest.location` must be non-blank.

Search Plan and its limits

Type: grilling Status: resolved Assignee: smayankulkarni05 Blocked by: 01

Question

How does `plan` produce a Search Plan that the Gateway can run within its quota?

- Which fields the LLM fills and which limits apply.
- Where those limits live, given the LLM's output, a user's edit and Collection's `broaden` all produce plans.
- What happens to seniority, work mode and employment type.

Answer

Decided by grilling, in two steps: the limits first, then where they live.

- **The strong tier fills `SearchPlan` directly.** The model fills every field, so Kernel §3's schema policy allows it, with no separate draft schema.
- **Limits go on the Kernel's `SearchPlan`,** so the LLM's output, an edit at `plan_review` and `broaden` all pass the same checks. A violation from the LLM is fed back on the retry. A violation in an edit is `InvalidAnswer`, which re-prompts. This replaced an earlier choice to put them on a stage-private draft schema.

Field	Limit	Why
`titles`	1–4	Each title is its own Fetch, and Adzuna allows 25 calls a day
`country`	in `ADZUNA_CURRENCY`	`Fetch` rejects anything else
`radius_km`	0–100	A wider radius stops being one market
`max_days_old`	1–60	USAJOBS's `DatePosted` caps at 60
`target_postings`	20–200	Below 20, a Stat means little. Above 200, Sufficiency can force a broaden on a healthy market.
`category`	in `ADZUNA_CATEGORIES`, or null	A tag the source doesn't know would silently find nothing

- **`ADZUNA_CATEGORIES`** sits in `kernel/types.py` beside the currency table. The tags come from `DOMAINS` in `gateway/usajobs.py`, and a test keeps the two equal. Adzuna's list is per country, so the check is only that a tag is known.

- **Seniority** goes into `titles` only if the user stated it. **Work mode and employment type** can't be searched on, so they stay on `hiring_request` and reach the Run Notes.
- **To verify in the build:** both chain models must accept `maximum` and `maxItems` under strict `json_schema`. If one rejects them, the limits move into a `model_validator`, which leaves them out of the schema.

Pause payloads and answers

Type: grilling Status: resolved Assignee: smayankulkarni05 Blocked by: 01, 02, 03

Question

What are the final payload and answer fields for `clarify` and `plan_review`, replacing the stub shapes in Runtime §5?

- How `clarify` matches answers to questions, given the Controller validates without seeing the payload.
- What `plan_review` shows and which answers are invalid.
- Which state fields `plan_review` writes, and how pass 1 behaves before the real Pause exists.

Answer

Decided by grilling.

- **`ClarifyPayload`:** only the unanswered questions.
- **`ClarifyAnswer(answers: list[str])`:** matched by position. The Controller validates without the payload, so it can't check the count. The node pads missing answers with blanks and drops extras. A blank counts as vague, so Enter skips a question and uses up that field's asks.
- **CLI:** typing a number picks that option, and anything else is free text.
- **`PlanReviewPayload`:** the plan plus the Assumptions, because this is the last moment the user can correct one before quota is spent.
- **`PlanReviewAnswer(action, plan, feedback)`:** validators require `plan` on `edit` and `feedback` on `reject`, so a bad answer re-prompts.
- **New private fields:** `plan_decision` and `plan_feedback`. Rejects have no cap, since a human drives each one.
- **Pass 1:** `plan_review` is a pass-through returning `{}`, and a null `plan_decision` routes to `fetch_agent`.

What focus changes

Type: grilling Status: resolved Assignee: smayankulkarni05 Blocked by: none

Question

HLD §12 defers to this LLD what each `--focus` value changes. What does it change in Request & Plan, and does the CLI need to change for the spec's `--focus salary` example?

Answer

Decided by grilling.

- **`focus` changes nothing in the search.** The same Postings answer every Dimension and Stats are cheap Python, so narrowing the fetch would save nothing and would weaken the rest of the Market Brief. `plan` doesn't read it.
- **Brief uses it** to order and emphasise Insights (⏳ Brief LLD).
- **Alias.** The spec's example is `--focus salary`, but the CLI's choices are `Dimension` values, where that would be rejected. The CLI accepts `salary` as an alias for `compensation`.

Prompts

Type: grilling Status: resolved Assignee: smayankulkarni05 Blocked by: 01, 02, 03

Question

How much of the prompt text does this LLD fix, and what does each prompt get?

Answer

Decided by grilling.

- **The LLD fixes each prompt's inputs, rules and schema descriptions.** The exact wording lives in `request_plan/prompts.py` and is tuned during the build. The write-up has to say what changed after testing, so the wording should stay easy to change.
- **`intake` (cheap):** `request_text` plus the answered questions, with examples from the spec's cases.
- **Question drafting (cheap):** the draft plus the flagged fields.
- **`plan` (strong):** `hiring_request`, Assumptions, and on a reject the previous plan and feedback. Guidance suggests about 100 target Postings for a broad role, about 30 for a niche one, and `max_days_old` of 30.
- **Jev:** the raw request, the draft and the Q&A.
- **Caveats:** a Jev call with `llm_fallback` or `truncated` logs a Caveat, as Kernel §11 requires. Those are the only Caveats this stage raises.

Write docs/lld/request-plan.md

Type: task Status: resolved Assignee: smayankulkarni05 Blocked by: 01, 02, 03, 04, 05, 06

Question

Is `docs/lld/request-plan.md` written, with per-pass build checklists and future-scope entries, and with the upstream ⏳ Request & Plan items closed?

Answer

Yes. docs/lld/request-plan.md is written, with checklists for passes 1 and 2 and a Request & Plan section in `future-scope.md`. The ⏳ items in Kernel §4, Runtime §1, §5 and §17, Gateway §3 and HLD §12 are closed.

No code changed. `kernel/types.py`, `gateway/types.py` and `State` still have the old shapes until step 1 of the pass 1 checklist.

Decisions made while writing, beyond the tickets:

- **Runtime comes first in the build,** since this stage needs `runtime/deps.py`, `runtime/pause.py` and the stub `types.py`.
- **Future scope gained three items:** `focus` shaping the Search Plan, fetching Adzuna's per-country category list, and multi-country Runs.
- **Unproven until the live smoke:** the 0.5 clarity cut on real requests.

request-plan-build

request-plan-build/issues/01-kernel-contract-and-types.md

01: Kernel contract and Request & Plan types

What to build: the prefactor that makes the stage easy to add, with no behaviour change. The shared country and category tables live in the Kernel, `HiringRequest` carries a validated country, `SearchPlan` enforces its limits so the LLM, an edit and `broaden` share one set of checks, and the stage's types are filled in. Spec: Request & Plan LLD §3, §6, §11, §12 pass 1 steps 1-2 (the `runtime/graph.py` edge changes belong to tickets 02 and 03).

Blocked by: None (can start immediately).

Status: done

- [x] `ADZUNA_CURRENCY` moves to the Kernel and `ADZUNA_CATEGORIES` is added, with the Gateway re-using both
- [x] `HiringRequest.country` is required and must be in `ADZUNA_CURRENCY`, and existing fixtures and stubs are updated
- [x] `SearchPlan` enforces the LLD limits on `titles`, `country`, `radius_km`, `max_days_old`, `target_postings` and `category`
- [x] `State` gains `request_draft`, `plan_decision` and `plan_feedback`, owned by Request & Plan
- [x] `default_country` setting exists, defaults to `us` and must be in `ADZUNA_CURRENCY`
- [x] `RequestDraft` is added with every field nullable, and `PlanReviewAnswer` requires `plan` on edit and `feedback` on reject
- [x] Test: each limit rejects its bad value, and `ADZUNA_CATEGORIES` matches the tags in the USAJOBS domains table
- [x] Test: `PlanReviewAnswer` rejects `edit` with no plan and `reject` with no feedback, and all the models survive the Runtime serde test
- [x] The existing test suite still passes

Notes: `check_country` in `kernel/types.py` is the one country check, shared by `HiringRequest`, `SearchPlan` and the `default_country` setting. `RequestDraft` and `WorkMode` are added to `ALLOWED_MSGPACK` so the draft survives the serde test. The `clarify` edge and `clarity_check` targets were already in `runtime/graph.py`. `Fetch` keeps its own country check and message.

02: A clear request reaches `plan`

What to build: "Data Analyst, Bangalore" goes through the real `intake` and `clarity_check` with no question asked, and the final `hiring_request` is set for the next stage. `plan` stays a stub. Fields that need asking are flagged and routed, but drafting questions and settling at the cap belong to ticket 03. Spec: Request & Plan LLD §4, §5 (`intake`, `clarity_check` steps 1-2 and 5), §7, §9, §12 pass 1 steps 3-4 and part of 7.

Blocked by: 01: Kernel contract and Request & Plan types.

Status: done

- [x] `request_plan/prompts.py` and `request_plan/nodes.py` exist, with `make_nodes(deps)` and `ROUTERS`, and the prompts follow the rules in §7
- [x] `intake` uses the cheap tier, writes `request_draft` and one `note`, and is safe to rerun
- [x] Code flags a null `role`, `location` or `country`, or a `country` outside `ADZUNA_CURRENCY`, with no Jev call
- [x] One Jev request judges each field that exists and isn't already flagged: `role`, `location`, and `seniority` only if the user gave one. "Yes" means too ambiguous, and no call is made if no field qualifies
- [x] A missing `seniority` is not ambiguous
- [x] `clarity_check` alone writes `hiring_request`, and logs a `route` entry with each field's outcome and reason
- [x] A `Judgement` with `source="llm_fallback"` or `truncated=True` logs a Caveat
- [x] `after_clarity` reads only state, and `runtime/graph.py` has the new `clarity_check` targets
- [x] Test with fake chains: `intake` writes the draft, and answered questions reach the prompt
- [x] Test with `fake_classifier`: a clear request gets no question and sets `hiring_request`, a missing field skips Jev, and a fallback `Judgement` logs a Caveat
- [x] Test: all `after_clarity` outcomes

Notes: `build_graph` wires the real stage unless `Deps.stub_stages` is set, which `fake_deps` does, so the Runtime stub tests are unchanged. A country problem is flagged against `location`, since `country` is not a `RequestField`. `nodes.py` holds tickets 02-04 and 06 together.

03: The clarify loop, re-ask cap and settling

What to build: "Engineer, Texas" pauses with one targeted question per flagged field, the answer goes back to `intake` for a new draft, and the Run carries on. A field asked twice settles with an Assumption or aborts the Run, with the reason visible in `scout show`. `plan` stays a stub. Spec: Request & Plan LLD §4, §5 (`clarity_check` steps 3-5, settling, `clarify`), §12 pass 1 steps 4, 5 and 7.

Blocked by: 02: A clear request reaches `plan`.

Status: done

- [x] A flagged field is asked while it has fewer than 2 earlier questions with the same `about`, and settled at 2
- [x] The cheap tier drafts one private `QuestionDraft(text, options)` per field, with at most 4 options, and code sets `about` and `answer=None`
- [x] A question about an unsupported country lists the supported ones, from code
- [x] `clarify` pauses with only the unanswered questions and matches answers by position: missing answers become blank, extras are dropped, and a blank counts as vague
- [x] `clarify` returns to `intake`, and `runtime/graph.py` has the new edge
- [x] Assumptions are written only when `clarity_check` finishes, so a field settled and later cleared leaves none
- [x] An abort writes no `hiring_request`, logs a `route` entry with the reason and goes to END
- [x] Test: only flagged fields are asked, and a third ask settles instead
- [x] Test: each row of the settling table: a vague value is kept with an Assumption and no LLM call, a null `location` or `country` uses `default_country` with an Assumption, an unsupported `country` aborts, and a null `role` aborts
- [x] Test in a one-node graph with a checkpointer: padding, extras dropped, and a `Command(resume=ClarifyAnswer(...))` round trip
- [x] Graph test with other stages stubbed: "Engineer, Texas" goes through a real `clarify` loop to `fetch_agent`

Notes: the unsupported-country question takes its options from `sorted(ADZUNA_CURRENCY)`, so the user sees country codes. A field settles as a country-wide search on `default_country`, replacing a non-null `location` too, as the LLD table reads.

04: Search Plan and the pass-through `plan_review`

What to build: a final `hiring_request` becomes a `search_plan` within the limits, written by the strong tier, and the Run carries on to `fetch_agent` through a pass-through `plan_review`. Spec: Request & Plan LLD §4, §6, §7, §9, §12 pass 1 steps 6-7.

Blocked by: 02: A clear request reaches `plan`.

Status: done

- [x] `plan` uses the strong tier with `SearchPlan` directly, and writes `search_plan` and one `note` with the titles and target
- [x] Its input is `hiring_request` and the Assumptions, plus the previous plan and `plan_feedback` when `plan_decision` is `reject`, and it does not read `focus`
- [x] `PLAN_SYSTEM` follows the rules in §7
- [x] `make_nodes` exports `plan_review` as a pass-through returning `{}`, so `plan_decision` stays null
- [x] `after_review` reads only state: `abort` goes to END, `reject` to `plan`, and anything else, including null, to `fetch_agent`
- [x] Test with fake chains: a bad plan then a good one passes on the feedback retry
- [x] Test: all `after_review` outcomes
- [x] Graph test with other stages stubbed: a clear request reaches `fetch_agent` with a `search_plan` set

Notes: `plan_review` is now the real Pause from ticket 06, so the stage graph tests assert the plan reaches `plan_review` and then `fetch_agent`.

05: Pass 1 live smoke

What to build: a script (not in CI) runs the spec's cases through the real models, proving the stage behaves on real output. This is the first real check of the 0.5 clarity cut, and of both chain models accepting the `SearchPlan` limits under strict `json_schema`. Spec: Request & Plan LLD §12 pass 1 step 8.

Blocked by: 03: The clarify loop, re-ask cap and settling; 04: Search Plan and the pass-through `plan_review`.

Status: done

- [x] "Data Analyst, Bangalore" asks nothing and gives a valid `search_plan`
- [x] "Engineer, Texas" asks, and gives a valid `search_plan` once answered
- [x] Both chain models accept the new `maximum` and `maxItems` keywords under strict `json_schema`, or the limits move into a `model_validator` that leaves them out of the schema
- [x] Any change to the clarity cut or the prompts is noted for the write-up

Notes: `scripts/smoke_request_plan.py [model]` ran on both chain models, `minimax/minimax-m3.1-flash` and `deepseek/deepseek-v4.1-flash`, each pinned for both tiers. Both accepted the `maximum` and `maxItems` limits under strict `json_schema`, so no `model_validator` move was needed. `scripts/smoke_cli.py` now answers by prompt and ran `scout run "Engineer, Texas"` end to end through the real Controller and CLI to `done` with exit code 0, including the real `plan_review`. Findings for the write-up: (1) Jev at the 0.5 cut kept rating "Texas" and even "Austin, Texas" as ambiguous after the user answered, so `location` settled with an Assumption after two asks on both models; the clarity cut looks too eager for `location`. (2) The question writer put numbered options in the question text, fixed in `QUESTION_SYSTEM`. (3) The plan added Junior and level-number titles with no seniority stated, fixed in `PLAN_SYSTEM` and gone on the deepseek re-run.

06: Real `plan_review` and the `salary` alias

What to build: pass 2. A user sees the Search Plan and Assumptions before any quota is spent, and can approve, edit, reject with feedback or abort. `--focus salary` is accepted as an alias for `compensation`. Spec: Request & Plan LLD §6 (`plan_review`), §8, §12 pass 2.

Blocked by: 04: Search Plan and the pass-through `plan_review`; Runtime build 03: CLI and `--focus`.

Status: done

- [x] `plan_review` pauses with `PlanReviewPayload(plan, assumptions)`, and writes `plan_decision`, `plan_feedback` and a `pause` log entry
- [x] An `edit` replaces `search_plan` with the user's plan
- [x] Rejects have no cap
- [x] Test: approve, edit, reject with feedback reaching the next `plan`, abort, and a bad edit re-prompting
- [x] Test: `--focus salary` lands in the checkpoint as `compensation`

Notes: the `salary` alias was already in the CLI from Runtime build 03, with its checkpoint test in `tests/test_cli.py`. A bad edit is refused by `PlanReviewAnswer` validation before it reaches the node, which is what the CLI re-prompts on.

collection-lld

collection-lld/map.md

Collection LLD

Label: wayfinder:map Parent: Talent Market Scout: HLD (component D in docs/HLD.md §3, §4)

Destination

`docs/lld/collection.md`: an LLD of Collection precise enough to build straight from, covering pass 1 (`search_jobs`, `fetch_agent`, `tools`, `collect`, `relevance`, Sufficiency, `broaden`, a pass-through `low_data_review`), pass 2 (the real `low_data_review`, USAJOBS-related Caveats) and pass 3 (tool event summaries), with nothing left to decide. Building the code is a separate effort after this map.

Notes

- Domain: Collection turns an approved Search Plan into a Sample of Relevant Postings, broadening within a fixed limit and pausing when it can't get enough. Vocabulary: `GLOSSARY.md`. Spec: `Joveo_Talent_Market_Agent- Assessment.pdf`. Deadline Mon 2 Nov 2026, 9:00 AM, so this map stays lean.
- Fixed upstream: docs/lld/kernel.md §4–§7 (state, resets, routing), §9 (Caveats), §11 (Jev), §12 (Tiers), docs/lld/runtime.md §3–§5 (stage contract, wiring, Pause protocol), §10 (events), docs/lld/gateway.md §3, §5, §6, §17 (`Fetch`, errors, `coalesce`, hand-ons), docs/lld/request-plan.md §6 (`SearchPlan` limits), HLD §4, §6.
- Skills: `langchain-router` first for any framework question (per `CLAUDE.md`); `grilling` + `domain-modeling` for grilling tickets.
- Standing preferences: inherited from the Kernel map (Pydantic v2 everywhere, `uv`, Python 3.12, `src/talent_scout/`, `pytest`, root `.env`).
- All grilling tickets were resolved in one session, in two rounds, on 2026-10-31.

Decisions so far

<!-- one line per closed ticket -->

- The fetch agent and its source tools: two tools, `search_adzuna` and `search_usajobs`, each taking only `title` and `page`, cheap tier, 6 Fetches per Attempt, with results in their own slice and only a summary in messages.

- What Relevant means and how it is judged: role only, one Jev request per Posting, verdicts kept across Attempts, and the Sample draws on every Attempt.
- Sufficiency, broaden and blocked Attempts: `len(sample) >= target` with 2 broadens, USAJOBS counted, `broaden` can't touch country or target, and `fetch_blocked` skips straight to the Pause.
- The low_data_review Pause: the payload shows counts, plans and the block reason; the answer is an action plus optional guidance, and `broaden` is always allowed.
- Who builds the Sends: Collection's routers return `"analyse"`, and Runtime turns it into Analysis's `fan_out(state)`.
- Caveats: eight Caveats with fixed triggers and writers, source `total` as a `note`, and no Caveat for a normal L2 read.
- Prompts: the LLD fixes each prompt's inputs and rules, wording lives in `collection/prompts.py`, and "yes" on the relevance Noul means really this role.
- Write docs/lld/collection.md: the LLD is written and the HLD, Kernel, Gateway, Runtime, Request & Plan and future-scope docs agree with it, with `FetchRecord` replacing `fetch_results` while writing.

Not yet specified

Out of scope

- Batching, `fan_out`'s body and federal pay in Stats: Analysis LLD.
- Rendering Caveats and the `total` disclosure: Brief LLD.
- Choosing the three sample runs within the Adzuna budget: Delivery.
- A daily budget shared across concurrent Runs: Infra shell LLD.
- Writing the Collection code: separate build effort after this map.

The fetch agent and its source tools

Type: grilling Status: resolved Assignee: smayankulkarni05 Blocked by: none

Question

What does the model control when fetching, how do Postings get from a tool call to `collect` without filling the message history, and what stops the agent?

Answer

Decided by grilling.

- **Revised: two tools, `search_adzuna` and `search_usajobs`, each `(title, page=1)`.** The original single `search_jobs` is superseded: the spec evaluates tool calling, so the model must choose the tool, visibly in the trace. `search_usajobs` is bound only for `country == "us"`. The Gateway call becomes `fetch(f, source)`, and `FetchRecord` gains `source`. Reasons and costs are in the Collection LLD §5. Bullets below that say "one tool" are the superseded decision.
- **`search_jobs(title, page=1)` (superseded).** Country, location, radius, `max_days_old` and category are pinned by code from `current_plan`, so the model can't drift from the plan the user approved. The title may be any string, not only one from the plan, because the model's judgement on variants is the point of having an agent.
- **One tool, disclosed.** The spec says "use tool calling so the model decides which tool to use", while the Gateway LLD fixed one tool. The model decides titles, pages and when to stop, but never which tool. A second tool such as `get_search_status` would only repeat what tool messages say, so it isn't built. The write-up says so.
- **`fetch_results: list[FetchResult].`** A new Collection slice written by `tools`, read by `collect` and cleared by `broaden`. `ToolMessage.artifact` is untyped, so strict msgpack blocks models there, and 50 Postings per call don't belong in history.
- **The model sees a summary:** new-Posting count, each source's total, `calls_left`, failures and a few sample titles.
- **`collect`** re-applies `coalesce` across Fetches. The mappers already normalise.
- **Cheap tier, 6 Fetches per Attempt.** Over 3 Attempts that is at most 18 Adzuna calls from the 25-a-day limit, not counting retries. At the cap, `tools` refuses further calls with a `ToolMessage` and the router goes to `collect`.
- **Over-fetching.** The agent can't see relevance until later, so its prompt asks for about 1.5× `target_postings` raw.

Executor errors:

- Bad args, an unknown tool name or a `GatewayError` become a `ToolMessage` the model can read.
- `SourceRejected` and bugs propagate and stop the Run.
- Several calls in one turn run concurrently, and any over the cap are refused individually.
- An Attempt where the model makes no tool call counts as a normal Attempt with a logged `warning`, rather than adding a retry path for something rare.
- `LLMExhausted` propagates from `fetch_agent`.

What Relevant means and how it is judged

Type: grilling Status: resolved Assignee: smayankulkarni05 Blocked by: none

Question

What makes a Posting Relevant, how is Jev called for it, and what happens to verdicts and Postings across Attempts?

Answer

Decided by grilling.

- **Role only.** A Posting is Relevant if it's really the researched role, at any seniority and in any location the search returned. Judging seniority too would empty the Sample of unlevelled Postings and bias the seniority mix, which is itself a reported Dimension. For "Senior Data Analyst, Bangalore":
  - "Data Analyst" with no level: Relevant
  - "Data Entry Analyst": not Relevant
  - a FAA "Public Notice" talent pool announcement: not Relevant, because it isn't a job advertisement for the role
- **One `jev.judge` request per Posting.** Batching saves little: 500 Postings cost under a cent at $0.042/M, and a shared request has to fit the 32 KB limit, which USAJOBS descriptions of up to 4,000 characters use up quickly.
- **`relevance_verdicts: dict[str, bool]`.** A new Collection slice, so later Attempts judge only new Postings. This is safe because verdicts judge against `hiring_request`, which doesn't change. `sample` is recomputed from verdicts each Attempt, as Kernel §6 says.
- **The Sample draws on every Attempt,** including Postings a later plan wouldn't have found. Relevance doesn't depend on the plan, so the Posting stays good Evidence, and this avoids spending quota to find it again.
- **No cap when the Sample overshoots the target.** Large Samples are Analysis's batching problem.

Sufficiency, broaden and blocked Attempts

Type: grilling Status: resolved Assignee: smayankulkarni05 Blocked by: none

Question

When is the Sample enough, what can `broaden` change, how many Attempts are allowed, and what happens when an Attempt can't get more data at all?

Answer

Decided by grilling.

- **Sufficiency:** `len(sample) >= current_plan.target_postings`. There is no separate noise rule, since noise already shows up as too few Relevant Postings.
- **USAJOBS counts** toward the target the same as Adzuna. They're real open jobs, and filtering them would discard the only Evidence some roles have. A Caveat fires when they dominate (Caveats).
- **2 broadens, so 3 Attempts,** before `low_data_review`. Attempts left = 2 − `len(broadened_plans)`.
- **`broaden` runs on the strong tier and fills a whole `SearchPlan`,** given the current plan, counts per title and a few example titles `relevance` rejected.
  - Code pins `country` and `target_postings`, so the model can't reach Sufficiency by lowering the target or leave the market the user asked about.
  - Its prompt offers the moves: more title variants, a wider radius, an older `max_days_old`, dropping `category`, a country-wide search, and replacing a noisy title. The last covers the spec's "broaden or refine".
  - `fetch_agent` can still page deeper in the new Attempt, since repeated Fetches are free L1 hits.
  - `LLMExhausted` propagates.
- **`fetch_blocked: str | None`.** `relevance` writes the reason when an Attempt can't get more data: Adzuna at 0 calls outside the US, or every Fetch in the Attempt failing. A set value goes straight to `low_data_review`, skipping remaining Attempts.
  - A USAJOBS-backed Run in the US carries on normally.
  - **`gateway.calls_left(source)`** is a new one-line method. `failures` holds plain strings, and when every source fails `fetch` raises with no result, so nothing else carries the signal.
  - The check is skipped when `replay_cache` is set, otherwise a recorded Run could take a different route on replay.

The low_data_review Pause

Type: grilling Status: resolved Assignee: smayankulkarni05 Blocked by: none

Question

What does the user see at `low_data_review`, what can they answer, and what does pass 1's pass-through do?

Answer

Decided by grilling.

- **`LowDataPayload`:** `count`, `target`, `attempts`, the plans tried, how many Postings were fetched and rejected, `calls_left`, and `fetch_blocked`.
- **`LowDataAnswer`:** `action` (`broaden`, `continue`, `abort`) plus optional `guidance`, which feeds the next `broaden`.
- **New slices:** `low_data_decision` and `low_data_guidance`, written by the Pause and read by the router.
- **`broaden` is always allowed and has no cap,** because a human drives each one, the same reasoning as `plan_review` rejects. It can't be refused for lack of quota:
  - The Controller validates an answer without seeing the payload (Runtime §5), so a validator can't know the budget is out.
  - A user returning the next day has fresh quota, while the stored Pause still says it's gone.
  - If quota is still out, the next Attempt finds nothing new and returns to the same Pause, at the cost of one `broaden` call, one `fetch_agent` turn and no quota.
- **`abort`** goes to END. **`continue`** logs a Caveat and goes to analysis.
- **Pass 1:** a pass-through that doesn't interrupt, leaves the decision null, and still logs the Too few Postings Caveat. The router treats null as `continue`.

Who builds the Sends

Type: grilling Status: resolved Assignee: smayankulkarni05 Blocked by: none

Question

`relevance`'s router must return `Send` × N to `batch_extract`, but the payload and batching belong to Analysis, and a stage may import only `runtime/deps.py` and `runtime/pause.py`. Who builds them?

Answer

Decided by grilling.

- **Collection's routers return a plain `"analyse"` outcome,** and `runtime/graph.py` turns it into `analysis.fan_out(state)`.
- **`fan_out` also sends zero Postings to `stats`,** which moves the HLD's "continue with 0 postings" edge case out of Collection.
- This changes the `CONDITIONAL` handling in Runtime §4 slightly. ⏳ The Analysis LLD owns `fan_out`'s body.

Caveats

Type: grilling Status: resolved Assignee: smayankulkarni05 Blocked by: none

Question

Gateway §17 and Kernel §9 hand Collection every Caveat's trigger and wording. Which does it raise, when, and from which node?

Answer

Decided by grilling.

Caveat	Fires when	Written by
Postings the mapper dropped	`dropped` > 0, with the count	`collect`
One source failed	`failures` has a source and another source answered, or L2 served a partial page	`collect`
Mostly federal	over 50% of the Sample is USAJOBS	`relevance`
Old data	over half of the Sample is older than 30 days	`relevance`
Too few Postings	the user continues below target, or pass 1's pass-through does	`low_data_review`
Broadened	any `broaden` ran, with the count and what changed	`relevance`, once Sufficient or at the Pause
Jev degraded	any `llm_fallback` or `truncated` verdict, one per Attempt with count and keys	`relevance`
Replay age	any `cache == replay`, with the oldest `fetched_at`; live calls during replay reported too	`collect`

- **No Caveat for a normal L2 read,** because up to 7 days old is within the data's expected freshness.
- **Source `total` is a `note`,** leaving out `cache == store`, and Brief renders it as labelled disclosure.
- Merged Duplicates are a `note`, not a Caveat (Gateway §6).

Prompts

Type: grilling Status: resolved Assignee: smayankulkarni05 Blocked by: none

Question

How much prompt text does the LLD fix, and what does the relevance Noul ask?

Answer

Decided by grilling.

- **Same treatment as Request & Plan.** The LLD fixes each prompt's inputs and rules, and the exact wording lives in `collection/prompts.py`, tuned during the build. The write-up reports what changed.
- **Relevance Noul:** "yes" means the Posting really is this role, which is the side that must clear the bar (Kernel §11). Each request sends the role and any seniority, plus the Posting's title, company, Domain and description.
- **Glossary:** gains **Attempt**, which these decisions use throughout.

Write docs/lld/collection.md

Type: task Status: resolved Assignee: smayankulkarni05 Blocked by: 01, 02, 03, 04, 05, 06, 07

Question

Assemble the resolved tickets on this map into `docs/lld/collection.md`, in the shape of `docs/lld/request-plan.md` and including a build checklist per pass, and make the other docs agree with it. Run `langchain-router` first for the framework details: the `tools` executor, `ToolMessage`, and binding the tool through `tiers.chat`.

Known changes elsewhere:

- **Kernel:** add `fetch_results`, `relevance_verdicts`, `fetch_blocked`, `low_data_decision` and `low_data_guidance` to `State` and the owner table; `broaden` clears `fetch_results`; update §7's routing rows and §9's Caveat list.
- **Gateway:** add `calls_left(source)` to `JobsGateway` and `FakeGateway`; mark the Collection hand-ons in §17 as settled.
- **Runtime:** `"analyse"` → `analysis.fan_out(state)` in §4, the `LowDataPayload` and `LowDataAnswer` shapes in §5, and the hand-ons in §17.
- **HLD:** §3 row D, §4's diagram labels, and the attempt cap.
- **future-scope.md:** a Collection section, with at least a second tool, a cap on the Sample, and judging seniority.

Answer

docs/lld/collection.md is written, with checklists for passes 1, 2 and 3 and a Collection section in `future-scope.md`. The HLD, Kernel, Gateway, Runtime and Request & Plan docs agree with it.

No code changed. `State`, `collection/types.py`, `JobsGateway` and `FakeGateway` keep their old shapes until step 1 of the pass 1 checklist.

Checked offline on the pinned versions: `InjectedToolArg` hides `plan` and `attempt` from the schema, a tool invoked with plain args returns its raw value, bad args raise `ValidationError`, and `FetchRecord` with `Source`-keyed dicts survives a checkpoint under strict msgpack.

Decisions made while writing, beyond the tickets:

- **`fetches: list[FetchRecord]` replaces `fetch_results: list[FetchResult]`.** A `FetchResult` can't say which Fetch it answered, and a Fetch where every source failed has none. The record carries the Fetch, the result or error, and the Attempt.

- **`broaden` doesn't clear the records.** `fetch_agent` needs to know what's been searched and `broaden` needs each title's totals, so `collect` strips their Postings instead. This reverses the ticket's "cleared by `broaden`".
- **`kernel/state.py` imports `gateway/types.py`,** which changes the Kernel and Gateway import rules.
- **`broaden` gets source totals per title,** not Relevant counts per title, because `Posting` doesn't record which title found it.
- **Closing Caveats.** Mostly federal, Old data, Broadened and Too few Postings fire once, when Collection finishes: in `relevance` when Sufficient, and in `low_data_review` on `continue`. This narrows the Caveats ticket's "at the Pause", which would repeat them for each broaden the user chose.
- **"Old" is measured from the newest `fetched_at`,** because a clock would change the Caveat text on replay and miss the LLM cache.
- **One source failed** also fires when a whole Fetch failed while another Fetch in the Attempt answered.
- **Unproven until the live smoke:** the 0.5 relevance cut, and whether 1.5× raw reaches the target in one Attempt.

collection-build

collection-build/issues/01-contract-and-fetching.md

01: Collection contract and the model's searches

What to build: given an approved Search Plan, the cheap model chooses tools, titles and pages and searches up to the cap. Each call leaves a `FetchRecord` in state and only a short summary in `messages`. This includes the prefactor the stage needs: the Gateway's per-source `fetch`, its public `calls_left`, the Collection `State` fields and the final Collection types. Spec: Collection LLD §2, §3, §5, §12, §13, §14 pass 1 steps 1-3.

The `fetch(f, source)` change is already uncommitted in the working tree (`gateway.py`, `testing.py`, `test_gateway.py`, `smoke_gateway.py`). Keep it and land it with this ticket instead of redoing it.

Blocked by: None (can start immediately).

Status: done

Contract:

- [x] `JobsGateway.fetch(f, source)` and `FakeGateway.fetch(f, source)` replace the fan-out `fetch(f)`, and the Gateway tests and smoke script use them
- [x] `JobsGateway.calls_left(source)` and `FakeGateway.calls_left(source)` are public
- [x] `State` gains `fetches`, `relevance_verdicts`, `fetch_blocked`, `low_data_decision` and `low_data_guidance`, owned by Collection, and `kernel/state.py` can import `gateway/types.py` with no import loop
- [x] `FetchRecord`, `LowDataPayload` and `LowDataAnswer` have their final fields, at the existing module path
- [x] The `low_data_review` stub, `ANSWER_FOR` and the clients still work with the larger payload, and `guidance` stays optional
- [x] `MAX_FETCHES = 6` and `MAX_BROADENS = 2` exist, and `make_nodes(deps)` and `ROUTERS` are wired into `build_graph`, overriding only the stubs of nodes that have landed
- [x] A stage imports only `runtime/deps.py` and `runtime/pause.py` from Runtime
- [x] Test: `FetchRecord`, including `FetchResult`'s `Source`-keyed dicts, and the Pause models survive the Runtime serde test

Tools and `tools`:

- [x] `search_adzuna` and `search_usajobs` each expose only `title` and `page` to the model, with `plan` and `attempt` as `InjectedToolArg`, and their docstrings state each API's real differences

- [x] `Fetch` is built from the tool's `title` and `page` plus the plan's `country`, `location`, `radius_km`, `max_days_old` and `category`, and `FetchRecord.attempt` is `len(broadened_plans)`
- [x] A US plan binds both tools and any other plan binds only `search_adzuna`, chosen from `current_plan.country` on each call
- [x] `tools` runs the calls under `asyncio.gather`, looking each tool up among those bound this Attempt, with injected values applied last
- [x] Each outcome follows §5: a good call gives a record and a summary, `GatewayError` gives a record with `error` and a `ToolMessage` with its text, an unknown tool or `ValidationError` gives `status="error"` and no record, over the cap gives a refusal and no record, and `SourceRejected` propagates
- [x] Every `ToolMessage` counts toward the cap, including refusals and bad args
- [x] The summary follows §5's format: source, title, page, Posting count and new count, then total, then at most 5 sample titles, with `calls left` only for Adzuna, and a failed call replaces the middle lines with `adzuna failed: <failure>.`
- [x] "New" means not in `postings` and not in an earlier record of this Attempt
- [x] `fetches` is written as a whole list on each update, and a Run killed mid-node reruns as L1 hits at no quota cost
- [x] Test: `plan` and `attempt` are absent from both bound schemas
- [x] Test: model args can't override `plan` or `attempt`
- [x] Test: a non-US plan binds only `search_adzuna`, and a `search_usajobs` call there is an unknown tool
- [x] Test: each tool passes its own `Source` to `gateway.fetch`
- [x] Test: `GatewayError` gives a record with `error`, bad args give an error `ToolMessage`, a 7th call is refused, and `SourceRejected` propagates
- [x] Test: no Posting text beyond sample titles reaches a message

`fetch_agent`:

- [x] Uses `tiers.chat("cheap", tools=[...], name="fetch_agent")`, and `LLMExhausted` propagates
- [x] An Attempt's first turn (`messages` empty) writes a `SystemMessage` and a `HumanMessage` holding `current_plan`, the raw goal of 1.5 × `target_postings`, the Sample so far, and already-searched `(source, title, page, total)` for records whose other Fetch fields match the current plan
- [x] Later turns send `messages` and append the reply
- [x] At `MAX_FETCHES` or more `ToolMessage`s it writes a `note` and makes no LLM call
- [x] No tool call on a first turn logs a `warning` and carries on as a normal Attempt, with no retry path
- [x] `after_fetch_agent` returns `tools` when the last message has `tool_calls`, else `collect`, including at the cap
- [x] `FETCH_SYSTEM` in `prompts.py` follows every §12 rule, with stable text first
- [x] Test with `FakeToolChatModel`: the first turn holds the plan, the already-searched list leaves out records with different Fetch settings, a turn at the cap makes no LLM call, and no tool call logs a `warning`

- [x] Test: a graph run with later stages stubbed loops `fetch_agent` → `tools` → `fetch_agent` → `collect`
- [x] The existing test suite still passes, and no test calls an LLM or a jobs API

02: Fetches become a Sample, and a Sufficient Sample moves on

What to build: an Attempt's records merge into `postings`, Jev judges each new Posting against the researched role, and the Relevant ones form the Sample. A Sufficient Sample leaves Collection through the `"analyse"` outcome and reaches `stats` in the graph. `fetch_blocked` is set here so that the next ticket can route on it. Spec: Collection LLD §4, §6, §7, §8, §11, §12, §14 pass 1 steps 4-5, and the `"analyse"` part of step 7. Runtime LLD §4.

Blocked by: 01: Collection contract and the model's searches.

Status: done

`collect`:

- [x] Plain code that reads only the current Attempt's records, with no LLM call
- [x] Merges by key first, with the first copy seen winning, then applies `coalesce` to the result
- [x] Strips Postings from those records and keeps everything else on them, such as totals and `error`
- [x] Writes a `note` with the count kept and Duplicates merged. Merged Duplicates are never a Caveat
- [x] Writes a `note` with source totals as `{title: {source: total}}`, leaving out `cache == store`
- [x] Test: the same key from two Attempts stays one Posting with no self-merge
- [x] Test: a Duplicate across Fetches collapses
- [x] Test: records lose their Postings but keep totals, and `store` totals are left out of the note

`relevance`:

- [x] Each key in `postings` with no verdict gets one `jev.judge("relevance", state, {"relevant": RELEVANT_NOUL})` under `asyncio.gather`, with the Kernel's semaphore limiting concurrency
- [x] The state sent is `role` and `seniority` from `hiring_request`, and the Posting's `title`, `company`, `domain` and `description`
- [x] Verdicts are kept across Attempts, so later Attempts judge only new Postings
- [x] `sample` is `[k for k in postings if relevance_verdicts[k]]`, recomputed each Attempt, drawn from every Attempt and never capped
- [x] Relevant means role only: unlevelled and junior Postings are Relevant, while a keyword collision or a FAA "Public Notice" talent pool isn't
- [x] `RELEVANT_NOUL` in `prompts.py` follows every §12 rule, with "yes" meaning really this role and both or neither criteria sides set

- [x] `fetch_blocked` is written on every pass: "Every search in this Attempt failed: <error>" when the Attempt has records and all have `error`, and "Adzuna's daily limit is used up, until the next UTC day" when `calls_left(Source.adzuna) == 0`, `country != "us"` and `replay_cache` isn't set. Otherwise it's `None`
- [x] Writes a `route` log with the count, target and outcome, e.g. "38 of 142 Postings relevant (route: broaden)"
- [x] Jev errors that survive its own retries and fallback propagate, and `scout resume` reruns the node
- [x] Test with `fake_classifier`: only unjudged Postings are sent, and the Sample drops a key that `coalesce` merged
- [x] Test: both `fetch_blocked` reasons fire, a US Run isn't blocked by an empty Adzuna budget, and replay skips the budget check

Routing:

- [x] `after_relevance` returns `analyse` when `len(sample) >= current_plan.target_postings`, then `low_data_review` when blocked or at `MAX_BROADENS`, otherwise `broaden`. USAJOBS Postings count the same as Adzuna's, with no separate noise rule
- [x] `build_graph` wraps Collection's two routers so `"analyse"` becomes `fan_out(state)`, and until Analysis lands the stub in `runtime/testing.py` returns `"stats"`
- [x] `CONDITIONAL["relevance"]` gains `stats`, which the stub `fan_out` returns and the current target list lacks
- [x] Test: every `after_relevance` outcome
- [x] Test: a graph run with other stages stubbed goes from `fetch_agent` to `stats` when the first Attempt is Sufficient

03: Broadening, pass 1 Caveats and the live smoke

What to build: finishes pass 1. A short Attempt broadens within the limit and tries again, and when it still can't get enough, the Run carries on through a pass-through `low_data_review` to analysis. The Run log says plainly what happened to the data. A live run through the real models and Adzuna then proves the whole stage. Spec: Collection LLD §4, §9, §10 (pass 1), §11, §12, §14 pass 1 steps 6-9.

Blocked by: 02: Fetches become a Sample. Also from `.scratch/request-plan-build`: 01 (`SearchPlan` limits), 03 (the clarify loop) and 04 (Search Plan and pass-through `plan_review`), since the live smoke needs a real `search_plan`.

Status: done

`broaden`:

- [x] Uses `tiers.structured("strong", SearchPlan, system=BROADEN_SYSTEM, name="broaden")`, and `LLMExhausted` propagates
- [x] Code pins `country` and `target_postings` to the current plan's values, and a violation of the other limits is fed back on the retry
- [x] Appends to `broadened_plans`, clears `messages` to `[]`, and writes a `note` with what changed
- [x] The input holds `hiring_request`, the current plan, the Sample count and target, each title and page searched with its source totals, up to 10 rejected Posting titles, and `low_data_guidance` when set
- [x] `BROADEN_SYSTEM` in `prompts.py` follows every §12 rule, with stable text first
- [x] Test: the LLM's `country` and `target_postings` are overwritten, `messages` is cleared, a bad plan passes on the feedback retry, and guidance reaches the prompt

Pass-through `low_data_review` and routing:

- [x] `make_nodes` exports a `low_data_review` that doesn't interrupt, leaves `low_data_decision` null and still logs the closing Caveats
- [x] `after_low_data` returns END on `abort`, `broaden` on `broaden`, and `analyse` on `continue` or null
- [x] Test: every `after_low_data` outcome
- [x] Test: a graph run with other stages stubbed has too few Postings on the first Attempt, broadens, then reaches Sufficiency
- [x] Test: a graph run that stays short gets 3 Attempts, passes through `low_data_review` and reaches `stats`, including with a Sample of zero
- [x] Test: `fetch_blocked` skips straight to `low_data_review` with broadens left

Pass 1 Caveats, each with a trigger and a non-trigger test:

- [x] Postings the mapper dropped: `dropped` > 0, with the count, written by `collect`
- [x] Replay age: any `cache == replay`, with the oldest `fetched_at` and the number of live calls during replay, written by `collect`
- [x] Jev degraded: any `llm_fallback` or `truncated` verdict, one per Attempt with the count and keys, written by `relevance`
- [x] Old data: over half of the Sample is older than 30 days, measured from the newest `fetched_at` and never from today, as a closing Caveat
- [x] Broadened: any `broaden` ran, with the count and what changed, as a closing Caveat
- [x] Too few Postings: the Sample is below target, as a closing Caveat
- [x] `_closing_caveats` runs once, when Collection finishes: in `relevance` when Sufficient and in `low_data_review` when passing through
- [x] Test: a normal L2 read raises no Caveat
- [x] Test: Caveat text doesn't change with the clock

Live smoke (a script, not in CI):

- [x] It spends a few Adzuna calls, so get the user's OK in chat before running it
- [x] "Data Analyst, Bangalore" runs through the real models and Adzuna to a Sample, with the route visible in the Run log
- [x] Records whether the 0.5 relevance cut holds and whether 1.5× raw reaches the target in one Attempt
- [x] Any change to the cut or the prompts is noted for the write-up

Live smoke result ("Data Analyst, Bangalore", minimax-m3.1-flash, real Adzuna)

- Plan target was 100. Attempt 1 gathered 98 raw (1.5x = 150), so 1.5x raw did **not** reach the target in one Attempt: the cap of 6 searches yielded fewer Postings than the goal. 88 of 98 relevant, still short of 100, so it broadened twice. Third Attempt: 220 raw, 138 relevant, Sufficient.
- The 0.5 cut held: 63% relevant overall (90% on Attempt 1). Rejections looked right (accounting, tax and data-centre "analysts"), except "Business Data Analyst", which is borderline.
- No change to the cut or the prompts.
- Jev degraded via `llm_fallback` has no test: `make_deps` has no fallback chain. The `truncated` path is tested.
- Script: `scripts/smoke_collection.py`.

04: The real low_data_review Pause and pass 2 Caveats

What to build: when Collection can't get enough, the Run stops and shows the user the counts, the plans tried and any block reason. The user broadens with optional guidance, continues with what's there, or aborts, from either client. USAJOBS-related Caveats also land here. Spec: Collection LLD §3, §10, §11, §14 pass 2. Runtime LLD §5.

LLD §14 doesn't list the client work, but the CLI and Streamlit render this Pause today and don't ask for guidance, and §10 requires it.

Blocked by: 03: Broadening, pass 1 Caveats and the live smoke.

Status: done

The Pause:

- [x] `low_data_review` interrupts with a `LowDataPause` carrying `count`, `target`, `attempts`, `plans` (every plan tried, oldest first), `fetched`, `rejected`, `calls_left` and `blocked`
- [x] It writes `low_data_decision` and `low_data_guidance`, replaced at each review, and a `pause` log entry
- [x] `broaden` is always allowed with no cap, and is never refused for lack of quota
- [x] `calls_left` is for information only, and nothing routes on it
- [x] Closing Caveats are raised only on `continue`, not at each Pause
- [x] `continue` with a Sample of zero reaches `stats`
- [x] Test in a one-node graph with a checkpointer: the payload's fields, and each action
- [x] Test: guidance reaches `broaden`
- [x] Test: a further broaden past the cap returns to the Pause
- [x] Test: closing Caveats fire only on `continue`

Clients:

- [x] `render_pause` shows the counts, the plans tried and the block reason, shared by every client
- [x] The CLI offers `broaden`, `continue` or `abort`, and on `broaden` asks for optional guidance, where Enter skips
- [x] Streamlit offers the same three actions with optional guidance
- [x] Test: both clients send `guidance` on `broaden` and leave it null when skipped

Pass 2 Caveats, each with a trigger and a non-trigger test:

- [x] One source failed: L2 served a partial page (the record's `failures` holds its source), or a call failed while another call in the Attempt answered, written by `collect`
- [x] Mostly federal: over 50% of the Sample is USAJOBS, as a closing Caveat
- [x] Test: both on `FakeGateway` results with `failures` and L2 partial pages

05: Tool events in the live view

What to build: a client watching a Run sees each search as it starts and what it returned, because the `tools` executor emits its own events. Spec: Collection LLD §5, §14 pass 3. Runtime LLD §10, §16 pass 3 step 2.

Blocked by: 01: Collection contract and the model's searches. Also `.scratch/runtime-build` 05: Event stream, which adds `emit()` and the `ToolCalled` and `ToolReturned` mapping.

Status: done

- [x] The executor emits `{"type": "tool_called", "name": "search_adzuna", "summary": '"data analyst" page 1'}` when a call starts
- [x] The executor emits `{"type": "tool_returned", "name": "search_adzuna", "summary": "50 Postings, 38 new"}` when it returns, or the error text when the call failed
- [x] Test inside a one-node graph: both events are delivered for a good call and for a failed one
- [x] Test: the Controller maps them to `ToolCalled` and `ToolReturned` rather than dropping them as unknown types

analysis-lld

analysis-lld/map.md

Analysis LLD

Label: wayfinder:map Parent: Talent Market Scout: HLD (component E in docs/HLD.md §3, §4)

Destination

`docs/lld/analysis.md`: an LLD of Analysis precise enough to build straight from, covering `fan_out`, `batch_extract`, `normalise_skills` and `stats`, with nothing left to decide. Building the code is a separate effort after this map.

Notes

- Domain: Analysis turns a Sample into per-Posting Extractions and a list of Stats that each carry their Evidence. The LLM extracts, and Python computes every number. Vocabulary: `GLOSSARY.md`. Spec: `Joveo_Talent_Market_Agent- Assessment.pdf`. Deadline Mon 2 Nov 2026, 9:00 AM, so this map stays lean.
- Fixed upstream: docs/lld/kernel.md §3 (`Stat`, `MarketStats`, LLM-schema policy), §5 (the `extractions` reducer), §9 (Caveats), §10 (msgpack rules), §12 (Tiers), docs/lld/runtime.md §3, §4 (stage contract, `"analyse"`), §10 (events), docs/lld/collection.md §4, §15 (hand-ons), docs/lld/gateway.md §7, §17, HLD §5.
- Skills: `langchain-router` first, then `langgraph-fundamentals` for `Send` and reducers (per `CLAUDE.md`).
- **The user delegated every decision on this map to the agent** ("make all decisions yourself, ensure they are rational"), so the grilling tickets were answered from the spec, the upstream LLDs and live probes rather than a live exchange. Each ticket gives its reasoning so any of them can be challenged later.
- Live probes ran on 2026-10-31 on the 100 Adzuna and 25 USAJOBS fixture Postings with `minimax/minimax-m3.1-flash` and `typesafe/jev-1.14`, spending no Adzuna quota. The scripts were throwaway. The findings are in Live probe of extraction and skill merging.
- Standing preferences: inherited from the Kernel map (Pydantic v2 everywhere, `uv`, Python 3.12, `src/talent_scout/`, `pytest`, root `.env`).

Decisions so far

<!-- one line per closed ticket -->

- Live probe of extraction and skill merging: the default token limit fails even 10 Postings, citation slips grow with Batch size, all Adzuna pay in hand is estimated, and a loose merge prompt inflates counts while a tight one doesn't.
- What is extracted, and how it stays grounded: verbatim skills with a requirement, seniority, `years_min`, employer type, with code dropping any skill or year that isn't in the Posting's text.
- Batching and fan out: at most 20 Postings or 20,000 characters per Batch, 8,192 output tokens, positional refs, and `Send` from pass 1.
- When extraction fails: transient failures stop the Run and resume reruns only the failed Batch, while bad output skips that Batch with a Caveat.
- Normalising skills: one LLM call after the fan-in, in a new `normalise_skills` node, with no Jev veto and every merge listed in the run log.
- Normalising pay: advertised, estimated and federal pay are three separate groups, never pooled, annualised and shown in one currency using a dated fixed-rate table.
- The Stat catalogue: fixed ids per Dimension, skills ranked by distinct employers, employer type decided per employer, and a trend only on a window of 28 days or more.
- Caveats and the run log: nine Analysis Caveats with fixed triggers, all written by `stats` except the skill-merge one.
- Prompts: the LLD fixes inputs and rules, wording lives in `analysis/prompts.py`, and the probed prompt text is the starting point.
- Write docs/lld/analysis.md: the LLD is written and the HLD, Kernel, Runtime, Collection, Gateway, glossary and future-scope docs agree with it.

Not yet specified

Out of scope

- Confidence Tags, the Hiring Outlook, Insight emphasis from `focus`, and rendering Caveats and Stats: Brief LLD.
- Whether a Client Recommendation may use an estimated salary band: Brief LLD, with Analysis's labels as its input.
- Choosing the three sample runs within the Adzuna budget: Delivery.
- Writing the Analysis code: separate build effort after this map.

Live probe of extraction and skill merging

Type: task Status: resolved Assignee: smayankulkarni05 Blocked by: none

Question

Batch size, the token limit, the grounding rule and the skill-merge design all depend on how a real model behaves on real Postings. What does it do on the fixtures, and does `Send` behave as the failure policy needs?

Answer

Run on 2026-10-31 from throwaway scratchpad scripts, using `tests/fixtures/jobs/` (100 Adzuna, 25 USAJOBS), `minimax/minimax-m3.1-flash` through the real `Tiers.structured`, and `typesafe/jev-1.14`. No Adzuna or USAJOBS calls.

Batch size

Batch	`max_tokens`	Time	Tokens in / out (reasoning)	Missing	Skills not in the Posting
10 Adzuna	2,048	16 s	failed: `LLMExhausted` after 2 attempts		
20 Adzuna	2,048	18 s	failed: `LLMExhausted` after 2 attempts		
20 Adzuna	8,192	15 s	2,604 / 4,092 (2,080)	0	3 of 157 (1.9%)
40 Adzuna	8,192	30 s	4,782 / 6,199 (2,886)	0	24 of 240 (10%)
5 USAJOBS, 17,101 chars	8,192	3 s	3,324 / 745 (0)	0	0 of 53

- The cheap tier's default 2,048 tokens can't hold even 10 Postings, because reasoning takes about half the output.
- Skills not found in the cited Posting grow with Batch size, which points to attribution slipping between Postings. This is a single run per size, so it's a direction, not a measured rate.
- No Posting was missing from any response.

Pay in the fixtures

All 100 Adzuna salaries are `predicted`, with `min == max`, ranging from 89,379 to 363,162 USD, so `unknown` period reads as annual for estimates. No advertised Adzuna salary was available, so the same

reading for advertised pay is **unproven until a live Run**. All 25 USAJOBS rows are `PA` ranges.

Skill merging

A first merge prompt, "group names that mean exactly the same skill", over-merged on 221 names: `AWS` took `S3`, `Lambda` and `Glue`, `Git` took `Bitbucket`, `Azure Data Factory` took `SnapLogic`, and `Machine Learning` took `Big Data`. These would inflate top-skill counts directly. The extraction prompt also returned lists as one name ("AWS: Glue, S3, Athena, Data Catalog") and duties as skills ("data engineering" 19, "pipelines").

A tightened extraction and merge prompt (say what a variant is, name what stays apart, default to leaving out, and split lists) gave 166 names and 12 proposed merges, all correct: `Airflow→Apache Airflow`, `Spark→Apache Spark`, `Azure→Microsoft Azure`, `data lakes→data lake`. Jev agreed with all 12 (p 0.79 to 0.98), so it never had to veto anything. 15 of 335 skills (4.5%) still weren't found in the cited Posting.

Boilerplate. `IoT` (15), `fraud detection` (9) and `cancer research` (8) reached the top of the raw counts, almost all from Booz Allen's 23 Postings, which share a standard opening paragraph.

Employer type. Per-Posting answers disagree within one employer (Booz Allen: 9 agency, 14 direct). Well-known staffing firms were also missed: Randstad Digital and Mitchell Martin came back direct.

`Send`, offline, on the pinned versions. With a Pydantic payload and 4 branches, one raising and one returning `{}`: the fan-in node runs once, a rerun re-executes only the failed branch, the finished branches' writes survive, and `{}` is a valid update. The checkpoint stores the `Send` payload class, so it needs an entry in `allowed_msgpack_modules` under strict msgpack.

What is extracted, and how it stays grounded

Type: grilling (delegated to the agent) Status: resolved Assignee: smayankulkarni05 Blocked by: 01

Question

Which per-Posting facts does the LLM return, and what stops an invented fact reaching a Stat, given that "every number traces back to real postings" is 20% of the marks?

Answer

Only facts a Dimension needs and code can't get from the Posting:

Field	Values	Why
`skills`	`name` plus `requirement`: `must`, `nice` or `unstated`	Skills Dimension, must-have vs nice-to-have
`seniority`	`entry`, `mid`, `senior`, `lead`, `executive`, `unknown`	Seniority mix. From the title or explicit wording only, never guessed from years
`years_min`	int or null	Typical years of experience, explicit only
`employer_type`	`direct`, `agency`, `unknown`	Direct vs staffing split

Left out: salary (structured fields exist, and an LLM reading amounts out of text is the risk to avoid), location, employment type and work mode (no Dimension uses them).

Skill names are verbatim. Normalising inside each Batch can't be consistent across Batches, and it would make a name impossible to check. So extraction copies, and one later step normalises (Normalising skills).

Code checks what it can. A skill whose folded name isn't a whole-word match in the Posting's title and description is dropped and counted. A `years_min` whose number isn't in the text, or that is over 40, becomes null. The probe saw 2% to 10% of skills fail this check depending on Batch size. Those are the very cases that would otherwise become false Evidence.

`seniority` and `employer_type` are judgements, so they can't be checked this way. The spec accepts that, because they're reported as shares, not cited facts.

Folding keeps `+` and `#`, so `C`, `C++` and `C#` stay three skills. Whole-word matching stops `R` matching inside another word.

`Extraction` is the stored model, with `ungrounded: int` added, because a branch can't log. The LLM model `PostingFacts` adds `ref` instead. Both share a base so the two can't drift.

USAJOBS Postings get `employer_type = direct` from code in `stats`, since a federal agency hires for itself.

Batching and fan_out

Type: grilling (delegated to the agent) Status: resolved Assignee: smayankulkarni05 Blocked by: 01

Question

How is the Sample split into Batches, what does a `Send` carry, and how does zero Postings reach `stats`? The Sample has no cap, so 3 Attempts can bring up to 900 Postings.

Answer

`MAX_BATCH_POSTINGS = 20`, `MAX_BATCH_CHARS = 20_000`, `max_tokens = 8192`.

- 20 Postings used 4,092 of 8,192 output tokens in the probe, which leaves room for a heavier reasoning turn. 40 used 6,199, with five times the rate of skills not in the cited Posting.
- A smaller Batch also loses less when one is skipped, and a rerun repeats less.
- The character cap is what handles USAJOBS: 20,000 characters is about 5 of its 4,000-character descriptions, and about 40 Adzuna snippets, so for Adzuna the count limit applies first. Well within the context window either way, which is the spec's edge case.
- 900 Postings is 45 Batches. At 15 s each and the existing `llm_concurrency` of 20, that's under a minute.
- The 60 s call timeout holds at this size.

Greedy packing over Postings sorted by key, so the same Sample always gives byte-identical prompts, which the replay cache needs. A single oversized Posting still gets its own Batch.

`ExtractBatch(postings: list[Posting])` is the payload, because `owned()` requires a `Send` arg to be a model. It carries no Hiring Request: extraction doesn't depend on the request, which also lets identical Batches hit the replay cache across Runs.

Refs are positions in the Batch (0, 1, 2…), not Posting keys. They cost fewer tokens, and the model can't return a key that looks real but isn't. A `ref` outside the Batch is ignored, the first answer wins on a repeat, and a Posting with no answer simply has no Extraction (When extraction fails).

`fan_out(state)` returns `"stats"` when the Sample is empty, otherwise one `Send` per Batch. A zero-branch fan-out would stall the Run (HLD §4).

`Send` from pass 1, reversing HLD §11's "sequential extraction" then "Send fan-out" split. The two differ by a few lines, and building sequential first means writing the failure and progress paths twice, days before the deadline.

When extraction fails

Type: grilling (delegated to the agent) Status: resolved Assignee: smayankulkarni05 Blocked by: 01

Question

A Batch can fail because the provider is down, or because the model keeps returning output that won't validate. What happens to the Run in each case? The spec says neither a crash nor garbage is acceptable.

Answer

The two need opposite handling, so `LLMExhausted` gains `bad_output: bool`, true when the last error in the chain was bad output.

Failure	What happens	Why
Transient, every model (`bad_output` false)	Propagates. The Run stops at `batch_extract`, and `scout resume` reruns only that Batch	Waiting fixes it. A Brief built without those Postings would be worse than a stopped Run, and the probe confirmed finished Batches aren't repeated
Bad output, every model (`bad_output` true)	The Batch returns `{}` and its Postings have no Extraction	The same text would fail the same way on every resume, so propagating would trap the Run forever
A Posting left out of a good response	No Extraction for it	Too rare for a retry path
`normalise_skills` bad output	Skills stay merged by folding alone, with a Caveat	Same reasoning, and folding alone still gives true counts, only more split

No new state. The missing Postings are `set(sample) - set(extractions)`, so `stats` raises the one Caveat. Nothing has to be recorded by a branch, which couldn't log anyway.

`stats` never fails on this. Skills, seniority and agency Stats use the Postings that were extracted, and their labels say so ("of 118 extracted"). Pay, demand, employers and titles need no Extraction at all, so even a fully failed extraction yields a usable, Caveated Brief.

No splitting a bad Batch in half to retry. It's the only way to recover those Postings, but bad output hasn't been seen at 8,192 tokens, so it would be code for a case with no evidence. It's listed in `future-scope.md` with what would bring it back.

Normalising skills

Type: grilling (delegated to the agent) Status: resolved Assignee: smayankulkarni05 Blocked by: 01, 02

Question

"Postgres" and "PostgreSQL" must count once, across parallel Batches that can't see each other. Where does that happen, and how is over-merging prevented?

Answer

A new LLM node, `normalise_skills`, between the fan-in and `stats`, on the cheap tier.

- **Why not inside each Batch:** Batches would choose different canonical names, and verbatim names are what make grounding possible.
- **Why not code alone:** synonyms like K8s and Kubernetes can't be computed, and an alias table only covers the roles it was written for. This agent takes any role.
- **Why not inside `stats`:** `stats` stays pure Python, per HLD §5, so it's deterministic and cheap to test.

Steps

1. Code folds every name and counts them. Case, spacing and punctuation variants merge here with no LLM.
2. The top `MAX_SKILLS_TO_MERGE = 300` folded names go to the model, sorted, so the prompt is stable for replay. Only 15 skills are reported, and a name outside the top 300 can't plausibly reach them. No names, no call.
3. The model returns only groups of two or more. Code keeps only variants that were in the input, and the first group wins if a name appears twice.
4. `skill_aliases: dict[str, str]` maps folded name → canonical name. A name absent from it stands for itself, displayed in its most common spelling.

Over-merging is the failure that matters, because an inflated count is a wrong number in a client Brief, while a split one only ranks a skill lower. The probe showed the prompt decides this: the loose wording merged `S3` into `AWS`, and the tight wording made 12 merges, all correct. So the prompt defines a variant, names what stays apart, and defaults to leaving a name out.

No Jev veto. It was tested as a check on each merge, and it agreed with all 12, so on this evidence it would add a fourth placement, a threshold and up to hundreds of calls to catch nothing. It's in `future-scope.md`, for if over-merging shows up in live runs. Note that it was never tried against a bad merge, so its veto power is unmeasured.

Every merge goes in a run log `note`, so a reader of the run log or a `samples/` transcript can see "Spark → Apache Spark" and catch a bad one. The live smoke asserts that known traps stay apart.

The fan-in means one `normalise_skills` per Run. `fan_out`'s zero-Postings path goes straight to `stats`, so `skill_aliases` stays empty there.

Normalising pay

Type: grilling (delegated to the agent) Status: resolved Assignee: smayankulkarni05 Blocked by: 01

Question

Settle the HLD §12 items: currency and period normalisation, `salary_is_predicted`, and USAJOBS vs Adzuna in compensation Stats.

Answer

Three groups, never pooled. Each gets its own median and middle 50%.

Group	Which Postings	Why apart
`advertised`	Adzuna, not `predicted`	What employers actually said
`estimated`	Adzuna, `predicted`	Adzuna's own model, not an employer's statement. All 100 fixture salaries are this
`federal`	USAJOBS	Fixed pay tables, which the Gateway LLD and HLD §7 say must never mix into market pay. Also wide GS ranges

Dropping estimates would leave a US Brief with no pay figure at all. Pooling them would present a model's guess as market evidence. Keeping them labelled gives the Brief a figure and an honest strength. ⏳ The Brief LLD decides how a Client Recommendation uses them, with the expectation of preferring `advertised` and a lower Confidence Tag on `estimated`.

The disclosure share counts advertised only, over Adzuna Postings. An estimated salary isn't disclosed pay, and counting USAJOBS would make any federal-heavy Sample look transparent because the law requires it.

A Posting's figure: the midpoint when both bounds exist, otherwise the one bound.

Period: annual, as ×2080 hourly, ×260 daily, ×52 weekly, ×26 biweekly, ×12 monthly. `unknown` (every Adzuna salary) is read as annual, the evidence for estimates being the fixtures. A Caveat fires when advertised pay relied on that reading.

Implausible figures, for `unknown` periods only, are excluded and counted, because an hourly rate read as annual would wreck a group:

- below the equivalent of 1,000 USD a year, which catches hourly and most low monthly figures, or
- below one sixth of the group's median, which catches monthly figures (a ratio of 12) while leaving real junior pay alone.

Currency: a Run has one country and so one market currency, and USAJOBS is USD inside a US Run. The Stat currency is `hiring_request.currency` when the table has it, otherwise the market currency. If the user asked for a currency the table lacks, a Caveat says so and pay stays in the market currency.

A fixed `UNITS_PER_USD` table with a date, not a live rate lookup. It covers the 13 Adzuna currencies, all in the ECB's reference rates. A live call adds a dependency and a failure mode, and would change Stats on replay. Pay is rounded to whole units, so the conversion doesn't suggest false precision. A Caveat states the date whenever a conversion happens, and rates are refreshed by hand.

Minimums: a group needs 5 figures, below which it has no Stats rather than a misleading median. The same applies per seniority level, so `comp.estimated.senior.median` exists only with 5 or more figures. That per-level split is what lets a "Senior Data Analyst" Brief recommend senior pay, even though Relevance deliberately keeps every level in the Sample.

The Stat catalogue

Type: grilling (delegated to the agent) Status: resolved Assignee: smayankulkarni05 Blocked by: 02, 05, 06

Question

Which Stats exist, with which ids and which rules, so the Brief can cite every figure it uses? Kernel §3 leaves the catalogue to Analysis.

Answer

The catalogue is in the LLD §8. These are the decisions behind it.

Rank skills by distinct employers, then Postings. Booz Allen's shared opening paragraph put `IoT` and `cancer research` near the top of a data engineer's raw counts. Counting employers first demotes one company's boilerplate without any boilerplate detection. The label shows both ("Python: 15 of 100 extracted Postings, from 9 employers").

Must-have and nice-to-have are separate counts per skill. Most Adzuna snippets are too short to state which, so `unstated` is common and neither count is forced.

Employer type is decided per employer, not per Posting. The probe's Booz Allen split of 9 to 14 is the model reacting to one Posting's wording, not a fact about Booz Allen. Rule, in order:

1. USAJOBS: direct.
2. The name has "staffing" or starts a word with "recruit", or is in a short `KNOWN_AGENCIES` list: agency. The probe called Randstad direct, and a list is cheap, exact and easy to explain.
3. Otherwise the majority of that employer's Extractions, with a tie or nothing known giving `unknown`.

Employers group by folded name minus legal suffixes (Inc, LLC, Ltd, Pvt and so on), shown in the most common spelling. `coalesce` already treats company names as words, but the suffix rule is new because grouping needs it and Duplicates don't.

Titles group by folded title only. Stripping locations or levels from a title with code would also damage real variants like "Data Engineer - Java", and the variants employers really use are what the spec asks for.

The trend is a labelled comparison, computed only when it can mean something. The newest 14 days against the 14 before, needing `max_days_old >= 28` and at least 20 Postings across both. Rising at

≥ 1.25×, falling at ≤ 0.8×, otherwise flat. Its label says it counts ads still live, because filled ads vanish and that always tilts the view towards recent. A Caveat doesn't add more than the label, and the Brief quotes labels.

"Now" is the newest `fetched_at`, the same reasoning as Collection's Old data Caveat: a clock would change Stats and miss every LLM cache entry downstream on replay.

Shares carry their percentage in the label, because the Brief may not compute figures. **Every Stat's Evidence is the Postings it counted,** including each median's group.

Rounding is fixed so replay stays byte-identical: pay to whole units, shares to 3 places. Quartiles use `statistics.quantiles(n=4, method="inclusive")`, since `exclusive` can land outside the data on small groups.

`focus` changes nothing here, as Request & Plan §8 already says. Every Stat is cheap Python, and the Brief does the emphasis.

Caveats and the run log

Type: grilling (delegated to the agent) Status: resolved Assignee: smayankulkarni05 Blocked by: 04, 05, 06, 07

Question

Which limitations found in Analysis become Caveats (Kernel §9), and who writes them, given that `batch_extract` branches can't log?

Answer

Caveat	Fires when	Written by
Postings not extracted	Any Sample key has no Extraction, with the count	`stats`
Skills merged by spelling only	`normalise_skills` hit bad output	`normalise_skills`
Skills from snippets	Any Adzuna Posting in the Sample, with the count. Their 500-character snippets undercount skills	`stats`
Pay is estimated	No `advertised` Stats, but `estimated` Stats exist	`stats`
Too little pay data	No group reached 5 figures	`stats`
Pay period assumed	Any `advertised` figure had an `unknown` period, with the count	`stats`
Implausible pay excluded	Any figure was excluded, with the count	`stats`
Pay converted	The Stat currency differs from a Posting's, with the rate date	`stats`
Currency unavailable	The requested currency isn't in the table	`stats`

Federal pay kept apart isn't a Caveat. The Stats are labelled, and it's by design. Collection's "Mostly federal" Caveat covers the tilt.

Skills not in their Posting are a `note`, not a Caveat. They were dropped, so what's reported is grounded, and nothing about the Brief is less trustworthy.

`stats` **is the natural writer** because it runs once and sees everything, and it computes all of these from state, so branches record nothing but `ungrounded`.

Other log entries

Node	Writes
`normalise_skills`	`note` listing every merge
`stats`	one `note`: Sample size, Extractions, Stats produced, skills dropped

A transient failure stopping the Run isn't a Caveat either (Kernel §9).

Prompts

Type: grilling (delegated to the agent) Status: resolved Assignee: smayankulkarni05 Blocked by: 01, 02, 05

Question

What must each Analysis prompt get and do?

Answer

As in Collection and Request & Plan: the LLD fixes inputs and rules, the wording lives in `analysis/prompts.py`, and it's tuned on real runs. The write-up reports what changed.

The probe gives the build a tested start, and the write-up a real "what I changed after testing" story. Both of these changes came from reading its output:

- **`EXTRACT_SYSTEM`:** went from "each concrete skill" to "things a person could list on a CV, one to four words, copied exactly, one per entry, split lists, leave out duties and the role's own name". The top entries went from "data engineering" and "pipelines" to named tools. Still imperfect: "fraud detection" and "cancer research" get through, which ranking by employer handles.
- **`MERGE_SYSTEM`:** went from "mean exactly the same skill" to a definition of a variant (usable in place of the canonical name in any sentence), named things that stay apart, and "if unsure, leave it out". Over-merges went from at least 5 clear cases to none in 12.

Both treat Posting text as data, never instructions, since job ads come from third parties and reach the model verbatim.

Stable text comes first, as `Tiers.structured` already enforces (Kernel §12), so Batches share a cached prefix.

Write docs/lld/analysis.md

Type: task Status: resolved Assignee: smayankulkarni05 Blocked by: 01, 02, 03, 04, 05, 06, 07, 08, 09

Question

Assemble the resolved tickets on this map into `docs/lld/analysis.md`, in the shape of `docs/lld/collection.md` and including a build checklist, and make the other docs agree with it. Run `langchain-router` first for the framework details: `Send`, the fan-in and the reducer.

Known changes elsewhere:

- **Kernel:** `skill_aliases` on `State` and the owner table, `LLMExhausted.bad_output`, the Caveat list.
- **Runtime:** `normalise_skills` in the edges and owners, `ExtractBatch` in `ALLOWED_MSGPACK`, the `fan_out` hand-on.
- **HLD:** §3 row E, §4's diagram, §5, §11's sequential extraction, §12's Analysis row.
- **Collection, Gateway:** mark their Analysis hand-ons as settled.
- **Glossary:** Extraction, and advertised vs estimated pay.
- **future-scope.md:** an Analysis section.

Answer

docs/lld/analysis.md is written, with one build checklist, since Analysis has nothing to hold back for later passes. The other docs above agree with it, and `future-scope.md` has an Analysis section.

No code changed. `State`, `analysis/types.py`, `LLMExhausted` and `runtime/graph.py` keep their old shapes until step 1 of the checklist.

Decisions made while writing, beyond the tickets:

- **`relevance` can't take the zero path,** since its Sample is at least the target of 20, so it keeps its existing targets. Only `low_data_review` needs `stats`.
- **`.unknown_share` is what Postings with no company count towards,** rather than dropping them from the split.
- **With no Postings in the older window, the trend is `rising`,** because a ratio can't be computed and every Posting being new is what the data shows. Its label still says it counts ads still live.

Unproven until a live Run: that advertised Adzuna pay with an `unknown` period is annual (the fixtures have only estimates), and whether the tight merge prompt holds on roles beyond data engineering, since the probe covered only that role.

analysis-build

analysis-build/issues/01-postings-become-grounded-extractions.md

01: Postings become grounded Extractions

What to build: a Sample fans out into Batches through `Send`, the cheap tier extracts them, and code drops any skill or year that isn't in the Posting's text, so `extractions` holds only grounded facts. A transient failure stops the Run and `resume` reruns only the failed Batch, while bad output skips that Batch. `normalise_skills` and `stats` stay stubs here. Spec: Analysis LLD §2, §3, §4, §5, §6, §11, §13 steps 1-4 and the resume half of step 9.

Blocked by: None (can start immediately). Collection pass 1 is done, so `sample`, `fetches` and `"analyse"` exist.

Status: done

Kernel and Runtime changes:

- [x] `LLMExhausted.bad_output` is set from whether the last failure was bad output
- [x] `State` gains `skill_aliases`, owned by Analysis, defaulting to empty
- [x] `ExtractBatch` is in `ALLOWED_MSGPACK`
- [x] `normalise_skills` is in `EDGES` and `OWNERS` with a stub node, and runs between `batch_extract` and `stats`
- [x] `analysis.fan_out` replaces `stub_fan_out`
- [x] `analysis/types.py` holds the §3 types, every LLM field required with no defaults
- [x] Test: `Extraction` and `ExtractBatch` survive the Runtime serde test under strict msgpack
- [x] Test: `bad_output` is right for each kind of failure

`_batches` and `fan_out`:

- [x] Batches hold at most 20 Postings or 20,000 description characters, sorted by key
- [x] An empty Sample routes to `"stats"`, otherwise one `Send` per Batch
- [x] Test: 45 Batches for 900 Adzuna-sized Postings
- [x] Test: the count limit, the character limit with USAJOBS-sized text, and one oversized Posting
- [x] Test: the same order whatever the input order

Grounding:

- [x] A skill whose folded name isn't a whole-word match in the folded title and description is dropped and counted in `ungrounded`
- [x] A `years_min` over 40, or whose number isn't in the text, becomes null

- [x] Test: `C` isn't grounded by `C++`, and `R` isn't found inside a word
- [x] Test: an invented skill is counted in `ungrounded`
- [x] Test: `years_min` of 5 passes on "5+ years", fails when absent and fails at 41

`batch_extract`:

- [x] Uses `tiers.structured("cheap", BatchFacts, system=EXTRACT_SYSTEM, name="batch_extract", max_tokens=EXTRACT_MAX_TOKENS)`
- [x] The input is one §6 block per Posting, with `ref` as its position, and no Hiring Request
- [x] `EXTRACT_SYSTEM` in `prompts.py` follows every §11 rule, starting from the probed wording in the Prompts design ticket
- [x] The branch writes only `extractions` and never logs
- [x] Test with fake chains: refs map to the right keys, and a repeat and an out-of-range ref are ignored
- [x] Test: bad output returns `{}`, and a transient `LLMExhausted` raises

Resume:

- [x] Test with a checkpointer and other stages stubbed: one Batch raising transiently stops the Run, `resume` reruns only that Batch, and `normalise_skills` runs once

02: Skill spellings merge into canonical names

What to build: after the fan-in, a real `normalise_skills` node groups spellings of the same skill into `skill_aliases` and lists every merge in the Run log, so a reader can catch a bad one. When the model gives bad output, folding alone still gives true counts, and a Caveat says so. Spec: Analysis LLD §7, §10, §11, §13 step 5.

Blocked by: 01: Postings become grounded Extractions.

Status: done

- [x] Uses `tiers.structured("cheap", SkillGroups, system=MERGE_SYSTEM, name="normalise_skills", max_tokens=MERGE_MAX_TOKENS)`
- [x] Folds every skill name across `extractions`, counts them, and sends the top `MAX_SKILLS_TO_MERGE` as sorted lines
- [x] Writes each variant's folded name → `canonical`, for variants in the input only, and the first group wins when a name repeats
- [x] A variant equal to its canonical name is skipped
- [x] A `note` lists every merge, like "Spark → Apache Spark"
- [x] `MERGE_SYSTEM` in `prompts.py` follows every §11 rule, starting from the probed wording in the Prompts design ticket
- [x] Transient `LLMExhausted` propagates
- [x] Test: no names means no call, and it writes `{}` plus its log entry
- [x] Test: a variant not in the input is ignored, and the first group wins
- [x] Test: the merge `note` is written
- [x] Test: bad output leaves aliases empty and raises the "Skills merged by spelling only" Caveat, and good output doesn't raise it

03: Demand, skills, competition and title Stats

What to build: a real `stats` node turns Extractions into `MarketStats`, with Evidence on every Stat, so every number traces to the Postings it counted. Python computes everything, and two runs on the same state give identical Stats. Pay comes in the next ticket. Spec: Analysis LLD §8 (all but pay), §10, §13 step 7, the matching part of step 8 and the Evidence half of step 9.

Blocked by: 01: Postings become grounded Extractions. Tests set `skill_aliases` by hand, so this doesn't need 02.

Status: done

General rules:

- [x] `compute_stats(state)` returns the Stats, the Caveat messages and the summary data, and the node writes them to `stats` and `run_log`
- [x] Ids, Minimums and omissions follow the §8 catalogue for the demand, skills, competition and titles_seniority rows
- [x] "Now" is the newest `fetched_at`, never the clock
- [x] Stats from Extractions use the Sample keys with an Extraction as their denominator, and their labels say so
- [x] Shares carry their percentage in the label and round to 3 places
- [x] A Sample of zero gives `MarketStats(sample_size=0, stats=[])`
- [x] Test: "now" comes from `fetched_at`, and two runs give identical Stats
- [x] Test: every Stat's Evidence is within the Sample

Skills:

- [x] Names resolve through `skill_aliases`, with `must` winning over `nice` over `unstated`, and a skill with no alias shows in its most common spelling
- [x] Ranked by distinct employers, then Postings, then name, and a Posting with no company counts as its own employer
- [x] The label shows both counts
- [x] Test: an alias and its canonical name in one Posting count once
- [x] Test: skills rank by employers before Postings

Employers and titles:

- [x] Employers group on the folded name without trailing legal suffixes, and Postings with no company count only towards `.unknown_share`
- [x] Employer type is decided per employer in the §8 order, using `KNOWN_AGENCIES`
- [x] Titles group by folded title only

- [x] Test: a Booz-style split resolves by majority, and a tie is `unknown`
- [x] Test: Randstad is an agency with any model answer, and USAJOBS is direct

Trend:

- [x] Compares the newest 14 days against the 14 before that, and the label carries the live-ads wording
- [x] Test: the minimums (`max_days_old >= 28`, 20 Postings across both windows) and the thresholds (1.25×, 0.8×, and `rising` with an empty older window)

Caveats and the Run log, each Caveat with a trigger and a non-trigger test:

- [x] Postings not extracted: any Sample key has no Extraction, with the count
- [x] Skills from snippets: any Adzuna Posting in the Sample, with the count
- [x] One `note` holds the Sample size, Extractions, Stats produced and skills dropped as `ungrounded`

04: Pay Stats

What to build: `MarketStats` gains compensation: advertised, estimated and federal pay as three groups that never mix, annualised and shown in one currency, with seniority slices and the disclosed and estimated shares. The Caveats say what was estimated, assumed, excluded or converted. Spec: Analysis LLD §8 catalogue compensation rows, §9, §10, §13 step 6 and the rest of step 8.

Blocked by: 03: Demand, skills, competition and title Stats, because this extends `compute_stats`.

Status: done

- [x] `UNITS_PER_USD` holds the 13 Adzuna currencies from ECB reference rates, with `FX_DATE`
- [x] A figure is the midpoint of `min` and `max` or the one bound, annualised with `ANNUAL`, then converted to the Stat currency
- [x] The Stat currency is `hiring_request.currency` when the table has it, otherwise `ADZUNA_CURRENCY[current_plan.country]`
- [x] For an `unknown` period only, a figure below the equivalent of 1,000 USD a year, or below a sixth of its group's median, is excluded and counted
- [x] Quartiles use `statistics.quantiles(figures, n=4, method="inclusive")`, and pay rounds to whole units
- [x] `unit` is like `"USD/year"`, and labels name the group, the count and any conversion
- [x] Seniority slices exist for `advertised` and `estimated` only
- [x] `comp.disclosed_share` counts `advertised` over Adzuna Postings, and `comp.estimated_share` is the same over `estimated`

Tests:

- [x] Each period annualises
- [x] The three groups never mix
- [x] A figure of 25 with an `unknown` period is excluded
- [x] A monthly figure is excluded by the sixth rule, and a real junior figure isn't
- [x] Conversion and the currency fallback
- [x] 4 figures give no Stats, and 5 do
- [x] The seniority slices, with the same Minimum per level

Caveats, each with a trigger and a non-trigger test:

- [x] Pay is estimated: `estimated` Stats exist and `advertised` ones don't
- [x] Too little pay data: no group reached 5 figures
- [x] Pay period assumed: any `advertised` figure had an `unknown` period, with the count
- [x] Implausible pay excluded: any figure was excluded, with the count
- [x] Pay converted: the Stat currency differs from a Posting's, with `FX_DATE`

- [x] Currency unavailable: the requested currency isn't in the table

05: Analysis live smoke

What to build: a script runs fixture Postings through the real model and prints what Analysis made of them, which proves the whole stage and shows any bad merge. It spends no job API calls. Spec: Analysis LLD §7, §11, §13 step 10.

Blocked by: 02: Skill spellings merge into canonical names, 03: Demand, skills, competition and title Stats, and 04: Pay Stats.

Status: done

- [x] `scripts/smoke_analysis.py` (not in CI) runs fixture Postings through `minimax/minimax-m3.1-flash`
- [x] It prints the top skills, every merge and the employer types
- [x] It asserts that `AWS` and `S3`, and `Git` and `GitLab`, stay apart when both appear
- [x] Results and any change to the prompts are recorded in this ticket for the write-up

Results

Run on 2026-10-31 with `minimax/minimax-m3.1-flash` on both tiers, replay off, over `tests/fixtures/jobs/` (100 Adzuna, 25 USAJOBS, no job API calls). `python scripts/smoke_analysis.py [model]` takes about 15 s: 10 Batches, 125 Postings, 63 to 77 skills dropped as ungrounded per run. Extraction was 119, 125, 125 and 120 of 125 Postings across the runs. The script now prints any Batch that returns nothing; the 120 run printed none, so the model skipped Postings inside Batches that did return, and the 119 run's cause was not captured.

- **Merge prompt over-merged.** With the LLD's wording, the cap of 300 names gave 79 merges, many wrong: `t sql` and `pl sql` into `sql`, `aws` into `aws cloud services`, `flight dynamics` and `statics` into one, `llms` into `generative pretrained transformers gpts`, `engineer in training eit` into `fundamentals of engineering fe`. The earlier probe (166 names, 12 merges) did not show this. This run sent a longer list and a wider fixture set (including USAJOBS engineering Postings); which of those, or model variance, caused it was not isolated.
- **Prompt change** (`MERGE_SYSTEM` in `analysis/prompts.py`): added "Most names belong to no group, so expect only a handful of groups" and named three failing shapes ("pl sql" and "sql", "flight dynamics" and "statics", "aws" and "aws cloud services") as not spellings of each other. Two of the examples repeat the SQL and AWS rule above and one is specific to the USAJOBS fixtures, so a run without the examples was tried: it gave 52 merges, including `t sql → sql` and `secret → top secret sci clearance`. The examples stay. The next run gave 17 merges, mostly correct (`airflow`, `spark`, `azure`, `glue`, `etl`, `nlp`). This is one run per wording, so the improvement is a direction, not a measured rate.

- **Assertions pass** on the last run after the change (the output of the first post-change run was not read): `AWS` and `S3` stay apart, `Git` and `GitLab` stay apart (both pairs appear in this Sample; on the first run `Git` and `GitLab` did not both appear, so that check was skipped).
- **Employer types** came out at about 50% direct and 49% agency, with 2 Postings of unknown type. Not checked against the individual employers here.
- **Reciprocal and chained aliases** (`data analytics → analytics` with `analytics → data analytics`) left a skill split under two keys. `_aliases` now aims each group at the end of any chain, so neither can form; one test covers it.
- **Remaining weak spot:** `spark pyspark → pyspark` still gets through.
- **Step 11** (is an `unknown` pay period on advertised Adzuna pay annual) was unproven here because the fixtures have only estimated pay; ticket 06 checked it on a live Run and it is annual.

06: Check the advertised pay period assumption

What to build: on a live Run with real advertised Adzuna pay, confirm that an `unknown` period really is annual. The fixtures only hold estimates, so this is unproven until then, and the "Pay period assumed" Caveat states the assumption in the meantime. Spec: Analysis LLD §9, §13 step 11.

Blocked by: 05: Analysis live smoke.

Status: done

- [x] It spends Adzuna calls, so get the user's OK in chat before running it, and it can share a Run with Delivery's sample runs
- [x] Advertised figures with an `unknown` period from a live Run are inspected and recorded in this ticket
- [x] If they aren't annual, the `ANNUAL` rule and the Caveat text change, with a test (they are annual, so no change)

Results

Run on 2026-10-31 with the user's OK for 3 Adzuna calls: one page of 50 each for `data engineer` (us), `warehouse associate` (us, chosen because its pay is mostly hourly) and `software engineer` (gb), country-wide, 30 days. Every `advertised` (non-predicted) salary came back with `period="unknown"`, as the mapper sets it. Adzuna's response carries no period field, so the check is on the magnitudes.

- **Annual, confirmed.** 60 advertised figures in usable ranges, all annual-sized: US data engineers 70,000 to 200,000 USD; GB software engineers 45,000 to 120,000 GBP.
- **Hourly pay arrives already annualised.** The 20 US warehouse figures run 20,800 to 66,560, and the common ones are exact multiples of 2,080 (20,800 = 10/h, 31,200 = 15/h, 41,600 = 20/h, 52,000 = 25/h). So Adzuna converts hourly to annual itself, and `ANNUAL[unknown] = 1` is right. Applying the hourly factor again would have been wrong.
- **Junk does appear, and the existing rule catches it.** Four GB postings had a lone `max` of 5.0 or 60.0 (an hourly or day figure that was not annualised). Against GB medians of about 65,000 they fall below both the 1,000 USD floor and a sixth of the group median, so step 4 excludes them and the "Implausible pay excluded" Caveat counts them. Not run through `stats` live, only checked against the rule by reading it.
- **Outliers kept:** `348000` USD for a cleared role (its title says `$348k/yr`) and a GBP 208,000 to 286,000 director are real, so nothing to change.
- **Changes:** none to `ANNUAL` or the Caveat text. The LLD §9 "unproven" note now says it was checked. The Caveat still reads "treated as annual", which is now an observed fact rather than a guess, but it stays because it tells a reader the period is inferred, not stated.

- **Not covered:** one page per query, two countries, and Adzuna's other 11 countries were not sampled. The Adzuna budget for the day is down to 1 call.

brief-lld

brief-lld/map.md

Brief LLD

Label: wayfinder:map Parent: Talent Market Scout: HLD (component F in docs/HLD.md §3, §4)

Destination

`docs/lld/brief.md`: an LLD of Brief precise enough to build straight from, covering `write_brief`, `critic`, `render` and building the Run Notes, with nothing left to decide. Building the code is a separate effort after this map.

Notes

- Domain: Brief turns Stats, Caveats and the run's slices into a Market Brief, as JSON and Markdown. The LLM chooses and phrases Insights but never invents figures. Vocabulary: `GLOSSARY.md`. Spec: `Joveo_Talent_Market_Agent- Assessment.pdf` (Step 6). Deadline Mon 2 Nov 2026, 9:00 AM, so this map stays lean.
- Fixed upstream: docs/lld/kernel.md §3 (Stat ids expand into Evidence, LLM-schema policy), §4, §6, §7 (critic routing), §9 (every Caveat in the Run Notes), §12 (Tiers), §14 (replay banner), docs/lld/analysis.md §8, §9, §10, §14, docs/lld/collection.md §11, §15, docs/lld/request-plan.md §8, docs/lld/runtime.md §3, §4, §10, HLD §5, §6.
- Skills: `langchain-router` first, then what it points to (per `CLAUDE.md`).
- The user resolved the first two `grilling` tickets live, then on 2026-10-31 told the agent to make every remaining decision itself and finish the map. So tickets 04 to 11 keep their `grilling` type but were decided by the agent, and each says so.
- Live calls use `minimax/minimax-m3.1-flash` and `typesafe/jev-1.14`, spending no Adzuna quota.
- Standing preferences: inherited from the Kernel map (Pydantic v2 everywhere, `uv`, Python 3.12, `src/talent_scout/`, `pytest`, root `.env`).

Decisions so far

<!-- one line per closed ticket -->

- Live probe of the writer and critic: the writer cites real Stats, but leaves recommendations empty, varies its outlook on identical input and can run away at zero Postings, while a strict critic rejects accurate Insights and loosened wording fixes much of that.

- The Market Brief schema: the LLM drafts only the Summary and 1 to 8 Insights with their Stat ids, and `render` builds the typed `MarketBrief` in code, with Evidence as the union of cited Stats' Postings and one new `State` field, `brief_draft`.
- Confidence Tag rules: code rates each cited Stat (High at 30+ Postings from 10+ employers, Medium at 10+ from 5+), an Insight takes its weakest Stat, estimated pay, snippet skills and old data cap it at Medium, and only Insights are tagged.
- Live check of the draft design: a strict figure and coverage check exhausted 7 of 8 runs, while the adopted design got 8 of 8 valid drafts and 51 of 54 Insights confirmed, and a runaway can happen with Stats too.
- Hiring Outlook rules: code rates it from employers, how demanding the profile is and the trend, says it's a proxy because Postings don't show candidates, and defaults to a stated Competitive under 20 Postings.
- Client Recommendations: code picks the title by employers, the band as the middle 50% of advertised then estimated pay at the request's level, and the top 5 skills, each `None` when it lacks evidence.
- What focus changes in the Brief: `focus` sorts Insights and steers the writer's emphasis only, and code adds an Insight for any Dimension left uncovered.
- Thin and empty Samples: zero Postings makes no LLM or Jev call and gives a code-built Brief, and under 10 Postings caps Insights at 3 with no coverage rule.
- Writer failures and bad citations: a per-Run validator retries unknown ids and invented figures through Kernel's feedback retry, transient failure stops the Run, and bad output from every model gives a rule-written draft with a Caveat.
- The critic: Jev judges each Insight against its cited Stats at 0.5, up to 2 revisions replace only flagged Insights, and what's unconfirmed at the cap ships capped at Medium with a Caveat.
- render and the Run Notes: Markdown leads with answers, each part points to an Evidence list by reference, and Run Notes come from the slices, with a banner on replay.
- Brief prompts: the LLD fixes the writer's input order and rules, values go in formatted, wording stays in `brief/prompts.py`, and two Analysis Caveats get reworded.
- Write docs/lld/brief.md: the LLD is written and the upstream docs and glossary are in line with it.

Not yet specified

Nothing: the destination is reached.

Out of scope

- Writing the Brief code: separate build effort after this map.
- The Supervisor stretch goal: ruled out in HLD §1.
- How long-term memory preferences reach the Brief: Long-term memory LLD.
- Choosing the three sample runs, the `samples/` transcripts and the write-up: Delivery.

Live probe of the writer and critic

Type: task Status: resolved Assignee: smayankulkarni05 Blocked by: none

Question

Analysis code isn't built, so what do a throwaway script and hand-built Stats from the 100 Adzuna and 25 USAJOBS fixture Postings show?

- Does the writer cite only real Stat ids, and does it keep figures in its prose faithful to them?
- Does Jev separate supported Insights from deliberately unsupported ones, and with what scores?
- Does a critic request stay under Jev's 32 KB limit, given the Evidence it has to carry?
- What output token limit does the writer need?

This gathers facts for the decisions below and decides nothing. It spends no Adzuna quota, and the script isn't kept.

Answer

Run on 2026-10-31 with `minimax/minimax-m3.1-flash` and `typesafe/jev-1.14`, for about $0.09 of OpenRouter credit and no job API calls.

Setup and its limits

- 46 hand-built Stats in Analysis's id and label shape, from one Adzuna fixture of 50 Postings. Skills came from a fixed word list, not real Extractions. USAJOBS wasn't used, so federal pay is untested.
- A rough draft schema and a hand-written prompt, not a design. Nothing here fixes either.
- 4 full runs and 1 at `max_tokens=2048`, 3 thin (8 Postings) and 2 zero through `Tiers`, then 5 raw zero calls.
- The prompt said Washington DC, but the fixture is mostly Manhattan and New Jersey. The thin Caveat also said 50 Postings when the Sample had 8, and the writer repeated that. Neither affects what was measured.
- The number check ignores single digits, so it can miss small wrong counts. Treat the figure result as strong, not proven.

The writer

Question	Result
Invented Stat ids	0 across all 8 successful runs
Figures in no Stat	None. The only ones flagged were 50 and 500, both from the prompt's Caveats
Figures from a Stat it didn't cite	1 run, where a summary landed in `job_title.value`
Parts with no citation at all	0 to 2 per run
Time and size with Stats	5 to 11 s, 1,215 to 2,476 output tokens, no reasoning tokens
Token limit	2,048 worked for the full run at 1,523, but a thin run reached 2,476, so 2,048 is too tight

Three failures the schema let through:

- **Empty or wrong recommendations.** In the 4 full runs at 8,192 tokens, `job_title.value` was empty in 3 and held a summary in the fourth. `salary_band.value` was empty in 2. A plain `str` validates all of these. The prompt's "leave lists empty if thin" rule may be leaking into full runs, but this wasn't isolated.
- **Free-text fields.** `dimension` came back as "Demand" and "Title and Seniority", and once as "data_availability".
- **`value` holds prose.** Bands appeared as "$133,676 - $203,199", "133,676-203,199" and a sentence. Titles and bands can come from code instead.

The outlook isn't stable. On the same input the full runs gave competitive 4 times and hard to fill once. The thin runs gave favourable once and competitive twice. One raw zero-Posting call answered **favourable with no Postings at all**. Reasons cite employer spread and recency in both directions.

Thin Samples don't change the writer's behaviour. With 8 Postings it wrote the maximum of 8 Insights, a salary band, and up to 9 top skills, despite the prompt's rule. Figures stayed honest, but the volume didn't shrink.

Zero Postings

Both `Tiers` calls failed, after 164 and 464 s, using 25K to 52K tokens each and about $0.03.

The cause is a whitespace runaway. In the raw repeat, 1 of 5 calls ran to `finish_reason=length` at 8,192 tokens, with 43,564 characters that were 99% whitespace and invalid JSON. The other 4 finished in 156 to 454 tokens. No call with Stats did this.

The feedback retry makes it worse, because it resends the whole bad output as input. This is a Kernel `Tiers` behaviour, so any strict-schema call that runs away pays the same way.

The critic

Scores are Jev's `p_yes`, and the cut is 0.5. The states and questions below were tried on 12 labelled claims and the 6 Insights from one real draft.

State and question	Exact claims passed	Bad claims rejected	Real writer Insights passed
Stats only, strict wording	3 of 3 (0.83 to 0.98)	7 of 8	1 of 6
Stats + 20 Postings, strict	3 of 3	7 of 8	0 of 6
Stats + all Postings, strict	3 of 3	7 of 8	0 of 6
Stats + hiring request, loosened wording	3 of 3 (0.92 to 0.97)	**8 of 8** (0.01 to 0.07)	3 of 6

- **A strict critic rejects accurate writing.** The real Insights were accurate, yet scored 0.04 to 0.64. Used as written, the revise loop would mostly spin to its cap.
- **Wording mattered most.** The loosened question names the request as a source, allows descriptive words, and names estimated-as-advertised as false. That last case passed the strict wording at 0.65 and scored 0.07 after.
- **Long claims score lower.** Two single-figure claims scored 0.94 and 0.89. The real Insights stating many figures scored 0.16 and 0.17, even loosened. They also add judgement such as "can add staffing flexibility". Which of the two lowers the score wasn't separated.
- **Postings don't help.** Adding them changed no outcome on the labelled claims, and lowered real Insights' scores.
- **My "paraphrase" label was doubtful.** "Python is the most widely requested skill" cited only Python's Stat, which can't show rank, so it's left out of the table's counts. It scored 0.06 strict and 0.42 loose.
- **Cuts remain untuned.** These 20 claims show direction and aren't a basis for fixing a threshold.

Size: Stats only tops out at 2 KB per Insight. All Stats and all Insights in one state is 8.9 KB. With all Postings a request reaches 30.5 KB, and the largest was truncated, so Jev's limit only matters if Postings are sent. Calls took 1 to 3 s in total.

What the open tickets should weigh

Facts only. The decisions remain the user's.

- The Market Brief schema: empty recommendations, free-text `dimension`, and prose in `value`.
- Confidence Tag rules and Thin and empty Samples: the writer doesn't adjust to thin evidence on its own.
- Hiring Outlook rules: an LLM-chosen outlook varied on identical input and said favourable at zero.
- Thin and empty Samples: an LLM call at zero is slow, failure-prone and has nothing to say.
- Writer failures and bad citations: citations held up, while whitespace runaway and the size of the feedback retry didn't.

- The critic: Stats-only state, loosened wording, and claim length.
- Brief prompts: the first prompt's outcomes above are the write-up's "what changed after testing" starting point.

The Market Brief schema

Type: grilling Status: resolved Assignee: smayankulkarni05 Blocked by: 01

Question

What are `MarketBrief` and its parts, which fields does the LLM fill in its draft, and which does code fill, including how cited Stat ids expand into Evidence? How many Insights are allowed, and what new `State` fields does Brief add?

Answer

Decided by grilling. Types live in `brief/types.py`.

```
class BriefDraft(Model):                # all the LLM fills, for now
    summary: str
    summary_stat_ids: list[str]         # at least 1
    insights: list[InsightDraft]        # 1 to 8; each is text + stat_ids (at least 1)

class MarketBrief(Model):
    hiring_request: HiringRequest
    replayed: bool
    summary: str
    summary_stat_ids: list[str]
    insights: list[Insight]             # text, dimension, stat_ids, evidence, confidence
    recommendations: ClientRecommendations
    outlook: HiringOutlook              # rating, reason, stat_ids, evidence
    run_notes: RunNotes
    stats: MarketStats                  # whole, so every cited id can be checked

class ClientRecommendations(Model):     # each has reason, stat_ids, evidence
    job_title: TitleRecommendation | None    # + title
    salary_band: BandRecommendation | None   # + low, high, unit, copied from Stats by code
    top_skills: SkillsRecommendation | None  # + skills

class RunNotes(Model):
    questions: list[ClarifyingQuestion]
    assumptions: list[Assumption]
    final_plan: SearchPlan              # state.current_plan
    earlier_plans: list[SearchPlan]
    postings_analysed: Evidence         # the Sample
    caveats: list[str]                  # messages only: state.py imports this file, so no
RunLogEntry
```

- `render` **alone turns the draft into the Brief,** and so alone expands Evidence. `critic` reads the draft and Stats directly.
- **Evidence** is the union of the cited Stats' Postings: what a part rests on, not what's behind each figure. A share never gives a count of 0: `comp.*_share` Evidence is its denominator, so 0% disclosed still rests on every Adzuna Posting, and other shares are omitted when empty (corrected from the real Stats while resolving Confidence Tag rules).
- **`dimension`** is the first cited Stat's, so the LLM has no field to get wrong.
- **`hiring_request`** is there because broadening can change what the plan implies. **`replayed`** shows a replay in `brief.json` alone. There's no date, since a clock would make replayed Briefs differ.
- **"2 to 3 sentences"** stays in the prompt, since a validator would retry over abbreviations. The final `insights` has no minimum.
- **`State`** gains only `brief_draft: BriefDraft | None`, replaced by `write_brief` on each revision.

Left to later tickets:

- Confidence Tag rules, Hiring Outlook rules and Client recommendations add a draft field only if the LLM should write that part.
- Thin and empty Samples: `outlook` is required even at zero Postings, a zero-Posting Run must avoid a draft with no Insights, and thin runs may tighten the cap of 8.
- The critic names its own `State` fields.
- Render and Run Notes adds the `total` disclosure and places the replay banner.
- Write the Brief LLD doc: required fields break the `MarketBrief()` stub in `runtime/testing.py`.

Confidence Tag rules

Type: grilling Status: resolved Assignee: smayankulkarni05 Blocked by: 02

Question

How is an Insight tagged High, Medium or Low: computed by code, set by the LLM, or an LLM tag capped by code? What thresholds apply, and how do they answer the spec's "you set and justify the thresholds"? How do Caveats, Sample size, and estimated or snippet-based Evidence affect the tag?

Answer

Decided by grilling. **Code computes the tag in `render`,** and the LLM has no field for it. The probe's writer didn't adjust to thin evidence, and a rule is what lets the thresholds be stated and justified.

Each cited Stat is rated on its own Evidence:

Rating	Postings	Distinct employers
High	30 or more	10 or more
Medium	10 or more	5 or more
Low	anything else	

- **Employers** use the grouping in `stats`, and a Posting with no company counts as its own employer. `competition.employer.*` Stats are exempt from the employer minimum, since they cover one employer by construction.
- **Counts are absolute,** not shares of the Sample: the spec says "many postings", and 8 of 8 is still thin. Sample size therefore needs no rule of its own. A Run at the minimum target of 20 Postings can't produce a High.

The Insight takes the lowest rating among its cited Stats, because it's only as strong as its thinnest figure. Evidence stays the union (see The Market Brief schema), so citing `demand.sample_size` beside a 6-Posting skill can't inflate the tag. A broad Insight citing many Stats tags low, and the prompt can nudge the writer to split it.

Three things then cap it at Medium, worked out from the same facts that raise the Caveats, never by matching Caveat text:

1. It cites `comp.estimated.*`, which is Adzuna's guess.
2. It cites `skills.*` and any Adzuna Posting is in the Sample, since 500-character snippets are partial evidence.

3. The Old data Caveat fired, which applies to every Stat.

Other Caveats only appear in the Run Notes: Too few Postings (the counts show it), Mostly federal (federal pay is kept apart), Pay converted (a small effect) and Pay period assumed (held on the live Run for us and gb).

Why these numbers: 30 is a common rule of thumb for when a median stops moving much with each new Posting. Below 10, quartiles and shares move a lot with each Posting. The employer minimums stop one company's repeated ads counting as a market pattern, which is what "consistent" means here: the AI skill has 13 Postings from 4 employers (Low), and SQL has 14 from 12 (Medium). Pay spread was rejected as a second rule covering one Dimension. These numbers are judgement checked against one role's fixture, not tuned across roles.

Checked on the 125 fixture Postings (real `stats` code, 2026-10-31, shown as Postings/employers):

Result	Stats
High	`competition.direct_share` 68/42, `titles.variant.data-engineer` 35/19, `comp.disclosed_share` 100/49
Medium, by count	`comp.federal.median` 25/12, `skills.sql.count` 14/12, `skills.python.count` 18/17, Booz Allen's employer count 23/1 (exempt)
Medium, capped from High	`comp.estimated.median` 100/49
Low	`skills.artificial-intelligence.count` 13/4, `skills.pyspark.count` 8/8, `comp.estimated.lead.median` 5/5

In practice, a Run with only estimated pay never gets a High pay Insight, and an Adzuna Run never gets a High skills Insight.

`Insight` gains `confidence_reason: str`, written by code and absent from `BriefDraft`, such as "Medium: weakest cited Stat has 18 Postings from 17 employers; skills come from snippets". Without it, Evidence of 125 Postings beside a Low tag would go unexplained.

The rationale lives in `docs/lld/brief.md`, and the Markdown Brief carries a two or three line key stating the rule. `brief.json` has no key, since `confidence_reason` explains each tag there.

Only Insights are tagged. Client Recommendations and the Hiring Outlook stay untagged, as the spec and schema have them.

Left to later tickets:

- Client recommendations: with no tag, a salary band's `reason` has to say when pay is estimated.
- Brief prompts: whether the writer is told this rule, so it phrases claims to match and splits broad Insights.
- Render and Run Notes: places the key in the Markdown.

- Write the Brief LLD doc: Analysis §14 expects a recommendation's pay to use "`estimated` at a lower tag", which no longer holds, so it needs correcting in the upstream sweep.

Hiring Outlook rules

Type: grilling Status: resolved Assignee: smayankulkarni05 Blocked by: 01

Question

How is exactly one of Favourable, Competitive or Hard to fill chosen, and who writes its one-line reason? The spec defines the values by talent relative to demand, but Postings only measure demand, so which Stats stand in for supply, and how does the Brief say so honestly? What happens when the Sample is too thin to support any of the three?

Answer

Decided by the agent, on the user's instruction to make every decision on this map. **Code picks the rating and writes the reason,** and `BriefDraft` gains no field. The first probe's LLM gave different ratings on identical input, and said favourable with no Postings.

Nothing stands in for supply. Postings don't show candidates, so the rule reads demand pressure and how demanding the profile is, and every reason says so. First match wins:

#	Condition	Rating
1	Under 20 Postings	Competitive, stated as a default and not a finding
2	20+ employers, a high bar, and the trend isn't falling	Hard to fill
3	Under 10 employers, a known bar that isn't high, and the trend isn't rising	Favourable
4	Otherwise	Competitive

- **Employers** is `competition.distinct_employers`. Many employers is what the spec calls "many employers chasing the same profile".
- **A high bar** is any of: the request's own level is senior, lead or executive; `skills.years_median` is 5 or more; or `seniority.senior`, `.lead` and `.executive` sum to 50% or more. The bar is **known** when the request states a level, or either kind of Stat exists. With neither, the rule can't call the talent plentiful.
- **The trend only blocks.** It can stop Hard to fill or Favourable, but never earns either, because Analysis says it tilts recent.

- **20 Postings** is the lowest target a Search Plan can have, so anything below it is weak by any plan's standard.
- **Pay isn't used.** Nothing in the Stats says what standard pay is.

The reason is one line, like: "61 employers across 125 Postings, with a median of 4 years asked and a rising trend. Postings show demand, not candidates, so this is an estimate of the talent market." The default reads: "Only 8 Postings, too few to judge the talent market, so Competitive is a default rather than a finding."

The enum stays at three values. The spec says exactly one, and `outlook` is required. A fourth "not rated" would be more exact but breaks that, so the honesty lives in the reason.

The writer sees a rule-based rating so the Summary can't contradict it, and never sees the default, since Live check of the draft design showed it repeats a default as a finding. The rule is one pure function called by both `write_brief` and `render`, with no new `State` field.

Evidence is the union of the Stats that decided it. A default has none, which `stat_ids=[]` makes plain.

Checked on the fixtures in that same ticket: Competitive at 125, Hard to fill at the Adzuna 100, whose median is one year higher, and a default at 8. These are judgement calls, not tuned, and Favourable will be rare: since Postings can't see plenty of talent, the rule leans to Competitive. That belongs in the write-up's failure modes.

Client Recommendations

Type: grilling Status: resolved Assignee: smayankulkarni05 Blocked by: 02

Question

How are the suggested job title, salary band and top skills chosen, and how is each tied to Evidence? Which pay group may set the band (Analysis expects `advertised`, then `estimated` at a lower tag, and never `federal` alone for a private-sector role), which seniority slice fits the request, and what is recommended when no pay group has enough figures?

Answer

Decided by the agent, on the user's instruction. **Code chooses all three and writes each reason,** so `BriefDraft` gains nothing. The first probe's LLM left these empty or put prose in them, and each is a pick from Stats.

The request's level comes from words in `hiring_request.seniority`, first match: junior, entry, graduate, intern → `entry`; mid → `mid`; senior, sr → `senior`; lead, principal, staff → `lead`; director, head, vp, chief, executive → `executive`. Anything else, such as "5+ years", is no level.

Job title:

- Candidates are `titles.variant.*` with at least 3 Postings from at least 2 employers, so one company's repeated ad can't win.
- Ranked by distinct employers, then Postings, then id, as Analysis ranks skills.
- With a request level, a candidate whose title has one of that level's words wins over one without.
- The name is the Stat's label before its last ": ", which a test pins, since `Stat` has no name field.

Salary band:

- **Group:** `advertised`, then `estimated`. `federal` only if over half the Sample is USAJOBS, the same fact as the Mostly federal Caveat, since no field says whether a role is private-sector.
- **Level:** within the first group with Stats, that level's slice if it exists, otherwise the whole group, with the reason saying "all levels". A stated salary at the wrong level beats a guess at the right one: it's the group order Analysis expected, and the reason discloses it.
- **Band:** `.p25` to `.p75`, the spec's "middle 50%". The cited Stats are `.p25`, `.median` and `.p75`.
- **Tags:** none. Analysis's "`estimated` at a lower tag" no longer holds, since Confidence Tag rules tags only Insights. So the reason has to say it: "Middle 50% of 100 Adzuna-estimated figures, at senior level. These are Adzuna's estimates, not pay employers stated."

Top skills: the first 5 `skills.*.count` Stats in Stat order, which is already by employers, with at least 3 employers each. The reason gives Posting counts, and says skills come from snippets when any Adzuna Posting is in the Sample.

No evidence means `None`, for each of the three alone. Markdown then says "Not enough evidence to suggest a job title". A plan's title or an LLM's guess would have no Evidence, and the spec requires each be tied to it. When no pay group has Stats, the band is `None` and Analysis's Too little pay data Caveat explains it.

Evidence on each is the union of its cited Stats' Postings, expanded by `render`.

`focus` **changes none of this:** a client gets all three whatever the emphasis.

What focus changes in the Brief

Type: grilling Status: resolved Assignee: smayankulkarni05 Blocked by: 02

Question

Request & Plan and Analysis leave `focus` to Brief. What does `--focus` change in the Market Brief: the order of Insights, how many cover that Dimension, the Summary, or the recommendations? How does it reach the writer, and what guarantees the other Dimensions still appear?

Answer

Decided by the agent, on the user's instruction. `focus` changes emphasis only:

Part	With `focus`
Insight order	`render` moves Insights of the focused Dimensions first, keeping the writer's order otherwise. This is code, so it always holds
Insight count	The writer is asked to give the focus most Insights. Prompt only: a count isn't worth a retry
Summary	The writer is asked to lead with it. Prompt only
Recommendations, Outlook, Confidence Tags, Stats	Nothing
Run Notes	`focus` is listed, so a reader knows why the Brief leans that way

It reaches the writer as one line of the input, such as `Focus: compensation`, and all Stats are still sent. An empty `focus` says `Focus: none`.

The guarantee is code, not a retry. After the critic, `render` adds one Insight for every Dimension with Stats that no Insight cites, stating that Dimension's first Stat label as written, cited to that Stat. Stat order means it's the top item: the sample size, the first pay median, the top skill, the top employer, the top title. Live check of the draft design found that a coverage error fed back didn't fix the draft, and 4 of 6 non-thin runs left a gap, so a retry would spend money and end in the rule-written Brief. This applies with or without `focus`, because the spec's five dimensions are the Brief's job either way. Thin and empty Samples sets when it doesn't apply.

Thin and empty Samples

Type: grilling Status: resolved Assignee: smayankulkarni05 Blocked by: 03, 04, 05

Question

What does the Market Brief say when a Run continues at `low_data_review` with a weak Sample, or with zero Postings and `MarketStats(sample_size=0, stats=[])`? Does `write_brief` still call the LLM at zero, and does `critic` run? What fills the required parts, such as the outlook and recommendations, when no Evidence exists?

Answer

Decided by the agent, on the user's instruction.

Zero Postings: no LLM and no Jev. The first probe's zero-Posting calls took up to 464 s, ran away, and said favourable about nothing, and there's nothing to say.

Part	At zero
`write_brief`	Writes a code-built draft with no call: "No Relevant Postings were found for {role} in {location}, so this Brief has no market findings." `summary_stat_ids` and `insights` are empty
`critic`	Runs as a node, judges nothing and routes to `render`. No special route, because its list of unjudged Insights is empty
Insights	`[]`. This resolves the question left by The Market Brief schema: the final Brief has no minimum, and the draft's minimum applies only to what the LLM returns
Outlook	Competitive, by default, with the reason saying so and `stat_ids=[]`
Recommendations	All three `None`
Run Notes	Whole, since they matter most here: the plans tried, and Too few Postings and any other Caveats

Weak Samples: the writer runs, with two regimes, because the probe showed it doesn't shrink its output by itself:

Sample	At most	Every Dimension covered
1 to 9 Postings	3 Insights	No, unless no Insight survives

Sample	At most	Every Dimension covered
10 or more	8 Insights	Yes

Below 10 Postings, no Stat can rate above Low, and filling all five Dimensions is padding. The cap goes into the input and the check, and 2 thin runs obeyed it.

The rest needs no thin rule, since it already follows from the others: tags from Confidence Tag rules, a default outlook below 20 Postings from Hiring Outlook rules, and `None` where Client Recommendations minimums fail.

One line sits below the Summary in Markdown when the Sample is under `current_plan.target_postings`: "Weak Sample: 8 Postings against a target of 30. Treat this Brief as directional." It's code, so it holds whatever the writer says, and the Too few Postings Caveat holds the detail in the Run Notes.

Writer failures and bad citations

Type: grilling Status: resolved Assignee: smayankulkarni05 Blocked by: 01, 02

Question

What happens when the draft cites a Stat id that doesn't exist, states a figure in prose that no cited Stat holds, or leaves a required part empty? Which of those are retried with the error fed back, dropped, or raised as a Caveat? What happens on transient failure, and on bad output from every model, given that a Run should never end without a Brief or return garbage?

Answer

Decided by the agent, on the user's instruction.

A per-Run validator makes Kernel's feedback retry do the work. `checked_draft(...)` returns a `BriefDraft` subclass with a `model_validator` that closes over this Run's Stats. `Tiers.structured` feeds its errors back with no new retry code. The node then copies the result into a plain `BriefDraft`, because a checkpoint stores the class path and a local class can't rehydrate. The limits stay out of the JSON schema, which keeps it the same across Runs.

Fault	Result
Unknown Stat id	Retried
Summary or Insight with no Stat ids, or blank	Retried
No Insights, or over the cap	Retried
A figure in no Stat, Caveat or the request	Retried
A figure from a Stat it didn't cite	Passes here. The critic flags it
A Dimension with no Insight	Not checked here. `render` fills it (What focus changes in the Brief)
Transient on every model	Propagates. The Run stops, and `scout resume` reruns only this node (Kernel §12)
Bad output on every model, including a runaway	The rule-written draft below, with a Caveat

The figure check is deliberately loose. Tied to cited Stats, it exhausted a thin run, while Live check of the draft design got 8 of 8 valid with any Stat, and every figure the check rejects is one the Sample doesn't contain. So it catches inventions, and the critic catches wrong attribution.

- Numbers are read with `(?<![\w.])\d[\d,]*(?:\.\d+)?[kKmM]?(?!\w)`, so `S3` isn't a number, and `$134k` is 134,000.
- A number also passes as a whole-number rounding of an allowed one, so 38% matches 38.4%.
- Number words like "five" aren't caught. The critic is the only check on them.

Never end without a Brief, and never return garbage. The rule-written draft is plain code: a Summary giving the Sample size and one Insight per Dimension with Stats, reusing the fill in `render`, so its figures are all true. Bad output would repeat on every resume, as in Analysis's Batch rule. It raises the Caveat "Brief written by rule: every model's draft failed its checks, so Insights state computed figures directly" and sets `revisions` to the cap, so no further LLM call is spent.

Salvaging the valid Insights from a failed draft was rejected, since `LLMExhausted` doesn't carry the output and adding that isn't worth a rare path.

Runaway. The live check saw one at 125 Postings, so `WRITE_MAX_TOKENS = 4096` limits its cost. The largest reply the first probe saw was 2,476 tokens, and that was on a bigger schema. `finish_reason=length` already moves to the next model with no retry (Kernel §12).

The critic

Type: grilling Status: resolved Assignee: smayankulkarni05 Blocked by: 01, 02, 08

Question

What exactly is Jev asked per Insight, with what input and threshold, and does it also check recommendations? What does `write_brief` get back on a revision, what is the revision cap, which fields reset and which decide the route? What ships when the cap is hit with an Insight still unsupported? Keep this small enough that the build can skip the critic without reopening anything.

Answer

Decided by the agent, on the user's instruction.

What's judged: each Insight, one Jev request each, gathered at once. Not:

- **Recommendations and the Outlook,** which code builds from Stats.
- **The Summary,** which scored 0.14 and 0.15 even when accurate, so would force every Run to the cap. The figure check covers its numbers, and it stays short by prompt. That it isn't judged belongs in the write-up.
- **Insights added by `render`,** which are Stat labels as written.

The state, per Live check of the draft design, is `hiring_request`, the Caveat messages, `claim` and `stats` (`id`, `label`, `value`, `unit` of each cited Stat). No Postings: the first probe showed they change nothing and lower scores. That narrows HLD §6's "the Postings it cites", and the requests stay under 2 KB, so Jev's limit never comes up.

The question says yes if each figure and comparison in the claim matches the cited Stats, the request or the Caveats, and that fair descriptive wording and stating only some figures are fine. No if a figure is wrong, a comparison or ranking isn't shown by the cited Stats, or estimated pay is presented as advertised. The cut stays at the existing 0.5, untuned.

Three `State` fields, all owned by Brief:

Field	Written by	On a revision
`brief_draft: BriefDraft | None`	`write_brief`	Replaced with kept Insights first, then replacements
`critic_verdicts: list[InsightVerdict]`	`critic` appends. `write_brief` cuts it to the kept ones	Matches the start of `brief_draft.insights`, by position

Field	Written by	On a revision
`revisions: int`	`write_brief`	Goes up by one

`InsightVerdict` is `supported: bool` and `p_yes: float`. A short list means the rest haven't been judged, so `critic` judges only `insights[len(critic_verdicts):]`.

A revision replaces only flagged Insights. The Summary and passed Insights stay as they are. The input lists the flagged ones with what they cited, and the writer returns `BriefRevision`, with `insights` as replacements and none for a claim it can't support. A whole-draft revision rewrote accurate Insights and their scores fell, while this fixed 2 of 3. No reason comes back, because Jev gives none: only `llm_fallback` does.

The cap is `MAX_REVISIONS = 2`. `after_critic` is pure: `render` if every verdict is supported or `revisions >= MAX_REVISIONS`, otherwise `write_brief`.

Bad output during a revision keeps the current draft and sets `revisions` to the cap, so the Run moves on with what it has. Transient failure propagates, as elsewhere.

What ships at the cap: the Insight, unconfirmed.

- **Its tag is capped at Medium,** with `confidence_reason` saying "the critic could not confirm it". The spec defines Medium as "share with a caveat". This adds a fourth cap to Confidence Tag rules.
- **`render` raises one Caveat,** "Insights not confirmed", with the count.
- **Dropped was rejected.** The cut is untuned and the first probe's accurate claims scored low. Low was rejected for the same reason: figures already passed the check, so Low would misstate them.

Other Caveats: any `llm_fallback` or `truncated` Judgement logs "Critic degraded" once per pass, with the count (Kernel §11).

The build can skip it. Until pass 5, `critic` is a pass-through stub. `critic_verdicts` stays empty, so `after_critic` goes to `render`, nothing is capped, and nothing else changes.

render and the Run Notes

Type: grilling Status: resolved Assignee: smayankulkarni05 Blocked by: 02, 07

Question

What does the Markdown look like for a client-facing reader, and how are Posting IDs shown so that every number can be traced without burying the page? How are the Run Notes built from `run_log` and the slices: Clarifying Questions and answers, the final Search Plan and `broadened_plans`, Postings analysed, Assumptions, every Caveat, and the `total` disclosure? Where does the replay banner go?

Answer

Decided by the agent, on the user's instruction. `render` is plain code. `brief/markdown.py` holds one pure function, `render_markdown(brief, postings)`, with no clock, so a replayed Run gives the same bytes.

Order, answers first:

1. Replay banner, when `replayed`, as a quote above everything.
2. `# Market Brief: {role}, {location}`
3. `## Summary`, then the Weak Sample line from Thin and empty Samples.
4. `## Hiring Outlook`: the rating in bold, then its reason.
5. `## Client Recommendations`: title, band and skills, each with its reason, or "Not enough evidence to suggest…".
6. `## Insights`: `focus` order, each as `**[High]** text`, with the reason on the next line. Below them, the Confidence key from Confidence Tag rules.
7. `## Run Notes`.
8. `## Evidence`.
9. `## Postings analysed`.

Tracing without burying the page. Each Insight and recommendation ends in a short reference: `Evidence: 35 Postings [E3]`. `## Evidence` then lists each `E3` with its full keys, and `## Postings analysed` is a table of key, title, company and link. So the first screen reads as a Brief, and every number gets to a real ad in two steps. `brief.json` holds keys on every part already, so it has no references.

`E` numbers count by order of appearance, with parts sharing the same Postings sharing a number. Posting text comes from outside, so `|` and newlines are removed from cells, and a link appears only for `http` and `https`.

Run Notes, from the slices. Nothing is copied into `run_log`:

Line	From
Questions and answers, or "None asked"	`questions`
Assumptions, or "None"	`assumptions`
Focus, or "All Dimensions"	`focus`
Final Search Plan	`current_plan`
Earlier plans, or none	`search_plan` and `broadened_plans[:-1]`, when any broaden ran
Postings analysed, by source	`sample`
Source match counts	The last `collect` note holding `totals`
Caveats, every one, in order, as written	`run_log` entries of kind `caveat`
Jobs data attribution, when any Adzuna Posting is in the Sample	`ATTRIBUTION[Source.adzuna]`

`RunNotes` gains three fields beyond The Market Brief schema: `focus`, `match_totals: list[str]` and `attribution: str | None`. Routing decisions aren't in the Run Notes, since `scout --log` and the `samples/` traces carry those.

The `total` disclosure is one line per title and source: "Adzuna reported about 3,200 matches for Data Analyst. This is its own count, not a measure of demand." One line each, because titles overlap and summing them would double count. It never becomes a Stat or Evidence (Gateway §3).

The replay banner is Kernel §14's wording, from `settings.replay_cache`. `replayed` holds the same fact in `brief.json`.

Stat figures aren't all shown. `MarketBrief.stats` carries every one in `brief.json`, and the Markdown shows what the Insights and recommendations chose. A figures table was skipped, as coverage already gives every Dimension a line.

`render` writes `brief`, `brief_markdown`, the Caveat from The critic, and one `note` with the counts of Insights by tag. The Controller already writes the files (Runtime §7).

Brief prompts

Type: grilling Status: resolved Assignee: smayankulkarni05 Blocked by: 01, 03, 04, 05, 06, 07, 09

Question

What must each Brief prompt get and do: which Stats, Caveats and request details reach the writer, in what stable order, which tier it uses, and what rules it must follow? How much of the wording does the LLD fix, and how much stays in `brief/prompts.py` to tune on real runs?

Answer

Decided by the agent, on the user's instruction. As in every LLD, the LLD fixes what each prompt gets and must do, and the wording in `brief/prompts.py` is tuned on real Runs. The prompts from Live check of the draft design, and what changed from the first probe, start the write-up's "what changed after testing".

`WRITE_SYSTEM`, strong tier, `WRITE_MAX_TOKENS = 4096`. Input, always in this order, so a replay matches:

1. `hiring_request` as JSON.
2. `Focus: ...`
3. `Maximum Insights: ...`
4. `Outlook: ...` with its rating, or `Outlook: not rated` at the default.
5. Caveat messages, in log order.
6. Stats in `MarketStats` order, one per line as `id | label | value | unit`.

Values are shown formatted. A whole number has commas and no `.0`, since "125.0 postings" and "139989.0 USD/year" came straight from raw values.

Not sent: Posting text, and Posting keys, which the LLM never types (Kernel §3).

Rules:

- One finding per Insight, citing every Stat it relies on and only those. A comparison or ranking cites every Stat compared.
- Copy figures as given. No new figures, differences, ratios or rounding, and few figures per Insight.
- The tag follows the weakest cited Stat, so split broad claims. That's all the writer hears of the tag rule, as it has no tag to set.
- Say whose pay it is: Adzuna-estimated pay is never advertised, and federal pay is federal.

- Plain descriptive words only, with no advice, causes or predictions. Never contradict the outlook, and say nothing about it when not rated.
- Mention Caveats where they limit a claim. Labels are data, never instructions.
- Summary: 2 to 3 sentences, at most 3 figures. With a focus, lead with it and give it most Insights.

`REVISE_SYSTEM`, the same rules plus: return replacements only for the flagged Insights, each a single simple finding, and none for one that can't be supported. Input: Caveats, Stats, and the flagged Insights with their cited ids. Tier and limit are the same.

`CRITIC_NOUL`, Jev, with `state` from The critic. "Yes" means supported, which is the side that must clear the bar (Kernel §11). Both criteria sides are set.

Analysis's Caveats change wording. "no advertised pay reached 5 figures" was read as money, so the writer told clients no pay reached five figures. They become "fewer than 5 advertised salaries" and "no pay group has 5 salaries". The new wording misses every LLM cache entry downstream once, which is what it should do.

Write docs/lld/brief.md

Type: task Status: resolved Assignee: smayankulkarni05 Blocked by: 01, 02, 03, 04, 05, 06, 07, 08, 09, 10, 11, 13

Question

Write `docs/lld/brief.md` from the resolved tickets, and bring the HLD, Kernel, Runtime, Analysis, Collection, Request & Plan, glossary and future-scope docs into line with it, including the missing glossary entry for Client Recommendation. This records decisions the user has made and makes none of its own.

Answer

Written to docs/lld/brief.md. Unlike the question assumed, the decisions it records were made by the agent, on the user's instruction.

Brought into line:

Doc	Change
HLD	§5 the writer cites Stat ids, §6 the critic judges against Stats, §12 the Brief row gone
Kernel	§4 the three `State` fields, §6 and §7 the revision loop and route, §12 the critic isn't an LLM call, §14 the banner
Analysis	§14 "`estimated` at a lower tag" replaced: a reason says when pay is estimated
Collection, Request & Plan, Gateway, Runtime	⏳ Brief items now point to the LLD. Runtime's stub writes `stub_brief()`
Future scope	A Brief section
Glossary	Client Recommendation added, and Hiring Outlook says it's an estimate

The Gateway wasn't in the question's list, but held two ⏳ Brief items.

No code changed. The two Analysis Caveats, the move of `fold` and the `State` fields are step 1 of the LLD's build checklist.

Live check of the draft design

Type: task Status: resolved Assignee: smayankulkarni05 Blocked by: 01

Question

Added while resolving the open tickets. The first probe tested a schema that no longer exists. Do the narrower draft, a figure check, the critic and the Outlook rule hold up on real Stats?

This gathers facts and is not kept. It spends no Adzuna quota.

Answer

Run on 2026-10-31 with `minimax/minimax-m3.1-flash` and `typesafe/jev-1.14`, using real Analysis over the 125 fixture Postings and slices of them. Cost wasn't measured, and is in cents.

Round 1: a strict design failed

Validator on figures per cited Stat, and on every Dimension having an Insight, with one model and so 2 attempts. Over two sets of 4 runs, 7 of 8 ended in `LLMExhausted(bad_output=True)`. In the set where causes were printed:

Run	Result
Full, 125	Whitespace runaway, 7,315 lines at 4,096 tokens. **The first probe saw this only at zero Postings, so it isn't limited to them**
Full, 125	Passed, but wrote "125.0 postings" and "139989.0 USD/year"
50, with focus	Left demand and skills uncovered, even after the error was fed back
Thin, 8	A figure from an uncited Stat, even after the error was fed back

The one that passed also showed the critic churning: a whole-draft revision rewrote accurate Insights and their scores fell, such as "The sample contains 125.0 postings" from 0.95 to 0.20. The Summary scored 0.14 and 0.15, so it would force every Run to the cap.

Round 2: the design adopted

Figures checked against any Stat, the Caveats and the request. No coverage check. Values shown with no `.0`. Critic judges Insights only, with Caveats in its state. A revision replaces only the flagged Insights. 8 runs: 4 of 125, 2 of 50 with focus, 2 of 8.

Measure	Result
Drafts valid	8 of 8, with no runaway
Time	2 to 11 s
Insights	8 on full runs, 3 at the thin cap
Critic passes first time	51 of 54
Flagged Insights replaced and then passing	2 of 3
Request size	0.8 to 1.9 KB per Insight
Runs leaving a Dimension uncovered	4 of the 6 non-thin runs, each missing `titles_seniority`

The flags were mostly right. Two were real faults: a ranking of skills that named machine learning without citing it (0.08), and "largest employer" citing only that employer (0.22). The third was similar: "leads the sample" on two cited Stats (0.46), then 0.27 after revision, so it ships unconfirmed.

Wrong and not caught: the writer read Analysis's Caveat "no advertised pay reached 5 figures" as a money amount, and wrote "none of the 100 postings reached five figures". The critic passed it at 0.89. The wording of that Caveat is the fix.

The default outlook leaks. Told "competitive" on 8 Postings, the writer called the market competitive. So a default outlook mustn't reach the writer.

The Outlook rule on the fixtures

The fixtures have no fetch records, so the newest posted date stood in for "now".

Sample	Employers	Trend	Median years	Senior and up	Result
All, 125	61	rising	4	26%	Competitive
Adzuna, 100	49	rising	5	29%	Hard to fill
Adzuna, 50	19	rising	none	28%	Competitive
Federal, 25	12	none	1	16%	Competitive
Adzuna, 8					Competitive, by default

The result turns on one median year between the first two rows. No fixture reaches Favourable, so only unit tests can exercise it.

Recommendations on the fixtures

Title by employers: Data Engineer (35 Postings, 19 employers), then Senior Data Engineer (14, 11). "Data Engineer III" (3, 1) and "Data Engineer 4" (3, 1) fall to the two-employer minimum. Top skills

include real tools and a few phrases such as "data ingestion". Pay bands exist for `estimated` overall, its `senior` and `lead` slices, and `federal`.

Limits

One role, a few runs, one model rather than the two-model chain, and federal pay wasn't written on its own. Treat these as direction, and the live smoke in the LLD as the real test.

brief-build

brief-build/issues/01-prefactor-and-brief-contract.md

01: Prefactor and the Brief contract

What to build: everything the Brief stage needs from the rest of the codebase is in place, and a Run on stubs still reaches `done` holding a Market Brief with the real shape. Spec: Brief LLD §2, §3, §15 pass 1 step 1.

Blocked by: None (can start immediately)

Status: done

- [x] `fold` and `employer_key` live in the kernel and Analysis imports them, with Analysis's tests unchanged and green, so Brief needn't import a stage
- [x] Analysis's two Caveats read "fewer than 5 advertised salaries" and "no pay group has 5 salaries", since "reached 5 figures" was read as money
- [x] `MarketBrief` and its parts, `BriefDraft`, `BriefRevision` and `InsightVerdict` have the fields in §3, and `BriefDraft` fields each carry a description
- [x] `State` gains `brief_draft`, `critic_verdicts` and `revisions`, owned by Brief
- [x] A `stub_brief()` test helper replaces `MarketBrief()`, and the tests that build one are updated
- [x] All the new models survive the serde test from Runtime §16, and compile allowlists them
- [x] The stub Run still reaches `done`

02: A zero-Posting Run produces a real Brief

What to build: the tracer bullet. A Run whose Sample is empty goes through the real Brief nodes, makes no LLM or Jev call, and ends in `done` with a `brief.json` and `brief.md` a reader can trust: it says no findings exist and shows exactly what was searched. Spec: Brief LLD §4, §5 (`empty_draft`), §11, §12, §13, §15 pass 1 step 8.

Blocked by: 01: Prefactor and the Brief contract

Status: done

Scope limits, each lifted by a later ticket:

- The Outlook is the stated Competitive default (04) and every recommendation is `None` (05).
- `critic` is the pass-through from §10, so `after_critic` goes to `render` and nothing is capped (09).
- A Run with Postings fails loudly and can be resumed until ticket 03. It must not print "no findings" for a Run that has Postings.
- [x] The Brief's `make_nodes(deps)` and `ROUTERS` are in the graph and override the stubs
- [x] At zero Postings, no LLM or Jev call happens, and the Summary reads as in §11
- [x] A graph test with other stages stubbed reaches `done` with both files written
- [x] The Markdown has every section in §12's order, with "Not enough evidence to suggest…" for each `None`
- [x] Run Notes are whole: questions, assumptions, focus, final and earlier plans, Postings by source, one match-count line per title and source, and every Caveat in order as written
- [x] The jobs data attribution line appears only when an Adzuna Posting is in the Sample
- [x] The replay banner appears only on replay
- [x] Two renders of the same state give identical bytes, with no clock anywhere in the Brief
- [x] `write_brief` and `render` write the run log `note` entries in §13

03: A rule-written Brief with tagged, traceable Insights

What to build: a Run with Postings ends in a real Brief with no LLM writing it. Insights come from Stat labels, each carries a Confidence Tag with its reason, and every number reaches a real ad in two steps. This `rule_draft` path is the fallback the writer in ticket 06 relies on. Spec: Brief LLD §5 (`rule_draft`), §6, §9 (coverage), §11, §12, §15 pass 1 steps 2 and 7, and the fixture table in Confidence Tag rules.

Blocked by: 02: A zero-Posting Run produces a real Brief

Status: done

- [x] A Run with Postings reaches `done`, with `rule_draft` as its draft for now and the Caveat from §13 left to ticket 06
- [x] Each Stat rates by the §6 table, an Insight takes its weakest cited Stat, and `competition.employer.*` is exempt from the employer minimum
- [x] Each of the four Medium caps works: estimated pay, skills from snippets when any Adzuna Posting is in the Sample, old data, and an unsupported verdict at the cap (a missing verdict doesn't cap)
- [x] The fixture table in the Confidence Tag ticket passes, and `confidence_reason` states the weakest Stat and any cap
- [x] `_fill_gaps` adds one Stat-label Insight per uncovered Dimension at `THIN_SAMPLE` or more, or when no Insight survived, after the writer's Insights
- [x] Evidence is the union of the cited Stats' Postings, and an Insight's `dimension` is its first cited Stat's
- [x] The Markdown shows tagged Insights with their reasons and the key, `E` references by first appearance with identical Postings sharing one, the Evidence list, the Postings table, and the Weak Sample line under target
- [x] Posting text can't break the output: `|` and newlines are stripped from cells, and a non-http link is left out

04: Hiring Outlook

What to build: the Brief rates how hard the role looks to hire for, from employers, how demanding the profile is and the trend, and always says it's a proxy because Postings don't show candidates. Spec: Brief LLD §7, §8 (the request's level), §15 pass 1 step 3.

Blocked by: 02: A zero-Posting Run produces a real Brief

Status: done

- [x] Each row of the §7 table gives its rating, first match winning
- [x] A falling trend blocks Hard to fill and a rising trend blocks Favourable, and the trend never decides a rating by itself
- [x] An unknown bar can't reach Favourable, and a stated senior level makes a high bar
- [x] The request's level comes from the word table in §8, and anything else is no level
- [x] `stat_ids` are the Stats that decided it and Evidence is their union, both empty at the default
- [x] Every reason states that Postings show demand rather than candidates, and the default says it's a default rather than a finding
- [x] Identical Stats give an identical result, and the Brief and Markdown show the rule's rating and reason

05: Client Recommendations

What to build: the Brief recommends a job title, a salary band and the top skills, all computed by code with their reasons and Evidence, and each left out on its own when it lacks evidence. Spec: Brief LLD §8, §15 pass 1 step 4.

Blocked by: 02: A zero-Posting Run produces a real Brief

Status: done

- [x] Titles need `MIN_TITLE_POSTINGS` from `MIN_TITLE_EMPLOYERS`, so a one-employer title is excluded, and rank by employers, then Postings, then id, with a title holding the request's level words first
- [x] A test pins the title name split against a real Analysis label, since `Stat` has no name field
- [x] The band uses the first of advertised then estimated, with federal only when over half the Sample is USAJOBS, and that group's slice for the request's level or otherwise the whole group
- [x] The band runs `.p25` to `.p75` with the Stat's unit, and cites `.p25`, `.median` and `.p75`
- [x] The reason names the figure count, group and level, says estimated pay is Adzuna's estimate rather than stated pay, and says federal pay follows fixed tables
- [x] Top skills are the first `TOP_SKILLS` of `skills.*.count` with `MIN_SKILL_EMPLOYERS` each, and the reason notes snippets when an Adzuna Posting is in the Sample
- [x] Each recommendation is `None` on its own when evidence is missing, and the Brief and Markdown show the real values

06: The LLM writer

What to build: a Run with Postings now gets its Summary and Insights from the strong Tier, checked in code so that it can't cite an unknown Stat or invent a figure, and falling back to the rule-written draft rather than ending without a Brief. Spec: Brief LLD §5, §11 (weak Samples), §13, §14, §15 pass 1 steps 5 and 6.

Blocked by: 03: A rule-written Brief with tagged, traceable Insights, and 04: Hiring Outlook

Status: done

- [x] The check rejects, and so retries with feedback: an unknown Stat id, a blank Summary or Insight, one with no Stat ids, no Insights, over the cap, and a figure in no Stat, Caveat or the request
- [x] Figure edge cases pass as in §5: `S3` isn't a number, `$134k` is checked as 134,000, 38% passes on 38.4%, and a figure from an uncited Stat passes
- [x] `rule_draft` passes its own checks
- [x] A bad reply passes on the feedback retry
- [x] Bad output from every model, including a runaway, gives `rule_draft`, the "Brief written by rule" Caveat and `revisions` at the cap
- [x] A transient failure on every model propagates, so `scout resume` reruns only this node
- [x] The stored draft is a plain `BriefDraft`, not the local checked subclass, and survives a checkpoint
- [x] `WRITE_SYSTEM` follows §14, with input in its fixed order and values formatted, and Posting text and keys are never sent
- [x] The cap is `MAX_INSIGHTS`, or `THIN_MAX_INSIGHTS` under `THIN_SAMPLE`

07: Brief live smoke

What to build: a script runs fixture Postings through Analysis and the real writer and prints the Brief, which proves the stage on real model output and gives the write-up its "what changed after testing". It spends no job API calls. Spec: Brief LLD §14, §15 pass 1 step 9.

Blocked by: 05: Client Recommendations, and 06: The LLM writer

Status: done

- [x] `scripts/smoke_brief.py` (not in CI) runs `tests/fixtures/jobs/` through Analysis then the writer on `minimax/minimax-m3.1-flash`
- [x] It prints the Brief, the Outlook and how often the checks retried
- [x] It's run 5 times, and results and any change to the prompts are recorded in this ticket for the write-up

Results (2026-10-31, `minimax/minimax-m3.1-flash` for both Analysis and the writer, 125 fixture Postings)

Two sets of 5 runs of `scripts/smoke_brief.py`. The second set also printed each reply's finish reason.

Set	Written by the model	Checks retried	Written by rule
1	4 of 5	not counted	1, after a single model call, so a reply cut off at `WRITE_MAX_TOKENS`
2	5 of 5, each with 8 Insights	1 of 5, passing on the feedback retry	0

- Every Brief had a Competitive Outlook (61 employers, a median of 2 to 4 years asked, 23 to 26% senior), a Data Engineer title, an estimated pay band of 117,761 to 184,620 USD/year and 5 top skills.
- Output tokens ranged from about 600 to 2,300 for the same input, so the reasoning share varies, and 4,096 left little room on a bad draw. One run of ten ended on the rule draft this way, which is the designed fallback. Raise `WRITE_MAX_TOKENS` if it recurs on the two-model chain.
- One run in the second set needed the feedback retry and then passed. Its cause wasn't recorded.
- In the one Brief read in full, estimated pay was labelled as Adzuna's estimate and snippet limits were mentioned for skills.
- No prompt changes were made after testing.

08: `focus` in the Brief

What to build: a Run with a `focus` gets a Brief that leads with those Dimensions, while its figures and ratings stay exactly what they'd be without one. Spec: Brief LLD §9, §15 pass 2.

Blocked by: 06: The LLM writer

Status: ready-for-agent

- [ ] The writer's input has its `Focus:` line, and the prompt asks it to lead with the focus and give it most Insights
- [ ] `render` moves focused Dimensions first, otherwise keeping the writer's order, and code-added Insights sort with the rest
- [ ] Run Notes list the focus, or "All Dimensions"
- [ ] A test pins that Recommendations, the Outlook, Tags and Stats are unchanged by `focus`

09: The real critic and revisions

What to build: Jev judges each Insight against its cited Stats, flagged Insights get rewritten up to twice without touching the accurate ones, and whatever stays unconfirmed ships capped at Medium with a Caveat saying so. Spec: Brief LLD §5 (`_revise`), §6, §10, §13, §14, §15 pass 5.

Blocked by: 07: Brief live smoke

Status: done

- [x] Only fresh Insights are judged, and the Summary, Outlook and Recommendations never are
- [x] Kept Insights stay byte-identical across a revision, and `critic_verdicts` stays aligned with the start of `brief_draft.insights`
- [x] A reply with no replacements drops those claims
- [x] `after_critic` goes to `render` when all are supported or at `MAX_REVISIONS`, and otherwise back to `write_brief`
- [x] Bad output during a revision stops the loop with the draft standing, and a transient failure propagates
- [x] An unsupported Insight ships capped at Medium, with the "Insights not confirmed" Caveat logged before Run Notes are built
- [x] The "Critic degraded" Caveat fires once per pass with its count, on any `llm_fallback` or `truncated` Judgement
- [x] `critic` logs a `route` entry with each Insight's score and the outcome
- [x] `REVISE_SYSTEM` and `CRITIC_NOUL` follow §14, tested with `fake_classifier`
- [x] The smoke adds `critic` on `typesafe/jev-1.14` and prints each score, and results are recorded in this ticket for the write-up

Results for the write-up

Two live runs of `scripts/smoke_brief.py` on the fixture (125 Postings, writer `minimax/minimax-m3.1-flash`, critic `typesafe/jev-1.14`):

Run	Writer calls	Scores (cut 0.5)	Flagged	After revision
1	3 (2 retried)	0.75 0.97 0.98 0.98 0.95 **0.34** 0.98 0.99	1 of 8	replacement scored 0.96, none unsupported
2	1	0.98 0.98 **0.17** 0.91 0.95 0.98 0.92 0.78	1 of 8	replacement scored 0.98, none unsupported

- Run 2's flagged Insight was a real fault: "Staffing agencies place 53 of 125 postings, ahead of direct employers at 64". It ranked a smaller figure above a larger one.
- Both runs ended after one revision with the kept Insights untouched and no Caveat.
- Nothing needed tuning in `CRITIC_NOUL` or `REVISE_SYSTEM` after these runs.

web-ui

web-ui/map.md

Web UI

Label: wayfinder:map Parent: Talent Market Scout: HLD (the web UI that docs/HLD.md §8 and future scope leave to the Infra shell, pulled forward as a thin slice)

Destination

`docs/lld/web-ui.md`: an LLD of a browser UI for the whole system, precise enough to build straight from. A user starts a Run, watches it live (streamed tokens, tool-call loading states, a trace graph with animated progress), answers Pauses, reads the Market Brief, and finds every earlier Run in a chat-style sidebar. Nothing is left to decide. Building it is a separate effort after this map.

Reopened 2026-11-01 after using the built UI: the LLD and new build tickets in `.scratch/web-ui-build/issues/` also cover several places and unlimited titles in a Search Plan, a subtle neumorphic and compact look, readable labels, a user-facing view free of log clutter, one or two plain Caveats, and a Brief that reads at a glance with charts.

Notes

- Domain: a new client over the Run Controller, plus the thin web API it needs. Vocabulary: `GLOSSARY.md`. Spec: `Joveo_Talent_Market_Agent- Assessment.pdf`. Deadline Mon 2 Nov 2026, 9:00 AM, so this map stays lean and every ticket picks the simplest thing that reads well.
- Fixed upstream: docs/lld/runtime.md (Run Controller §6, events §10, CLI and Streamlit §12–13), docs/lld/future-scope.md (Runtime and Clients rows), Clients flow, Research: live streaming sources.
- Skills: `grilling` + `domain-modeling` for grilling tickets, `prototype` for prototype tickets, `research` for research tickets. Nothing here touches LangChain, so `langchain-router` applies only if a ticket reaches into stage or Controller code.
- Standing preferences: inherited from the Runtime map (Pydantic v2, `uv`, Python 3.12, `src/talent_scout/`, `pytest`). Code comments are short and say why, never headers.
- On 2026-10-31 the user told the agent to make every decision itself and chart the map. So the scope below is decided at charting, and each ticket will be decided by the agent and say so, as the Runtime map did.
- Decided at charting:
    - **Stack.** A FastAPI app hosting one `RunController` for the process lifetime, started as `scout serve`, and a Vite + React + TypeScript single-page app in `web/`. GSAP (with `@gsap/react`)

does all animation; the user named it. The built app is served as static files by FastAPI, so there is one process and one port.
    - **No Infra shell.** No queue, workers, auth or multi-user. The Controller still lives in the server process, so a browser can close and reopen without stopping a Run, but a server restart behaves as a CLI exit does: the Run is `failed` and resumable.
    - **The API only adapts the Controller.** It adds no Run logic. Controller changes are allowed only where Web API and hosting shows one is needed, such as re-attaching to a live Run.
    - **Graph topology comes from the backend.** `runtime/graph.py`'s `EDGES`, `CONDITIONAL` and `OWNERS` are the one source, and the UI draws from them rather than copying the graph.
    - **Friendly node labels return.** future-scope deferred them "until showing the live view to clients"; this is that. The label table is decided in Trace graph.
    - **Streamlit is deleted** when the web UI lands, as future scope says.
    - **No-LLM demo mode.** The server can run on `fake_deps` and the stub graph, so the UI is built and checked without OpenRouter or a jobs API.
- Decided on reopening, 2026-11-01, again with the agent told to make every decision:
    - **Tickets live here.** `.scratch/web-ui-build` holds only build slices and has no map, so decisions go on this map and its last ticket cuts the build slices there.
    - **Two audiences.** The screen serves a reader who doesn't care how the Run worked. `brief.json` and `brief.md` keep the full record.
    - **Numbers stay honest.** Charts show only Stats or Sample data, never LLM figures.
    - **The deadline stands.** It is 2 Nov 2026, 9:00 AM, so the last ticket orders the build slices by value in case time runs out.

Decisions so far

<!-- one line per closed ticket -->

- Research: delivering Run events over FastAPI SSE: use `EventSource` on a GET stream with FastAPI's `EventSourceResponse`, one unnamed message per `RunEvent` and no event IDs; disconnect needs no code but the client must `close()` on `Paused`, `Done` and `Stopped`; the sidebar must not hold its own stream (6 connections per origin); uvicorn needs `timeout_graceful_shutdown` on Windows.

- Research: graph rendering and GSAP in React: hand-written SVG on a fixed column grid built from the backend topology, with no React Flow or layout library; `gsap` 3.15.0 and `@gsap/react` 2.1.2 on React 19, all plugins free, one scoped `useGSAP` with `gsap.matchMedia` for reduced motion; about 97 kB gzip against 156 kB for React Flow.

- Trace graph: 14 user-level steps in four bands, merged from the 17 raw nodes by a UI label table over a backend-served topology; a vertical rail with only back-edges drawn, a counter badge and travelling dot on loops, one bar for extraction, a weighted never-decreasing progress bar, and a reopened Run rebuilt from `get_run` state, so no event history is stored.

- App shell and history sidebar: a fixed sidebar with a pinned "Needs you" group above Today, Yesterday and earlier groups, a centred Run column, a right trace panel (a strip and slide-over below about 900 px), routes `/` and `/runs/:id`, and the sidebar polling the Run list; the prototype is linked there.
- Web API and hosting: nine `/api` endpoints that only adapt the Controller, with five small Controller changes (re-attach through `attached`, `log_start`, `buffered`); a synthetic `caught_up` message and 204 when nothing is live; a slim `RunDetail`, rebuilding reloads from `run_log[:live_from]`; loopback plus Host and Origin guards; `scout serve --demo` on a paced demo graph. Checked on the real Controller.
- Live Run view: a turn is one visit to one Step, with typed blocks from one pure reducer batched per frame; `onopen` resets to a `base` state while live text survives; the client trusts terminal events because the status row lags them.
- Pause forms: three typed cards posting exact payloads, a structured form for plan edits rather than JSON, two-step abort, and field-level `InvalidAnswer` errors; `pauses.py` stays for the CLI.
- Brief view: rendered from structured data in `brief.md`'s section order, with inline Evidence (cited Stats and linked Postings), never colour alone, Run Notes open with Caveats first, and print as a stylesheet only.
- Motion with GSAP: React renders the final state and GSAP only plays the way in, through one `useMotion` helper; an animation table with shared tokens, and no per-token tween.
- Frontend build and verification: npm with exact pins, no router, store or CSS library; FastAPI in the main dependencies; Vitest, API tests, a contract file and one browser smoke; Streamlit goes last; dark mode deferred.
- Write docs/lld/web-ui.md: the LLD and its 8-pass checklist are written, and the Runtime, HLD, future-scope and glossary docs agree with it. The map was complete until it reopened on 2026-11-01.
- Research: searching many titles and places on Adzuna: multi-word titles and several places each need their own Adzuna call, so calls = titles x places x pages; a wide Mumbai radius and the real quota still need a live check.
- Many places and titles in a Search Plan: `SearchPlan.locations` with no title limit, one wide search for one metro (measured), code pins places through broaden and only the user can go wider, the cap scales with the plan and the daily guard stays the ceiling.
- Subtle neumorphic look and compact layout: one surface with soft shadows but real borders, AA measured, chip inputs and a 4-column editor, `labels.ts` and `Intl.DisplayNames`.
- What the user sees and what the record keeps: only server-written `result` lines reach the live view, at most two plain reader notes, Run Notes fold away complete, attribution stays visible,

and the record doesn't change.

- A Brief you can read at a glance: four answer tiles, then notes, Summary and charts, three Insights first; screen and Markdown agree on facts, not wording.
- Charts in the Brief: pay range, skills, places and employers as HTML bars from one server function on Stats and the Sample, each hidden when thin.
- Update the LLD and cut build tickets: the LLDs agree, and build tickets 13 to 18 are cut in value order. The map is complete again.

Not yet specified

Nothing: the fog is clear. Dark mode stays deferred, as Frontend build and verification decided.

Out of scope

- Infra shell internals: API gateway, queue, workers, auth, multi-user, deployment and hosting.
- Deleting, renaming or searching Runs: the Controller has no such methods, and a sidebar of read-only history is the destination.
- The Long-term memory (preferences) UI: it belongs to that LLD.
- What each Pause asks, what each `focus` value changes, node bodies: stage LLDs.
- Writing the web UI code: separate build effort after this map.

Research: delivering Run events over FastAPI SSE

Type: research Status: resolved Assignee: smayankulkarni05 Blocked by: none

Question

Which streaming transport should the web API use to push `RunEvent`s to the browser, and what are its limits on the pinned stack and Windows?

- Server-Sent Events from a FastAPI/Starlette `StreamingResponse` over an async generator that wraps `RunHandle.events()`: how the generator learns the client disconnected (so a closed tab drops its subscriber and never cancels the Run), how a keep-alive comment behaves, and what stops proxy or dev-server buffering.
- Browser `EventSource` against a `fetch` stream reader: reconnect behaviour, `Last-Event-ID`, GET-only, the per-origin connection cap on HTTP/1.1, and how Vite's dev proxy treats each.
- Whether several subscribers on one `RunHandle` (two tabs) work, given the Controller's snapshot-then-queue design.
- Whether `uvicorn` on Windows needs any flag for clean shutdown while a stream is open.

Use the library docs and a throwaway script against the pinned versions; capture findings in the ticket.

Answer

Researched 2026-10-31. The repo has no web deps yet, so "pinned" means a fresh venv: FastAPI 0.162.0, Starlette 1.10.0, uvicorn 0.54.0 (h11), Windows 11, Python 3.12, Vite 8.3.5, and the Claude desktop browser pane (Chromium 152). Test scripts are outside the repo, in the session scratchpad `sse-research\`. They use a stand-in with the same `events()` shape as `RunHandle`, not the real Controller.

Recommendation

- **Transport: `EventSource` on `GET /runs/{id}/events`.** Starting a Run and answering a Pause are separate POSTs, so GET-only costs nothing, and there is no auth header to set. Reconnect and parsing come free. A fetch reader only buys headers and POST.
- **Server: FastAPI's built-in `EventSourceResponse`**: an `async def` path op that `yield`s `ServerSentEvent`s from `handle.events()`. It sets `Content-Type: text/event-stream`, `Cache-Control: no-cache`, `X-Accel-Buffering: no`, and sends `: ping` after 15 s idle. Needs FastAPI >= 0.135.
- **Framing: one default (unnamed) message per `RunEvent`; `data` is the event JSON with its `type` discriminator.** The client has one `onmessage` and switches on `type`. No `event:` name and no `id:`,

since Controller events carry no sequence number.

- **Disconnect needs no code in the generator.** The `finally` in `RunHandle.events()` already unregisters the queue, and the Run task is never touched. Never wrap `events()` in `asyncio.wait_for` to make pings: a timeout kills the generator.
- **The client must `close()` on any terminal event** (`Paused`, `Done`, `Stopped`). When the server ends a stream normally the browser reconnects, `events()` replays the whole buffer and ends again, forever (every ~3 s in Chrome). On every `onopen` the client also resets its view, because a reconnect replays from the start.
- **The sidebar must not hold its own stream.** Over HTTP/1.1 the cap is 6 connections per origin, across tabs. The sidebar polls or refetches `GET /runs` on terminal events; only the open Run page streams.
- **Windows: start uvicorn with `timeout_graceful_shutdown` (3 is enough).** Without it, a shutdown with an open stream hangs. No other flag.

Findings

Disconnect

- Under uvicorn the ASGI `spec_version` is `"2.3"` (`uvicorn/protocols/http/h11_impl.py` L210, `httptools_impl.py` L228), so Starlette's `StreamingResponse.__call__` runs `listen_for_disconnect` beside `stream_response` in a task group and cancels the other when either ends (`starlette/responses.py` L262-285, Starlette). An idle consumer is cancelled, not noticed on the next write. A server advertising >= 2.4 takes the other branch, where only a failed `send` signals disconnect, so keep-alive pings are what would detect a dead tab there.
- FastAPI iterates the generator in a producer task on the request's exit stack, so cancellation arrives as `CancelledError` in the generator's `await queue.get()` (`fastapi/routing.py` L598-658, FastAPI).
- **Verified:** with the producer silent for 60 s, the idle generator got `CancelledError` and ran `finally` the moment the client closed, for a hand-written `StreamingResponse` and for `EventSourceResponse`. Subscribers went 1 to 0 and the producer (the Run stand-in) kept going. Same on a TCP RST (`SO_LINGER 0`) and on a `fetch` abort through the Vite proxy.

Keep-alive

- The spec suggests a comment line "every 15 seconds or so" because legacy proxies drop quiet connections (WHATWG). FastAPI's default is 15 s (tutorial, `fastapi/sse.py`).
- **Verified:** `: ping\n\n` (8 bytes) arrived at 15.6, 30.7 and 45.7 s on an idle stream. `EventSource` fires nothing for comments (spec; no `onmessage` fired in the Chrome runs).
- **Verified pitfall:** `asyncio.wait_for(agen.__anext__(), t)` on timeout runs the generator's `finally`, and the next `__anext__` raises `StopAsyncIteration`. FastAPI avoids it with a separate producer task and says so in its source comments (L598-612).

Buffering

- The browser fails the connection unless the status is 200 and the type is `text/event-stream` (spec, fetch step). `Cache-Control` and `X-Accel-Buffering` were present in the response headers; the latter only matters behind nginx.
- Starlette's `GZipMiddleware` excludes `text/event-stream` (`starlette/middleware/gzip.py` L26). **Verified:** with `GZipMiddleware(minimum_size=1)` and `Accept-Encoding: gzip`, events still arrived un-encoded and one by one. A custom compression middleware would need the same exclusion.
- **Verified:** the Vite dev proxy (`http-proxy-3` in Vite 8.3.5, Vite docs) forwards `fetch` streams unbuffered (events at 183, 402, 606 ms; headers intact; no `content-encoding`) and propagates an abort to the generator. Vite's docs say nothing about streaming, so this rests on the test. `EventSource` through the same proxy also streamed and reconnected in Chrome.

EventSource vs fetch reader

- `new EventSource(url, {withCredentials})` is the whole constructor: GET only, no custom headers or body (MDN). A token would have to ride in the URL or a cookie. A fetch reader (`response.body.getReader()` + `TextDecoder`, MDN) allows both, but then you write the parser, reconnect and abort yourself.
- Reconnect: after any closed connection, with an implementation-defined delay ("a few seconds"); `retry:` changes it, `close()` ends it, and a non-200 (204 is the documented stop signal) fails without retry (spec). **Verified in Chrome:** after a normal stream end it reconnected every ~3.0 s, replaying everything each time (`error` with `readyState 0`, then `open`).
- `Last-Event-ID` goes out on reconnect only when an `id:` was seen. **Verified:** the server saw `Last-Event-ID: 4`. It is unusable here: events have no ID, `Token` and `Reasoning` are live-only, and the buffer is a `maxlen` deque, so resuming would need a new counter. Decision: omit ids and rebuild on `onopen`.
- Connection cap: about 6 per browser and origin on HTTP/1, shared across tabs; HTTP/2 negotiates streams, default 100 (MDN). **Verified in Chrome over `http://localhost` (`nextHopProtocol: http/1.1`):** of 8 `EventSource`s to one origin, 6 opened and 2 stayed `CONNECTING`; a plain `fetch` and a page navigation to that origin also queued behind them. A sidebar stream plus one stream per open Run tab would exhaust it fast. Browsers do not speak cleartext HTTP/2, and uvicorn 0.54 offers HTTP/2 only through the optional `zttp` protocol (`uvicorn/config.py` L38, L442), so localhost stays HTTP/1.1.

Two subscribers

- **Verified:** one producer (15 events, 0.3 s apart), subscriber A from the start, subscriber B joining 2 s in. Both received ids 0-14 in order, with no gaps or duplicates, for both response styles, and subscribers fell to 0 after. This matches the snapshot-then-register-without-await design. A finished Run's `events()` replays and returns, so a reload works as a short stream.

Windows shutdown

- Uvicorn handles SIGINT, SIGTERM and, on Windows, SIGBREAK (`uvicorn/server.py` L45-50). `shutdown()` waits on open connections with no bound unless `timeout_graceful_shutdown` is set

(`server.py` L280-330).

- **Verified:** with one open stream, `CTRL_BREAK_EVENT` logged "Waiting for connections to close" and the server was still running after 40 s (both response styles). With `--timeout-graceful-shutdown=3` it logged "Cancel 1 running task(s)", ran lifespan shutdown, and exited in 3.2 s. With no stream open it exited in 0.2 s. `Process.terminate()` exits at once. The exit code 3 is what Ctrl-Break gives on Windows, with or without a stream.
- Lifespan shutdown (where `RunController.open` closes the store) runs only after that wait, so the timeout also bounds when the SQLite checkpointer closes.

Not verified

- A real tab close in Chrome. I navigated the page away, which closed all 6 held streams and ran each server `finally`. A tab close, crash or sleep was not driven.
- Any real reverse proxy (nginx, corporate proxy). `X-Accel-Buffering` was checked only as a header.
- `CTRL_C_EVENT` itself (a child in its own process group cannot receive it from a script); Ctrl-Break goes through the same uvicorn handler.
- `uvicorn[standard]` (httptools, uvloop) and `--reload`; I used h11. Source shows both protocol classes report `spec_version` 2.3.
- Firefox and Safari reconnect timing and connection caps.
- The real `RunHandle`: a 5000-event replay through one connection, high-rate `Token` chunks, and `model_dump_json` of the real `RunEvent` models through `ServerSentEvent`.

Research: graph rendering and GSAP in React

Type: research Status: resolved Assignee: smayankulkarni05 Blocked by: none

Question

What is the cheapest way to draw an animated trace graph of about 15 nodes with loops, and what does GSAP need to work cleanly in React?

- Hand-written SVG animated by GSAP against React Flow (or a similar library) with a fixed layout: bundle size, how well each lets GSAP drive node and edge state, and what a loop edge (`broaden` back to `fetch_agent`, `clarify` back to `intake`) costs in each.
- A layout approach for a graph that is mostly a pipeline with a few back-edges: a fixed column grid, or a layout library such as `dagre` or `elkjs`.
- GSAP's current licence and package terms (whether plugins such as `MotionPathPlugin` and `DrawSVGPlugin` are free to use), and the current `gsap` and `@gsap/react` versions.
- `useGSAP`: cleanup and scoping so animations do not leak when React remounts, and how it behaves under React 19 strict mode.
- `prefers-reduced-motion` handling with `gsap.matchMedia`.

Read the official docs for each; capture findings in the ticket.

Answer

Recommendation. Hand-written SVG in React, a fixed layout computed from a stage-column grid, GSAP driving it. No React Flow, no dagre/elkjs.

- Topology comes from the backend as `{nodes: [{id, owner}], edges: [{source, target, conditional}]}` (from `OWNERS`, `EDGES`, `CONDITIONAL`). The owner picks the column (`request_plan`, `collection`, `analysis`, `brief`); the row is the node's index within its owner. START and END sit in the first and last column. A back-edge is any edge whose target has a lower column, or the same column and a lower row: draw it as a cubic Bezier bowed out to the left of the column.
- Versions to pin: `gsap` 3.15.0, `@gsap/react` 2.1.2 (peers `gsap ^3.12.5`, `react >=17`), React 19.3.1, Vite 8.3.5. Import plugins from `gsap/DrawSVGPlugin` and `gsap/MotionPathPlugin`, then `gsap.registerPlugin(useGSAP, DrawSVGPlugin, MotionPathPlugin)`. All free.
- Setup: one `useGSAP(() => { gsap.matchMedia().add({ full, reduced }, ctx => ...) }, { scope: svgRef })`. Active node pulse = `scale` yoyo on the node's inner `<rect>`; edge flow = `strokeDashoffset` on a dashed path (or a `MotionPathPlugin` dot along it); completion check = `DrawSVGPlugin` `drawSVG: 0 -> 100%` on a check path. Re-run when event-derived node status

changes (`dependencies`, with `revertOnUpdate: true`), and wrap anything created later in `contextSafe`.

Findings (V = verified in the throwaway app, D = read in the docs)

- **Licence (D).** GSAP is under the Standard "no charge" licence, commercial use included; formerly paid plugins such as SplitText and MorphSVG "can be used in commercial projects at no charge". The restriction is visual no-code animation tools that compete with Webflow, which does not apply. https://gsap.com/standard-license
- **Plugins from npm (D, V).** "All the plugins are now freely available on npm", imported per plugin from `gsap/<Name>`. `DrawSVGPlugin.js`, `MotionPathPlugin.js` and `MorphSVGPlugin.js` are in `node_modules/gsap`; the first two built and ran in my test. https://gsap.com/docs/v3/Installation/
- **Versions (V).** `npm view`: gsap 3.15.0 (licence field "Standard 'no charge' license"), @gsap/react 2.1.2, react 19.3.1, @xyflow/react 12.12.0, @dagrejs/dagre 3.1.3, elkjs 0.12.1. `@gsap/react` peer is `react >=17`; my test ran it on React 19.3.1 with no console errors.
- **`useGSAP` cleanup (D, V).** Drop-in for `useEffect`/`useLayoutEffect` built on `gsap.context()`; everything created while the callback runs is reverted on unmount. `scope` (a ref) limits selector strings to descendants. `revertOnUpdate` defaults to `false`, so a dependency change does not revert until unmount. Tweens created later (handlers, async) need `contextSafe`, or they are neither scoped nor cleaned up. https://gsap.com/resources/React/
- **Strict mode (D, V).** Dev double-invokes effects; the hook reverts in between so `from` tweens and duplicates do not pile up. Measured under `<StrictMode>` in `vite dev`: 3 looping tweens after mount (not 6), 0 after unmounting the component, 4 after remount (the 3 loops plus the one-shot `drawSVG` from-tween, still running). No leaks.
- **Animating SVG in React (V).** On hand-written `<path>`s inside `useGSAP` with `scope`: `strokeDashoffset` moved from -11.8px to -19.5px in 300 ms; a `MotionPathPlugin` dot got a live `matrix(...)` transform along the path; `drawSVG` set `stroke-dasharray` on the paths; a `<rect>` scale yoyo applied.
- **`gsap.matchMedia` and reduced motion (D, V).** `mm.add({ full: '(prefers-reduced-motion: no-preference)', reduced: '(prefers-reduced-motion: reduce)' }, ctx => ...)`; `ctx.conditions` says which matched. It uses `gsap.context()` internally; setup reverts when the query stops matching, and a returned function is for non-GSAP cleanup only. https://gsap.com/docs/v3/GSAP/gsap.matchMedia()/ Verified by stubbing `window.matchMedia` to report `reduce` and remounting: 0 running tweens, edges set to the static colour, `strokeDashoffset` stayed 0. The `useGSAP` page does not mention `matchMedia`; it worked inside `useGSAP` with `scope` here.
- **React Flow cost (V, D).** 12.12.0, 1.2 MB unpacked, +85 kB gzip over the hand-SVG build (table). Default routing with top/bottom handles gave long overlapping swoops for back-edges and same-column edges in my 19-node/30-edge test, so loops need a custom edge path plus left/right handles: the same hand computation as plain SVG, inside a framework. Its own animation docs use SVG `<animateMotion>` or the Web Animations API on custom edges, not GSAP. https://reactflow.dev/examples/edges/animating-edges

- **GSAP and React Flow node wrappers (V).** `.react-flow__node` carries RF's inline `translate3d(...)`; GSAP parsed it and rewrote it as `translate3d(...) scale(...)`, so tweening the wrapper can fight RF's positioning on later moves. Animate an inner element in a custom node instead. Edge `<path>` dashoffset tweens worked.
- **Layout libraries (D, V).** React Flow's layout page: dagre is simple and fast, with no edge routing; elkjs is the most configurable, async, and "by far the largest". https://reactflow.dev/learn/layouting/layouting dagre has `acyclicer: 'greedy'` (default off) to break cycles by reversing or dropping a feedback arc set, so which edge it treats as the "back" edge is the heuristic's choice, not ours. https://github.com/dagrejs/dagre/wiki Our stage order is known, so a column grid gives a deterministic layout for free.
- **Loop-edge cost by option.** Hand SVG: one extra branch in the path function (about 6 lines). React Flow: custom edge component, handle placement and path maths. dagre: node coordinates only (no edge routing per the RF page), so back-edges are still drawn by hand. elkjs: routes edges but is async and +453 kB gzip.

Measured production-build sizes (Vite 8.3.5, minified, gzip)

Build	gzip
React 19 + react-dom only (baseline)	68.6 kB
+ `@gsap/react` + GSAP core + `matchMedia`, hand-written SVG (a)	97.0 kB (+28.4)
(a) + `DrawSVGPlugin` + `MotionPathPlugin`	107.2 kB (+10.3)
React + `@xyflow/react` + GSAP core, no plugins (b)	156.4 kB (153.9 JS + 2.6 CSS)
baseline + `@dagrejs/dagre`	84.3 kB (+15.7)
baseline + `elkjs` bundled	522.1 kB (+453.5)

Test apps (19 nodes, 30 edges from the real `EDGES`/`CONDITIONAL`) are in `C:\Users\SMAYAN~1\AppData\Local\Temp\claude\d--Joveo\82a0791c-35de-4261-b0fd-f9b6c47d5b48\scratchpad\graph-research\{a,b,c}`. Hand SVG saves about 49 kB gzip against React Flow; the larger win is control over loops and no wrapper-transform clash.

Not verified

- Visual quality of the hand-SVG layout: drawn for size and animation tests only; spacing, label widths and back-edge clearance for the real 17 nodes are untuned, and loop overlap is unchecked.
- The real OS `prefers-reduced-motion` setting: I stubbed `window.matchMedia`, not the OS or browser flag.
- StrictMode in a production build: dev double-invocation was tested; the production build was only measured.
- `MorphSVGPlugin`: present in the package, not built or run (not needed for this graph).

- Licence detail: the licence page does not name DrawSVG or MotionPath; the "all plugins free on npm" claim is from the Installation page and the files ship in the npm package.
- Fan-out display for `batch_extract`, mid-session topology changes, narrow viewports, and whether pan/zoom is wanted: design questions, not tested.

Web API and hosting

Type: grilling Status: resolved Assignee: smayankulkarni05 Blocked by: Research: delivering Run events over FastAPI SSE

Question

What is the HTTP surface that lets a browser do everything the CLI does, and what does it need from the Run Controller?

- Endpoints and shapes: start a Run (request text, focus), list Runs, read one Run (`RunView`), resume with an answer, cancel, the event stream, the graph topology, and the Brief files. Status codes for `UnknownRun`, `RunLocked`, `NotResumable`, `InvalidAnswer` and `BalanceTooLow`.
- The wire format of an event, given Research: delivering Run events over FastAPI SSE. `RunEvent` is already a discriminated union on `type`; say whether it goes out unchanged.
- How a browser re-attaches to a Run that is still live after a reload or in a second tab. The Controller has no method for this today, and future scope says re-attach is "built on its own API": decide the smallest Controller change, or why none is needed.
- How a client builds a Pause form: whether the API exposes each answer model's JSON Schema from `ANSWERS`, or the UI hard-codes the three kinds.
- Hosting: how `scout serve` opens one Controller in the app lifespan, serves the built `web/` files, and runs in the no-LLM demo mode on `fake_deps` and the stub graph.
- How the server handles shutdown with a Run live (the Run ends `failed` and resumable, as a CLI exit does).

Resolution is the endpoint table, the Controller change if any, and the demo-mode switch.

Answer

Decided by the agent (the user delegated every decision on 2026-10-31), then checked with a throwaway spike on the real `RunController` over a paced demo graph: real HTTP, the browser pane's `EventSource`, Starlette's `TestClient`, and the Vite proxy. FastAPI 0.162.0, uvicorn 0.54.0, Vite 8.3.5. The spike is outside the repo (session scratchpad, `web-spike\`). "Verified" below means it was run there.

Endpoints, all under `/api`, all `async def`. A sync path op would run on a thread pool, but the lock connection and `aiosqlite` belong to the loop thread.

Method, path	Body	Success	Refusals
`GET /meta`		`{graph, dimensions, countries, categories, confidence_key, demo}`	

Method, path	Body	Success	Refusals
`GET /runs`		`list[RunRecord]`, newest first	
`POST /runs`	`{request_text, focus}`	201 `{thread_id}`	402, 502, 422
`GET /runs/{id}`		`RunDetail`	404
`POST /runs/{id}/resume`	`{answer: dict or null}`	200 `{thread_id}`	404, 409, 422, 402, 502
`POST /runs/{id}/cancel`		200 `Stopped`	404, 409 `not_live`
`GET /runs/{id}/events`		200 SSE, or 204 when nothing is live here	404
`GET /runs/{id}/brief.json`, `brief.md`		the file, as a download	404

Refusals share one body, `{code, message, errors?}`: `unknown_run` 404, `run_locked` 409, `not_resumable` 409, `not_live` 409, `no_brief` 404, `invalid_answer` 422, `invalid_request` 422 (FastAPI's own validation errors, reshaped by one handler), `balance_too_low` 402, `upstream` 502 (`httpx.HTTPError` from `check_balance`).

`InvalidAnswer` must be cleaned. Pydantic's raw `errors()` hold exception objects in `ctx`, which JSON can't carry. The handler sends only `{loc, msg}` and strips the "Value error, " prefix. **Verified** shapes: a field error has `loc: ["plan", "radius_km"]`, a model-level error (reject with no feedback) has `loc: []`, and a refused answer leaves the Run `paused`.

`RunDetail`, not `RunView`. `RunView.state` holds LangChain messages and every Posting's full text, so the API returns a slim model built from it: `record`, `request_text`, `focus`, `pause`, `stopped_at`, `run_log` (`node`, `kind`, `message` only), `live_from`, `brief`, and `sample` (`key`, `title`, `company`, `location`, `url`, `posted_at`, only when there is a Brief). `url` is null unless the scheme is http or https, the same guard as `brief/markdown.py`'s `_link`, so the client never receives a `javascript:` link.

Wire format. `RunEvent` goes out unchanged, as `model_dump_json()` in one unnamed message, plus one synthetic message that is not a `RunEvent`: `{"type": "caught_up"}`, sent once the buffered replay has been written. The client needs that boundary to apply a replay with no animation and to know where lost tokens could be. **Verified exact:** on attaching mid-Run, 0 `Token` or `Reasoning` events came before `caught_up`, because they are queue-only and the queue is read after the replay. `Done.paths` is sent unchanged and the UI ignores it.

Controller changes, the only ones.

1. `_live` becomes `dict[str, RunHandle]`, with `attached(thread_id) -> RunHandle | None`. This is re-attach. The handle already supports several subscribers, so nothing else is needed.
2. `RunHandle.log_start`, the `log_len` already passed to `_spawn`. `RunDetail.live_from` reports it, or null with no handle.

3. `RunHandle.buffered`, the buffer's length, read with no await before `events()` takes its snapshot.
4. `_brief_paths` becomes public as `brief_paths`.
5. The demo switch below.

A refinement to Trace graph: rebuild from `run_log`, not State fields. A resumed Run gets a new handle with a fresh buffer, so a replay covers only the events since the last resume. The client's starting view for a reload is therefore built from `run_log[:live_from]`, and the replay supplies everything after it. That has no overlap and no race, because `live_from` is fixed per handle, and it replaces the State-field rule, so `RunDetail` needs no route facts. Three nodes write no log line in normal use (`fetch_agent`, `tools`, `batch_extract`), so the rebuild infers them: a `collect` line means Search ran before it, and a `normalise_skills` line means Read postings ran before it.

Stream lifecycle, verified in the browser pane. A live handle gives SSE. With no handle, a dependency raises `HTTPException(204)` and `EventSource` goes to `readyState` 2 (closed) with no retry in 7 s. An unknown ID gives 404, which also stops it. `Paused` reaches the client just before the task ends, so right after it `GET /runs/{id}` still said `running` with a live handle. **The client trusts the event and never refetches to confirm it.** The sidebar's refetch after a terminal event waits 300 ms, and `Done`'s fetch of the Brief retries while the status still says `running`. An immediate resume right after `Paused` returned 200 in the check, but the client retries once at 300 ms on `run_locked`, which also covers reconcile's probe (Runtime LLD §8).

Cancel is `handle.cancel()` on the attached handle. **Verified:** it gives `Stopped(node, "cancelled")` and `failed`, and Resume with `answer: null` goes on from that node. On a paused Run it is 409 `not_live`, since abort is the Pause's own answer.

Pause forms: the API does not expose JSON Schema. The UI renders three typed kinds, because a generated form could not read well, and `/meta` serves only the lists it can't hard-code without drift (countries, categories, dimensions, the Confidence key).

A Run started by the CLI shows `running` with `live_from: null`. The UI says it is running elsewhere and polls `GET /runs/{id}`.

Local security, which the delegation did not remove. The server has no auth, can spend OpenRouter and Adzuna quota, and any web page can send requests to localhost.

- Binds `127.0.0.1` only, with no `--host` flag.
- `TrustedHostMiddleware(allowed_hosts=["127.0.0.1", "localhost"])` stops DNS rebinding. **Verified:** a foreign `Host` gets 400.
- A `same_origin` dependency on every POST: if `Origin` is present and its host differs from `Host`, 403. **Verified**, including a body-less cross-origin cancel, which JSON content-type checks alone would miss. A `text/plain` JSON POST is also refused by FastAPI (422).
- No `BaseHTTPMiddleware`, so nothing sits between the SSE response and disconnect handling. Both guards are pure ASGI or a dependency.
- **Vite's proxy must be `{target, changeOrigin: false}`.** With the string shorthand, **verified**, every dev POST gets 403, because `Host` no longer matches `Origin`. With the object form, same-origin

POSTs give 201, a foreign one 403, and SSE and 204 pass through.

Hosting. `scout serve [--port 8000] [--demo]` is a new subcommand in `clients/cli.py`, importing `clients/web.py` lazily. `uvicorn.run(app, host="127.0.0.1", port=port, timeout_graceful_shutdown=3)`. The lifespan opens one `RunController` on `app.state`. Static files: `/assets` mounted from `web/dist/assets`, and a catch-all GET returns `index.html` so `/runs/:id` reloads work, with `/api/*` staying a real 404. If `web/dist` is missing, the catch-all says to run the build. Uvicorn runs one worker and no `--reload`, since a reload would stop live Runs.

Shutdown. Verified: with a Run live, the lifespan exit cancels it, and after a restart it reads `failed`, `cancelled`, with no live handle, and it is resumable. The existing `open()` already does this.

Concurrent Runs. Verified with two at once on one Controller. There is no Run limit. On the real Gateway they share the atomic daily budget counter, so the first come is served first, which settles the HLD's open question for this slice.

Demo switch. `Settings.demo: bool = False`, and `demo_delay: float = 0.4`. `open_deps` yields `fake_deps` marked `demo=True`, `_check_balance` skips, and `build_graph` overlays `demo_nodes()` and `DEMO_ROUTERS` from a new `runtime/demo.py`. The bare stubs emit no tokens, tools or delays, so a UI can't be built against them. **Verified:** a small `BaseChatModel` scripted with `_astream` delivers real `Token` and `Reasoning` events through `messages` mode, and six `Send`s give real `ExtractProgress` 1/6 to 6/6 with one `NodeStarted`. `--demo` builds its settings with `make_settings` so it needs no `.env`, and uses `.cache/demo/` for `runs_db` and `runs_dir`, so demo Runs never reach real history. A request containing "fail" raises once in `normalise_skills`, which exercises `Stopped` and Resume.

Tests. `TestClient(app, base_url="http://127.0.0.1")` works and reads SSE to a Pause, since the stream is bounded (**verified**). Its default host is refused by the Host guard, which is why the base URL is set.

Noticed: each load logs "Blocked deserialization" for `HiringRequest`, `LogKind`, `RunLogEntry`, `ClarifyingQuestion`, `SearchPlan` and `MarketStats`. `State.model_validate` copes, so it is noise, but a long-lived server would show it in its log. Pass 1 lists them in `ALLOWED_MSGPACK`.

Not verified: a real tab close, or a real reverse proxy. Neither changes the design for one local process.

App shell and history sidebar

Type: prototype Status: resolved Assignee: smayankulkarni05 Blocked by: none

Question

How does the app look and navigate, so that earlier Runs sit in a sidebar the way Claude and ChatGPT list chats?

- The page frame: sidebar, a main column for the Run (request, live stream, Pauses, Brief), and where the trace graph sits in relation to it (side panel, top strip or collapsible drawer).
- A sidebar row: the title (`RunRecord.summary`, first 80 characters), a status badge for `running | paused | done | aborted | failed`, a relative time, and the grouping (Today, Yesterday, Earlier). Which rows are attention-worthy (`paused`, `failed`), and how a `running` row shows itself live.
- A "New Run" entry and the composer: request box, focus chips for the `Dimension` values plus the `salary` alias, and the empty state.
- Routing: `/` for a new Run and `/runs/:id` for an opened one, so a Run is a link; what opening a finished, paused, failed or running Run shows first.
- How the sidebar learns of changes (poll `list_runs`, or ride the event stream), given the Controller offers no global feed.

Raise fidelity cheaply: a static React/HTML mock with fake data covering each status, reviewed in the browser by the agent. The resolution records the layout, the sidebar row design and the routes, and links the mock.

Answer

Decided by the agent (the user delegated every decision on 2026-10-31) after reviewing three structurally different frames on one page: the prototype, switched with `?variant=A|B|C` or the arrow keys. It is throwaway and holds fake data for every status.

Winner: frame A's skeleton, with C's attention grouping and B's progress bar.

- A (fixed sidebar, centred column, trace panel at the right) is the closest to the Claude and ChatGPT idiom the user asked for, and gives the Run the full height.
- B hides history behind a button, which is not a sidebar. Kept from it: a thin full-width progress bar and the compact stage strip, but only as the narrow-screen layout.
- C's "Needs you" group is the best idea in the set: a user with five Runs open needs to see which ones are waiting on them. Its flat History list lost the date groups, so those stay from A.

Layout.

- Left: sidebar, 272 px. Centre: one column, max 740 px, holding the request bubble, the live stream or the Pause card, then the Brief. Right: the trace panel, 316 px, shown on any opened Run and absent on a new one.
- Under about 900 px wide, the sidebar becomes a slide-over opened from a header button (with a count of Runs needing the user), and the trace panel collapses to a one-line stage strip under the header, with the thin progress bar below it. This settles most of the "responsive depth" fog.

Sidebar.

- Top: product name, then a "New Run" button.
- A **Needs you** group is pinned above the date groups, shown only when non-empty: every `paused` and `failed` Run, newest first. These rows are bold. They appear only here, not again in their date group.
- Below it, `running` and finished Runs under Today, Yesterday, Previous 7 days and Earlier, by calendar day of `updated_at` in the browser's timezone, newest first.
- A row is the title (`RunRecord.summary`, one line, ellipsised by CSS), then a status badge and a relative time. No rename, delete or search (out of scope).
- Badge words: `running` "Running" (blue, pulsing dot, the one live signal), `paused` "Needs you" (amber), `failed` "Stopped" (red), `done` "Done" (green), `aborted` "Aborted" (grey). Each badge pairs its colour with a word, so colour is never alone.
- The selected row gets a white background and a hairline border.

Composer (the New Run page). A centred heading, one textarea, the five `Dimension` values as toggle chips (Demand, Compensation, Skills, Competition, Titles & seniority), and a Start button. The CLI's `salary` alias is not offered: the Compensation chip is the same Dimension. Enter submits and Shift-Enter adds a line. A blank request is not submittable.

Routes. `/` is the composer, `/runs/:id` an opened Run, so every Run is a link. Start navigates to `/runs/:id` once the API returns the thread ID. An unknown ID shows "Run not found" with a link to `/`. What an opened Run shows first, by status:

- `running`: the request, then the live stream attached from the buffer (needs re-attach, decided in Web API and hosting).
- `paused`: the request and the Pause card.
- `failed`: the request and a "Stopped at <step>" banner with Resume.
- `aborted`: the request and the reason, no Brief.
- `done`: the request, then the Brief.

How the sidebar learns of changes. It polls `GET list Runs`: every 3 s while any row is `running` and every 15 s otherwise, plus a refetch on window focus and right after the user starts, resumes, cancels or answers. Polling needs no Controller change and the list is small (one SQLite read, no lock). The open Run's own state still comes from its event stream, and a stream's terminal event triggers one immediate refetch. A global feed is not built.

Not settled here (left to the named tickets): the stream and Pause card content (Live Run view, Pause forms), the Brief (Brief view), and the trace panel's real contents, which are a placeholder of four stage rows in the prototype (Trace graph). Dark mode stays in the fog.

Trace graph

Type: prototype Status: resolved Assignee: smayankulkarni05 Blocked by: Research: graph rendering and GSAP in React

Question

How does the user-level trace graph show which tool or step the system is on, and what remains?

- The node set a user sees: all of the HLD §4 graph, or grouped by owner stage (`request_plan`, `collection`, `analysis`, `brief`) with the Pause nodes drawn as "waiting for you". The friendly label for each raw node name and each tool (`search_jobs` and the like), and how API calls (`ApiCalled`, per source, cached or not) appear on the Collection step.
- Node states and what each looks like: idle, active, done, failed, skipped, waiting on a Pause.
- Loops and branches: `broaden` back to `fetch_agent`, `clarify` back to `intake`, plan edit back to `plan`, critic back to `write_brief`. Drawn as a back-edge, a counter on the node ("Attempt 2"), or both.
- The fan-out of `batch_extract` as one node with an inline progress ring or bar from `ExtractProgress`, not per-branch nodes.
- The overall progress bar: what it measures (nodes passed, weighted by stage) and what it does on a loop.
- A finished or reloaded Run: whether the graph can be rebuilt from `get_run` (`run_log` and state) with no stored events, or the Controller must persist them. This settles the fog item on persisted event history.
- The renderer chosen from Research: graph rendering and GSAP in React.

Prototype the graph with a scripted event feed, reviewed in the browser by the agent. The resolution records the node and label table, the state model, the progress rule and the renderer, and links the prototype.

Answer

Decided by the agent (the user delegated every decision on 2026-10-31) by building the prototype: the real `RunEvent` shapes replayed through one pure reducer into a hand-written SVG animated by GSAP 3.15.0, with three scripted Runs (a happy path with one question and one critic revision, a widen-twice-then-ask Run, and a Run that fails during extraction) and three reopened Runs. Serve the folder over http to open it. It is throwaway; the reducer is the part worth carrying into the build.

What the user sees: 14 steps in four bands, not 17 raw nodes. The bands are the owner stages, and a label table in the UI merges plumbing nodes into one step, so the graph reads as what the system is doing rather than how it is wired.

Band	Step (label)	Raw nodes	Optional
Understand	Read your request	`intake`, `clarity_check`	
	Ask you	`clarify`	yes, a Pause
	Plan the search	`plan`	
	Your approval	`plan_review`	a Pause
Collect	Search job boards	`fetch_agent`, `tools`	
	Filter postings	`collect`, `relevance`	
	Widen the search	`broaden`	yes
	Too few postings?	`low_data_review`	yes, a Pause
Analyse	Read postings	`batch_extract`	
	Tidy skill names	`normalise_skills`	
	Compute the numbers	`stats`	
Brief	Write the brief	`write_brief`	
	Check every claim	`critic`	
	Finish	`render`	

- **Topology stays on the backend; labels stay in the UI.** The server's graph endpoint returns `{nodes: [{id, owner}], edges: [{source, target}]}` from `OWNERS`, `EDGES` and `CONDITIONAL`. The UI orders steps by owner band then node order, maps each edge through the label table, drops self-edges (the `fetch_agent` and `tools` loop) and `START`/`END`, and treats an edge to an earlier step as a back-edge. The prototype asserts the five back-edges this yields: `ask→read`, `approve→plan`, `widen→search`, `few→widen`, `check→write`. A node missing from the table should be a build-time test failure, so a new node can't silently go unlabelled.
- **The spine is the timeline, and only back-edges are drawn as edges.** Steps sit on a vertical rail in order; the rail segment into the live step is a flowing dashed blue line, and segments already passed turn green. Forward edges that skip a step (`read` to `plan` past `ask`) are not drawn: the skipped step shows "Not needed" instead.
- **Layout: a fixed vertical rail,** one 214 px-wide node per step, 22 px between nodes, back-edges bowed to the right (the panel is 316 px). The `Search` node is taller and holds source pills. This replaces the shell's 4-row placeholder, so the panel scrolls and keeps the live step in view.

States, each readable without colour: idle (hollow circle), active (spinner arc, pulsing ring, flowing rail), done (filled green circle with a drawn check), waiting for the user (amber, pause bars, slow ring, "Waiting for you to approve" and so on), failed (red cross, "Stopped. Resume retries it"), skipped (faded, dashed, "Not needed"). Optional steps have a dashed outline from the start. A step is done when the Run moves to a different step, or on `Done`; a Pause only ever marks its own step waiting.

Tools and API calls.

- A tool call shows on the `Search` step, not as its own node: while a call is in flight the step's line reads `Adzuna · "data engineer" p1`, and on `ToolReturned` it falls back to the last note.
- Per source, a pill under the step: filled blue with the count when live, hollow with "· cached" when from cache, red with a cross when failed. Tool names map `search_adzuna` to Adzuna and `search_usajobs` to USAJOBS; an unknown tool shows its raw name.

Loops. A step that runs again gets a counter badge ("×2"); the back-edge turns amber and a dot travels back along it. Steps between the target and where the Run was reset to idle with their notes cleared, but optional steps that ran (`ask`, `widen`, `few`) stay done so they are never marked skipped. The progress bar never goes backwards.

Fan-out. `batch_extract` is one step, "Read postings", with a bar and "3 of 6 batches" from `ExtractProgress`; the bar turns green when done. No per-branch nodes, as no event carries a branch ID.

The progress bar is a weighted share of the main path: Read your request 1, Plan 1, Your approval 0.5, Search 3, Filter 1, Read postings 4 (by batch fraction), Tidy 1, Compute 1, Write 2, Check 1, Finish 0.5. Optional steps weigh 0. An active step counts 35% of its weight (the extraction fraction for Read postings), a done step counts all of it, and `Done` snaps to 100%. It never decreases on a loop. It turns amber while waiting on a Pause and red on `Stopped`. Under the bar, a line names the live step ("Search job boards…", "Waiting for you: Your approval", "Stopped at …").

Reload and reconnect. The reducer is pure and the buffer replays from the start (see Research: delivering Run events over FastAPI SSE), so on reconnect the client applies the buffered events at once with no animation and then goes on live. Checked: replaying the events gives an identical state.

A reopened Run needs no stored events, which settles the fog item: rebuild from `get_run`. (Web API and hosting later refined the source from State fields to `run_log`, so one rule also serves a reload mid-Run. No events are stored either way.) The status, `stopped_at` or the pending Pause kind place the cursor; steps before it are done; the optional steps use State fields: `questions` non-empty means Ask you ran, `len(broadened_plans)` is how many times Widen ran (Search shows ×N+1), and `low_data_decision` set means the low-data Pause ran. Checked on a done, a failed and a paused Run. What a reopened Run loses: tool and source lines, per-step notes and attempt detail. The trace shows the route and outcome, which is all the shell's reopened Runs need. No Controller change.

Narrow screens use the same state: four stage chips (done, active, waiting or failed) plus the live-step line and the progress bar.

Motion shown here, for Motion with GSAP to formalise: spinner rotation, pulse ring, rail dash flow, a drawn check (`DrawSVGPlugin`), a dot along the back-edge (`MotionPathPlugin`), a counter pop, a failure shake, bar width tweens and smooth scrolling to the live step. State drives every tween. Under reduced motion no repeating tween runs (3 live loops became 0 in the check) and every change is a plain state change. Checked in the browser pane: no console errors.

Renderer: hand-written SVG, as Research: graph rendering and GSAP in React recommended. The prototype is vanilla JS; the build uses React with one scoped `useGSAP`, and a reducer that is this one rewritten in TypeScript.

Not settled here: how the real UI places the Pause card relative to its waiting step (Live Run view), and the full animation inventory and timings (Motion with GSAP).

Live Run view

Type: grilling Status: resolved Assignee: smayankulkarni05 Blocked by: Web API and hosting, App shell and history sidebar

Question

How does a running Run read, as a chat-style stream with real-time tokens and tool-call loading states?

- The message model for the stream: what is a turn, and how `NodeStarted`, `NodeDone` notes, `Token`, `Reasoning`, `ToolCalled`/`ToolReturned` and `ApiCalled` become blocks in it.
- Token streaming: only `fetch_agent` sends `Token`s, the other LLM nodes send only `Reasoning`, and structured output is never shown as fragments. What the user sees while `clarity_check`, `plan`, `broaden` and `write_brief` draft, and how `Reasoning` shows (collapsed "thinking", dimmed) and copes with being absent.
- Tool and API loading: the card for a tool call from `ToolCalled` to `ToolReturned` (spinner, then summary and count), and a failed `ApiCalled`.
- Extraction progress as a bar from `ExtractProgress`.
- Reconnecting: the client attaches mid-Run and gets every node, tool and API line from the buffer but no earlier tokens. How the stream avoids a visible gap, and what a dropped connection shows.
- The `Stopped` and `Done` events: the failed-Run banner with Resume, the Cancel button, and the hand-off to the Brief view.
- Auto-scroll, with a "jump to latest" control when the user scrolls up.

Resolution is the block types, the reducer from event to view state, and the reconnect behaviour.

Answer

Decided by the agent (the user delegated every decision on 2026-10-31).

A turn is one visit to one Step, using Trace graph's label table. A `NodeStarted` whose Step differs from the current turn's opens a new turn. `fetch_agent` and `tools` alternating stay in one "Search job boards" turn, and a loop opens a new turn, because the stream is the chronological record and the trace is the map. The active turn is expanded. A finished turn collapses to its check, label and last note, and clicking it shows the rest. Once a Run is `done`, every turn folds under one "How this Run went" `<details>`, closed, above the Brief.

Blocks in a turn, and the event behind each.

Event	Effect
`NodeStarted`	New turn if the Step changed. If the node drafts, a `drafting` line with a spinner
`Reasoning`	Appends to a `thinking` block: a closed `<details>` reading "Thinking…", then "Thought", body dimmed. Created on the first fragment, so no reasoning means no block
`Token`	Appends to a `prose` block as plain text with `white-space: pre-wrap`, and ends the drafting line. No Markdown parsing
`ToolCalled`	Opens a `tool` card: spinner, source name, summary
`ApiCalled`	Sets the source pill on the first card named `search_<source>` without one, or a turn-level pill if none: count, "cached", or red "failed"
`ToolReturned`	Resolves the oldest open card of that name: check, then the summary
`ExtractProgress`	Sets the turn's `progress` bar and "3 of 6 batches"
`NodeDone`	Appends its notes, and ends the drafting line
`Paused`	Pause card below the stream
`Done`, `Stopped`	See below
`caught_up`	Sets `animate: true`, and adds a `gap` line if needed

- Tools pair first-in-first-out by name, since no event carries a call ID. `tools` runs calls in a `gather` but emits `tool_returned` in call order, so this is exact.
- A card still open at `Stopped` reads "Interrupted".
- **Drafting phrases:** `clarity_check` "Working out what's unclear…", `plan` "Drafting the search plan…", `broaden` "Working out how to widen the search…", `write_brief` "Drafting the brief…", `critic` "Checking claims against the numbers…". JSON fragments are never shown. Other nodes show only their label.
- `Reasoning` stays closed by default: it is a terse scratchpad (Runtime LLD §10), and the reader hands Briefs to clients.

The reducer is one pure function over `RunState = {turns, trace, pause, ending, connection, animate}`, and the trace's slice is the prototype's reducer in TypeScript. **Batching:** events go into a queue flushed once per frame (`requestAnimationFrame`, with a 250 ms timeout because frames stop in a hidden tab), and the reducer folds the batch. So one React render per frame carries any token rate, and a replay settles in one go. Finished turns are `memo`ised.

Starting state. `fromLog(detail)` groups `run_log[:live_from]` into turns by consecutive Step, and places the trace. With no live handle it reads the whole log and uses `record.status`, `pause` and `stopped_at`, so a reopened Run has its whole route and notes but no tokens or tool cards.

Connecting, with one rule for every path. Each stream starts from a `base` state, and `onopen` resets to `base`, because a replay starts from the current handle's start:

- First open of a new Run: empty base.
- Reload: `fromLog`.
- After answering a Pause or pressing Resume: the state on screen, with the Pause marked answered, becomes the base. This local action also takes the Step out of waiting, since the rerun Pause node's `NodeStarted` wouldn't.
- Live text sits in a separate `texts` map keyed by turn index, which `onopen` does not reset. Replays are deterministic, so indices line up, and a dropped connection keeps what was already on screen.

Avoiding a visible gap. Everything but tokens returns from the buffer. Text sent during the outage is lost, so `caught_up` adds one muted `gap` line, "Some live text wasn't kept", when the active turn is one that could have streamed and it was reached by replay.

Dropped connections. On `error` with `readyState` 0 (reconnecting), after a 2 s grace, a small "Reconnecting…" pill shows at the end of the stream. On `readyState` 2 (204 or 404), the client fetches `GET /runs/{id}` and rebuilds from it. That also covers a server restart mid-Run, which ends in the failed banner.

Endings. The client calls `close()` on every terminal event.

- `Paused`: the card appears from the event's own payload, and the sidebar refetches after 300 ms.
- `Done` with `done`: fetch `RunDetail`, retrying up to 5 times at 200 ms while it says `running`, then hand off to the Brief view. `aborted`: a notice with the reason, the last `run_log` message or `record.error` (the same rule as `scout show`), and a link to `/`.
- `Stopped` with a node: a banner, "Stopped at <Step>. Nothing is lost; Resume retries that step", with Resume (`answer: null`) and the raw reason in a `<details>`. If the reason is `cancelled`, it reads "You stopped this Run at <Step>". With `node: null`, nothing was checkpointed, so there is no Resume, only a link to `/`.

Stop. While a Run is live here, the Run header has a Stop button, with no confirmation, since Resume undoes it. A `running` Run with `live_from: null` shows "Running in another window" and polls `GET /runs/{id}` every 3 s, with no Stop.

Auto-scroll. The stream sticks to the bottom while a sentinel at the end is within 80 px of view (`IntersectionObserver`). Scrolling up unsticks it and shows a "Jump to latest" pill, which reads "Waiting for you" when a Pause arrived off-screen. The scroll write happens once per flush.

Pause placement. The card sits inline, at the end of the stream, so it is where the reader's eyes already are. The trace's waiting Step is clickable and scrolls to the card.

Accessibility. The stream is `role="log"` with `aria-live="off"`, since announcing every token would be unusable. A visually hidden `aria-live="polite"` region announces Step changes, Pauses and endings. Pause headings take focus on arrival when the stream is stuck to the bottom.

Pause forms

Type: grilling Status: resolved Assignee: smayankulkarni05 Blocked by: Web API and hosting

Question

How does each Pause kind render as an inline card in the stream and take an answer?

- `clarify`: each `ClarifyingQuestion` with its options as chips plus a free-text field, and skipping a question.
- `plan_review`: the Search Plan and Assumptions shown readably, with approve / edit / reject (feedback) / abort. How `edit` works in a browser: a structured form over `SearchPlan`'s fields, or the JSON editor the CLI uses with inline validation.
- `low_data_review`: the counts, plans tried and any block reason, with broaden (optional guidance) / continue / abort.
- `InvalidAnswer`: how the errors show against the same card, with no state changed.
- A Pause on reopening a paused Run from the sidebar, and a Pause answered in another tab.
- Reuse of `clients/pauses.py`'s text formatting, or a typed rendering that replaces it for the web.

Resolution is one card design per kind and the answer payload each posts.

Answer

Decided by the agent (the user delegated every decision on 2026-10-31). The refusal shapes were checked in the spike described in Web API and hosting. Every card posts to `POST /api/runs/{id}/resume` as `{answer: <payload>}`. `PlanReviewAnswer` has no field defaults and forbids extras, so all three of its keys are always sent (**verified:** omitting them is refused).

Kind	Action	`answer` payload
`clarify`	Send	`{answers: string[]}`, one per question in order, `""` for a skipped one
`plan_review`	Approve	`{action: "approve", plan: null, feedback: null}`
	Edit	`{action: "edit", plan: <SearchPlan>, feedback: null}`
	Reject	`{action: "reject", plan: null, feedback: "<text>"}`
	Abort	`{action: "abort", plan: null, feedback: null}`
`low_data_review`	Search wider	`{action: "broaden", guidance: "<trimmed>" or null}`

Kind	Action	`answer` payload
	Continue	`{action: "continue", guidance: null}`
	Abort	`{action: "abort", guidance: null}`

`clarify`: "A quick question" or "A few quick questions". Each question has its text, its `options` as real radio inputs styled as chips, and a free-text box. Picking a chip fills the box, and typing clears the chip, so one value is sent. Blank is a skip, which `clarify` already handles by position. The button reads "Send answers", or "Continue without answering" when all are blank.

`plan_review`: "Check the search plan". A readable list, not field names: Titles as chips, Where (`location` or "Country-wide", with the country code in capitals), Radius, Posted within N days, Target N Postings, Category (or "Any"). Below it, Assumptions as "seniority: Senior, because …". Actions: Approve (primary), Edit, Ask for a different plan, Abort.

`edit` is a structured form, not the CLI's JSON editor. A customer success manager should not be editing JSON, and `SearchPlan` is seven flat fields:

- `titles`: 1 to 4 text rows, with add and remove
- `country` and `category`: selects from `/api/meta`, the same sets the validators check, with `category` also offering "Any"
- `location`: text, blank meaning country-wide
- `radius_km` (0 to 100), `max_days_old` (1 to 60), `target_postings` (20 to 200): number inputs with those limits

The HTML limits only help. The server stays the check. "Use this plan" submits.

`low_data_review`: "Too few Postings". "Only 3 relevant Postings found; the target is 20." "Fetched 50, rejected 47 as not the role, over 1 Attempt." Then `blocked` if set, Adzuna calls left if known, and the plans tried in a `<details>`. A guidance textarea, optional, feeds "Search wider". Then "Continue with 3 Postings" and Abort. Guidance is blank-to-null as in `low_data_answer`, in TypeScript.

Abort is two-step everywhere, an inline "Abort this Run? Yes / No", because it is the one irreversible answer.

`InvalidAnswer`. The card stays, values kept. `errors` with `loc[0] === "plan"` show under the field named by `loc[1]`, anything else at the top of the card. Reject's feedback is also blocked client-side when blank. Buttons disable while posting.

Other refusals.

- `run_locked`: retry once after 300 ms, then "This Run is open elsewhere".
- `not_resumable`, or `invalid_answer` saying the Run isn't waiting: "Answered elsewhere", then refetch `RunDetail` and rebuild.
- `balance_too_low` or `upstream`: the message at the top of the card, and the Run stays `paused`, so it can be retried.

After answering, the card collapses to a one-line record in the stream ("You approved the plan", "You answered 2 questions"), then the new stream opens. On a reload, the record is the raw message of the `pause` entry in `run_log` ("plan approve").

Reopening. A `paused` Run's card renders from `RunDetail.pause`, identical to a live one. **Another tab:** the sidebar poll sees the open Run's `status` or `updated_at` change, and the page refetches and re-attaches. So a stale card lasts at most one poll interval, and anything posted in that gap meets the refusals above. A stale answer can only pass if the Run is at a later Pause of the same kind, which needs a reject loop plus a second tab, and that is accepted.

`clients/pauses.py` stays, for the CLI alone. The web renders typed payloads, since text formatted for a terminal has no chips or fields to style. Its "shared by every client" docstring and `event_line`'s mention of Streamlit get corrected when Streamlit goes.

web-ui/issues/08-brief-view.md

Brief view

Type: grilling Status: resolved Assignee: smayankulkarni05 Blocked by: Web API and hosting

Question

How does the finished Market Brief read in the browser, for a customer success manager who will hand it to a client?

- The layout of Summary, Insights (Confidence Tag, Evidence count and the cited Stats), Client Recommendations (title, salary band, top skills), the Hiring Outlook rating with its reason, and Run Notes (questions and answers, final Search Plan, Postings analysed, Assumptions, Caveats).
- How a Confidence Tag, the three Outlook ratings and a Caveat are made visible without relying on colour alone.
- Whether Evidence expands to show the cited Stats, and what a user can click through to.
- Rendering from `brief.json` structured data or from `brief.md`, and the JSON and Markdown downloads.
- An `aborted` Run, which has no Brief: what the page shows instead.
- A print-friendly view, if it costs a stylesheet only.

Resolution is the section order, the component per section and the data each reads.

Answer

Decided by the agent (the user delegated every decision on 2026-10-31).

Render from structured data, `RunDetail.brief` and `sample`, not `brief.md`. Evidence needs to expand, and tags need components. It also means no Markdown renderer and no raw HTML, so Posting text from outside stays escaped by React. Downloads are the server's files.

Section order is `brief.md`'s, so the screen, the download and the print agree: the answer first, the working last.

#	Section	Component	Reads
1	Title, Downloads	`BriefHeader`	`hiring_request.role`, `.location`; links to `brief.json`, `brief.md`; Print
2	Replay banner, Weak Sample notice	`BriefNotices`	`replayed`; `stats.sample_size < run_notes.final_plan.target_postings`
3	Summary	`Summary`	`summary`

#	Section	Component	Reads
4	Hiring Outlook	`Outlook`	`outlook`
5	Client Recommendations	`Recommendations`	`recommendations.job_title`, `.salary_band`, `.top_skills`
6	Insights	`Insights`	`insights`, and `/meta`'s `confidence_key` as a closed `<details>`
7	Run Notes	`RunNotes`	`run_notes`
8	Postings analysed	`PostingsTable`, closed `<details>`	`sample`

The Weak Sample and "Not enough evidence" wording match `brief/markdown.py`. The replay banner is its `REPLAY_BANNER`.

Never colour alone.

- Confidence Tag: a pill with the word and a three-dot meter (●●● High, ●●○ Medium, ●○○ Low), then `confidence_reason` in plain text beneath.
- Hiring Outlook: a large rating word with a shape, ▲ Favourable, ◆ Competitive, ▼ Hard to fill, then the `reason`.
- Caveat: a warning icon and the word "Caveat" on each, in Run Notes.

Run Notes stay open, not folded, with Caveats first. The glossary says Caveats are always shown, and a closed fold would hide what most affects trust. Then Questions and answers, Assumptions, Focus, Final Search Plan, earlier plans, Postings analysed by source (counted from `posting_keys` prefixes), source match counts, attribution.

Evidence is one `Evidence` component, used by Insights, Recommendations and the Outlook. "Evidence: 38 Postings" is a button that expands inline to:

- The cited Stats, from `stat_ids` looked up in `brief.stats`: label, value and unit. These are what a figure rests on.
- The Postings, from `posting_keys` looked up in `sample`: title and company, linked to the source, ten at first with "Show all 38". `url` is already null unless http(s), and a null shows no link. Links use `target="_blank" rel="noopener noreferrer"`.

Two details from the types: `Evidence.count` is a Python property and isn't in the JSON, so the UI uses `posting_keys.length`. An Outlook at its default has empty `stat_ids` and shows no Evidence, as the Markdown does.

Formatting. A salary band is `low to high unit` with no decimals via `toLocaleString`, and a numeric Stat likewise, matching the Markdown. Missing recommendations show "Not enough evidence to suggest…". Zero Insights shows "None: this Brief has no findings."

An `aborted` Run shows the request, then "This Run ended without a Brief", the reason, a link to `/`, and the turns from `run_log`. No download links, and those endpoints return 404 `no_brief`.

Print costs a stylesheet only. `@media print` hides the sidebar, trace panel, the "How this Run went" fold and all buttons, and puts the column at full width. Closed `<details>` stay closed, so the complete Posting list belongs to the Markdown download.

Motion with GSAP

Type: grilling Status: resolved Assignee: smayankulkarni05 Blocked by: Trace graph, Live Run view

Question

Which animations does the UI have, and how does GSAP drive each one without making a streaming page jittery?

- The inventory: node activation and completion, edge flow along the active path, the overall progress bar, extraction progress, tool-card enter and resolve, token reveal, sidebar row enter and status change, Pause card attention, and the Brief arriving.
- Timing and easing tokens shared by all of them, and a motion budget (what may run at once while tokens stream at speed).
- Mapping state to animation: state change drives a tween, never the other way round, so a burst of events or a reconnect replay settles on the final state rather than playing back every step.
- `useGSAP` scoping and cleanup per component, from Research: graph rendering and GSAP in React.
- `prefers-reduced-motion`: what is removed, and what remains as a plain state change.
- Performance: avoiding layout thrash on the token stream, and pausing off-screen tweens.

Resolution is the animation table (trigger, target, tween, duration) and the shared tokens.

Answer

Decided by the agent (the user delegated every decision on 2026-10-31). The timings are the ones reviewed in the trace prototype, collected into tokens.

The one rule: React renders the final state, and GSAP only plays the way into it. Classes and attributes carry every state, so tweens are `from` or decorative. It follows that:

- Reduced motion is simply no tweens, with nothing else to write and no state lost.
- A burst or replay can't play back steps, since a frame's batch renders once.
- With `animate: false` (before `caught_up`, and on any reopened Run) no one-shot fires.

One helper, `useMotion(fn, deps, scope)` in `motion.ts`, wraps `useGSAP` with `revertOnUpdate: true`, and runs `fn` inside `gsap.matchMedia().add("(prefers-reduced-motion: no-preference)", ...)`. Each animated component owns one call scoped to its own element, with `deps` being its state, so a loop ends itself when the state changes. Plugins register once in `motion.ts`. Handlers use `contextSafe`. Previous state comes from a ref, to tell an entry from a re-render.

Tokens.

```
DUR  = { fast: 0.15, base: 0.25, slow: 0.5 }
EASE = { out: "power2.out", pop: "back.out(3)", linear: "none", settle:
"elastic.out(1.2,0.3)" }
```

Trigger	Target	Tween	Duration, ease
Step becomes active	Step icon arc	rotate 360, repeating	0.9 s, linear
	Step ring	opacity .55 to 0, scale 1 to 1.04, repeating	1.3 s, out
Rail segment is live	Segment `stroke-dashoffset`	10 to 0, repeating	0.6 s, linear
Step becomes done	Check path	`drawSVG` 0 to 100%	0.35 s, out
Step starts waiting	Step ring	opacity .6 to 0, scale 1 to 1.05, repeating	2 s, sine.out
Step fails	Step group	x -3 to 0	slow, settle
Counter changes	Badge	scale 1.6 to 1	0.4 s, pop
Loop taken (`fx`)	Dot on the back-edge	`motionPath` 0 to 1, then hidden	0.9 s, power1.inOut
Progress changes	Overall bar	`scaleX`, origin left	slow, out
`ExtractProgress`	Turn's bar and the Step's bar	`scaleX`	base, out
New turn	Turn	y 6 to 0, opacity 0 to 1	base, out
Tool card enters	Card	y 4 to 0, opacity 0 to 1	fast, out
Tool card resolves	Spinner out, check `drawSVG`	as Step done	0.35 s, out
Prose is streaming	Caret	opacity 1 to 0, repeating	slow, stepped
Sidebar row is `running`	Badge dot	opacity 1 to .25, scale to .7, repeating	0.6 s, sine.inOut
New sidebar row	Row	opacity 0 to 1	base, out
Row status changes	Badge	opacity 0 to 1	base, out
Pause card arrives	Card	y 8 to 0, opacity, then border pulse twice	base, out
Brief arrives live	Sections	y 8 to 0, opacity, stagger .06	base, out

Token reveal has no per-token tween. Text is appended per frame. A tween per fragment is exactly the jitter this ticket warns of, and the caret is the only motion on prose.

Budget. Only `transform`, `opacity` and SVG attributes tween, never layout, so the prototype's bar `width` becomes `scaleX`, and `<details>` opens instantly with no height tweens. At any moment the repeating tweens are: the active Step's three (arc, ring, rail), one caret, and one dot per `running` sidebar row. One-shots on stream content fire per turn or per card, never per token. At most one loop dot plays per batch.

Scrolling uses native `scrollIntoView({behavior: "smooth", block: "nearest"})` for the live Step, with `"auto"` under reduced motion, so `ScrollToPlugin` isn't needed.

Off-screen. Below 900 px the trace panel is not mounted, so its loops don't exist, and the strip has only its bar. GSAP's ticker follows `requestAnimationFrame`, so a hidden tab costs nothing and `lagSmoothing`'s default avoids a jump on return. No `IntersectionObserver` per tween: the live Step is kept in view anyway.

Reduced motion leaves: the static icons (arc, pause bars, check, cross), final colours, instantly set bars, the counter, and instant scrolling. No repeating tween runs, which the prototype checked (3 live loops became 0).

Bundle: `gsap` with `DrawSVGPlugin` and `MotionPathPlugin` on React 19 built to 106.7 kB gzip with the pinned toolchain, in line with the research estimate.

Frontend build and verification

Type: grilling Status: resolved Assignee: smayankulkarni05 Blocked by: Web API and hosting

Question

How is the frontend laid out, built, served and checked, so that someone can build it step by step?

- `web/` layout, the package manager and pinned versions (Vite, React, TypeScript, `gsap`, `@gsap/react`), and the state approach for the Run reducer (plain `useReducer` and context, or one small store).
- The `web` dependency group in `pyproject.toml` (FastAPI, uvicorn) and how `scout serve` fits `[project.scripts]` beside the `scout` CLI.
- Development: Vite's dev server proxying `/api` to the Python server, and the demo mode as the default way to work.
- Production: the built `web/dist` served by FastAPI, with a single `uv run scout serve`.
- Verification: unit tests for the event reducer with scripted event feeds (Vitest), API tests with FastAPI's test client over the stub graph, and one browser smoke through a stub Run (start, each Pause, Done). The skip-list: no pixel tests.
- Deleting Streamlit (`streamlit_app.py`, `streamlit_bridge.py` and the `streamlit` group) and when.
- A build checklist in passes, each leaving one check behind.

Resolution is the layout, the commands and the checklist.

Answer

Decided by the agent (the user delegated every decision on 2026-10-31). The pins were installed together in a scratch project, where `tsc`, `vite build` and `vitest run` passed, with a `<StrictMode>` GSAP component in the build, on Node 22.14.0 and npm 10.9.2.

Package manager and pins. npm, which needs no install, with a committed `package-lock.json` and exact versions: `react` and `react-dom` 19.3.1, `gsap` 3.15.0, `@gsap/react` 2.1.2; dev: `vite` 8.3.5, `@vitejs/plugin-react` 6.1.2, `typescript` 7.0.2, `vitest` 5.0.5, `@types/react` and `@types/react-dom` 19.3.0.

No router, store or CSS library. Two routes need about 15 lines over `history.pushState` and `popstate`. Each Run's state is one `useReducer` in `useRun`, and the sidebar's is one `useRuns`, with a small context only to let the page ask the sidebar to refetch. Plain `styles.css` with custom properties, lifted from the shell prototype.

`web/` layout.

```
web/
  package.json  package-lock.json  index.html  vite.config.ts  tsconfig.json
  src/
    main.tsx  App.tsx  router.ts  styles.css  motion.ts
    api/      client.ts (fetch, refusal shape)  stream.ts (EventSource lifecycle)  types.ts
    run/      steps.ts (label table, weights)  reducer.ts  fromLog.ts  useRun.ts  RunPage.tsx
Stream.tsx  Turn.tsx  ToolCard.tsx
    pause/    PauseCard.tsx  ClarifyForm.tsx  PlanReviewForm.tsx  PlanEditor.tsx
LowDataForm.tsx
    brief/    BriefView.tsx  Evidence.tsx  RunNotes.tsx
    trace/    TracePanel.tsx  TraceStep.tsx  ProgressBar.tsx  StageStrip.tsx
    sidebar/  Sidebar.tsx  useRuns.ts
    composer/ Composer.tsx
    test/     reducer.test.ts  steps.test.ts  demo-run.json
```

Types are hand-written in `api/types.ts`, kept honest by a contract check rather than a generator. SSE events aren't in FastAPI's OpenAPI anyway. A pytest runs a Demo Run and compares its events and `/meta` with the committed `web/src/test/demo-run.json`, which the Vitest suite feeds to the reducer. A backend shape change fails pytest, and regenerating the file then fails the TypeScript side.

Python dependencies. `fastapi==0.162.0` and `uvicorn==0.54.0` join the main `dependencies`, not a group. The ticket suggested a `web` group, but `uv run` only includes `dev` by default, so a group would break the required single `uv run scout serve`, and this is now the product's main client. `scout serve` is a subcommand of the existing script, with no new `[project.scripts]` entry. `.gitignore` adds `web/node_modules/` and `web/dist/`.

Development.

```
uv run scout serve --demo
npm --prefix web run dev
```

Vite runs on 5173 with `server.proxy["/api"] = {target: "http://127.0.0.1:8000", changeOrigin: false}`. The string shorthand fails every POST, as Web API and hosting records. Demo mode is the default way to work. Backend changes need a manual restart.

Production.

```
npm --prefix web ci
npm --prefix web run build
uv run scout serve
```

Verification.

- **Vitest, pure logic:** the Demo Run feed reaches the expected end state; replaying a feed all at once equals applying it one by one; a reset keeps `texts`; `fromLog` places the cursor per status and

infers Search and Read postings; the label table covers every node in `/meta` and yields the five back-edges; progress never decreases; tool pairing is first-in-first-out.

- **`tests/test_web_api.py`, over the demo graph with `demo_delay=0`:** the whole of Web API and hosting's table through `TestClient(app, base_url="http://127.0.0.1")`: every refusal and its shape, `caught_up` before any token, 204, `live_from`, cancel then resume, downloads, both guards, an http-only `url`, the catch-all, the contract file.
- **`scripts/smoke_web.py`, not in CI:** one browser path through a Demo Run: start, each Pause including a refused edit, reload mid-Run, stop and resume, Done, Brief. `uv run --with playwright python scripts/smoke_web.py`, using `chromium.launch(channel="msedge")`, so no browser download or repo dependency.
- **Live:** one real Run by hand before calling it done, since the demo proves the plumbing and not how real notes read.
- **Skip-list:** no pixel tests, no component-render tests, no animation assertions. `useMotion`'s rule makes state the thing to test.

Streamlit is deleted in the last pass, once the smoke passes, so a working UI always exists: `clients/streamlit_app.py`, `clients/streamlit_bridge.py`, the `streamlit` group, then `uv lock`.

Dark mode, the last fog item: not in this effort. Colours are all custom properties, so it stays cheap to add, but a second palette needs its own contrast checks against a Mon 2 Nov deadline. It goes in future scope.

Checklist: `docs/lld/web-ui.md` holds the passes, each leaving one check, and the order to cut in if time runs short.

Write docs/lld/web-ui.md

Type: task Status: resolved Assignee: smayankulkarni05 Blocked by: App shell and history sidebar, Trace graph, Live Run view, Pause forms, Brief view, Motion with GSAP, Frontend build and verification

Question

Does `docs/lld/web-ui.md` say all that was decided, precisely enough to build from, and do the other docs agree with it?

Write the LLD from the closed tickets (the web API, shell, trace graph, live view, Pause forms, Brief view, motion, and build and verification) with a build checklist. Then bring the neighbours in line: runtime.md §12–13 (Streamlit replaced, any Controller change), the HLD's client and Infra shell rows, the future-scope Runtime and Clients rows this brings back (web UI, friendly labels, re-attach, Cancel, token display) and any new deferrals, and `GLOSSARY.md` if a new term entered. Resolved when no doc contradicts another and no decision is missing.

Answer

Done by the agent on 2026-10-31. Wrote docs/lld/web-ui.md from the closed tickets, with an 8-pass build checklist and a cut order. Brought the neighbours in line:

- runtime.md: §1, §2 and §11 name the web UI; §6 gains `attached`, `log_start`, `buffered` and `brief_paths`; §13 is now a pointer to the new LLD; §14 gains the two demo settings; §17's Infra shell line no longer claims the API.
- HLD: the client diagrams, the Clients row, the registry note, §10's scope and the build order. It records the web UI as pulled forward, with the Infra shell unchanged.
- future-scope.md: removed what this effort brings back (web UI, re-attach, Cancel, token display, friendly labels) and added a Web UI section of new deferrals.
- `GLOSSARY.md`: added Step and Demo Run.

Checked by searching the docs for what each change could contradict. One correction came out of writing it: Trace graph said 13 Steps, but its table and prototype have 14, so the ticket and LLD now say 14.

Runtime §10's "Gateway's `fetch` emits `api_called`" and the build-order history were left alone: they are accurate or historical, and not touched by this effort.

No code was changed, as the map's destination is the LLD. Streamlit still exists until the build's last pass, so runtime.md's pass 2 checklist keeps its history.

Research: searching many titles and places on Adzuna

Type: research Status: resolved Assignee: smayankulkarni05 Blocked by: none

Question

What can one Adzuna call cover, and so what do many titles and many places cost against the quota?

- Can several titles share one call (`what_or`, or `what` with phrases), and how does that change `count`, ranking and relevance?
- Can `where` take several places, or does each place need its own call? How do `where` and `distance` behave for Indian places such as Navi Mumbai and Thane, and does a wide radius around Mumbai already cover them?
- What are the current free-tier limits? The code assumes 25 calls a day (`MAX_FETCHES` in `collection/nodes.py`).
- Does USAJOBS accept several `LocationName` values or several keywords in one call?

Use primary sources: Adzuna's and USAJOBS's own API docs. Say which answers are documented and which still need a live call.

Answer

Full findings, with sources: research-adzuna-many-titles-and-places.md. No API search was called. Budget on calls = titles x places x pages.

- **Several titles in one call: documented no, for multi-word titles.** `what_or` ORs single keywords and `what_phrase` takes one phrase, so `count` would stop measuring a title. Sharing is only sensible for single-word variants. **Needs a live check:** `what_or` combined with `title_only`, quoted phrases, and whether `title_only` needs every word.
- **Several places in `where`: documented no.** `where` is one centre, `distance` is in km, and `locationN` is one hierarchy path, though it can cover a region.
- **Navi Mumbai, Thane and a wide Mumbai radius: needs a live check.** Nothing about Indian places is documented. About 4 calls would settle it.
- **Free-tier limits: no public number is documented.** The terms defer to the account. The project records 25 a day from the dashboard, worth re-reading. `MAX_FETCHES` is actually 6 per Attempt; the 25 is `adzuna_daily_limit`.
- **USAJOBS: documented** that `LocationName` takes several `;`-separated values, and `Keyword` and `PositionTitle` are not marked multi-value. **Needs a live check:** several locations with `Radius`.

Many places and titles in a Search Plan

Type: grilling Status: resolved Assignee: smayankulkarni05 Blocked by: Research: searching many titles and places on Adzuna

Question

How does one Run search several places and as many titles as the user wants, without breaking the Adzuna quota?

"Mumbai" can mean Mumbai, Navi Mumbai and Thane, but `HiringRequest.location` and `SearchPlan.location` each hold one string, and `Fetch` carries one place. Titles are capped at 4 in three places: `SearchPlan.titles`, the plan prompt and `MAX_TITLES` in `PlanEditor.tsx`. `MAX_FETCHES` (6 per Attempt) is the real ceiling, so removing the visible cap alone would let a user add titles that never get searched.

- The Search Plan's shape: `locations: list[str]` or something else, and what `HiringRequest.location` becomes.
- How a multi-select Clarifying Question answer travels: `ClarifyForm` sends one string per question, and only some questions should allow several answers. Which ones, and who says so?
- How the fetch budget spreads across titles × places, and what the user is told if some combinations go unsearched. Titles must not have a hard limit in the UI.
- Broaden's limits. The screenshot Run asked for Mumbai, then broaden blanked the location, so the Brief headed "Mumbai" rests on a country-wide search. It also produced the joined title "Brand Marketing Intern Content Marketing Intern". May broaden ever leave places the user chose?
- Compatibility: earlier Runs in the sidebar and their checkpoints hold the single `location`, and the replay cache keys on it. They must still open.
- What changes in the CLI Pause text and `brief/markdown.py`.

Answer

Decided by the agent, as the user told it to make every decision on 2026-11-01. Open to change once the user reads it.

Live check run for this ticket, 5 Adzuna calls on `in` with `title_only=marketing`: Mumbai 5 km 110, Mumbai 50 km 122, Thane 10 km 6, Navi Mumbai 10 km 9, `location0=India&location1=Maharashtra` 204. The 50 km page returned ads whose `area` is Navi Mumbai and Thane, and 110 + 6 + 9 less the 3 Mumbai-area ads the small searches also returned is 122. So one wide search covers that metro. Details.

- **Shape.** `SearchPlan.locations: list[str]` replaces `location`, with `[]` meaning country-wide and one shared `radius_km`. `HiringRequest.location` stays one string, the user's words ("Mumbai, Navi Mumbai, Thane"), since it only heads the Brief and feeds prompts. `Fetch.location` stays one place, so L1, L2 and Gateway replay keys don't change.
- **Same metro means one search.** `PLAN_SYSTEM` merges places in one metro area into one centre with a wider radius (Mumbai 50 km for those three). Separate cities stay separate. There is no gazetteer, so this is the model's judgement, visible and editable at plan review.
- **Multi-select.** Only `about == location` questions allow several answers, decided by the UI from `about`. Chips become checkboxes, and the answer is sent joined with ", ", so `ClarifyAnswer.answers: list[str]` doesn't change and intake copies it as usual. `QUESTION_SYSTEM` is told to offer the areas a place could include, and never an "Other" option, because the text box is always there. That also settles the fog on how other ambiguous places get their options.
- **Titles have no limit.** `max_length` goes from `SearchPlan.titles` and `MAX_TITLES` from the editor. `PLAN_SYSTEM` still suggests about 4, because each one costs a call.
- **Budget.** The per-Attempt cap becomes `max(MAX_FETCHES, titles x places)`, so a title the user adds does get searched. The real ceiling stays the daily guard in `gateway/store.py`, and running out already reaches `low_data_review` through `fetch_blocked`. `_fetch_input` lists the combinations in order, the primary title in every place first, so what a short budget drops is the least important.
- **The search tools take `location`.** A new model argument, blank meaning the first place. A value not in the plan is refused with an error `ToolMessage`, so the model still can't leave the approved plan.
- **What the user is told.** Plan review shows "3 titles x 2 places = 6 searches. 5 Adzuna calls left today." It is information, not a refusal. `PlanReviewPayload` gains `calls_left: int | None = None`, and the count is arithmetic in the client. A chosen place with no search at all raises the Caveat "Places not searched: Thane".
- **Broaden may never drop places.** Code pins `locations`, as it pins `country` and `target_postings`, and "a country-wide search" leaves `BROADEN_SYSTEM`. Only the user can go wider: `low_data_review` gets a "Search the whole country" checkbox, sent as `LowDataAnswer.whole_country: bool = False`. Broaden may still widen the radius up to the 100 km limit.
- **Joined titles.** "Brand Marketing Intern Content Marketing Intern" came from the model fitting five titles into a cap of four. The cap is gone, and `BROADEN_SYSTEM` and `PLAN_SYSTEM` say each title is one job title. `PLAN_SYSTEM` also says a bare place name, as "Mumbai, India" never matches a region in `store.py`'s `_fits`.
- **Compatibility.** A `model_validator(mode="before")` on `SearchPlan` turns an old `location` into `locations`, so old checkpoints and Briefs open. It isn't in the schema the LLM sees. New fields all have defaults. `Fetch` keys survive, but LLM replay entries for prompts holding the plan will miss, so sample Runs are recorded after this ships.
- **CLI and Markdown.** `render_plan` prints the list, or "country-wide" when empty. `_plan` in `brief/markdown.py` does the same. `match_totals` is keyed per title and place, as totals for one title would overwrite each other. USAJOBS makes one call per place like Adzuna, because several-location calls are unverified and it isn't metered.

Subtle neumorphic look and compact layout

Type: prototype Status: resolved Assignee: smayankulkarni05 Blocked by: none

Question

What should the app look like so it stops feeling tall, blocky and unfriendly?

Prototype the Search Plan editor, the clarify card with multi-select chips, and one Brief section in `.scratch/web-ui/prototype/`, then decide from them.

- Tokens in `web/src/styles.css`: surface colour, raised and inset shadows, radii, and how far the neumorphism goes. It stays subtle.
- Text and control contrast. Neumorphism is known to lose edges, so inputs, focus rings and the disabled state must remain clear (WCAG AA).
- The editor's layout: titles as removable chips with an always-available add box, and short fields sharing rows, so it fits one screen at desktop width and stacks on a phone.
- Readable labels for every raw value: "IT Jobs" for `it-jobs`, "PR, Advertising & Marketing Jobs" for `pr-advertising-marketing-jobs`, and country names for `IN`. Where the label table lives, with the wire values unchanged.
- Whether radius, days and target stay number boxes or become something friendlier.
- Print still works. Dark mode stays deferred.

Answer

Decided by the agent, as the user told it to make every decision on 2026-11-01. Open to change once the user reads it.

Prototype, on real Brief data, with `?depth=flat|subtle|strong`. Chose **subtle**: strong makes cards look like buttons, and flat doesn't change how the app feels.

- **Tokens** in `web/src/styles.css`: `--bg #eef0f3` as the one surface, `--ink #1f2430`, `--muted #535c6b`, `--edge #7d8796`, `--accent #2f5d50`, `--focus #1f5fd0`; `--raised 5px 5px 12px rgba(150,160,178,.38), -5px -5px 12px rgba(255,255,255,.85)`, a smaller `--raised-sm`, and `--inset` for inputs. Radii 14 for cards, 8 for inputs, 99 for chips.
- **Neumorphism stops at the surface.** Controls keep a real 1px `--edge` border, because shadow alone loses the edge. Measured against `--bg`: ink 13.59, muted 5.91, edge 3.18, focus 5.10, white on accent 7.49, warning 6.09 on its own tint. All meet AA. Focus is a solid 2px outline. Disabled is a dashed border and muted text, not `opacity: .4`.

- **Editor.** Titles and Places are chip boxes with an always-open add input (Enter adds, x removes), no limit. Short fields share a 4-column grid, two columns on a phone. At 665 px wide it is 451 px tall, so it fits one screen. No overflow at 375 px.
- **Friendlier fields.** Radius ("Within") and days ("Posted in the last") become selects with presets, 5, 10, 25, 50, 100 km and 7, 14, 30, 60 days, adding the plan's own value when it isn't a preset. Radius is disabled with no place. Target stays a number box, as any value is meaningful.
- **Labels.** One new `web/src/labels.ts` with a 20-row category table with a raw fallback. Country names come from native `Intl.DisplayNames`, so no table. Wire values don't change.
- **Print** drops the shadows, uses white and gives cards a hairline. Dark mode stays deferred.
- **Sidebar and Run header** (the fog): they carry no log text, so they need only the tokens. No ticket.

What the user sees and what the record keeps

Type: grilling Status: resolved Assignee: smayankulkarni05 Blocked by: none

Question

Which Run details reach the user on screen, and which stay only in the record?

The UI currently prints log text meant for debugging: raw plan diffs under "Filter postings", route names such as "(route: analyse)", "Some live text wasn't kept", and every Caveat with its internals.

- One rule for what is user-facing, applied to `turn.notes` in the live view and to the trace panel. Does the server send friendly text, or does the UI map or drop log lines?
- Removing the "Some live text wasn't kept" line (`Stream.tsx`, the `gap` flag in `reducer.ts`). It is sticky, so it stays at the foot of the stream after the turn it was about, and its check counts drafting nodes whose tokens are never shown.
- Which Caveats matter to a reader, with a limit of one or two plain lines, and their wording. For example, "Broadened 2 time(s): titles: [...]" becomes "We widened the search to find enough postings", and a dropped location must be said plainly. A Weak Sample and a search wider than the place asked for look like the ones that change how far to trust the Brief.
- What is left of Run Notes on screen. Plans, source counts and internal Caveats stay complete in `brief.json` and `brief.md`, and the attribution line must stay visible.

Answer

Decided by the agent, as the user told it to make every decision on 2026-11-01. Open to change once the user reads it.

The rule: a line reaches the screen only if the reader can act on it or it changes how far to trust the Brief. Everything else stays in the record.

- **The server writes the friendly text.** A new `LogKind.result` marks lines meant for readers, and `NodeDone` gains `results: list[str] = []`. The live view and trace panel show only these, and never `notes`. Mapping debug strings in the UI would break whenever a message changed. Existing `run_log` messages stay as they are for `scout show`.
- **The lines:** `collect` "Found 80 postings" (with ", 12 duplicates" if any), `relevance` "51 are the role", `broaden` "Widened the search". Other Steps show their label and check alone. `fromLog` filters on `kind`. An old Run shows the same labels without these lines.
- **Tool cards.** The model still gets "Refused: ...", while the emitted summary becomes "Skipped: search limit reached". Bad arguments and unknown tools read "Didn't run". Pills say "Adzuna"

rather than `adzuna`.

- **"Some live text wasn't kept" goes:** the line in `Stream.tsx`, the `gap` flag and its check in `reducer.ts`. It told the reader nothing they could act on.
- **At most two plain Caveats on screen, from a new pure `reader_notes(brief)`,** first matches in this order:
    1. Places widened: "We searched the whole country, not only Mumbai."
    2. A "Places not searched" Caveat: "We couldn't search Thane."
    3. Weak Sample: "Only 12 postings against the 20 we aimed for, so treat this as a rough guide."
    4. Any broaden: "We widened the search to find enough postings."
    5. "Old data" Caveat: "Most of these postings are over 30 days old."
    6. "Mostly federal" Caveat: "Most of these postings are US federal jobs."
    7. "One source failed" Caveat: "One job source didn't answer, so this rests on the other." Skills from snippets, assumed pay periods and critic or Jev degradation stay in the record, and the charts say what they rest on.
- **Computed on read, not stored.** `reader_notes` goes on `RunDetail`, so earlier Runs get them and `brief.json` keeps its bytes.
- **Run Notes on screen** move into a closed "How this was researched", unchanged and complete, including every Caveat. This reverses "Run Notes stay open" from Brief view, because the reader notes now carry what affects trust. The attribution line moves out of the fold to the foot of the Brief, always visible. `brief.json` and `brief.md` don't change.

A Brief you can read at a glance

Type: grilling Status: resolved Assignee: smayankulkarni05 Blocked by: Subtle neumorphic look and compact layout, What the user sees and what the record keeps

Question

How does the Market Brief give its answer in seconds, with detail only on request?

- The first screen: the Outlook, the recommended title, salary band and top skills, as scannable cards rather than one long column.
- Insights: the limit on how many show first, one line each, with the confidence reason and Evidence folded away. Does the `write_brief` prompt need shorter text, or is this display only? The Critic and citations must keep working either way.
- Where Summary, Evidence, Postings analysed and the remaining Run Notes go.
- How the Weak Sample notice and the short Caveats from the ticket above appear without taking over the page.
- Whether screen, Markdown and print still agree on wording, given `BriefView.tsx` copies `brief/markdown.py`'s wording on purpose.

Answer

Decided by the agent, as the user told it to make every decision on 2026-11-01. Open to change once the user reads it.

- **First screen, in order:** title; four tiles (Hiring Outlook with its shape and word, Job title, Salary band, Top skills, up to 4); reader notes; the Summary; then charts. Outlook and recommendation reasons open from a "Why" fold on each tile, with Evidence inside. A missing recommendation shows "Not enough evidence" in muted text. Four tiles become two columns on a phone.
- **Insights.** "What we found" shows 3, highest confidence first and stable within a level, with "Show all 8" below. Each is its pill and text, with the reason and Evidence in a fold. Display only, nothing is dropped.
- **One small prompt change.** `InsightDraft.text`'s description asks for one sentence of at most 20 words. Text is never truncated on screen, as cutting could remove a figure. The Critic and citation checks don't change, as they read the same fields.
- **Below:** "How this was researched" holds Run Notes, then "Postings analysed", both closed. Downloads stay top right, quieter. The attribution line is last.
- **Notices.** The replay banner stays. A reader note is one slim tinted line with a warning icon, at most two. The Weak Sample notice is one of those notes, so it can't appear twice.

- **Screen and Markdown agree on facts, not wording.** Screen is for the reader and Markdown is the record, so `BriefView.tsx` drops its copied strings and comment. A pytest checks that the figures in `reader_notes` come from the Brief. Print follows the screen, with folds closed, and the Markdown is the full version. Also fixes "with a entry level" in `outlook()`'s reason.

Charts in the Brief

Type: prototype Status: resolved Assignee: smayankulkarni05 Blocked by: A Brief you can read at a glance, Many places and titles in a Search Plan

Question

Which charts earn a place in the Brief, what data backs each one, and how are they drawn?

Consult the `dataviz` skill and prototype against a real Brief from `runs/`.

- Locations: `PostingRef.location` is already on the wire, but raw strings need grouping, and `Posting.region` is not sent.
- Pay: `MarketStats` only holds p25, median and p75, so a range bar works today. A true distribution needs per-Posting figures added server-side. Salary is often missing or estimated, so the chart must show how many Postings it rests on and hide below a minimum.
- Skills: top skills from the `skills.*.count` Stats, split into required and nice to have.
- Also consider posting age and employer or seniority mix, only if each answers a question a client asks.
- Every number must come from a Stat or the Sample, never from the LLM, and should link to its Evidence.
- Drawing: hand-written SVG, as the trace graph is, unless the charts show that a library pays for itself. Reading without colour alone, screen-reader text, phone width and print.
- `brief.md` has no charts: is the data there as text already?

Answer

Decided by the agent, as the user told it to make every decision on 2026-11-01. Open to change once the user reads it.

Prototype on both real Briefs, checked at phone width. Palette run through the `dataviz` validator.

Four small charts earn a place, each answering a client's question:

Chart	Question	Data	Hidden when
Pay, middle 50%	What should we pay?	The Salary band's own `stat_ids`, so it can't disagree with its tile. p25 to p75 box, a median line, both ends labelled, "from 28 postings"	No Salary band, which already needs 5 figures

Chart	Question	Data	Hidden when
Skills employers ask for	What leads the job description?	`skills.*.count`, top 8 from 2+ employers, the required part from `skills.*.must`	No such skills
Where the postings are	Is the market where I asked?	Sample `region`, grouped	Under 2 groups
Who is hiring most	Who am I up against?	Top 5 `competition.employer.*`, plus `competition.distinct_employers`	No employers

- **Rejected.** A per-Posting pay distribution needs new server data and salaries are sparse. A posting-age chart: the trend is one word already and tilts recent, so it's a line in the Insights. Seniority mix: the real Marketing Run is 100% unknown.
- **Skills from one employer are left out and the caption says so.** In the real Marketing Run, one employer alone accounted for reels and X (Twitter), 9 postings each.
- **Places group from the data.** Strip what all Postings share, then group by the next `region` level, so a metro search shows areas and a country-wide one shows states. Real result: Mumbai 25, Thane 3, Navi Mumbai 2. Top 6, then "N other places". A Posting whose region is only the country counts as "Area not given": 15 of 51 in the real country-wide Run, and they must not stop the grouping.
- **One pure server function, `brief/glance.py`.** `glance(brief, postings)` returns `reader_notes` and chart rows, each with its Evidence keys, and goes on `RunDetail`. The UI only draws. Numbers come from Stats or the Sample, never the LLM, and earlier Runs get charts.
- **Drawing: HTML and CSS, not SVG.** Every chart is horizontal bars or one range, so widths on real text rows are the shortest thing that works, and give screen readers and print the labels for free. No library. Clicking a row opens its Evidence. A native `title` gives the hover.
- **Colour.** One hue: `--chart #00795f`, and `--chart-2 #2fb08f` for the non-required part of a skill bar. They pass the validator's lightness, chroma and colour-blind checks on `--bg`. `--chart-2` is 2.38:1, a warning that needs visible labels, and every bar has its number, so it's met. The "dark is required" caption shows only when a required count exists. Print forces colour with `print-color-adjust: exact`.
- **`brief.md`** has no charts and only part of this data, so `render_markdown` gains `## Market figures`: pay, skills, places and employers as plain lists from the same `glance`.

Update the LLD and cut build tickets

Type: task Status: resolved Assignee: smayankulkarni05 Blocked by: Many places and titles in a Search Plan, Charts in the Brief

Question

Are `docs/lld/web-ui.md` and the upstream LLDs (request-plan, collection, brief, glossary) updated to these decisions, and are the changes cut into vertical build tickets numbered from 13 in `.scratch/web-ui-build/issues/`, ordered so the most useful ship first if time runs short?

Answer

Done. `docs/lld/web-ui.md`, `request-plan.md`, `collection.md`, `brief.md`, `kernel.md`, `runtime.md` and `GLOSSARY.md` carry these decisions. Build tickets 13 to 18 are in `.scratch/web-ui-build/issues/`, ordered so the most useful ship first.

Research: searching many titles and places on Adzuna

Researched 2026-11-01. No API search was called for this note; the only requests were for documentation pages and spec files. Each claim is marked **Documented** (a primary source says it), **Measured earlier** (recorded from this project's own past calls), **Inferred** (reasoned from the docs), or **Needs a live check**.

Sources

Key	Source	What it is
[OAS]	https://developer.adzuna.com/swagger/spec/test2.json	OpenAPI 3.1 spec behind Adzuna's interactive docs. Treated as the current parameter reference
[SW]	https://api.adzuna.com/v1/api-docs/adzuna	Older Swagger 1.2 spec, still served
[SEARCH]	https://developer.adzuna.com/docs/search	Adzuna search guide
[REGION]	https://developer.adzuna.com/docs/regional	Adzuna guide to `locationN`
[TOS]	https://developer.adzuna.com/docs/terms_of_service	Adzuna API terms
[UJ]	https://developer.usajobs.gov/api-reference/get-api-search	USAJOBS search reference
[UJR]	https://developer.usajobs.gov/guides/rate-limiting	USAJOBS rate limiting guide
[SMOKE]	Live smoke test	This project's measured calls, US only
[L1]	L1 cache and budget guard	Records the owner's dashboard limit

Bottom line

Budget on one Adzuna call per title, per place, per page. Several multi-word titles cannot share a call by any documented means, and `where` takes one centre. The way to save calls is to widen one search area rather than add places: one `where` with a wide radius, or one `locationN` region. Whether that actually catches Navi Mumbai and Thane needs a live check.

1. Can several titles share one call?

Documented

- `what_or`: "any keywords may be found", with terms space separated [OAS]. [SW] also allows commas. It takes keywords, and nothing describes grouping them into titles.
- `what_phrase`: "An entire phrase which must be found in the description or title" [OAS]. It is a single phrase, with no OR form.
- `what`: keywords, space separated [OAS]. Neither spec mentions quotes or phrase syntax.
- `title_only`: "Keywords to find, but only in the title", space separated [OAS]. Neither spec says whether several terms must all match or any may match.
- `count` is "The total number of advertisements found" [SW]. `sort_by` accepts `default`, `hybrid`, `date`, `salary` and `relevance` [OAS]. No primary source explains how relevance is computed.

Inferred

- `what_or=data engineer analytics engineer` means any of the words `data`, `engineer`, `analytics`, not either title. This agrees with the existing rule in `docs/lld/gateway.md`.
- OR-ing words can only grow `count` and bring in unrelated titles. So `count` stops measuring the market for a title, and pages fill with postings that must be filtered afterwards.
- The client sends `sort_by=date` (`adzuna.py`), so relevance plays no part in ordering today. Noise from an OR would be spread through the pages by date, not pushed to the end. Switching to `relevance` would break the "page N walks back in time" assumption.
- Single-word variants of one title (`developer`, `programmer`) are the only documented case where sharing a call is safe in principle.

Needs a live check

- Whether `what_or` combines with `title_only` so the OR applies to titles only. The docs do not describe combining them, and this is the one experiment that could change the conclusion.
- Whether quoted phrases work in `what`, `what_or` or `title_only`.
- Whether `title_only=data engineer` needs both words. `docs/lld/gateway.md` assumes it does.

2. Can `where` take several places?

Documented

- `where` is "The geographic centre of the search" [OAS]: one string, one centre. No multi-value form appears in either spec.

- `distance` is in kilometres from that centre [OAS][SW]. The default differs: 5 km in [OAS], 10 km in [SW]. This does not affect us, because the client always sends `distance` with `where`.
- `location0`..`location7` describe one place in a hierarchy, for example `location0=UK&location1=South East England&location2=Surrey` searches that county [OAS] [REGION]. Again that is one path per call, but it can cover a whole region.
- `in` is a supported country [OAS][SW].

Inferred

- Each separate place needs its own call, unless one radius or one region covers them all.

Indian places: nothing documented

No Adzuna source mentions Indian place names, how they resolve, or how Indian ads are geocoded. All of the project's measured calls were US [SMOKE].

As a rough estimate from general geography, not from a cited source, Thane and Navi Mumbai sit a few tens of kilometres from central Mumbai. A radius near 40 to 50 km around `where=Mumbai` should reach them geometrically, within the plan's 0 to 100 km limit. Whether it does in practice depends on things only a live call can show.

Needs a live check

- Whether `where=Navi Mumbai` and `where=Thane` resolve, and to what. Adzuna does not document what happens to an unknown place.
- Where Adzuna puts the centre of `Mumbai`, and whether `where=Mumbai&distance=50` returns ads whose `location.area` names Thane or Navi Mumbai.
- Whether radius search works for ads located only at city or state level.
- What `location.area` looks like for India, and so whether `location0=India&location1=Maharashtra` is a usable one-call region. Results from a Mumbai search will show this.

A small test would be about 4 calls: `where=Mumbai` at a wide radius, `where=Thane`, `where=Navi Mumbai`, and one `location0`/`location1` search. Compare `count` and the `area` values.

3. Free-tier limits

Documented: no public number. The terms say the free tier "is intended for evaluating the API" and "The plans available to you, and their limits, are shown in your account", and that limits may change [TOS]. The spec lists only 400, 404, 410 and 500 responses, with no quota error [OAS].

Project record: the owner read **25 calls a day** from the account dashboard, with no per-minute limit reported [L1]. That is the best figure we have, but it is not public, and the dashboard should be re-read before planning around it.

Unverified: third-party pages cite different figures (25 a minute with 250 a day, or about 1,000 a month). Search snippets attribute a similar set to Adzuna's terms, but the terms page fetched today

has no such numbers. Do not use these.

A correction to the ticket: `MAX_FETCHES` is not the 25. It is 6 searches per Attempt, so 18 over 3 Attempts (`collection/nodes.py`). The 25 is `adzuna_daily_limit`, enforced by the budget guard in `gateway/store.py`.

The cost

Calls = titles x places x pages, at up to 50 postings a page. The cap of 50 is enforced silently [SMOKE].

Titles	Places	Pages	Calls	Share of 25 a day
1	1	1	1	4%
2	3	1	6	24%, one full Attempt
4	1	1	4	16%
4	3	1	12	48%, exceeds `MAX_FETCHES`
4	3	2	24	96%
4	5	1	20	80%

Today's `MAX_FETCHES` of 6 allows only about 2 titles x 3 places, first pages only, in one Attempt. Broadening repeats the spend, and L1 and L2 reuse may reduce it. [SEARCH] adds nothing further on cost. Retries also count, since the guard counts each HTTP attempt (`store.py`).

4. USAJOBS

Documented

- `LocationName` accepts several values, separated by semicolons [UJ].
- `Keyword` and `PositionTitle` are not marked as multi-value, unlike the ten parameters that are [UJ]. `Keyword` searches for "all of the words specified (or synonyms of the word)", and `PositionTitle` is a "contains" match [UJ].
- `Radius` expands `LocationName` [UJ]. Its behaviour across several locations is not stated.
- Search limits: 500 rows a page, 10,000 a query. No call quota is stated [UJR].

Measured earlier

- `;` is the working separator on a multi-value series code: `2210;1550` gave 424, `2210:1550` gave 0 [SMOKE].
- `Radius` behaves like miles [SMOKE].
- Rows are about 85 KB each, so the client's page size of 50 is worth keeping [SMOKE].

Inferred: several places can share one call, but each title needs its own call. Since USAJOBS calls aren't limited in the code, this matters little for quota.

Needs a live check

- `LocationName=A;B` itself, as the measured test used a different parameter.
- `LocationName=A;B&Radius=N`.
- Whether `PositionTitle=A;B` is accepted despite not being documented.

Live check, 2026-11-01

Five real calls on `in`, `title_only=marketing`, `max_days_old=30`, first page, counted by the budget guard. This settles most of section 2 for this metro. One title and one day only, so it is evidence, not a rule for every place.

Search	`count`	`area` values on the page
`where=Mumbai&distance=5`	110	Mumbai and its suburbs
`where=Mumbai&distance=50`	122	Mumbai, plus Navi Mumbai and Thane
`where=Thane&distance=10`	6	Thane, Bhiwandi, Rabale, one Mulund West
`where=Navi Mumbai&distance=10`	9	Navi Mumbai, two Govandi
`location0=India&location1=Maharashtra`	204	Mumbai, Pune, Nagpur and others

- `where=Thane` and `where=Navi Mumbai` both resolve.
- Mumbai at 50 km reaches both. The three smaller searches overlap by the 3 Mumbai-area ads above, and 110 + 6 + 9 - 3 = 122, so nothing was missed here.
- `location0` and `location1` work for India, but Maharashtra is wider than a metro, so it isn't a substitute.
- `location.area` for India reads country, state, city, then locality. Some ads have the country alone.

Still unchecked: `what_or` with `title_only`, quoted phrases, and USAJOBS with several locations. No decision waits on them.

web-ui-build

web-ui-build/issues/01-controller-seams-and-demo-run.md

01: Controller seams and the Demo Run

What to build: the prefactor every other ticket stands on, with no UI yet. A client can find the live handle of a running Run and know where its replay begins, and a Demo Run walks the whole graph at human speed with no `.env`, LLM or jobs API, emitting every event type a real Run does. Spec: Web UI LLD §3 and §4 (Demo mode), §13 pass 1 steps 1 and 2.

Blocked by: None (can start immediately)

Status: done

Scope limits, each lifted by a later ticket:

- No HTTP. FastAPI and uvicorn join the dependencies here, but nothing serves yet (02).
- `demo_brief()` must be complete enough to develop the Brief view against (05), with all recommendations, each Confidence and Caveats.
- [x] `attached(thread_id)` returns the live handle while a Run's task runs, and none once it has ended
- [x] `log_start` equals the `run_log` length at the handle's start, including after a resume
- [x] `buffered` is the buffer's length, and `brief_paths` is public with its callers updated
- [x] `graph_topology()` returns the nodes and edges from the existing tables, in `OWNERS` order
- [x] `HiringRequest`, `LogKind`, `RunLogEntry`, `ClarifyingQuestion`, `SearchPlan` and `MarketStats` are in `ALLOWED_MSGPACK`, and a load logs no "Blocked deserialization" line
- [x] With `settings.demo`, the Controller uses the demo graph and skips `check_balance`
- [x] A Demo Run emits `Token`, `Reasoning`, `ToolCalled`, `ToolReturned`, `ApiCalled` (one source cached), `ExtractProgress`, all three Pauses, a critic loop and `Done`
- [x] The demo honours plan approve, edit, reject and abort, and low-data broaden, continue and abort
- [x] A request containing "fail" stops once in `normalise_skills`, and Resume then reaches `done`

- [x] `--demo`'s settings need no `.env`, and its Runs live under `.cache/demo/`, never in real history
- [x] Demo `brief.json` and `brief.md` are written through the real `render_markdown`
- [x] All of the above run in `pytest` with `demo_delay=0`, and the existing suite still passes

02: Start a Run and watch it reach a Pause

What to build: the tracer bullet, through every layer. A user runs `scout serve --demo`, opens the app, types a Hiring Request, picks focus chips, presses Start, lands on `/runs/:id`, and watches Step lines appear live until the Run stops at its first Pause. Spec: Web UI LLD §2, §4 (start, read, events, meta, security, hosting), §5 (frame, routes, composer), §7 (the stream's start only), §11, §13 pass 1 step 3 and pass 2.

Blocked by: 01: Controller seams and the Demo Run

Status: done

Scope limits, each lifted by a later ticket:

- The stream is one line per turn: Step label and notes. No tokens, tool cards or collapse (07).
- A Pause shows only "Waiting for you", with no form (03).
- No sidebar (06), trace panel (10) or animation (11). The layout leaves room for them.
- Opening a Run from a URL shows its request and status only. Rebuilding its history is 08.
- [x] `GET /api/meta`, `GET /api/runs`, `POST /api/runs`, `GET /api/runs/{id}` and `GET /api/runs/{id}/events` work as the LLD's table says, and all path ops are `async def`
- [x] Refusals use the one `{code, message, errors?}` body, including reshaped FastAPI validation errors
- [x] `RunDetail` is returned rather than `RunView`, with a `url` of null unless http or https
- [x] The stream sends each event unchanged, one `caught_up` after the replay, and 204 when nothing is live; an unknown ID gets 404
- [x] The client closes its `EventSource` on `Paused`, `Done` and `Stopped`, and resets to its base on `onopen`
- [x] The label table covers every `/meta` node and yields exactly the five back-edges, checked by a Vitest test, so a new node can't go unlabelled
- [x] A foreign `Host` gets 400; a POST with a foreign `Origin` gets 403, with and without a body
- [x] `same_origin` passes through the Vite dev proxy, which uses `changeOrigin: false`
- [x] The server binds loopback only and shuts down with an open stream inside `timeout_graceful_shutdown`

- [x] The production build is served by FastAPI; `/runs/:id` survives a reload; `/api/*` stays a real 404; a missing `web/dist` says to run the build
- [x] The composer refuses a blank request, Enter submits, Shift-Enter adds a line
- [x] An unknown Run ID shows "Run not found" with a link to `/`
- [x] API behaviour above is covered in `pytest` through `TestClient` with the loopback base URL
- [x] Checked by hand in the browser through a Demo Run to the first Pause

03: Answer a Pause and carry on

What to build: a user answers each Pause in an inline card and the Run continues on the same page, so a Demo Run goes from request to its end. Bad answers show against the same card with nothing changed. Spec: Web UI LLD §4 (resume, refusals), §7 (Connecting, the base after answering), §8.

Blocked by: 02: Start a Run and watch it reach a Pause

Status: done

Scope limits, each lifted by a later ticket:

- `plan_review` offers Approve, Ask for a different plan, and Abort. The Edit button is 04.
- The end of a Run shows a plain "Done" or "Aborted" line. The Brief is 05.
- A reopened paused Run renders its card, but its earlier history is 08.
- [x] `POST /api/runs/{id}/resume` works, with `InvalidAnswer` cleaned to `{loc, msg}` and its "Value error, " prefix stripped
- [x] `clarify`: option chips are real radio inputs, a chip fills the box and typing clears the chip, blank is a skip, and the answers stay in question order
- [x] The clarify button reads "Continue without answering" when all are blank
- [x] `plan_review` shows the Search Plan and Assumptions readably, with no field names
- [x] `low_data_review` shows the counts, any block reason, calls left and the plans tried, with optional guidance feeding Search wider
- [x] Every action posts the exact payload in §8's table, with all of `PlanReviewAnswer`'s keys
- [x] Abort is two-step on both cards
- [x] A refused answer keeps the values and the card, and leaves the Run `paused`
- [x] `run_locked` retries once after 300 ms, then shows "This Run is open elsewhere"
- [x] `not_resumable`, or an answer the Run isn't waiting for, shows "Answered elsewhere" and rebuilds from `RunDetail`
- [x] `balance_too_low` and `upstream` show their message and the Run can be retried

- [x] After answering, the card collapses to a one-line record, the Step leaves waiting, and a new stream opens with the state on screen as its base
- [x] Every refusal and payload is covered in `pytest`
- [x] Checked by hand through a Demo Run, all three Pauses to `done`, plus an abort of each kind

04: Edit the Search Plan

What to build: at the plan Pause, a user presses Edit, changes the Search Plan in a proper form, and the Run goes on with the edited plan. A value the server refuses shows under its own field. Spec: Web UI LLD §8 (`PlanEditor`, refusals).

Blocked by: 03: Answer a Pause and carry on

Status: done

- [x] The form starts from the Pause's plan and covers all seven fields as §8's table says
- [x] Titles take 1 to 4 rows, with add and remove
- [x] Country and category are selects from `/meta`, with category also offering "Any" (null)
- [x] A blank location means country-wide
- [x] Number inputs carry the model's limits, but the server stays the check
- [x] "Use this plan" posts `{action: "edit", plan, feedback: null}`, and the Run continues with that plan
- [x] An error with `loc[0] === "plan"` shows under the field named by `loc[1]`, and anything else at the top of the card, with values kept
- [x] Cancelling the edit returns to the read view with no request sent
- [ ] Checked by hand through a Demo Run: a refused edit (with HTML limits bypassed), a corrected one, and the edited plan visible in the Brief's Run Notes (refused and corrected edits checked; the Run Notes view is not)

05: Read the Market Brief

What to build: when a Run finishes, the Market Brief appears on the page for a customer success manager to read, check against its Evidence, download and print. A Run with no Brief says why. Spec: Web UI LLD §9, §4 (downloads), §7 (Endings).

Blocked by: 03: Answer a Pause and carry on

Status: in-progress (not yet checked in print preview or against a real Brief from `runs/`)

Scope limits, each lifted by a later ticket:

- Folding the turns under "How this Run went" needs turns with content, so it lands with 07 if that isn't done yet.
- [x] On `Done`, the client fetches `RunDetail`, retrying up to 5 times at 200 ms while it says `running`
- [x] The sections follow §9's order, rendered from structured data, with no Markdown renderer and no raw HTML
- [x] Confidence Tags, Outlook ratings and Caveats each read correctly without colour
- [x] Run Notes are open, with Caveats first
- [x] The replay banner and Weak Sample notice appear only when they apply, with `brief/markdown.py`'s wording
- [x] `Evidence` expands inline to the cited Stats and the Postings, ten at first, then "Show all"
- [x] Evidence counts use `posting_keys.length`, and an Outlook with empty `stat_ids` shows no Evidence
- [x] Posting links open in a new tab with `rel="noopener noreferrer"`, and a null `url` shows no link
- [x] Missing recommendations and zero Insights show the Markdown's wording
- [x] `brief.json` and `brief.md` download; a Run with no Brief gets 404 `no_brief`, and the page offers no links
- [x] An `aborted` Run shows the request, "This Run ended without a Brief", the reason and a link to `/`
- [x] `@media print` hides the app's chrome and buttons, and needs no script
- [x] Downloads and `no_brief` are covered in `pytest`

- [ ] Checked by hand with `demo_brief`, with one real Brief from `runs/`, and in print preview (`demo_brief` checked; the real Brief and print preview are not)

06: Find earlier Runs in the sidebar

What to build: every Run sits in a chat-style sidebar, with the ones waiting on the user pinned at the top. Clicking one opens it, and the list keeps itself current without a stream of its own. Spec: Web UI LLD §5.

Blocked by: 02: Start a Run and watch it reach a Pause

Status: done

Scope limits, each lifted by a later ticket:

- Opening a Run navigates to it, but what it shows for each status is finished by 08.
- The running dot is static until 11.
- [x] The frame is the 272 px sidebar and the centred column, with New Run returning to `/`
- [x] "Needs you" is pinned, shown only when non-empty, holds every `paused` and `failed` Run in bold, newest first, and they don't repeat below
- [x] Other Runs sit under Today, Yesterday, Previous 7 days and Earlier, by calendar day of `updated_at` in the browser's timezone
- [x] A row is an ellipsised title, a badge and a relative time; each badge has its word, so colour is never alone
- [x] The list polls every 3 s while any row is `running` and every 15 s otherwise, and on window focus
- [x] It also refetches after the user starts, resumes, cancels or answers
- [x] A stream's terminal event triggers one refetch after 300 ms, because the row lags the event
- [x] The sidebar opens no stream
- [x] When the open Run's `status` or `updated_at` changes and the page holds no stream, the page refetches and re-attaches
- [x] Under about 900 px the sidebar is a slide-over from a header button showing how many Runs need the user
- [x] Grouping, including the timezone day boundary, has a Vitest test
- [x] Checked by hand with Demo Runs in every status, and at phone width

07: See the Run think and search

What to build: a running Run reads as a live chat-style stream. Text streams in as the model writes it, a tool call shows a loading card that resolves to its result, extraction shows a bar, and finished turns tuck themselves away. Spec: Web UI LLD §7 (turns, blocks, State, auto-scroll, accessibility), §12 (contract).

Blocked by: 02: Start a Run and watch it reach a Pause

Status: done

Scope limits, each lifted by a later ticket:

- A reconnect may still lose streamed text, and a reload shows no history (08).
- Tokens are plain text with no caret animation (11).
- [x] A `NodeStarted` whose Step differs from the current turn's opens a new turn; `fetch_agent` and `tools` stay in one, and a loop opens another
- [x] The active turn is expanded; a finished one collapses to its check, label and last note, and expands on click
- [x] Every event in §7's table has its effect, through one pure reducer
- [x] `Reasoning` is a closed `<details>`, created on the first fragment, so none means no block
- [x] Drafting lines use §7's phrases, end on the first token or `NodeDone`, and JSON fragments are never shown
- [x] Tool cards pair first-in-first-out by name, and `ApiCalled` sets the source pill: count, "cached" or "failed"
- [x] A card still open at `Stopped` reads "Interrupted"
- [x] Events flush once per frame, with a 250 ms timeout for hidden tabs, and finished turns are `memo`ised
- [x] Once `done`, all turns fold under a closed "How this Run went"
- [x] Auto-scroll sticks within 80 px of the end, and scrolling up shows "Jump to latest", reading "Waiting for you" when a Pause arrived off-screen
- [x] The stream is `role="log"` with `aria-live="off"`, a polite region announces Step changes, Pauses and endings, and a Pause heading takes focus when stuck to the bottom

- [x] A pytest compares a Demo Run's events and `/meta` with the committed `demo-run.json`, and Vitest feeds that file to the reducer
- [x] Vitest covers: the feed's end state, one batch equals one by one, tool pairing, and no block for absent reasoning
- [x] Checked by hand through a Demo Run, and one fast token stream stays smooth

08: Reload or reconnect without losing the Run

What to build: a user can reload, open a Run in a second tab, lose the connection, or come back after a server restart, and the page rebuilds what happened and carries on live. A Run reopened from history shows its route and notes with no stored events. Spec: Web UI LLD §7 (`fromLog`, Connecting), §5 (what an opened Run shows first).

Blocked by: 03: Answer a Pause and carry on; 07: See the Run think and search

Status: done

- [x] `live_from` reports the attached handle's `log_start`, or null with none attached
- [x] `fromLog` groups `run_log[:live_from]` into turns by consecutive Step, or all of it with no live handle
- [x] A `collect` line implies `search` ran before it, and a `normalise_skills` line implies `postings` did
- [x] Each status opens as §5's table says, including a `running` Run live elsewhere, which polls every 3 s and offers no Stop
- [x] A reload mid-Run shows no duplicated or missing turns across the `live_from` boundary, including after a resume
- [x] `onopen` resets to the stream's base, while the `texts` map survives, so a dropped connection keeps text already on screen
- [x] `animate` stays false until `caught_up`, and on any reopened Run
- [x] "Some live text wasn't kept" appears only when the active turn could have streamed and was reached by replay
- [x] "Reconnecting…" shows after a 2 s grace at `readyState` 0
- [x] `readyState` 2 fetches `RunDetail` and rebuilds, including after a server restart mid-Run
- [x] Two tabs on one live Run both stream, and closing one leaves the Run and the other tab running
- [x] Vitest covers `fromLog` per status and its inferences, and a reset keeping `texts`
- [x] `pytest` covers `live_from`, and `caught_up` coming before any token on attach
- [x] Checked by hand: reload at each Pause and mid-search, a second tab, and a server restart mid-Run

09: Stop and Resume a Run

What to build: a user can stop a running Run, and when one stops for any reason, the page says where and offers Resume, which picks up from that step. Spec: Web UI LLD §4 (cancel), §7 (Endings, Stop).

Blocked by: 03: Answer a Pause and carry on; 07: See the Run think and search

Status: done

- [x] `POST /api/runs/{id}/cancel` returns `Stopped`, leaves the Run `failed` with `cancelled`, and gives 409 `not_live` with no attached handle
- [x] The Stop button shows only while the Run is live here, and needs no confirmation
- [x] `Stopped` with a node shows "Stopped at <Step>. Nothing is lost; Resume retries that step", with the raw reason in a `<details>`
- [x] A `cancelled` reason reads "You stopped this Run at <Step>"
- [x] `Stopped` with `node: null` offers no Resume, only a link to `/`
- [x] Resume posts `answer: null`, and its new stream's base is the state on screen
- [x] Resume's refusals show on the banner, as they do on a Pause card
- [x] A "fail" Demo Run stops at Tidy skill names and reaches `done` after Resume, rerunning only that step
- [x] `pytest` covers cancel then resume, `not_live`, and shutdown leaving a live Run `failed` and resumable
- [x] Checked by hand: Stop mid-search and resume, and a "fail" Demo Run

10: Trace panel and progress bar

What to build: beside the stream, a trace panel shows which of the 14 Steps the Run is on, what remains, where it looped, and how far along it is, live and after a reload. No animation yet: every state reads correctly on its own. Spec: Web UI LLD §6. The trace prototype holds a working reducer and layout to carry over.

Blocked by: 08: Reload or reconnect without losing the Run

Status: in-progress (not yet reloaded at each Pause by hand)

Scope limits, each lifted by a later ticket:

- Spinners, pulses, flowing rails and the loop dot are 11. Their static forms ship here.
- [x] The 316 px panel shows on an opened Run only, drawn as hand-written SVG from `/meta`'s topology and the label table
- [x] Only back-edges are drawn as edges; a skipped Step says "Not needed"
- [x] All six states read without colour, and optional Steps have a dashed outline from the start
- [x] A Step is done when the Run moves to a different Step or on `Done`, and a Pause marks only its own Step waiting
- [x] A tool in flight shows on the Search Step, with source pills for count, cached and failed; an unknown tool shows its raw name
- [x] A loop adds a "×2" badge, turns the back-edge amber, and resets the Steps between, except optional ones that ran
- [x] Read postings shows a bar and "3 of 6 batches"
- [x] Progress follows §6's weights, an active Step counts 35% (Read postings its batch fraction), `Done` snaps to 100%, and it never decreases
- [x] The bar is amber while waiting and red on `Stopped`, with a line naming the live Step
- [x] `fromLog` places the trace on a reload and reopen, with visit counts from `run_log`
- [x] The panel keeps the live Step in view, and clicking a waiting Step scrolls to its Pause card
- [x] Under about 900 px the panel becomes four band chips, the live-Step line and the bar
- [x] Vitest covers: progress never decreases, loop reset, and a replay giving the same state as live

- [ ] Checked by hand through a Demo Run and a reload at each Pause, and at phone width

11: Motion

What to build: the UI comes alive with GSAP: the trace animates, tool cards resolve, the sidebar's running rows pulse, and the Brief arrives, all without making a streaming page jittery. React still renders every final state, so the animations are never what the UI relies on. Spec: Web UI LLD §10.

Blocked by: 05: Read the Market Brief; 06: Find earlier Runs in the sidebar; 10: Trace panel and progress bar

Status: built; the two by-hand checks (reduced motion, fast stream) are still open

If time runs short, this ticket can be dropped, or cut down one-shots first, and the UI stays correct.

- [x] One `useMotion` helper wraps `useGSAP` and `matchMedia`, and each animated component owns one call scoped to itself
- [x] Every row in §10's table is implemented with the shared duration and easing tokens
- [x] GSAP only plays into a state React already rendered: disabling `useMotion` changes no state or layout
- [x] There is no per-token tween, and the caret is the only motion on prose
- [x] Only `transform`, `opacity` and SVG attributes tween, so progress bars use `scaleX`, not `width`
- [x] Repeating tweens end when their state changes, and none outlives its component under `<StrictMode>`
- [x] A reload, replay or reopened Run plays no one-shot, since `animate` is false until `caught_up`
- [x] At most one loop dot plays per batch
- [x] Scrolling to the live Step is native, and instant under reduced motion
- [ ] Checked by hand under OS reduced motion: no repeating tween runs, and every state still reads
- [ ] Checked by hand: a fast token stream stays smooth with the trace animating, and the console stays clean

12: Browser smoke, one live Run, delete Streamlit

What to build: proof that the whole UI works end to end, in a real browser on a Demo Run and by hand on a real Run, after which the web UI replaces Streamlit. Spec: Web UI LLD §12, §13 pass 8.

Blocked by: 04: Edit the Search Plan; 05: Read the Market Brief; 06: Find earlier Runs in the sidebar; 09: Stop and Resume a Run; 11: Motion

Status: built; the live Run by hand is still open

If 11 is dropped for time, this ticket waits on 10 instead.

- [x] A smoke script drives a Demo Run in a real browser: start, each Pause including a refused edit, reload mid-Run, stop and resume, `Done`, Brief
- [x] It uses the installed Edge through Playwright, with no browser download or repo dependency, and isn't collected by `pytest`
- [ ] One real Run by hand through `scout serve`, with notes on anything that reads badly, fixed or raised
- [x] A fresh production build serves with a single `uv run scout serve`
- [x] `streamlit_app.py`, `streamlit_bridge.py` and the `streamlit` group are deleted, and `uv lock` is refreshed
- [x] `event_line`'s docstring in the CLI and "shared by every client" in the Pause text helpers no longer mention what's gone
- [x] The CLI still works: `pytest` passes in full, and `npm test` passes
- [x] `docs/lld/web-ui.md` still matches what was built, with any change made during the build written back

13: Keep the places the user chose

What to build: a Brief headed "Mumbai" can no longer rest on a country-wide search. Broaden keeps the user's place, and only the user can go wider. Spec: Collection LLD §9, §10. Built on today's single `location`, so it ships even if the many-places ticket doesn't.

Blocked by: none

Status: ready

- [ ] `broaden` pins `location` in code beside `country` and `target_postings`
- [ ] `BROADEN_SYSTEM` loses the country-wide move and says each title is one job title
- [ ] `LowDataAnswer.whole_country` defaults to false, and a checkbox on the low-data card sets it, shown only when the plan has a place
- [ ] Tests: broaden can't blank the place, the checkbox can, and an old checkpoint still loads

14: Show only what a reader needs

What to build: the live view and trace show plain progress and no log text. Spec: Web UI LLD §7.

Blocked by: none

Status: ready

- [ ] `LogKind.result`, written by `collect`, `relevance` and `broaden`, and `NodeDone.results`
- [ ] Turns and the trace show results only, and `fromLog` filters on `kind`
- [ ] Refused, bad-argument and unknown-tool cards read "Skipped: search limit reached" or "Didn't run", while the model's text is unchanged
- [ ] Pills use source names
- [ ] Remove the "Some live text wasn't kept" line and `gap`
- [ ] Contract file updated. Vitest: no route name or plan diff reaches a turn. Old Runs still open

15: Subtle neumorphic look and readable labels

What to build: the whole app takes the new tokens, the plan editor fits one screen, and no raw value shows. Spec: Web UI LLD §15. The look prototype holds the CSS to carry over. Ships with a single place input until ticket 17 turns it into chips.

Blocked by: none

Status: ready

- [ ] Tokens in `styles.css`, with real borders, a solid focus ring and a clear disabled state
- [ ] `labels.ts` with the category table, and country names from `Intl.DisplayNames`, in the editor, plan card and Run Notes
- [ ] Editor: title chips with an add box, 4-column grid, radius and days selects that keep a non-preset value
- [ ] Print: no shadows, white background
- [ ] Checked by hand on a Demo Run at desktop and phone width, with keyboard focus visible everywhere

16: A Brief you can read at a glance

What to build: the answer in seconds, with detail on request. Spec: Web UI LLD §9, Brief LLD.

Blocked by: 15: Subtle neumorphic look and readable labels

Status: ready

- [ ] `brief/glance.py` with `reader_notes`, at most two by the §9 order, on `RunDetail` as `glance`
- [ ] Four tiles, each with a "Why" fold holding its reason and Evidence
- [ ] Reader notes under the tiles, and the Weak Sample notice only through them
- [ ] Three Insights first, highest confidence first, then "Show all", with reason and Evidence folded
- [ ] Run Notes and Postings under a closed "How this was researched", and attribution always visible
- [ ] `InsightDraft.text` asks for at most 20 words, and `outlook()` says "an entry level"
- [ ] Tests: the order and the limit of two, each note on a real Brief, every Caveat still in `brief.md`. Both Briefs in `runs/` open

17: Many places and as many titles as you want

What to build: a Run searches several places and any number of titles, and says what it costs. Spec: Request and Plan LLD, Collection LLD §5, Web UI LLD §8.

Blocked by: 13: Keep the places the user chose; 15: Subtle neumorphic look and readable labels

Status: ready

- [ ] `SearchPlan.locations` with the before-validator for old data, and no `max_length` on titles
- [ ] `PLAN_SYSTEM`: same metro means one centre and a wider radius, bare place names, each title one job
- [ ] `QUESTION_SYSTEM` offers a place's areas and never "Other"; checkboxes on `about == location`, joined with ", "
- [ ] Tools take a checked `location`; per-Attempt cap `max(MAX_FETCHES, titles x places)`; `_fetch_input` lists combinations in order; totals per title and place
- [ ] "Places not searched" Caveat, and `calls_left` with the arithmetic at plan review
- [ ] Editor: Places chips, no title limit. CLI, Markdown and `where()` print the list
- [ ] Tests: old `location` loads, an off-plan place is refused, cap arithmetic, the budget running out reaches the Pause. One live Run: "Mumbai" with Thane ticked

18: Charts in the Brief

What to build: four small charts from Stats and the Sample. Spec: Web UI LLD §9. The look prototype holds the markup and CSS.

Blocked by: 16: A Brief you can read at a glance

Status: ready

- [ ] `glance` gains chart rows with Evidence keys; `PostingRef` isn't changed
- [ ] Pay range from the band's `stat_ids`, skills, places and employers, each hidden by its rule
- [ ] `--chart` tokens, real text rows, every bar with its number, clicking a row opens Evidence
- [ ] `## Market figures` in `brief.md`
- [ ] Tests: places grouping with country-only regions, hide rules, one-employer skills left out, figures equal to Stats. Checked at phone width and in print